Farol — Sprint A documents

Multi-event coordination platform for academic festivals · LABDSOF 2026/27

Generated on 2026-11-01 from `9e9afc2`, in the order of `Docs/README.md`, *Sprint A deliverables*. Source: github.com/tomasoliveira21/LABDSOF. Code (backend/reports, infra/compose.yaml, .github/workflows/ci.yml) is in the repository.

Contents

- **1. Problem and Opportunity Report (with 3. User Research)**
  - Deliverables/problem-opportunity-report-queima.md
  - Deliverables/assumptions-and-dependencies.md
- **2. Market and Competitor Analysis**
  - Deliverables/market-and-competitor-analysis.md
  - Deliverables/market-and-competitor-analysis-product-specific.md
- **4. Product Vision**
  - Product/product-vision.md
  - Product/personas.md
  - Product/user-workflows.md
- **5. Product Backlog**
  - Product/features.md
- **6. Responsible AI Opportunity Assessment**
  - Research/architecture-investigation/responsible-ai.md
  - Technical/plans/ai-provider-evaluation.md
  - Research/architecture-investigation/evaluation/README.md
- **7. Technical Design**
  - Technical/architecture.md
  - Technical/domain-model.md
  - Technical/api-outline.md
  - Technical/local-delivery.md
- **8. Architecture Decision Records**
  - Technical/adr/0001-architectural-style.md
  - Technical/adr/0002-backend-and-frontend-technology.md
  - Technical/adr/0003-database-and-data-ownership.md
  - Technical/adr/0004-service-communication.md
  - Technical/adr/0005-ai-triage-provider-and-evaluation.md
  - Technical/adr/0006-identity-and-access.md
  - Technical/adr/0007-venue-map-representation.md
  - Technical/adr/0008-transport-data-source.md
  - Technical/adr/0009-architecture-documentation.md

    - Technical/adr/0010-cicd-strategy.md
    - Technical/adr/0011-deployment-and-infrastructure.md
    - Technical/adr/0012-observability.md
    - Technical/adr/0013-api-gateway.md
- **9. Security and Privacy Assessment**
    - Technical/security-and-privacy-assessment.md
- **10. Walking Skeleton**
    - Technical/plans/local-walking-skeleton.md
    - Technical/evidence/walking-skeleton-2026-10-31.md
- **Also asked for by the professor**
    - Deliverables/operating-cost-estimation.md
    - Product/non-functional-requirements.md
    - Process/team-and-process.md
    - Process/definition-of-ready-and-done.md
- **Annex: student survey script**
    - Research/inquerito-estudantes-google-forms.gs
- **Annex: architecture diagrams (architecture.md §14) and domain model**
    - Technical/domain-model.svg
    - Technical/diagrams/logical-view-level-1.drawio.svg
    - Technical/diagrams/logical-view-level-2.drawio.svg
    - Technical/diagrams/logical-view-level-3.drawio.svg
    - Technical/diagrams/logical-view-level-3-web.drawio.svg
    - Technical/diagrams/logical-view-level-4-reports.drawio.svg
    - Technical/diagrams/logical-view-level-4-transports.drawio.svg
    - Technical/diagrams/process-view-level-1-help-request.drawio.svg
    - Technical/diagrams/process-view-level-2-help-request.drawio.svg
    - Technical/diagrams/process-view-level-3-help-request.drawio.svg
    - Technical/diagrams/process-view-level-4-help-request.drawio.svg
    - Technical/diagrams/process-view-level-1-transports.drawio.svg
    - Technical/diagrams/process-view-level-2-transports.drawio.svg
    - Technical/diagrams/process-view-level-3-transports.drawio.svg
    - Technical/diagrams/process-view-level-4-transports.drawio.svg
    - Technical/diagrams/implementation-view-level-1.drawio.svg
    - Technical/diagrams/implementation-view-level-2.drawio.svg
    - Technical/diagrams/implementation-view-level-3.drawio.svg
    - Technical/diagrams/implementation-view-level-3-web.drawio.svg
    - Technical/diagrams/implementation-view-level-4-reports.drawio.svg
    - Technical/diagrams/implementation-view-level-4-transports.drawio.svg
    - Technical/diagrams/physical-view-level-1.drawio.svg
    - Technical/diagrams/physical-view-level-2.drawio.svg
    - Technical/diagrams/physical-view-level-3.drawio.svg

- Technical/diagrams/mapping-logical-implementation-level-1.drawio.svg
- Technical/diagrams/mapping-logical-implementation-level-2.drawio.svg
- Technical/diagrams/mapping-logical-implementation-level-3.drawio.svg
- Technical/diagrams/mapping-logical-implementation-level-3-web.drawio.svg
- Technical/diagrams/mapping-logical-implementation-level-4-reports.drawio.svg
- Technical/diagrams/mapping-logical-implementation-level-4-transports.drawio.svg
- Technical/diagrams/mapping-logical-physical-level-1.drawio.svg
- Technical/diagrams/mapping-logical-physical-level-2.drawio.svg
- Technical/diagrams/mapping-logical-physical-level-3.drawio.svg

1. PROBLEM AND OPPORTUNITY REPORT (WITH 3. USER RESEARCH)

`Docs/Deliverables/problem-opportunity-report-queima.md`

Problem and Opportunity Report

Project: Coordination platform for large-scale academic events
Pilot case: Queima das Fitas do Porto (Queimódromo, 8 nights, May)
Course: LABDSOF 2026/27, Sprint A (the assignment's Sprint 1)
Assignment direction: 4.3 *Local Event Coordination Platform*, applied to an event with crowds and road closures
Version: revised with the group's alignment questionnaire and the backlog decisions (October 2026)

Product vision (context)

We propose a **platform built from scratch** to coordinate large-scale academic events. It is a single application that brings together **several events** (Queima do Porto, Coimbra's Queima, an academic week). Each event has an **owner and a team** that configure and run it independently.

- **For the participant:** a web application (PWA) where they choose the event and find the information and help they need during the night.
- **For each event team:** a backoffice where they configure their event (for example, the venue map and transport) and receive and handle incidents.
- **Per-event permissions:** each team only sees and changes its own event.

Out of scope: the physical capacity of the entrances (how many gates there are and how the queue funnel is designed). That is solved with infrastructure, not software.

---

1. Target Community

Community

The academic community that organises and attends large student festivals in Portugal: Queimas das Fitas, academic weeks and similar events. In the **pilot case**, Queima das Fitas do Porto, there are about **250 thousand entries over 8 nights** (evidence E1).

The platform is not built for a single event. Each event joins the platform with its own team, so the community grows event by event without the product having to be redesigned.

Users

Group	Profile	Context of use
Event team (primary user)	The event owner (for example FAP) and its team: organisation, the Risk Behaviour Prevention committee (PCR), security coordination	Before the event, they configure it. During the event, they are in the control room and on the ground, need an overview of the venue and must respond quickly.
Participants (primary user)	Students aged 18 to 25, mostly in groups	At night, phone in hand, limited battery, congested network and often under the influence of alcohol. They need short, immediate information: where everything is, which transport to take, where to ask for help.
Operational partners (secondary users)	STCP, Metro do Porto, PSP, Civil Protection	They are a source or destination of information: reinforced timetables, road closures, incidents.

The two primary users have **the same weight**, by unanimous choice of the group. Each has its own interface: the backoffice for the event team and the web application for the participant.

Why this group

- The problem is concentrated in predictable, recurring, large-scale events, where a failure has safety consequences.
- Each event's organisation has the need, the budget (35% of Queima do Porto's 2026 budget went to security) and the channel to the students (ticketing, wristbands, social media) all at once.
- The format repeats in several cities and every year, which justifies a multi-event platform instead of a solution built for a single event.

---

2. Problem Statement

Statement

> **The organisers of large-scale academic festivals and their participants** face **fragmented and outdated operational information when** tens of thousands of people move each night between the venue, the entrances and transport, **which causes** slow responses to incidents, long and badly distributed queues, and participants who do not know where to go, how to get home or whom to ask for help.

From the participant's point of view:

> **Students who go to the Queima** have no access to reliable, real-time information **when** they are in or around the venue in the early hours, **which leads** to long waits and risky decisions (splitting from the group, walking home, driving).

Root causes

1. **There is no single source of operational information.** Organisation, PCR, security, PSP, transport operators and the City Council each communicate through their own channel: radio, website, Instagram, press releases, the municipality's traffic platform. Nobody has the full picture.
2. **Communication is static.** Timetables, road closures and rules are published before the event. What changes during the night (a congested gate, a late shuttle, an area to avoid) does not reach the participant.
3. **There is no shared spatial reference of the venue.** When there is one, the map is an image posted on social media that is not updated and cannot be used to show where each thing is or where an incident happened.
4. **Incident coordination depends on radio and phone.** This leaves no structured history and does not allow fast prioritisation and routing. It is a hypothesis to confirm in the interview with the security volunteer.
5. **Slow processes carried out at the gate itself.** In 2023, the chaos at the entrance was caused by exchanging the ticket for the wristband at the moment of entry.

Opportunity

Each **event configuration team can upload the venue map** to the platform. The map becomes the common reference: the participant finds their way with it, and the team uses it to place help requests and information. The same model applies to any event, because each team uploads and maintains its own map.

What the problem is and is not

It is the problem	It is not the problem
Slow and badly distributed access processes across gates	Lack of physical capacity at the entrances (infrastructure, out of scope)
Scattered and outdated information during the event	Lack of security: investment is already at a record (more than 500 staff per night, more than 100 cameras)
Incident coordination without a shared view	Risk behaviours themselves (alcohol, violence). They are cultural. The product can shorten the response time, but it does not remove the cause.
	"There is no app": FAP already has one, but it does not cover event operations

Consequences of not solving it

- Crushing episodes and panic attacks in the queues, as in 2022 and 2023.
- Longer time until assistance reaches those who need it.
- Loss of reputation for the organisation, with public complaints and negative press coverage.
- Journeys home in the early hours without information, with a higher risk of drink-driving and unsafe travel.

---

3. Stakeholder Map

Stakeholder	Role	Interest	Influence	Position
Academic federation (FAP)	Organiser, customer and decision-maker	High	High	For
PCR committee and support teams	Operational user	High	Medium	For
Student participants	End users	Medium	Low individually, high collectively (social media, complaints)	Neutral. They adopt it if it is useful and immediate.
Private security	Execution on the ground	Medium	Medium	Neutral
PSP and Civil Protection	Public order and emergency	High	High	Neutral or sceptical (they have their own systems and protocols)
INEM and fire brigades	Medical response	High	Medium	Neutral
STCP and Metro do Porto	Transport	Medium	Medium	For, if it does not require heavy integration
City Councils (Porto and Matosinhos)	Licensing and traffic	Medium	High	Neutral (they already have their own traffic platform)
Supplier of the federation's current app	Competitor or partner	High	Medium	Against, if it feels replaced
Ticketing (BOL, FNAC)	Partner needed to validate access	Medium	High	Neutral
Education institutions (ISEP, universities)	Institutional influencers, communication channel with students	Medium (student wellbeing and safety)	Medium	For
Each faculty's student associations	Co-organisers (parade, stalls) and direct channel to students	High	Medium	For, if they have a say in the configuration
Sponsors and concessionaires	Fund the event	Low	Medium	For, for the visibility
CNPD (data protection)	Regulator	Low, rises if there is location data	High	Neutral, but a constraint
Parents and families	Influencers	High on safety	Low	For

Influence × interest matrix

- **Manage closely** (high influence, high interest): FAP, PSP and Civil Protection.
- **Keep satisfied** (high influence, medium or low interest): City Council, ticketing, CNPD.
- **Keep informed** (low or medium influence, high interest): PCR, INEM, families, current app supplier, each faculty's student associations.
- **Involve as a channel:** education institutions (outreach and surveys).
- **Monitor:** sponsors, concessionaires.

Conflicts of interest

1. **New platform vs. current supplier (the most critical, according to the group).** The supplier of the federation's app may see the product as a threat and not cooperate on integration. Mitigation: come in as a complementary module, through a link or QR code, without requiring changes to the existing app.
2. **Transparency vs. reputation.** Showing real queue times helps the participant, but exposes the organisation's failures.
3. **Monitoring vs. privacy.** Density and location help operations, but require data minimisation and consent (GDPR).
4. **Revenue vs. safety.** Showing real-time occupancy may pressure the organisation to limit entries, which reduces revenue.
5. **Yearly mandates.** Student association boards change every year, which puts continued adoption at risk.

---

4. Problem Evidence

Strong evidence (verifiable public facts)

#	Evidence	Source
E1	About 250 thousand participants expected over 8 days (2025 and 2026)	Jornal da Maia, Observador
E2	Security takes 35% of the budget: more than 500 staff per day, 12 watchtowers, more than 100 cameras, PCR with 90 people	JN, Rádio Festival
E3	2022 and 2023: queues of more than 500 m, waits of more than 3 h, reports of crushing and panic attacks, complaints about "lack of information and signage"	Renascença, Público
E4	The cause identified in 2023 was exchanging the ticket for the wristband at the entrance. The organisation changed the venue after the complaints.	JN, Notícias ao Minuto
E5	Special transport operation (shuttle every 15 min or less, extra metro and bus services) and road closures, announced through separate channels	Andante, STCP
E6	The FAP app exists and covers tickets, discounts and sport, but not event operations	FAP
E7	Crowd density estimated from a festival app correlated above 0.8 with video measurement (technical feasibility)	arXiv 1306.3133

Medium or weak evidence

- Serious incidents in 2026: one rape complaint at the Queimódromo do Porto and two in Coimbra. They show the need for fast help-request channels, but do not prove that an app would have prevented anything.
- Event app adoption rates between 55% and 65% with promotion. The figures come from vendor blogs (Cvent, Lineupr), which have a commercial interest.
- The idea that "students want real-time information" is common sense. The student survey (n = 31, *User Research*) points the same way: 87 % would use the app and 39 % chose real-time announcements, but this is a small, non-random sample.

Gaps and primary validation plan

The group decided to go ahead with two validation methods:

Gap	Method
What information students look for during the night, whether the lack of information is still a problem and whether they would use an application for it	**Online student survey** (target: 100 or more answers)
How security coordinates incidents today, which channels it uses and where the delays are	**Interview with a security volunteer** at the Queima

Accepted limitation: the group has no direct access to the organisation (FAP) or to its internal queue and incident data. The view of operations comes from the volunteer interview and from public data. As the assignment requires, no user-research data in this report was invented. The limitations are documented.

---

5. Assumptions and Risks

Critical assumptions

#	Assumption	Risk	Ease of validation
A1	Organisations accept configuring and maintaining their event on the platform (map, transport, team)	High	Medium (interview and conversation with organisations)
A2	Participants check the application during the night, even in adverse conditions (alcohol, battery, noise)	High	Medium (survey and prototype)
A3	Participants adopt one application shared by several events, opened through a link or QR code, instead of one app per event	High	Easy (survey)
A4	Help requests made through the application reach the right team faster than through the current channels	High	Medium (volunteer interview and simulation)
A5	Each organisation has a usable venue map (image or PDF)	Medium	Easy

#	Assumption	Risk	Ease of validation
A6	Real-time information (transport, queues) can be entered or estimated by the event team, without direct integration with the operators	Medium	Medium
A7	AI features are more useful than a non-AI alternative and do not mislead users	Medium	Easy (simulated dataset and comparison with a non-AI alternative)
A8	The mobile network holds up at peak, or the application works in degraded or offline mode	Medium	Easy (tests and reports)
A9	The same product serves different events through configuration alone, without custom development	Medium	Medium

The most critical assumptions are **A2, A3 and A4**. If participants do not use the application during the night, or if help requests do not get faster, the product loses its reason to exist. The survey validates A2 and A3; the interview with the security volunteer validates A4.

Risks

Market and adoption

- Use is concentrated in a few days a year per event, with little retention. The multi-event platform softens this, because the same user comes back for other events.
- Convincing students to use yet another application. Mitigation: access through a link or QR code, no mandatory installation, with a single account for all events (mandatory login, decided on 9 October 2026).
- The supplier of each federation's current app may copy the features.
- Student association boards change every year, which makes each event's continuity on the platform uncertain.

Technical

- **Permissions across events:** a bug in the authorisation system can expose one event's data or incidents to another event's team. This requires automated tests dedicated to this isolation.
- Load peaks of tens of thousands of requests within minutes, for example at the end of a concert.
- Congested mobile network at the venue: requires caching, offline mode and short messages.
- Outdated or wrong information can make things worse (for example, a transport that no longer runs). Requires a clear indication of when each piece of data was updated.
- Phone location inside a crowded venue is inaccurate. The participant must be able to mark their location on the map.

Execution

- **Large scope for 3 sprints.** Mitigation: prioritise help requests, the map and transport information in the MVP. The rest comes in by order of value.
- Limited access to the real organisation during the semester. Mitigated with the volunteer interview, public data and simulation, documenting the limitation.

Regulatory and ethical

- GDPR: location and help requests (especially health-related ones) are sensitive data. Explicit consent, minimisation and short retention apply.
- A help button creates an expectation of response. The application must make clear that it is not an emergency service.
- AI-generated content cannot be presented as verified fact, as the assignment requires (5.6.2).

Key questions before the solution phase

1. **Would students use an application during the night**, and what information would they look for first? (survey)
2. **How does a help request reach the right team today**, and how long does it take? (interview with the security volunteer)
3. **Would event organisations be willing to configure and maintain their event** on a shared platform?

User Research

Research method

Item	Detail
Instrument	Online questionnaire in Google Forms, "*Queima das Fitas: como vives a noite do evento?*", 11 questions in Portuguese, about 3 minutes. Script: `inquerito-estudantes-google-forms.gs`
Anonymity	No name or e-mail collected; answers used only in aggregate
Collection	**3 to 6 October and 30 October 2026.** Shared by the team. **Collection closed on 30 October 2026**; later answers are not counted
Answers	**33** received, 1 marked invalid (see *Cleaning*), **32** valid. 1 person has never been to an academic festival and does not plan to go, so the form ended after question 1; 1 has never been but plans to go, so they skipped the questions about the experience. **n = 30** for questions 2 to 6 and **n = 31** for questions 7 to 10
Flow	Question 1 filters: people who have been answer about their last event (Q2–Q6) and about an application (Q7–Q10); people who plan to go answer only about the application; people who never went and do not plan to go end the form
Assumptions tested	A2 and A3 (Q6–Q9), A4 (Q3, Q5, Q10), the problem itself (Q3, Q4, Q6)
Cleaning	One answer was marked **invalid** and excluded entirely, because its free-text answers are jokes. Free-text answers in the *Outro* (other) option were also excluded, because they cannot be compared with the closed options (one in Q2, one in Q3); the closed answers of those two respondents were kept. Nothing else was changed
Analysis	Counts and percentages per option. Multiple-choice questions (Q2, Q3, Q4, Q8, Q9) add up to more than 100 %

The group also used public news, official event communications and transport information (§4). The interview with a security volunteer (§4, assumption A4) **has not been done yet**.

Results

Most respondents are experienced festival-goers: 25 of 32 have been several times and 5 once. Of the 30 who went, 25 have been to the **Queima do Porto**, 9 to Coimbra's and 1 to the Enterro da Gata in Braga.

What went wrong the last time (Q3, n = 30)

Situation	Answers
Waited more than 30 minutes to get in	**24 (80 %)**
Got separated from my group	14 (47 %)
Needed help (health, security, theft) and did not know whom to ask	8 (27 %)
Did not know where something was in the venue (WC, stage, first aid)	6 (20 %)
Did not know how to get home or when there was transport	5 (17 %)

Situation	Answers
Was the victim or witness of a theft	3 (10 %)
None of these	1 (3 %)

29 of 30 (97 %) went through at least one of these situations.

Where they looked for information (Q4, n = 30): the organisation's Instagram or social media 23 (77 %), friends or WhatsApp groups 18 (60 %), the federation app (e.g. FAP) 9 (30 %), the official website 8 (27 %), signs and staff at the venue 6 (20 %), *I did not look* 1 (3 %).

How easy it was to find information during the night (Q6, 1 = very hard, 5 = very easy, n = 30): 1 → 6 answers, 2 → 3, 3 → 9, 4 → 10, 5 → 2. **Average 2.97**, median 3.

First thing they would do if they needed help inside the venue (Q5, n = 30): ask friends 12 (40 %), look for a security guard or staff member 10 (33 %), go to the first-aid post 4 (13 %), *I don't know* 3 (10 %), call 112 1 (3 %).

Would they use an application like this during the night (Q7, n = 31): *yes, definitely* 13 (42 %), *probably yes* 14 (45 %), *probably not* 4 (13 %), *no* 0. **27 of 31 (87 %) positive.**

Most useful features (Q8, up to 3, n = 31)

Feature	Answers
Venue map (WC, stages, support points)	**16 (52 %)**
Help button (health, security, theft) that alerts the organisation	15 (48 %)
Timetable and location of transport home	14 (45 %)
Expected waiting time at the entrances	12 (39 %)
Real-time announcements from the organisation (changes, alerts)	12 (39 %)
Electronic ticket in the application	10 (32 %)
Assistant to answer questions about the event	3 (10 %)

What would stop them from using it (Q9, up to 2, n = 31): battery use 15 (48 %), having to install another app 12 (39 %), weak network at the venue 10 (32 %), not trusting that the information is up to date 7 (23 %), *nothing, I would use it* 5 (16 %), having to share my location 2 (6 %).

Location when asking for help (Q10, n = 31): share the phone's location 22 (71 %), only by marking the place on the venue map myself 8 (26 %), no 1 (3 %).

Q11 (optional, open): 1 answer, not about the product.

Against the validation targets (`product-vision.md` §5):

#	Target	Result	Reading
V1	≥ 60 % *yes, definitely* or *probably yes* in Q7	**87 %** (27/31)	Met. It is a stated intention, not observed use
V2	Q6 average ≤ 3 confirms that finding information is a problem	**2.97**	Met, by a small margin, and the result depends on the answers of 30 October (see *Limitations*). To confirm with a test in night conditions. The pain reported most is the wait at the entrance (80 %)

What this says about the assumptions

- **A2** (they use the app during the night): supported as an intention (87 %). The barriers are physical, not of interest: battery (48 %), installing another app (39 %) and weak network (32 %).
- **A3** (one app, opened by link or QR): 39 % name installing another app as a barrier, which supports the no-installation choice. The survey did not ask about one app for several events directly.
- **A4** (help through the app is faster): **not validated.** 8 of 30 needed help without knowing whom to ask, and the first reaction is people nearby (friends, staff, first aid), not a phone. 71 % would share the phone's location with a request and 26 % only a place on the map. The volunteer interview is still missing.
- **Root cause 1** (no single source): supported. Information comes from social media and WhatsApp groups (77 % and 60 %); official channels (website, FAP app, signs) are each 30 % or less.

Limitations

- **Small sample.** n = 30 against a target of 100 or more (§4). Each answer is about 3 percentage points; the percentages show direction, not precise values, and no statistical test was done.
- **Convenience sample.** Answers came from the group's own reach, not from a random sample of students. Most went to the Queima do Porto (25 of 30) and have been several times (25 of 32). The survey did not ask for age, course or institution.
- **When the answers were collected.** The 12 answers of 30 October came from contacts of a team member, after the first results were known; without them the Q6 average is 3.7.
- **Answered away from the event.** People answered at home, calmly, not during the night with noise, a crowd, low battery and a weak network. Q6 depends on how each person remembers the night. It needs a test in night conditions (usability test of SC1 and SC11).
- **Memory.** Q3 to Q6 ask about "*the last time*", which may be one or more years ago.
- **Intention is not behaviour.** Q7 measures what people say they would do. The form describes the app in a positive way before Q7 (*opens by link or QR, no installation required*), which may push answers up.
- **Forced choices.** Q8 allows at most 3 options and Q9 at most 2; a low count means *less chosen*, not *not wanted*.
- **Cleaning.** One answer marked invalid and two *Outro* free-text answers excluded. Counting the invalid answer, Q7 would be 84 % (27/32) and the Q6 average 2.97, so the conclusions do not change.
- **Only one side.** The survey covers participants. The event team's side (assumptions A1, A4, A6) has no primary data yet: the security-volunteer interview has not been done and the group has no access to FAP.

Resulting product decisions

Decided on 30 October 2026 from the survey data (#164), and presented to the team at the next planning meeting. Decisions 2, 3 and 4 are applied in `features.md` (30 October 2026); the board fields still have to

follow.

#	Decision	Data	Before → after
1	**The venue map stays the centre of the participant app** (F5, Must)	Map is the most chosen feature: 16 of 31 (52 %), with the help button close behind (15, 48 %)	Confirmed
2	**Entrance status per gate (F17) moves from Could to Should.** It stays a manual status set by the team; no automatic prediction (out of scope; assumption A6)	Waiting more than 30 min is the most common problem, 24 of 30 (80 %); waiting time chosen by 12 of 31 (39 %)	**Changed:** Could → Should
3	**Weak network and battery become design constraints of the participant pages:** light pages, few requests, the last-update time always visible. Weak-network mode (F11) goes back to sprint planning as a candidate for Should	Battery is the top barrier, 15 of 31 (48 %); install 12 (39 %); weak network 10 (32 %)	**Changed:** F11 reopened (Could, to review)
4	**The help request asks for the phone's location. If the participant allows it, it is used; if they refuse, they mark the place on the venue map.** The help screen also shows the nearest first-aid and staff points	22 of 31 (71 %) accept phone location, 8 only the map, 1 neither. 14 of 30 would first go to staff or first aid	**Changed:** map only → phone location first, map as the fallback. Phased on 1 November 2026 (#175): map only in Sprint B, phone location from Sprint C, after ADR-007 and assessment M-1 are amended
5	**AI stays on help-request triage, not on a participant assistant** (decision of 8 October 2026, `features.md` §5)	The assistant is the least chosen feature, 3 of 31 (10 %)	Confirmed

Problem statement: the report keeps the fragmented-information problem (root cause 1 is supported by Q4). Q6 meets V2 (2.97), but the result depends on the answers of 30 October, so the team confirms it with a test in night conditions. The strongest pain participants reported is the entrance wait, which is why decision 2 brings live entrance information forward. The statement is reviewed again after the volunteer interview.

Product vision

The product vision, personas, value proposition, principal user journeys, measurable success criteria and MVP definition are kept in their own documents, which replace the first draft that stood here: Product Vision, Personas, User Workflows and Main Features and Backlog. The first draft is kept below because those documents cite it; where they differ, they win.

Personas

The main personas are the event team and the student participant. The event team includes the owner, the organisation members, the prevention teams, security coordination and other operational roles. This team uses the backoffice before and during the event to configure the event, manage the map, update transport

information and handle incidents. The student participant is usually between 18 and 25 years old, goes to the event with friends and uses their phone during the night. This user needs short, reliable and easy-to-reach information, especially in a context where battery, network quality, noise and alcohol consumption can affect decision-making.

Value proposition

The platform gives participants a single, reliable place to find practical information about the event and to ask for help when needed. At the same time, it gives the event team a shared operational view, helping it understand what is happening, where problems are and how to respond faster. The product's value is not only in publishing information, but in keeping that information tied to the real situation at the event.

Main user journeys

A participant opens the web application through a link or QR code, selects the event, checks the map, finds entrances, services, transport options and safety information, and can ask for help if something goes wrong. The event team creates or selects an event in the backoffice, uploads or configures the venue map, adds key points such as entrances, stages, toilets, first aid and transport stops, and updates information during the night. When a participant sends a help request, the event team receives it with context and location, assesses its priority and routes it to the right operational team.

Success criteria

The product will be successful if students can quickly find the information they need during the event, if help requests are easier to submit and manage, and if the event team gains a clearer view of what is happening at the venue. Success should also be measured by adoption during the event, by less confusion around entrances and transport, by a faster operational response and by positive feedback from students and event teams.

MVP definition

The MVP should focus on the core coordination problem. It should include a web application for participants with event selection, a clear venue map, essential event information, transport information and a simple help-request flow. It should also include a backoffice for the event team, where each team can configure its own event, manage the venue map, update relevant information and view incoming incidents. Features that require heavy integrations, advanced crowd prediction or full automation should stay out of the first version, unless they are essential to validate the product.

Out of scope: the physical capacity of the entrances, including the number of gates and the physical design of the queue funnel. This is an infrastructure problem, not a software problem.

Assumptions and Dependencies

> LABDSOF 2026/27 · Sprint A · #162
> **Status (31 October 2026):** one list for the deck, put together from the documents linked in each row, with nothing new decided here (meeting with the professor, 22 October 2026, *Presentation contents*). When a source changes, change this list with it.

1. Product assumptions

From the Problem and Opportunity Report §5. The most critical are **A2, A3 and A4**: if any of them fails, the product loses its reason to exist. Validation status from `product-vision.md` §8.

#	Assumption	Risk	Planned validation	Status (31 October 2026)
A1	Organisations accept configuring and maintaining their event on the platform (map, transport, team)	High	Interview and conversation with organisations	Not validated: no conversation with an organisation is recorded
A2	Participants check the application during the night, even in adverse conditions (alcohol, battery, noise)	High	Survey and prototype	Supported as an intention: 87 % would use the app during the night (survey Q7, n = 31); the barriers are battery, installing another app and weak network (report, *User Research*)
A3	Participants adopt one application shared by several events, opened through a link or QR code	High	Survey	Partly supported: 39 % name installing another app as a barrier, which supports opening by link or QR; one app for several events was not asked directly (report, *User Research*)
A4	Help requests made through the application reach the right team faster than through the current channels	High	Volunteer interview and simulation	Not validated: the first reaction to a problem is people nearby, not a phone (survey Q5); the interview with a security volunteer is still to do (product vision §8)
A5	Each organisation has a usable venue map (image or PDF)	Medium	No method planned yet (the report rates it easy to validate)	Not validated. The design takes an image or a one-page PDF (ADR-007, proposed)
A6	Real-time information (transport, queues) can be entered or estimated by the event team, without direct integration with the operators	Medium	No method planned yet (the report rates it medium)	Changed for buses: live positions come from the Porto Digital API (ADR-008); event shuttles are still entered by the event team (ADR-008, decision 3)
A7	AI features are more useful than a non-AI alternative and do not mislead users	Medium	Simulated dataset, compared with a non-AI alternative	Dataset of 100 synthetic cases and a reproducible baseline in the triage evaluation; label review and model runs pending (guide K2, ADR-005)

#	Assumption	Risk	Planned validation	Status (31 October 2026)
A8	The mobile network holds up at peak, or the application works in degraded or offline mode	Medium	Tests and reports	Not validated. Designed for: data shown with its update time (SC7), light polling at the peak (`architecture.md` §10.1); offline mode is F11, a *Could* (SC10)
A9	The same product serves different events through configuration alone, without custom development	Medium	No method planned yet (the report rates it medium)	Not validated. Designed for: per-event data, roles and teams (`features.md` §4, ADR-003)

Other assumptions, kept in their own documents: the cost assumptions (`operating-cost-estimation.md` §2) and the peak-load assumptions (`architecture.md` §10.1).

2. External dependencies

Systems and services outside the team's control that the product or its delivery needs. The pinned third-party images that run inside the local stack (PostgreSQL, RabbitMQ and the observability stack) are part of the delivery, not external services (ADR-011). Keycloak and WireMock are pinned images too, but they are listed below because the product's behaviour depends on them.

#	Dependency	Used for	Decision	If it is unavailable	Status (31 October 2026)
X1	**Porto Digital urban platform** (FIWARE NGSI v2, open data, free)	Live bus positions (F9)	ADR-008, accepted by the professor	Degraded mode D2: the last valid data, with its time and a warning (`architecture.md` §9)	Accepted. 300 real responses recorded on 25 October 2026 for tests and the demo (`infra/wiremock/`). Not integrated yet
X2	**Local AI model:** Ollama with `qwen3:4b` Q4_K_M	Triage suggestions (F8)	ADR-005, guide K1	Degraded mode D1: the request reaches the operator untriaged, in order of arrival	Accepted as architecture. Measurements on the demo machine pending (guide K1)
X3	**Keycloak**, self-hosted	Login, tokens, account deletion (Admin API)	ADR-006	Tokens validated with cached keys (`architecture.md` §9.1); account deletion answers `202` and retries (guide D7)	Accepted; to be built in Sprint B. HTTPS for the phones is open (guide G4)
X4	**WireMock**	Simulator of Porto Digital and of the AI provider, for tests and the demo	Guide J4 (26 October 2026, #155), ADR-008	Point the URL at an unreachable address and restart the service (`architecture.md` §9.2)	Stubs tested with WireMock 3.13.2

#	Dependency	Used for	Decision	If it is unavailable	Status (31 October 2026)
X5	**Demo laptop, backup laptop and a LAN** (hotspot or router)	Local delivery and the demo	ADR-011, `local-delivery.md`	The backup laptop, with the same tag and the restored dumps (guide G4)	Minimum RAM and recovery time still to measure (ADR-011)
X6	**GitHub** (repository, Actions, Projects, Copilot) and **SonarQube Cloud**	Code, CI, backlog, code review, static analysis	ADR-010	Delivery builds locally from the git tag (guide H5)	In use. Branch protection on `main` has been active since 30 October 2026, with no required check yet (README §6)

2. MARKET AND COMPETITOR ANALYSIS

`Docs/Deliverables/market-and-competitor-analysis.md`

Market and Competitor Analysis (general)

> LABDSOF 2026/27 · Sprint A
> **What this document is:** the **general** view of the academic event management market in Portugal, phase by phase, with everything that exists today. The **specific** analysis of the features we want to include in the project is in the annex `market-and-competitor-analysis-product-specific.md` (annex 02a).
> **Aligned with the repository** (`Docs/Deliverables/problem-opportunity-report-queima.md`, commit `442ecc3`, 2026-10-03): a **multi-event** coordination platform for large-scale academic events, with a web application for the participant and a backoffice for each event's team. Pilot case: Queima das Fitas do Porto.
> **Target market:** Portugal. International references are for comparison only.
>
> **Legend:** no mark = fact with a source · "Not identified" = we did not find it in the sources (it does not mean it does not exist). Competitors' features are the ones they claim themselves; we did not test them.
>
> **Decisions taken since 6 October 2026** (added 31 October 2026, #162). This analysis was written on 6 October; the team decided later, and where the text below differs, these decisions win:
>
> - **Transport:** live positions of the city buses near the venue come from the Porto Digital urban platform, plus the event shuttles and temporary stops entered by the team (ADR-008, F9 in `features.md`). A link to Google Maps is no longer the plan for regular lines.
> - **AI:** help-request triage was chosen on 8 October 2026; the participant Q&A assistant was rejected (a wrong transport answer at night is a safety risk; `features.md` §5).
> - **Weak network:** offline mode (cached map and information) is F11, a *Could* since 9 October 2026; the time of the last update on each item stays in the MVP (SC7, product vision).

---

1. Context

Indicator	Value	Source
Students in higher education (2025/26)	447,680	ECO
Budgets of the big festivals (2023)	Porto €2.5 M · Coimbra €1.5 M · Braga €0.9 M · Aveiro €0.6 M	Notícias Magazine
Queima do Porto 2026	About 250,000 people over 8 nights; 35% of the budget on security; more than 500 staff per day	Observador
Queima de Coimbra 2026	€2.2 M; more than 210,000 visitors	As Beiras
Queues at Queima do Porto (2022 and 2023)	Queues of more than 500 m, waits of more than 3 h, reports of crushing; complaints about "lack of information and signage"	Renascença, Público

Today, each phase of the event is handled by a different tool, and the information that changes during the night does not reach the participant (Problem and Opportunity Report, section 2).

---

2. Alternatives by phase

Phase 1 — Availability and purchase *(outside the product; partners)*

Alternative	Strengths	Weaknesses	AI
BOL (Queimas do Porto and Coimbra, Enterro da Gata)	Leader in academic festivals; shop branded with the event; physical network; fights resale	6% fee + VAT; ends at the sale	Not identified
Ticketline / MEO Blueticket / Fnac	Well-known brands; physical points of sale	About 6% fee; only handle the sale	Not identified
Fever (+ DICE) / Shotgun	Event discovery; young audience; managed resale	Focused on consumption and marketing; organiser pricing is not public	**Yes** (Fever): recommendations and demand forecasting
Eventbrite	Self-service; check-in app	Expensive on cheap tickets (about 15.6% on a US$20 ticket)	**Yes**: generates descriptions, emails and ads
APP FAP	Official app; student discount; tops up the cashless wristband	A single federation; does not cover event operations	Not identified
Status quo (Instagram + Google Forms + MB Way)	Free and familiar	Payments confirmed by hand; no control of capacity (how many tickets are left)	—
International ref.: **FIXR / native** (United Kingdom)	Built for student unions; student-only tickets	Not present in Portugal; do not handle the venue	Not identified

Phase 2 — Entry and access control *(the product only informs; physical capacity is out of scope)*

Alternative	Strengths	Weaknesses	AI
Queima do Porto cashless wristband (entry + payment)	One wristband for everything; less fraud	In 2023, exchanging the ticket for the wristband at the gate caused hours of queuing (JN); the participant does not know which gate is less crowded	No
The ticketing platform's own scanners (BOL, Eventbrite, pretix)	QR validation included	Each platform has its own scanner; no real-time occupancy per zone	No

Alternative	Strengths	Weaknesses	AI
Paper or Excel lists (small events)	Free; work without a network	Fraud, queues, no data	—
International ref.: **Weezevent** (France)	Ticket, entry (QR/NFC, zones, offline) and cashless in one platform	Professional festivals; no known presence in Portugal	Not identified
International ref.: **Waitwhile** (virtual queue)	Join the queue by link or SMS, with an estimated wait time; more than 30,000 people managed at the Tribeca Film Festival (case)	Designed for experience and service queues, not for the gates of a venue with tens of thousands of people	Not identified

Phase 3 — During the event

3a. Participant information and wayfinding at the venue

Alternative	Strengths	Weaknesses	AI
Instagram, event website and map as an image (status quo)	It is where the audience is; zero cost	Static information, published before the event; the map is an image that is not updated; does not reach those already at the venue	—
APP FAP	Porto student base; official brand	Tickets, discounts and sport; no map, operational alerts or help (according to FAP's page)	Not identified
Portuguese festival apps (NOS Alive, Rock in Rio Lisboa)	Line-up, map, schedule, shuttle information	One app per festival; information only; no help request	Not identified
BindePaFesta (PT, launched in June 2026)	A single map of the country's village and popular festivals; reached the top of the Portuguese App Store (RFM)	For discovering festivals, not for wayfinding inside the venue or for operations	Not identified
International ref.: **Appmiral** (CM.com)	Mature festival app: map, timetables, push, SOS button with location	€375–935/month; one app per festival; requires installation	Not identified
International ref.: **Woov**	One app for many festivals; map, timetables, friends; has a Portuguese interface (App Store)	Requires installation and collects exact location; no help request or operations	**Yes**: automatic chat moderation
International ref.: **LineUpr** (Germany)	Event app as a PWA, no installation; yearly subscription for several events (pricing)	Built for conferences; no help or incidents	Not identified
International ref.: **42Chat FestivalBot** (USA)	SMS or web assistant that answers questions about the event (more than 60 topics) (site)	Questions and answers only; pricing not public; does not say how it handles what it cannot answer	**Yes**: chatbot

3b. Payments at the venue *(outside the product)*

Alternative	Strengths	Weaknesses	AI
Zpos (POS, PT)	Certified by the Tax Authority; MB Way; works offline for up to 5 days; from €19.90/month	Payments and invoicing only	Not identified
SumUp / ifthenpay	Cheap and well known	Payment only	No

3c. Help requests, incidents and team coordination

Alternative	Strengths	Weaknesses	AI
Radio + WhatsApp groups	Immediate; radio works without a network	No record, no priority, no precise location; do not reach the participant	—
WhatsApp/QR line + Ponto Lilás + colour-coded volunteers (Queima do Porto)	Human, familiar, no installation	Requests without category, place or status; leaves no data	—
Zello (walkie-talkie on the phone)	Replaces radios for US$7–13/user/month	Depends on the network; unstructured voice	Partial: transcription
International ref.: **Raven Controls** (COP26, FIFA)	Full incident → response cycle; the public reports by WhatsApp/QR	Enterprise; no PT; designed for professional control rooms	**Yes**: suggests category, priority and procedure
International ref.: **CriticalArc SafeZone** (campus)	Three help buttons; location only at the moment of the alert; about 25% of UK universities	Campus only; requires an app; institutional licence	Not identified
International ref.: **Rosterfy / VolunteerLocal**	Volunteer rosters and check-in	Do not handle incidents in real time	**Yes** (Rosterfy)

Phase 4 — Getting home and mobility

Alternative	Strengths	Weaknesses	AI
STCP, Metro and Andante announcements	Official information on shuttles and extra services (STCP, Andante)	Published before the event, on separate channels; do not say what changed during the night	—
Google Maps with STCP real time (since 9 June 2026)	Bus location and minutes to arrival at 2,517 stops (Renascença)	The news does not say whether it includes the Queima's special shuttles **[H: to verify]**; does not know the venue or the temporary stops	Not identified
Moovit / Anda app	Journey planning; Andante ticketing	Do not know the event context	Not identified
Ride-hailing (Uber, Bolt)	Trip sharing and emergency button	Price and wait at peaks; not connected to the organisation	Not identified

Phase 5 — After the event

Alternative	Strengths	Weaknesses	AI
Each supplier's reports (BOL, cashless, Zpos)	Reliable sales and consumption data	Each one shows only its own part	**Yes** (Fever)
Excel and manual reports	Flexible	Slow; incidents handled by radio leave no history; knowledge is lost when the board changes	—

Summary

Phase	Situation in the Portuguese market	Our product
Purchase	**Well served** (BOL, Ticketline, Fever), with about a 6% fee	Out. Partner
Entry	Technically solved at large events; the 2022 and 2023 queues show a lack of **information** for the participant	Information only (gates, alerts). Expected wait time at the gates is one of the features the survey asks about (component C8 of the annex)
During: information	**Static** (Instagram, map image); existing apps cover a single event or a single federation	**In**: map, information and alerts
During: help and incidents	**Fragmented and unrecorded** (radio, WhatsApp)	**In**: help request and backoffice
Getting home	Scattered official information; STCP real time reached Google Maps in 2026	**In**: the event's transport information
After the event	**Manual**	Side effect: the incident history is kept
AI	Exists in marketing (Eventbrite, Fever), in professional control rooms (Raven) and in Q&A assistants (42Chat). **We found no AI in the operation of academic events in Portugal**	Help-request triage, with a non-AI alternative (decided 8 October 2026; the Q&A assistant was rejected)

---

3. Differentiation opportunity

The market problem: those who inform the participant (Instagram, festival apps) do not handle incidents; those who handle incidents (radio, professional control rooms) do not talk to the participant. In Portugal, these two halves live on separate channels and neither is recorded.

Our proposal (repository): a multi-event platform with two sides connected by the same data.

Side	What it does	What it gains from being connected to the other
Participant web application (link or QR, no installation, with login)	Chooses the event; sees the map, essential information and transport; asks for help	The request reaches the team with context and a place on the map

Side	What it does	What it gains from being connected to the other
Event team backoffice	Configures the event (map, points, transport); updates information during the night; receives, prioritises and routes incidents	What the team updates appears to the participant immediately; each team only sees its own event

Why it is different from the alternatives:

1. **It connects participant and operations.** Festival apps stop at information; radio and WhatsApp stop at operations. The closest reference (Raven Controls) is for professional control rooms and has no Portuguese version.
2. **Multi-event with independent teams.** One application for several Queimas and academic weeks. Classic festival apps are one per event (Appmiral); multi-event ones (Woov, LineUpr) have no operations.
3. **The map as the common reference.** The participant finds their way with it and the team places requests on it. In the Portuguese status quo, the map is an image on social media; the festival apps that have one (NOS Alive, Rock in Rio Lisboa) cover a single festival and do not place help requests on it.
4. **No installation.** Appmiral, Woov and SafeZone require a native app. Every piece of information shows its last-update time; offline mode is a *Could* (F11).
5. **Information that changes during the night**, with the time of the last update. The status quo is static.
6. **Local:** European Portuguese and the event's special transport.
7. **AI with a non-AI alternative.** It is not a differentiator in itself (Raven and 42Chat already use it); what counts is that it is evaluated and has a fallback, as the assignment requires.

Risks to manage:

Risk	Response
"We already have Instagram and WhatsApp"	Show the gain in time to help and in up-to-date information; QR entry as fast as WhatsApp
The supplier of the federation's current app feels replaced	Come in as a complement, through a link or QR, without requiring changes to the existing app
An international supplier (Appmiral, Woov) moves down-market	Operations and incidents, which their products do not cover; Portuguese context
Use concentrated in a few days a year	Multi-event: the same user comes back for other events
Large scope for the MVP	Priority: help request, map and transport (as in the repository)

The detail per feature (alternatives, comparison matrix, prices and hypotheses to validate) is in annex 02a.

---

4. Main sources

Problem and Opportunity Report · Ticketing fees in Portugal (Observador) · BOL — Enterro da Gata 2026 · Fever — Data & Analytics · Shotgun · Eventbrite — AI tools · Eventbrite fees 2026 · APP FAP · Queima cashless wristband (JPN) · FIXR · native · Weezevent · Waitwhile — Tribeca · BindePaFesta (RFM) · Appmiral — pricing ·

Woov (App Store) · LineUpr — pricing · 42Chat FestivalBot · Zpos · SumUp PT · ifthenpay · Zello · Raven Controls · CriticalArc SafeZone · Rosterfy · STCP — Queima service · STCP real time on Google Maps (Renascença, 2026).

Remaining sources in annex 02a, section 9.

---

5. How AI was used in this research

5.1 Tool and role

Item	Description
Tool	**Claude** (Anthropic), in a project called "LABDSOF" that keeps the working documents, with web search and page reading enabled
AI's role	**Research and writing assistant.** It searched for alternatives, read supplier pages and the press, summarised and organised them into tables
Human role	The team member who wrote this analysis set the scope and the questions in each round, decided what went in and out, and aligned the result with the group's decisions
What the AI did **not** do	It did no user research, tested no products and made no product decisions

5.2 Process, by iteration

Date	Version	What was asked	What the AI did
2026-09-24	v1.x	Competitors and non-digital alternatives for security coordination, along the assignment's dimensions	Three research rounds (8 → 20 → 29 profiles), with a 15-capability matrix.
2026-10-01	v2.0	Widen it to academic event management as a whole	Research per stage of the event lifecycle: 10 families, more than 50 products, public prices
2026-10-01	v3.0	A short version, by phase, with strengths, weaknesses and AI use, Portuguese market only	Condensed the previous research into 2–3 pages
2026-10-06	**v4.0** (this document)	Iterate the analysis based on what is already in the repository	Read the Problem and Opportunity Report, the survey script and the repository's commit history; reorganised the phases; did new research on participant information, queues and getting home (sources P1–P10 of annex `02a`)

5.3 Rules followed to keep the evidence reliable

Rule	Where it shows in the document
Every fact has a linked source	Links in the tables and in section 4

Rule	Where it shows in the document
Absence of evidence is not evidence of absence	"Not identified" in the AI column
Competitors' features are commercial claims	Warning in the legend; no product was tested
No user data was generated by AI	This document has no survey or interview results
AI output is not presented as fact	The conclusions in section 3 are our reading of the tables, not a measured result

5.4 Limitations of the AI use

Limitation	Possible effect	Mitigation
The AI reads pages through an automatic summary	It may miss a detail or a price	Open the source before quoting a number in the final report
Prices and features change	Data may be out of date	Most source lists of the annex give the date they were checked; the list in §4 and annex §9.2 do not
Some searches returned nothing useful (for example, official apps of Queima de Coimbra and Enterro da Gata)	Gaps in coverage	Recorded as "not found"
AI can make mistakes or invent things	Risk of unfounded claims	Only claims with a link were kept, and the rules of §5.3 were applied by hand

Annex 02a — Product-specific market analysis

> LABDSOF 2026/27 · Sprint A
> **What this document is:** the market analysis **of each thing we want to include in the project**. The general document (`market-and-competitor-analysis.md`) covers the whole market, phase by phase; this annex goes down to feature level.
> **Basis:** what is in the repository on 2026-10-06 (`main`, commit `442ecc3`): `Docs/Deliverables/problem-opportunity-report-queima.md` (problem, vision, MVP, assumptions A1–A9) and `Docs/Research/inquerito-estudantes-google-forms.gs` (student survey, questions 1–11).
> **Assignment, Market section:** competitors and non-digital alternatives: users, features, strengths/weaknesses, business model, UX, privacy → value proposition.
>
> **Evidence legend**
>
> - No mark = fact with a source. IDs (all in section 9): `P#` = sources from 2026-10-06; `M#` = security, operations and app sources; `N#` = ticketing, payments and market sources; `F#` = reference case (Queima do Porto); `R:` = evidence cited in the repository's Problem and Opportunity Report.
> - Competitors' features are **supplier claims** (websites, prices, app stores). We did not test any product.
> - In the matrix, ❌ means "not found in the sources", not "does not exist".
>
> **Decisions taken since 6 October 2026** (added 31 October 2026, #162). This analysis was written on 6 October; the team decided later, and where the text below differs, these decisions win:
>
> - **Transport:** live positions of the city buses near the venue come from the Porto Digital urban platform, plus the event shuttles and temporary stops entered by the team (ADR-008, F9 in `features.md`). A link to Google Maps is no longer the plan for regular lines.
> - **AI:** help-request triage was chosen on 8 October 2026; the participant Q&A assistant was rejected (a wrong transport answer at night is a safety risk; `features.md` §5).
> - **Weak network:** offline mode (cached map and information) is F11, a *Could* since 9 October 2026; the time of the last update on each item stays in the MVP (SC7, product vision). The rows C5, C9 and C10 below are corrected accordingly.

---

0. Summary

Question	Short answer
What do we want to build?	A **multi-event** platform with two sides: a **participant web application** (map, information, transport, help request) and a **backoffice for each event's team** (configure the event, update information, handle incidents)
Is there a product that does all this for academic events in Portugal?	**We did not find one.** Each component exists in isolation: festival apps provide a map and alerts; professional control rooms handle incidents; Google Maps provides transport; WhatsApp receives help requests

Question	Short answer
Who comes closest?	**Appmiral** (map, alerts and SOS in a festival app) on the participant side and **Raven Controls** (incidents with public reporting by QR) on the team side. Neither joins the two sides for academic teams, in Portuguese and without installation
What is already common and does not set us apart?	Interactive map, timetables, push alerts, SOS button, incident dashboard, real-time transport
What sets us apart?	(1) the two sides connected by the **map**; (2) **multi-event with per-event permissions**; (3) **no installation**, with the last-update time on every item; (4) **the event's own transport** maintained by the team; (5) Portuguese context
Biggest threat	The free status quo: **Instagram + WhatsApp line + radio**

---

1. What we want to include (read from the repository)

#	Component	Status in the repository	Origin
C1	Multi-event platform: each event with its own owner, team and permissions	**MVP**	Report, "Product vision" and "MVP definition"
C2	Participant web application (PWA), opened by link or QR, no mandatory installation, with mandatory login (decided on 9 October 2026)	**MVP**	Report; survey, questions 7 and 9
C3	Venue map uploaded and configured by the team (entrances, stages, toilets, first aid, stops)	**MVP**	Report, "Opportunity"; survey, question 8
C4	Essential event information, updated by the team during the night (alerts)	**MVP**	Report, root cause 2; survey, question 8
C5	Transport and getting-home information	**MVP**	Report; survey, questions 3 and 8
C6	Help request by the participant, with the place marked on the map	**MVP**	Report; survey, questions 5, 8 and 10
C7	Backoffice: receive, prioritise and route incidents	**MVP**	Report, "Main user journeys"
C8	Expected waiting time at the entrances	**Candidate** (survey option)	Survey, question 8. The physical capacity of the gates is out of scope
C9	AI: support for prioritising help requests (triage); the participant Q&A assistant was rejected on 8 October 2026	**MVP** (triage; the assignment requires at least one AI workflow)	Survey, question 8; report, assumption A7
C10	Work on a weak network (cache, degraded mode) and show the time of the last update	**MVP:** the last-update time (SC7). **Could:** the offline cache (F11, since 9 October 2026)	Report, assumption A8 and technical risks

#	Component	Status in the repository	Origin
C11	Electronic ticket in the application	**Candidate** (survey option); ticketing is a partner, not part of the MVP	Survey, question 8; report, stakeholder map

Out of scope (repository): physical capacity of the entrances; heavy integrations; advanced crowd prediction; full automation.

---

2. Analysis by component

Each profile answers: who already does this, with which strengths and weaknesses, whether they use AI, and what it means for us.

C1 — Multi-event platform with per-event teams and permissions

Alternative	What it does	Strengths	Weaknesses for us	AI
One app per event (Appmiral; NOS Alive and Rock in Rio Lisboa apps)	An app branded for each festival (M2, M8, M9)	Own brand; mature	Each event pays for and maintains its own; the student installs one per festival	Not identified
Woov	One app for hundreds of festivals; the user chooses the event (M5, P2)	Proves the "one app, many events" model works; more than 1 M users per season	No operations; the organiser does not handle incidents there	Chat moderation
FEST App	Multi-festival "container" with a web dashboard for the organiser; free entry plan (P10)	Self-service; cheap	Distributed through app stores; information only	Not identified
LineUpr	Yearly "Flex" subscription: a quota of events and participants to spread over the year, up to 60% cheaper than buying event by event (P3)	Pricing model designed for those with several events a year	Conferences; no incidents	Not identified
BindePaFesta (PT, June 2026)	Map of village and popular festivals across the country; organisers manage their own content (P1)	Shows there is demand in Portugal for an app shared by many festivals; reached the top of the Portuguese App Store	Festival discovery, not event operations; popular-festival audience	Not identified
APP FAP	Official app of the Porto federation (M10)	Already has Porto's student base	A single federation; each federation would have to build its own	Not identified

Alternative	What it does	Strengths	Weaknesses for us	AI
MSL + native / FIXR (United Kingdom)	Platforms for students' unions with per-committee permissions (N15, N16, N17)	Prove the academic segment buys its own platforms	Not present in Portugal; focused on sales	Not identified
Non-digital: each organisation handles its event with its own tools	—	Full independence	Nothing is reused between events or between boards	—

What we learned: multi-event is **not new** on the participant side (Woov, FEST, BindePaFesta). What is new is that the **backoffice** is also multi-event, with **isolation between teams**. That is why the repository's report treats permissions across events as a technical risk: it is the point that has to be demonstrated.

C2 — Participant web application, no installation

Alternative	Entry channel	Strengths	Weaknesses for us
Native apps (Appmiral, Woov, FEST, SafeZone, APP FAP)	Installation from the app store	Reliable push; work better offline	Friction of installing yet another app
LineUpr	PWA, no installation (P3)	Opens by link; 20 languages	Push only on the most expensive plan
Raven Eye	The public reports through **WhatsApp with a QR code** (M13)	Zero installation; a channel everyone has	Exposes the phone number
42Chat FestivalBot	Assistant by **SMS or web** (P4)	No installation	Questions and answers only
FAP's WhatsApp/QR line	QR at the venue (F4, F5)	Already in use	No structure or metrics
Non-digital: signs, signage, staff	In person	Does not depend on the phone	In 2022 and 2023 there were complaints about "lack of information and signage" (R: Renascença, Público)

What we learned: the professional market has already gone after the participant **without asking them to install anything** (WhatsApp, SMS, PWA). The link or QR is the right choice; the cost is losing native push, so alerts (C4) must work with the application open and, if possible, with web notifications. Survey question 9 measures exactly this barrier ("having to install yet another app").

C3 — Configurable venue map

Alternative	What it does	Model	Strengths	Weaknesses for us
Map image on Instagram or the website (status quo)	Static image	Free	Zero cost	Not updated; cannot be used to point to a place; not connected to requests

Alternative	What it does	Model	Strengths	Weaknesses for us
Appmiral	Interactive map (Pro plan only, €935/month) (M2)	Yearly SaaS	Mature; find friends	Price; one app per festival
Woov	Festival map; 3D maps (M5, P2)	B2B, price not public	Rich experience	Collects the user's exact location (P2)
MapsPeople (MapsIndoors) / Mappedin	Indoor maps with turn-by-turn routes, point search and real-time updates; customers such as Fira Barcelona (P6)	Enterprise, price not public	Professional quality	Designed for trade fairs and buildings; does not say whether it serves temporary outdoor venues
Iventis	Geo-referenced venue planning for the organisation (M42)	SaaS, on request	Capacity and evacuation plans	For planning beforehand, not for the participant during the event
Google Maps / OpenStreetMap	City map	Free	Universal	Do not know the inside of a temporary venue
what3words (used in Controlled Events' control room)	Gives a name to each 3 m square (M18)	—	Precise location by voice	Yet another app; does not show the venue

What we learned: an interactive map is **common** in festival apps. What we did not find was the map used as a **shared reference between the participant and the team**: the participant marks on the map where they are, and the team sees the incident at that point. The repository's simple solution (the team uploads an image or PDF and marks points) avoids the cost of a map supplier and addresses the fact that phone location is inaccurate in a crowded venue.

C4 — Essential information and alerts during the night

Alternative	What it does	Strengths	Weaknesses for us
Instagram (stories) and website	Organisation's announcements	Reach	Does not reliably reach those at the venue; mixed with all other content
Festival app push (Appmiral, WMT Digital, Woov, FEST)	General and per-zone alerts (geofencing in WMT) (M2, M7, P10)	Immediate	Requires an installed app
Aloompa Presence (Crowd Connected)	Live heat maps and push segmented by venue zone, with consent-based location (P5)	Alert only those in a zone	Native app; the organisation must provide the map in CAD
Digital screens, megaphone and staff	On-site announcements (P8)	Do not depend on the phone	Limited reach; no record

What we learned: broadcasting alerts is a **commodity**. What we add is **reliability**: each piece of information with the time of its last update (the report identifies the risk of "outdated information making things worse") and a single source for what changes during the night.

C5 — Transport and getting home

Alternative	What it does	Strengths	Weaknesses for us
STCP, Metro and Andante announcements	Announce shuttles and extra services before the event (R: Andante, STCP)	Official	Static; on separate channels
Google Maps with STCP real time (since 9 June 2026)	Bus location and time to arrival at 2,517 stops (P9)	Free; already installed	The news does not say whether the Queima's special shuttles are included **[H: check in the GTFS]**
Moovit, Anda app	Planning; Andante ticketing (M36)	Well known	Do not know the event
Festival apps (Rock in Rio Lisboa)	Static shuttle information (M9)	Same place as everything else	Does not change during the night
SafeZone	Campus shuttle tracking (M22)	Real time	Campus, not events
Puntos Violeta (Spain)	Groups to "go home together" (M28)	Safe journey home	Another country; another focus
Ride-hailing (Uber, Bolt)	Trip sharing (M35)	Already installed	Price at peaks

What we learned: since June 2026, real time for STCP's regular lines is **already solved** by Google Maps; there is no point in rebuilding it. Our part is what mobility apps do not know: **temporary stops on the map, the event shuttle's timetable and what changed tonight**, entered by the team (assumption A6 of the report). For the rest, a link to Google Maps is enough. *Update 31 October 2026:* the team later chose to show live bus positions near the venue from the Porto Digital platform (ADR-008); temporary stops and the shuttle are still entered by the team.

C6 — Help request with the place on the map

Alternative	Users	UX	Privacy	Strengths	Weaknesses for us	AI
FAP's WhatsApp/QR line	Participant → operator	Familiar, no installation	Exposes the number; data at Meta	Already exists	No category, place or status	—
Finding a security guard, volunteer or the medical post	Participant	In person	Good	Human	Only serves those who see them and can reach them	—
Appmiral SOS	Participant → festival security	Button in the app	Real-time location	"Hundreds of alerts" handled (M1)	Native app; no version for volunteer teams	No

Alternative	Users	UX	Privacy	Strengths	Weaknesses for us	AI
SafeZone	Student → campus security	Three buttons: emergency, first aid, help	Location **only at the moment of the alert** (M22)	Good reference practice	Campus only	No
Raven Eye	Public → control room	WhatsApp + QR	Number exposed	Integrates with the control room (M13)	Enterprise; no PT	Yes, in the control room
AlertCops (Spain)	Citizen → police	App	Mandatory identification (M27)	Official	Does not accept anonymity	No

What we learned: the help button **exists** in festival and campus apps. Three patterns to follow: (1) entry without installation, like Raven Eye; (2) location only at the moment of the request, like SafeZone; (3) make clear it is not an emergency service, as the report already says. What is new is the participant **marking the place on the map** instead of relying on GPS. Survey question 10 measures the acceptance of this.

C7 — Incident backoffice

Alternative	Users	Features	Model	Strengths	Weaknesses for us	AI
Radio + control room with manual log	Coordination, security	Voice per channel	€20–25 per radio per week (M45)	Works without a network	No structured history; place described by voice	—
WhatsApp groups and spreadsheets	Team	Messages	Free	Everyone knows how to use them	No owner or status per case	—
Raven Controls	Professional control room	Log, maps, checklists, group notification	SaaS, on request (M13)	Used at COP26 and in a partnership for the FIFA World Cup	For professional control rooms; no PT	**Yes**: suggests category, priority and procedure
Momentus WeTrack	Venues	Incidents with severity and response plans (M16)	Enterprise	More than 4,000 venues	Enterprise	Not identified
Controlled Events	Civic events	Control room with map and weather; **per-event** licence (M18)	Per event or yearly	Pricing model useful for seasonal events	Not connected to the participant	Not identified

Alternative	Users	Features	Model	Strengths	Weaknesses for us	AI
Appmiral Emergency Reporting	Organiser	Brings reporting channels into one view (M3)	Included in the app	Same supplier as the app	Little public information	Not identified
Control room software (market)	Stadiums, cities	—	US$30k to more than US$1M (M19, a supplier's source)	—	Out of reach for a federation	—

What we learned: the incident → priority → routing cycle is **proven** in the professional market. What we did not find is a version for **academic teams** (organisation, prevention committee, volunteers), in Portuguese, connected to the participant application and at a federation's price.

C8 — Waiting time at the entrances *(candidate)*

Alternative	What it does	Strengths	Weaknesses for us
No information (status quo)	The participant only discovers the queue when they arrive	—	Queues of more than 500 m and more than 3 h in 2022 and 2023 (R: Renascença, Público)
Appmiral Venue Occupancy	Red, orange or green status per zone, **updated by hand** by staff (M4)	Simple; needs no sensors	Depends on someone updating it
Festival operations dashboards	Combine gate scans, queue length and shuttles; alerts when the wait passes a threshold, with push and signage (P8, a supplier's blog)	Practice described in the sector	Needs ticketing data
Waitwhile	Virtual queue by link or SMS with estimated time (P7)	More than 30,000 people managed at Tribeca	Not for the gates of a venue
Queue-it	Virtual queue for online sales, from US$1,499 per event (N34)	Handles purchase peaks	For the sale, not for the gate
Crowdscan	Measures density with sensors, no cameras (M43)	Objective data	Hardware and cost per event
Estimate from the app	A study shows a correlation above 0.8 between the density estimated by a festival app and the one measured on video (R: arXiv 1306.3133)	Technically feasible	Needs many users with location on

What we learned: the simple version (the team sets the status of each gate by hand, as in Appmiral) fits in the MVP and is consistent with assumption A6. The automatic version depends on ticketing or location data, which means heavy integration and raises privacy questions. It only makes sense to go ahead if the survey (question 8) shows demand.

C9 — AI *(candidate; the assignment requires an AI workflow and a fallback)*

Use	Who already does it	Strengths	Weaknesses and risks	Non-AI alternative
Participant Q&A assistant	**42Chat FestivalBot**: SMS and web, more than 60 topics, tiers up to 50,000 users (P4)	Answers immediately; relieves staff	The supplier does not publish accuracy or what it does when it does not know; a wrong transport answer can put someone at risk	FAQ page and search over the event information
Incident triage	**Raven Controls** suggests category, priority and procedure to an operator who decides (M13)	Validates the "AI suggests, the person decides" pattern	No PT; professional operator	Category chosen by the participant + order of arrival
Guided interview for sensitive reports	**Spot** (M32)	Helps describe what happened	Outside the moment of the event	Short form
Content moderation	**Woov**, with 95% less human work (M6)	Scales	Does not apply to our MVP	—
Marketing and demand forecasting	Eventbrite, Fever (N1; general document)	—	Outside our scope	—

What we learned: AI **does not set the product apart by itself**, because both candidate uses already exist. What counts is meeting the assignment better than suppliers show publicly: the assistant would answer **only from the information the team uploaded** (it was rejected on 8 October 2026; triage was kept); the AI never lowers a request's priority without a person; evaluation with a set of cases and comparison with the non-AI alternative (assumption A7).

C10 — Weak network and degraded mode

Alternative	How it deals with the network	Source
Radio	Does not depend on the mobile network	M45
Woov	RFID cashless working offline	M5
Zpos	POS offline for up to 5 days	N24
Weezevent	Offline entry terminals	N5
Zello, WhatsApp	Depend on the network	M40
Festival apps	We found no public information on offline mode	—

What we learned: only professional payment and entry systems advertise offline operation. Congestion in crowds is documented (Sónar is a case study, M46). Caching the map and the latest information and saying "data from HH:MM" is an assignment requirement that almost no participant-side alternative shows.

C11 — Electronic ticket in the application *(candidate; ticketing = partner)*

Alternative	Situation
BOL, Fnac, APP FAP	Already sell the tickets for the Queimas of Porto and Coimbra and for the Enterro da Gata (N8–N11, M10)
Cashless wristband	In Porto, entry is by wristband only (N22)

What we learned: selling or storing tickets would mean competing with suppliers already inside the customer, in a well-served phase. If the survey shows demand, the lowest-risk answer is a **link to the ticket** on the ticketing platform, not our own ticketing.

---

3. Comparison matrix (components of our product)

Legend: ✅ confirmed in the source · ◐ partial · ❌ not found in the sources · ? no information · — not applicable. The last row is the MVP **target**, not a fact.

Alternative	C1 Multi-event	C2 No install	C3 Map	C4 Alerts	C5 Transport	C6 Ask for help	C7 Incidents	C8 Wait time	
Instagram + website + map image	—	✅	◐ static	◐	◐ static	❌	❌	❌	❌
FAP's WhatsApp/QR line	❌	✅	❌	❌	❌	✅	◐ manual	❌	❌
Radio + control room	—	—	❌	❌	❌	❌	◐ verbal	❌	❌
APP FAP	◐ FAP events	❌	❌	◐	❌	❌	❌	❌	❌
Google Maps (STCP real time)	—	✅	❌	❌	✅ regular lines	❌	❌	❌	?
BindePaFesta	✅	◐ app and website	◐ festival map	✅	◐ directions	❌	❌	❌	❌
Appmiral	❌ one per festival	❌	✅ (Pro)	✅	?	✅ SOS	◐	◐ manual	❌
Woov	✅	❌	✅	✅	?	❌	❌	❌	◐ mo
LineUpr	✅	✅ PWA	◐	✅	❌	❌	❌	❌	❌

Alternative	C1 Multi-event	C2 No install	C3 Map	C4 Alerts	C5 Transport	C6 Ask for help	C7 Incidents	C8 Wait time	
FEST App	✅	❌	✅	✅	❌	❌	❌	❌	❌
42Chat FestivalBot	?	✅ SMS/web	❌	❌	◐ answers questions	❌	❌	❌	✅ ass
Raven Controls (+ Eye)	?	✅ WhatsApp	✅ for the control room	❌	❌	✅	✅	❌	✅
SafeZone	❌ campus	❌	◐	✅	◐ shuttles	✅	✅	❌	❌
Waitwhile	?	✅	❌	◐ SMS	❌	❌	❌	✅ virtual queue	❌
Us (MVP target)	✅	✅	✅	✅	✅ the event's and live buses	✅	✅	◐ candidate	◐ wi fal

Reading:

1. **No alternative covers C3 + C5 + C6 + C7 at the same time**, which is the core of the MVP in the repository (map, transport, help request, incidents).
2. Those with **C6 and C7** (Raven, SafeZone) have no Portuguese version; Raven has no participant information side, and SafeZone's alerts are for a campus, not an event. Those with **C3 and C4** (Appmiral, Woov, FEST) do not have an incident backoffice.
3. **C2 (no install)** appears mostly in messaging channels (WhatsApp, SMS); the most feature-rich festival apps are native.
4. The Portuguese status quo covers almost everything **partially and on different channels**. That is the real competitor.

---

4. Closest competitors along the assignment's dimensions

Dimension	Appmiral (CM.com)	Raven Controls	Woov	Queima do Porto status quo
Users	Participant; festival organiser	Control room; staff; public through Raven Eye	Participant; organiser	Participant; organisation; volunteers; security

Dimension	Appmiral (CM.com)	Raven Controls	Woov	Queima do Porto status quo
Relevant features	Map, timetables, push, SOS with location, manual occupancy	Incident log, maps, checklists, triage AI, reporting by WhatsApp/QR	Map, timetables, friends, chat	Instagram and website, APP FAP, WhatsApp/QR line, radio, colour-coded volunteers
Strengths	Mature; SOS in real use	Full cycle; credibility at large events	One app for many festivals; PT interface	Free; familiar; human
Weaknesses	One app per festival; marketing DNA	Enterprise; no PT; for professionals	No help or operations	Static; no record; separate channels
Business model	€375–935/month, billed yearly	Tiered SaaS, price on request	B2B, price not public	No software cost
UX	Native app branded for the festival	Operator console; public through WhatsApp	Social native app	Several channels; none shows the request's status
Privacy	Real-time location in SOS	Phone number exposed when reporting	Exact location and contacts tied to the user (P2)	Number exposed on WhatsApp; radio on an open channel

Us, along the same dimensions (target): participant and event team with the same weight · map, information, transport, help request and incidents · web application by link or QR · place marked by the participant on the map, shared only with the request, with short retention.

---

5. Differentiation by component

Component	Common in the market?	Where we stand out	How it is proven
C1 Multi-event	Yes, on the participant side	Multi-event backoffice with isolation between teams	Automated permission tests; a second event configured without development
C2 No installation	Partly (messaging channels)	Full web application, not just a reporting channel	Time between scanning the QR and seeing the map
C3 Map	Yes	Map uploaded by the team and used by both sides	Percentage of requests with a marked place
C4 Alerts	Yes	Time of the last update on every piece of information	Percentage of content with a visible time
C5 Transport	Yes, for regular lines	The event's shuttles and stops, plus live buses near the venue (ADR-008)	Comparison with the official announcements
C6 Help request	Yes, in native apps	No installation; place on the map	Time until the request is submitted

Component	Common in the market?	Where we stand out	How it is proven
C7 Incidents	Yes, in professional control rooms	For academic teams, in PT, connected to the participant	Time between request and routing
C8 Wait time	Rare	Manual version per gate	Only if the survey shows demand
C9 AI	Yes	Published evaluation and fallback	Accuracy against the non-AI alternative
C10 Weak network	Rare on the participant side	Cache and "data from HH:MM"	Demonstration of the degraded mode

In short: almost every component exists in some product. The differentiation lies in the **combination** (the two sides connected by the map), the **segment** (Portuguese academic event teams) and the **delivery model** (multi-event, no installation, resilient to a weak network).

---

6. Business model: relevant price anchors

Reference	Price	Source
Appmiral (festival app)	€375 / 625 / 935 per month, billed yearly	M2
LineUpr (event app as a PWA)	Three tiers; yearly subscription for several events up to 60% cheaper; price by calculator	P3
FEST App	Free entry plan; the company itself places festival tools between US$1,000 and US$25,000 and custom development between US$40,000 and US$400,000	P10 (a supplier's blog)
42Chat FestivalBot	Three tiers, up to 5,000, 15,000 and 50,000 users; price not public	P4
Controlled Events	Per-event or yearly licence; price not public	M18
Control room software	US$30k to more than US$1M	M19 (a supplier's source)
Radio rental (PT)	€20–25 per radio per week	M45
Zello	US$6.80–12.75 per user per month	M40
Queue-it	From US$1,499 per event	N34

---

7. Threats

Threat	Probability	Impact	Response
The organisation keeps using Instagram, WhatsApp and radio	High	High	Measure and show time to help and up-to-date information in a pilot

Threat	Probability	Impact	Response
The supplier of the federation's app adds a map and help	Medium	High	Come in as a complement by link; multi-event value
Appmiral or Woov win academic customers in Portugal	Low to medium	High	Operations and incidents; local context; no installation
Google Maps starts covering the event's shuttles	Medium	Low	Use the link; keep only the stops on the map
Participants do not open the application during the night (assumption A2)	Medium	Very high	Survey, questions 7 and 9; QR on the ticket and at the venue

---

8. What still needs validating, and where

Hypothesis in this analysis	Where it is validated	Linked to
Students look for information on Instagram and in WhatsApp groups today	Survey, question 4	C4
Map, transport and the help button are the most useful features	Survey, question 8	C3, C5, C6
Installing an app is a barrier	Survey, question 9	C2
Students accept marking their place on the map instead of sharing GPS	Survey, question 10	C6
There is demand for waiting time, an assistant and a ticket in the application	Survey, question 8	C8, C9, C11
Incidents are coordinated by radio and phone, with no history	Interview with the security volunteer	C7
The special shuttles do not appear on Google Maps	Technical check of STCP's GTFS	C5
Organisations would pay a yearly licence	Conversation with organisations	Section 6

Until then, these rows are hypotheses and do not go into the report as evidence.

---

9. Sources

Only the sources cited in this annex are listed. The IDs come from the original research rounds, so the numbering has gaps.

9.1 New sources (checked on 2026-10-06)

ID	Source	Type
P1	RFM — Portuguese app shows where the festivals are (BindePaFesta), 2026-08-27	Press

ID	Source	Type
P2	App Store — Woov: Your Festival Companion	App store
P3	LineUpr — Pricing	Supplier
P4	42Chat — FestivalBot	Supplier
P5	Aloompa — Presence (powered by Crowd Connected)	Supplier
P6	MapsPeople — Interactive indoor maps for events	Supplier
P7	Waitwhile — Tribeca Film Festival case study	Supplier
P8	Ticket Fairy — Real-time festival ops dashboards	Supplier blog
P9	Renascença — STCP makes real-time timetables available on Google Maps, 2026-06-09	Press
P10	FEST — How much does a festival app cost in 2026	Supplier blog

9.2 Repository evidence (Problem and Opportunity Report, section 4)

ID	Source
R: Renascença	Queues for the Queima do Porto (2022)
R: Público	Chaotic first night at the Queima do Porto (2023)
R: Andante, STCP	Special operations for the Queima 2025 · STCP with a dedicated service
R: arXiv	Crowd density estimation from a festival app

9.3 Security, operations and event apps (checked on 2026-09-24)

ID	Source
M1	CM.com — Festival App Introduces SOS Alert (2024-08-05)
M2	Appmiral — Pricing
M3	Appmiral Knowledge Center — Emergency Reporting
M4	Appmiral — Venue Occupancy Module
M5	Woov — official website
M6	Lasso Moderation — Woov case study
M7	WMT Digital (formerly Aloompa) — Live Events
M8	PCGuia — App of the Day: NOS Alive
M9	PCGuia — App of the Day: Rock in Rio Lisboa
M10	FAP — APP FAP, FAP in your pocket
M13	Raven Controls — Total Event Control
M16	Momentus — WeTrack Incident Management

ID	Source
M18	Controlled Events — Incident Management Software
M19	SIRV — Best control room software (2024; a supplier's source)
M22	CriticalArc — SafeZone Features
M27	Segurma — What is AlertCops?
M28	Puntos Violeta — official app
M32	Talk to Spot — FAQ
M35	Uber — Rider Safety Features
M36	Andante — Anda App
M40	Zello — Pricing
M42	Iventis — Pricing & Plans
M43	Crowdscan — Events
M45	Rádio Silva — Radio rental
M46	MDPI Sensors — Opportunistic network algorithms in music festival scenarios (2021)

9.4 Ticketing, payments and market (checked on 2026-10-01)

ID	Source
N1	TickPick — Eventbrite's 2026 changes: what organizers pay now
N5	Weezevent — Festivals
N8	BOL — Enterro da Gata 2026 (shop branded with the event)
N9	Bilheteira Fnac — Enterro da Gata 2026 · Bilheteira Fnac — Queima das Fitas do Porto 2026
N10	BOL — General Pass, Queima das Fitas do Porto 2026
N11	BOL — Queima das Fitas Coimbra 2026
N15	FIXR — 10 reasons to use FIXR for your student society
N16	MSL — About MSL
N17	native — Events & Ticketing (knowledge base)
N22	JPN — Queima das Fitas: at the Queimódromo, payments by wristband (2023)
N24	Zpos — Festivals
N34	Queue-it — Pricing

9.5 Reference case: Queima das Fitas (checked on 2026-09-24)

ID	Source
F4	JPN — "Pantones da Queima": students wear different colours to give assistance (2023)
F5	24 Notícias — Queima do Porto returns safer and more responsible (2022)

---

10. How AI was used in this research

10.1 Tool and role

Item	Description
Tool	**Claude** (Anthropic), with web search, page reading and read access to the project's local folder
AI's role	Read the repository, extract the list of product components, search for alternatives for each one and organise the comparison
Human role	The team member who wrote this analysis asked for the iteration based on the repository and for the split between the general document and the specific annex, and reviewed the result
What the AI did **not** do	It did not change the repository, test products or generate user answers

10.2 What was done in this iteration

Step	How	Result
1. Read the repository	The AI read the Problem and Opportunity Report, the survey script and the commit history of the local clone	List of components C1–C11 (section 1), each with its origin in the repository
2. Reuse the earlier research	The AI reused the relevant profiles from the two earlier rounds (security, 2026-09-24; general market, 2026-10-01)	Sources `M#`, `N#` and `F#` (sections 9.3 to 9.5)
3. New research	13 web searches on: multi-festival apps in Portugal, official Queima apps, Woov, LineUpr, venue maps, queues and waiting times, AI assistants for festivals, STCP transport	10 new sources (`P1` – `P10`)
4. Reading the pages	The AI opened 11 pages and extracted concrete facts (prices, features, dates)	Tables in section 2
5. Synthesis	Matrix by component, differentiation, threats and link to the survey questions	Sections 3 to 8

10.3 Searches without useful results

Search	What happened	How it was handled in the document
Official app of the Queima de Coimbra in 2026	No relevant results	We claim nothing about that app
Official app of the Enterro da Gata or of academic weeks	No relevant results	Not included
APP FAP news in 2026 (map, alerts)	Only consumer complaints and already known news	The description from FAP's page is kept
Woov, MapsPeople and 42Chat prices	Not public	"Price not public"
Greencopper and Crowd Connected	Only old or indirect pages	Greencopper not included; Crowd Connected cited through Aloompa (P5)

10.4 Rules and limitations

Rule or limitation	How it was handled
The pages were read through an automatic summary	Numbers must be confirmed at the source before they go into the final report
Two of the new sources are supplier blogs (P8, P10) and five are commercial pages (P3–P7)	Shown in the "Type" column and next to each use
The AI may misread a page	"❌" means "not found", not "does not exist"
The "MVP target" rating in the matrix is an intention, not a fact	Shown in the matrix legend
No user data comes from the AI	The hypotheses are left to the survey and the interview (section 8)

4. PRODUCT VISION

`Docs/Product/product-vision.md`

Product Vision

> LABDSOF 2026/27 · Sprint A (Sprint 1) · Assignment §5.4 and §7.1.2 (deliverable 4)
> **Status:** vision statement, pitch and success-criteria rule agreed in the team meeting of 8 October 2026 (minutes). Product name **Farol** decided on 9 October 2026. All target numbers were set by 26 October 2026 (NFRs).
> **Source of truth for the problem:** `problem-opportunity-report-queima.md`. This document summarises it and links back to it; it does not replace it.

Legend

Marker	Meaning
(no marker)	Already stated in the repository; the source is linked
[PROPOSED]	Drafted from repository content to fill a gap. Not yet agreed by the team
[DECISION NEEDED]	The team must choose before the item can be considered done
> ⚠️ **GAP**	Information that does not exist in the repository, and what is needed to complete it

---

1. Vision statement

> For the **organising teams of large academic festivals in Portugal** and the **students who attend them**, who today depend on scattered, static information and on radio and phone calls to get help to the right place, **Farol** is a **multi-event coordination platform** that connects a shared venue map, live event information and help requests in a single place. Unlike Instagram posts, WhatsApp help lines and single-festival apps, it links participants and the operations team through the same data, opens from a link or QR code without installation, with one account for every event.

One-line pitch: *One map, one source of truth, one tap to get help, for every academic festival.*

Vision statement and pitch **agreed by the team on 8 October 2026**. Updated on 9 October 2026: login is mandatory ("one account for every event"), and "keeps working on a weak network" was removed because offline mode (F11) is now a Could.

Product name: Farol (decided 9 October 2026)

Chosen from the candidates below. Still to check: the name is free on the domain.

Name	Idea
Farol	"Lighthouse": guides people home and shows where help is. Short, Portuguese, easy to say at night

Name	Idea
Recinto	"Venue": the map of the venue is the centre of the product
Malta	Portuguese slang for "the group of friends": community reports, students for students
Rumo	"Heading / direction": find your way in and home
Ponto	"Point" or "meeting point": every feature is a point on the map
SOS Festa	Says the safety part out loud, but it may sound like an emergency-only app

Check before choosing: the name is free on the domain, and it does not tie the product to one festival (it is multi-event).

2. Problem in one paragraph

Large academic festivals concentrate tens of thousands of people per night around a venue, its entrances and its transport. In the pilot case, the Queima das Fitas do Porto, that is about **250 000 entries over 8 nights** (evidence E1). Operational information is fragmented across organisers, the risk-prevention committee (PCR), security, police, transport operators and the city, each on its own channel, and it is published before the event, so nothing that changes during the night reaches the participant (root causes 1 and 2). In 2022 and 2023 this showed up as queues of more than 500 m, waits of over 3 hours, reports of crushing and panic attacks, and complaints about "lack of information and signage" (evidence E3).

Source: report §2 *Problem Statement* and §4 *Problem Evidence*.

3. Target community and users

Group	Role in the product	Interface	Source
Event team (primary)	Event organiser (e.g. FAP) and its team: organisation (EventAdmins), PCR, security coordination; operators in the control room and responders in the field, whose jobs come from the teams each event creates (`features.md` §4)	Backoffice	Report §1
Participants (primary)	Students aged 18–25, mostly in groups, on their phone at night	Participant web app (PWA)	Report §1
Operational partners (secondary)	STCP, Metro do Porto, PSP, Civil Protection: source or destination of information	Not in the MVP	Report §1
Platform team (internal) **[PROPOSED]**	The Farol team, with the global `FarolAdmin` role: creates events for verified organisations and names their first EventAdmin; never sees an event's operational data	Backoffice, platform area	Multi-event scope; `personas.md` P4

Both primary users carry **the same weight** (unanimous team decision, report §1). Detailed personas: `personas.md`.

4. Value proposition

For	Gain	Pain removed
Participant	A single, reliable place for practical information (map, entrances, services, transport home) and a way to ask for help with their location on the map	Not knowing where things are, how to get home or whom to ask for help
Event team	A shared operational picture: what is happening, where, and how to respond faster; one configuration reused across nights and events	Help requests arriving by radio and phone with no structured history, priority or precise location
Both	The information the team updates appears straight away to participants, with the time of the last update	Static information that is out of date as soon as the night starts

Source: report *Product vision* > *Value proposition*; market analysis §3 *Differentiation opportunity*.

Expected community benefit

Assignment §5.4. Beyond each user's gain, the festival community as a whole is expected to benefit in three ways, all taken from the report:

- **Help reaches people sooner:** requests arrive structured, with a point on the venue map, at the team that handles them, instead of by radio and phone (report §2, root cause 4 and *Consequences of not solving it*).
- **Less confusion at entrances and on the way home:** updates from the event team reach every participant at once, with their time, instead of being published before the night (report §2, root causes 1 and 2).
- **Problems are known before they become incidents:** confirmed community reports let other participants avoid queues and broken services, and tell the team early (`features.md` §6.4, question 2).

These need a real event to measure, so they stay as long-term outcomes (§5), not success criteria.

Why it is different (market analysis §3 and annex §5): no alternative found in Portugal covers the map, transport, help requests and incident handling together (annex §3, matrix). The difference lies in the **combination** (two faces linked by the map), the **segment** (Portuguese academic event teams) and the **delivery** (multi-event, no installation, fast on a slow network).

5. Measurable success criteria

Rule (team decision, 8 October 2026): a success criterion must **prove that the product works**. It must be measurable on the MVP by a test, a timer or a query, without depending on how fast people react. For example, "the AI triage routes a request in under 30 s" is a success criterion; "the event team responds faster" is not, because it depends on the operators, not on the product.

#	Success criterion	Metric	How it is measured	Target
SC1	The app opens fast, even on a weak network	Time from scanning the QR code to seeing the venue map, participant already logged in	Automated test with network throttling (slow 3G profile)	≤ 5 s cold, ≤ 1 s from cache (confirmed on 24 October 2026, #137)

#	Success criterion	Metric	How it is measured	Target
SC2	A help request reaches the event team at once	Time from "submit" on the phone to the request appearing in the backoffice, without reloading	Automated end-to-end test	p95 ≤ 5 s (confirmed on 24 October 2026, #137)
SC3	**The AI triage routes a request fast**	Time from submission to a stored suggestion of category, priority and team	Timer in the triage worker; load test with the evaluation set	**≤ 30 s** (team decision); in operation ≥ 95 % valid suggestions within 30 s over a one-hour window (NFR-03)
SC4	The AI triage is correct where it matters	Urgent cases under-prioritised; category accuracy	Evaluation set, compared with the non-AI fallback	0 urgent cases under-prioritised; category accuracy ≥ 80 % (confirmed on 24 October 2026, #137)
SC5	No request is lost when the AI fails	% of requests shown to the operator when the AI is off or slow	Automated test with the AI stopped	100 %, with the fallback after 30 s
SC6	Live updates reach participants	Time from publishing an announcement to it appearing on an open participant app	Automated end-to-end test	≤ 15 s (confirmed on 24 October 2026, #137: participant pages poll every 10 s, so the worst case is one interval plus the server time)
SC7	Information shows how fresh it is	% of information items showing their last-update time	Automated UI test	100 %
SC8	Each event's data stays private to its team	Cross-event access attempts that succeed	Automated authorisation tests (report §5, technical risks)	0
SC9	Every change to a request is recorded	% of request changes with an audit record (who, when, from, to)	Database query after the test suite	100 %
SC10	The app works without network	Map and information shown from cache after the network is cut	Automated offline test. Only if F11 (Could) is built; not a §6.8 scenario	Passes, with "data from HH:MM"
SC11	Asking for help is quick	Time from opening "Ask for help" to request submitted	Usability test with students	Median ≤ 30 s (confirmed on 24 October 2026, #137)

Validation of the problem (not success criteria)

These measure whether the product is worth building, not whether it works. They come from the user research:

#	Question	Metric	Target
V1	Would students use it? (assumptions A2, A3)	% answering "Sim, de certeza" (yes, definitely) or "Provavelmente sim" (probably yes) to survey Q7	≥ 60 %; below 40 % triggers a rethink

#	Question	Metric	Target
V2	Is finding information a real problem?	Survey Q6, ease of finding information (1–5)	Average ≤ 3 confirms the problem

Outcomes the MVP cannot measure (they need a real event): adoption during the event, faster response by the team, less confusion at entrances and transport, positive feedback from event teams (report, *Success criteria*). They stay as long-term outcomes.

SC11 is the only criterion measured with users rather than by the system: it measures how easy the product is to use, not how fast the operators react.

Targets: SC3 (30 s) set by the team on 8 October 2026; SC1, SC2, SC4, SC6 and SC11 confirmed on 24 October 2026 (#137), with SC6 raised from 10 s to 15 s to fit the 10 s polling of guide J3.

Results (student survey, 3 to 6 and 30 October 2026, n = 30 to 31): V1 met, 87 % (27/31). **V2 met**, 2.97, by a small margin and dependent on the answers of 30 October (report, *Limitations*); to confirm with a test in night conditions. The main pain reported is the wait at the entrance (80 %). Details, limitations and the decisions taken from them: report, *User Research*.

6. MVP boundary

In the MVP	Out of the MVP
Participant web app, behind login: event selection, venue map, essential information, transport information, simple help-request flow	Physical capacity of entrances (number of gates, queue funnel design): infrastructure, not software
Backoffice: each team configures its own event, manages the venue map, updates information, handles incoming help requests	Heavy integrations with operators
Multi-event with per-event permissions	Advanced crowd prediction, full automation
Accounts: login for every user, sign-up, consent and account deletion (F1, F2)	Own ticketing (link to the ticketing partner at most)
AI help-request triage, with the non-AI fallback D1 (F8; assignment §5.6)	Offline cache of the map and information (F11, Could)
Community reports, their confirmation by the responsible team, and badges (F12–F14; assignment §5.5)	Attendance badges, check-in, entrance status and friends (F15–F18, Should or Could)
Fast load on a slow network (SC1) and the last-update time on every item (SC7)	

The MVP is every Must of `features.md` §2, delivered as release R2 (`features.md` §8.2). Source: report *MVP definition* and *Out of scope*; annex §1. Feature-level detail and priorities: `features.md`.

Is this the smallest credible product experiment? (assignment §7.1.3) The first release that can be tried, R1 (Sprint B), is built around the help-request journey W2 + W3: it is the smallest product with which the critical assumptions A2 and A4 can be tried (report §5), and a participant joins the event by its QR code or link (US-12), which A3 needs. R1 also carries what the assignment requires in Sprint B: AI triage with its fallback D1 (§5.6,

§6.8) and the bus import, the external integration of §6.5. R2 adds what the assignment or privacy requires: community reports and badges, the engagement mechanism of §5.5, and sign-up, consent and account deletion (US-03 to US-05), which handle personal data. Everything else is Should, Could or out of the MVP.

7. Key risks (summary)

The three critical assumptions are **A2** (participants use the app during the night), **A3** (they accept one app for many events, opened by link or QR) and **A4** (help requests through the app reach the right team faster). If any of them fails, the product loses its reason to exist (report §5). Full list of assumptions A1–A9 and risks: report §5; with their status and dependencies: Assumptions and dependencies.

8. Research status

Method	Status	Validates	Source
Online student survey (Google Forms, 11 questions)	Done: 33 answers, 32 valid, n = 30 to 31 after the filter (3 to 6 and 30 October 2026)	A2, A3, A4, problem	`inquerito-estudantes-google-forms.gs`; results in the report, *User Research*
Interview with a security volunteer	**Not done yet**: planned for Sprint B (Sprint A deck, *Next*)	A4, root cause 4	Report §4
Public evidence (press, operators, FAP)	Done (E1–E7)	Problem	Report §4
Market and competitor analysis	Done	Differentiation	`market-and-competitor-analysis.md`

> ⚠️ **GAP: interview.** The report's *Results*, *Limitations* and *Resulting product decisions* are filled with the survey (30 October 2026): five decisions, three of which change the backlog once the team confirms them. The security-volunteer interview is still missing, so A4 and the event team's side have no primary data.

Personas

> LABDSOF 2026/27 · Sprint 1 · Assignment §7.1.2 (deliverable 4, *Personas*)
> **Status: proto-personas [PROPOSED]**. They are built only from what is in the repository: the user groups and context in the report (§1), the stakeholder map (§3), the public evidence (§4) and the hypotheses the survey is designed to test. **Only P1 is checked against user research so far:** the student survey of October 2026 (report, *User Research*; PE-1). The other personas are not based on user research yet (PE-2, PE-3). Names and ages are fictional. The assignment forbids fabricating research evidence (§5.3, §11), so every attribute below is either sourced or marked as a hypothesis.
> Legend: see `product-vision.md`.

Why five personas

The report has two primary user groups with the same weight: the **event team** and the **participants** (report §1). The event team, however, groups people with different jobs and different permissions: the person who **configures** the event before it starts and the people who **handle help requests** during the night. The journeys and the backoffice screens are different for each, so they are split here into two personas. Together with the participant, that makes three.

[PROPOSED] 10 October 2026: Farol hosts **many events** (Queima do Porto, Braga, Coimbra, Lisboa, …), and each organisation manages only its own. Someone outside every event has to create events, decide who runs each one, and step in when an event has no EventAdmin left. That is the fourth persona, **P4**, an internal user from the Farol team, with the global role `FarolAdmin`.

[PROPOSED] 12 October 2026 (professor feedback on the presentation: who receives the reports and how they are handled was not clear): the people who handle requests during the night do two different jobs. The **control-room operator** receives help requests and reports and **decides** what happens to each one; the **responder** is in the field, gets what the control room routes to their team and **acts** on it. They work in different places, on different screens, so the old P2 is split into **P2** and **P5**.

22 October 2026: the role model in `features.md` §4 (to confirm with the team) gives each persona a role or a job:

- **Roles.** P4 is the `FarolAdmin` (global). P3 is an `EventAdmin` of her event. P1 is a `RegularUser` of each event she joins. Roles belong to the membership of an event, so the same person can be EventAdmin in one event and RegularUser in another.
- **Jobs, not roles.** P2 and P5 are `RegularUser`s of the event. What they can do comes from the **teams** the EventAdmin puts them in: P2 is in a team with `HelpRequests.Triage` and `Reports.Review` (e.g. *Control room*); P5 is in a team with `HelpRequests.Respond` or `Reports.Respond` (e.g. *Nursing*, *Security*).

Persona	User group (report §1)	Role or job (features §4)	Main interface	Main journeys
P1 · Rita, the participant	Participants	`RegularUser`	Participant web app	W1, W2, W6, W7, W8
P2 · Miguel, the control-room operator	Event team (PCR, security coordination)	`RegularUser` in a team with `HelpRequests.Triage`, `Reports.Review`	Backoffice, live view (laptop)	W2, W3, W7
P3 · Sofia, the event admin	Event team (organisation)	`EventAdmin`	Backoffice, configuration	W4, W5
P4 · Daniel, the FarolAdmin [PROPOSED]	Farol team (internal user, not in report §1)	`FarolAdmin` (global)	Backoffice, platform area	W4 (first steps)
P5 · Inês, the responder [PROPOSED]	Event team (field teams, e.g. Nursing, Security, PCR, Logistics)	`RegularUser` in a team with `HelpRequests.Respond`, `Reports.Respond`	Backoffice on the phone, field view	W2, W3, W7

Journeys W1–W8 are described in `user-workflows.md`.

P1 · Rita, the participant

Who	21, university student in Porto, goes to the Queima with a group of friends for several nights
Context of use	At night, phone in hand, limited battery, congested network, loud environment, often after drinking (report §1)
Goals	Know where things are (WC, stages, first aid, entrances); know how and when to get home; know whom to ask for help, fast
Frustrations (sourced)	Long, badly distributed entrance queues and lack of information and signage (E3); information published before the event that does not reflect what changes during the night (root cause 2); the venue map is an image on social media (root cause 3)
What she does today (hypothesis)	Looks for information on Instagram, the event site and WhatsApp groups; looks for a security guard or staff member when she needs help. Survey (n = 30): information comes from the organisation's social media (77 %) and WhatsApp groups (60 %); in need, 40 % would first ask friends and 33 % look for staff (Q4, Q5; report, *User Research*)
Barriers to adoption (hypothesis)	Installing another app, battery use, sharing her location, weak network, not trusting that the information is up to date. Survey Q9 (n = 31): battery 48 %, installing another app 39 %, weak network 32 %, outdated information 23 %, sharing location 6 % (report, *User Research*)
Location sharing (hypothesis)	Survey Q10 (n = 31): 71 % would share the phone's location with a help request, 26 % only a place on the venue map. Product decision 4 of 30 October 2026 (report, *User Research*) asks for the phone's location first, with the map as the fallback; phased on 1 November 2026 (#175): map in Sprint B, phone location from Sprint C, after assessment M-1 and ADR-007 are amended

What success looks like	Opens the link from the QR code on her ticket or at the venue (annex §7), already logged in from the first time, sees the map in seconds, finds the next shuttle, and can ask for help for a friend in under 30 s (success criteria SC1, SC11)
Design implications	No installation; one account for every event, logged in once and kept logged in on the phone (login is mandatory, decided 9 October 2026); large tap targets; short text; works from cache; never asks for more data than needed (report §5, risks)

P2 · Miguel, the control-room operator

Who	24, volunteer in the risk-prevention committee (PCR) or security coordination, in the control room. **[PROPOSED] 12 Oct:** he decides what happens to each request and report; he does not go to the scene, the responders (P5) do
Role or job	A `RegularUser` of the event, in the *Control room* team, whose permissions are `HelpRequests.Triage` and `Reports.Review` (features §4.2, §4.4). Not a role: if the EventAdmin takes him out of the team, he is a participant like any other
Context of use	During the night, under time pressure, with many requests arriving at the same time; on a laptop in the control room
Goals	See every help request with what happened and where; decide the priority; route it to the right team of the event (e.g. *Nursing*, *Security*, *PCR*); know what is still open; confirm or reject the reports of the types his team reviews
Frustrations (hypothesis)	Requests arrive by radio and phone, with no structured history, no priority and no precise location. *Report root cause 4, explicitly marked as a hypothesis to confirm in the interview with the security volunteer*
What he needs from the product	A live list and a map of open requests; category, location and status on every request; a suggested priority he can accept or change; a record of who did what
Constraints	Must not depend on the product alone: radio stays as the backup channel (report §5, regulatory risks)
What success looks like	Sees each new request within seconds, with its exact location on the map and an AI-suggested priority and team within 30 s (SC2, SC3). Knows, for every report of his team, whether it is pending, confirmed, rejected or resolved

> ⚠️ **GAP:** almost everything about P2 is a hypothesis until the **interview with the security volunteer** happens (report §4). Needed: interview notes on how a request reaches the team today, which channels are used, how long it takes, and who decides the priority. Then update the *Frustrations* and *Constraints* rows.

P3 · Sofia, the event admin

Who	23, member of the organising team of the academic federation, responsible for operations or communication. She is an `EventAdmin` of her event and manages **only that event**: she cannot see or change any other festival on Farol. In another event she joins as a participant, she is a `RegularUser`

Context of use	Weeks before the event: sets it up on a laptop. During the nights: updates information (an entrance closed, a shuttle delayed) from the control room or her phone
Goals	Configure the event once (venue map, points of interest, transport, team members) and reuse it every night; publish changes that reach participants straight away
Frustrations (sourced)	Each partner communicates through its own channel; nobody has the complete picture (root cause 1); the venue map is a static image (root cause 3)
Constraints (sourced)	Board terms are annual, so knowledge is lost every year (stakeholder conflict 5); limited budget and no technical team; may already have an official app from another supplier (stakeholder conflict 1)
What she needs from the product	Upload the map as an image or PDF and place points on it; create the event's teams, give each one its permissions and help categories, and add people to them (features §4.2); share the management with other EventAdmins; publish updates that show their time; never see other events' data, and never let other events' teams see hers (report §5, technical risks)
What success looks like	A second night (or a second event) is ready without new development, only configuration (assumption A9)

> ⚠️ **GAP:** there is no direct access to the real organisation (FAP); this is a documented limitation (report §4). Needed: at least one conversation with someone who has organised an academic event, to validate assumption A1 (organisations accept to configure and maintain their event on the platform) and the configuration journey W4.

P4 · Daniel, the FarolAdmin [PROPOSED]

Who	27, member of the Farol team. Not part of any festival's organisation. He has the only global role, `FarolAdmin` (features §4.1)
Context of use	Office hours, on a laptop, mostly in the weeks before each festival and when an organisation changes its board. Not involved during the event nights
Goals	Bring a new festival onto Farol quickly and without new code; make sure every event is run by its real organisation; keep each event's data out of reach of everyone else, himself included
Frustrations / risks	Board terms are annual, so the people who run an event change every year and access gets lost (stakeholder conflict 5, sourced). Anyone could pretend to be a festival and publish a fake event that participants would trust (*hypothesis*: a risk of a multi-event platform, not in the report)
What he needs from the product	Create an event for a verified organisation and name its first EventAdmin (US-06); see every event's name, dates, organisation, EventAdmins and status (US-07); name a new EventAdmin when an event has none, e.g. when the board changes (US-08); suspend and reactivate an event that is abandoned or misused (US-09)
Constraints	**Never sees operational data**: no help requests, locations, reports or participant data of any event. Being `FarolAdmin` gives no access to any event's data (features §4.1). Isolation between events is a top technical risk (report §5) and privacy is a regulatory risk
What success looks like	A new festival goes live with configuration only (assumption A9), run by its real organisers, without Farol staff ever touching its incident data

> ⚠️ **GAP:** P4 is not in the report (§1 lists only the event team, participants and partners). It follows from the multi-event scope. Since 22 October only the FarolAdmin creates events, for an organisation that asks and is verified (features §4.1, US-06); this is part of the role model still to confirm with the team (PE-5).

P5 · Inês, the responder [PROPOSED]

Who	22, volunteer in one of the event's field teams (e.g. *Nursing*, *Security*, *PCR* or *Logistics*; each event creates its own), on foot around the venue
Role or job	A `RegularUser` of the event, in a team whose permissions include `HelpRequests.Respond` and/or `Reports.Respond` (features §4.2, §4.4). Not a role: the job comes from the team she is in
Context of use	During the night, on her phone, moving through the crowd, with noise, poor light and often her hands busy. The radio stays as her backup channel
Goals	Know straight away when something is sent to her team, and exactly where it is; tell the control room that she is on the way and when it is done
Frustrations (hypothesis)	Tasks arrive by radio, with a spoken description of the place that is hard to find in the crowd; nobody knows whether someone is already on the way. *To confirm in the interview with the security volunteer*
What she needs from the product	Every help request or report routed to her team on her phone, with its point on the venue map (US-31, US-52); one tap to accept ("on my way") and one to close ("resolved") (US-32, US-33, US-53); for a report, a way to answer "true / not true" when the control room asks her team to check it
Constraints	Sees only what is routed to her team, never the event's full list. Cannot set priorities, route requests or confirm reports: that needs `HelpRequests.Triage` or `Reports.Review`, the control room's permissions (P2)
What success looks like	Accepts a routed request within seconds and reaches the marked place without asking the control room where it is

> ⚠️ **GAP:** like P2, everything about P5 is a hypothesis until the **interview with the security volunteer** (how teams in the field get their tasks today, and who decides what).

Secondary users (not personas for the MVP)

Operational partners (STCP, Metro do Porto, PSP, Civil Protection) are a source or destination of information (report §1). In the MVP they do not use the product directly: transport information is entered by the event team or read from an external source (assumption A6). They are listed in the stakeholder map (report §3) and should become personas only if a later release gives them an interface.

Open items

#	Item	Needed to close it
PE-1	Validate P1's hypotheses (current information sources, barriers, location sharing)	Done with the survey (Q4, Q5, Q9, Q10; P1 above). Still open: reconcile the phone location (product decision 4) with assessment M-1 and ADR-007
PE-2	Validate P2's current workflow and pain points	Interview with the security volunteer
PE-3	Validate P3's willingness to configure and maintain the event	Conversation with an event organiser (assumption A1)
PE-4	Team agrees on splitting the event team into P2 and P3	Team decision
PE-5	Team agrees on P4 (FarolAdmin) and on who creates events. **Answered on 22 October** by the role model (features §4.1): only the FarolAdmin creates events, for an organisation that asks and is verified, and names its first EventAdmin (US-06)	Team confirmation of the role model (features §4)
PE-6	Team agrees on splitting the incident operator into P2 (control room, decides) and P5 (responder, acts). **Updated on 22 October:** the proposed event role `Responder` is dropped; P2 and P5 are jobs given by team permissions (`HelpRequests.Triage` and `Reports.Review` for P2, `HelpRequests.Respond` and `Reports.Respond` for P5, features §4.2), not roles	Team confirmation of the role model (features §4); interview with the security volunteer (which teams exist and who decides what)

Main User Workflows

> LABDSOF 2026/27 · Sprint 1 · Assignment §5.4 and §7.1.2 (deliverable 4, *Principal user journeys*)
> **Status:** draft. The three journeys in the report (*Main user journeys*) are expanded here into step-by-step flows with diagrams. Steps that are not in the report are marked **[PROPOSED]**.
> **Personas:** `personas.md` · **Features:** `features.md` · **Domain terms:** `domain-model.md`
> Diagrams use Mermaid, which GitHub renders directly.

Overview

ID	Journey	Persona	Source in the report	Demo priority
W2	**Participant asks for help**	P1 → P2 → P5	Third journey (help request)	**1, the core journey** (links both sides of the product)
W3	Control room handles a help request	P2 → P5	Third journey	1 (continuation of W2)
W1	Participant finds information	P1	First journey	2
W4	FarolAdmin creates an event, EventAdmin sets it up	P4 → P3	Second journey	3
W5	Team publishes a live update	P3 → P1	Second journey ("updates information during the night")	4
W6	Participant loses the network	P1	Risks: congested network, offline mode (report §5)	Could (F11). Assignment §6.8 is covered by scenarios D1 and D2 (`features.md` §3.1)
W7	Participant reports a problem and earns a badge	P1 → P2 → P5	Engagement mechanism (`features.md` §6)	Required by assignment §5.5
W8	Participant checks in at the venue	P1	Attendance badges (F15, F16)	Should

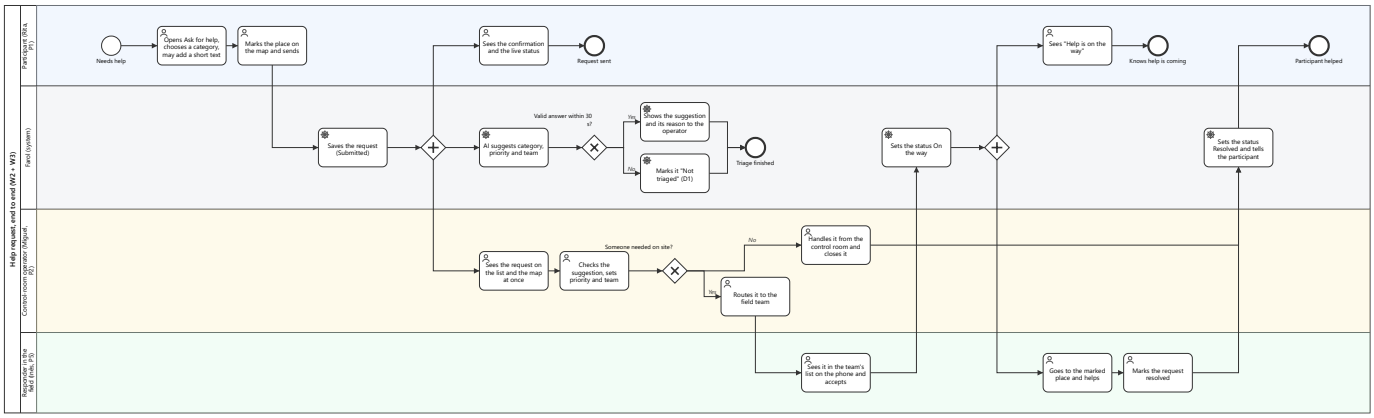
For the 5–7 minute presentation, **W2 + W3 tell the whole story**: a participant asks for help, and the event team sees it on the map, prioritises it and routes it.

Complete journeys (BPMN)

Added after the presentation of 12 October 2026 (professor feedback): one BPMN diagram per core journey, from the participant's first tap to the end, with one lane per actor. The Mermaid diagrams in W1–W8 stay as the step-by-step detail. Who receives what is in `features.md` §4.3. The lanes are personas, and since 22 October 2026 they are jobs given by team permissions, not roles (`features.md` §4.2): P2 stands for the members with

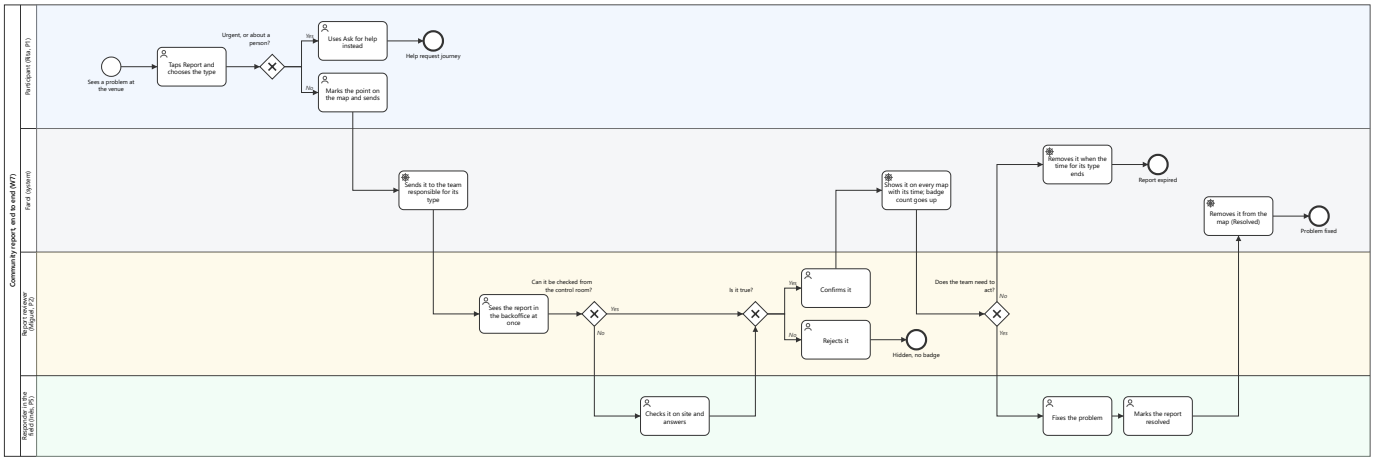
`HelpRequests.Triage` or `Reports.Review`, P5 for the members with `HelpRequests.Respond` or `Reports.Respond`.

Help request, end to end (W2 + W3)



- The request reaches the control room **at once**. The AI suggestion runs in parallel and never holds it back (scenario D1).
- The **operator decides** (priority, team, whether someone must go); the **responder acts** (accepts, goes to the point, resolves).
- The participant sees two changes: *On the way* and *Resolved*.
- Field teams see a routed request in their list on the phone, by polling; there is no push alert (US-29, US-31, guide C5).
- Not drawn, to keep it readable: the participant cancels, duplicates (see W3); the help-request consent was declined, the request fails or the rate limit is reached, where the app shows the first-aid points instead (NFR-19, US-23, US-25).

Community report, end to end (W7)

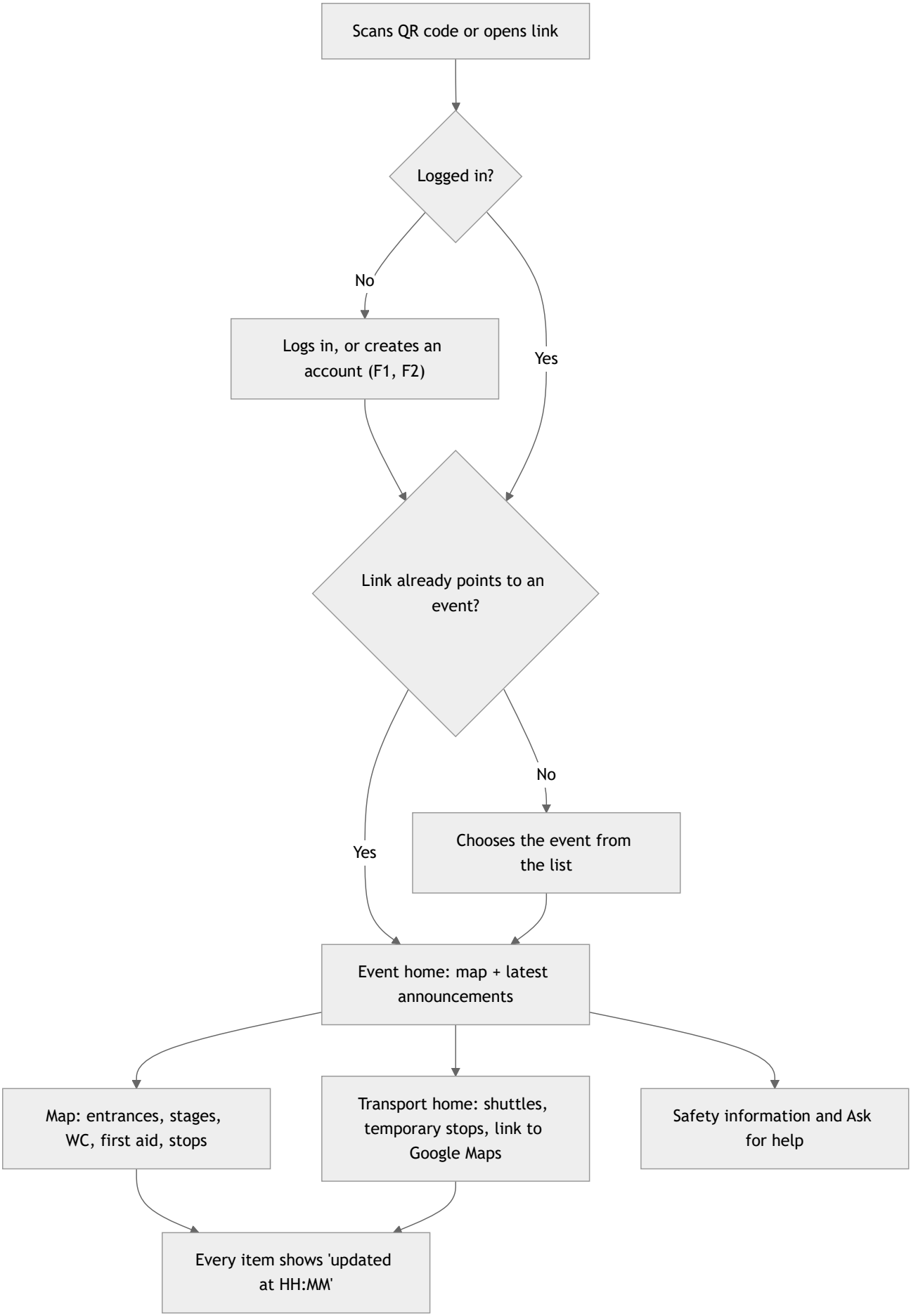


- A report skips the shared queue: its **type** says who is responsible, so it goes straight to the members with `Reports.Review` for that type (`features.md` §4.3).
- The **reviewer decides**: confirms or rejects, if needed after a team with `Reports.Respond` checks on site. A member of that team **acts** when the problem needs fixing.
- Not drawn: the team unconfirms a report later (see W7).

Files: `bpmn/` has each diagram as `.bpmn` (BPMN 2.0; open and edit it in bpmn.io or Camunda Modeler) and as `.svg` (for reading here). After editing a `.bpmn`, export the SVG again. These diagrams are for the documentation, not the slides: they are too detailed to read on a projector.

W1 · Participant finds information

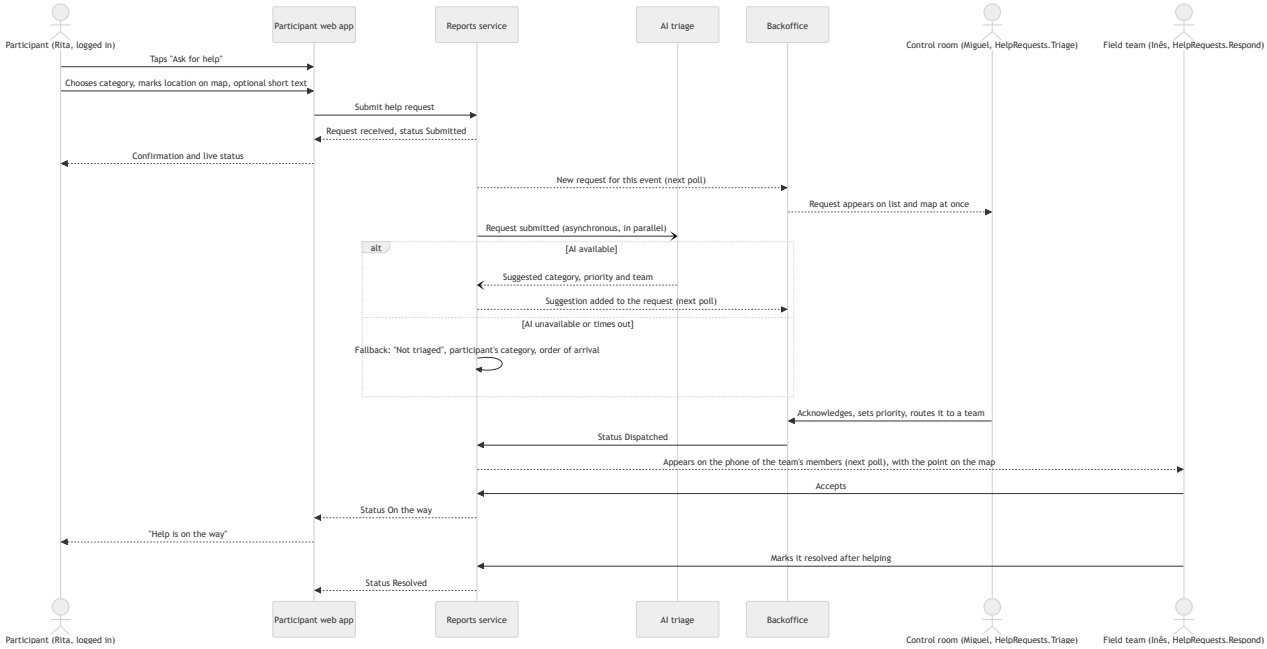
Source: report, first journey: "*opens the web application through a link or QR code, selects the event, checks the map, finds entrances, services, transport options and safety information*".


Scans QR code or opens link
Logged in?
No
Yes
Logs in, or creates an account (F1, F2)
Link already points to an event?
Yes
No
Chooses the event from the list
Event home: map + latest announcements
Map: entrances, stages, WC, first aid, stops
Transport home: shuttles, temporary stops, link to Google Maps
Safety information and Ask for help
Every item shows 'updated at HH:MM'

Login is mandatory (decided 9 October 2026): the participant logs in once and stays logged in on the device; one account works for every event. **[PROPOSED] details:** an event-specific QR code skips the selection step; the last-update time on every item comes from the risk "information that is out of date can make things worse" (report §5).

W2 · Participant asks for help (core journey)

Source: report, third journey: "*When a participant sends a help request, the event team receives it with context and location, assesses its priority and routes it to the right operational team.*" The AI step is the **help-request triage** chosen by the team (`features.md` §5); the suggestion must arrive within 30 s (SC3), otherwise the fallback applies.



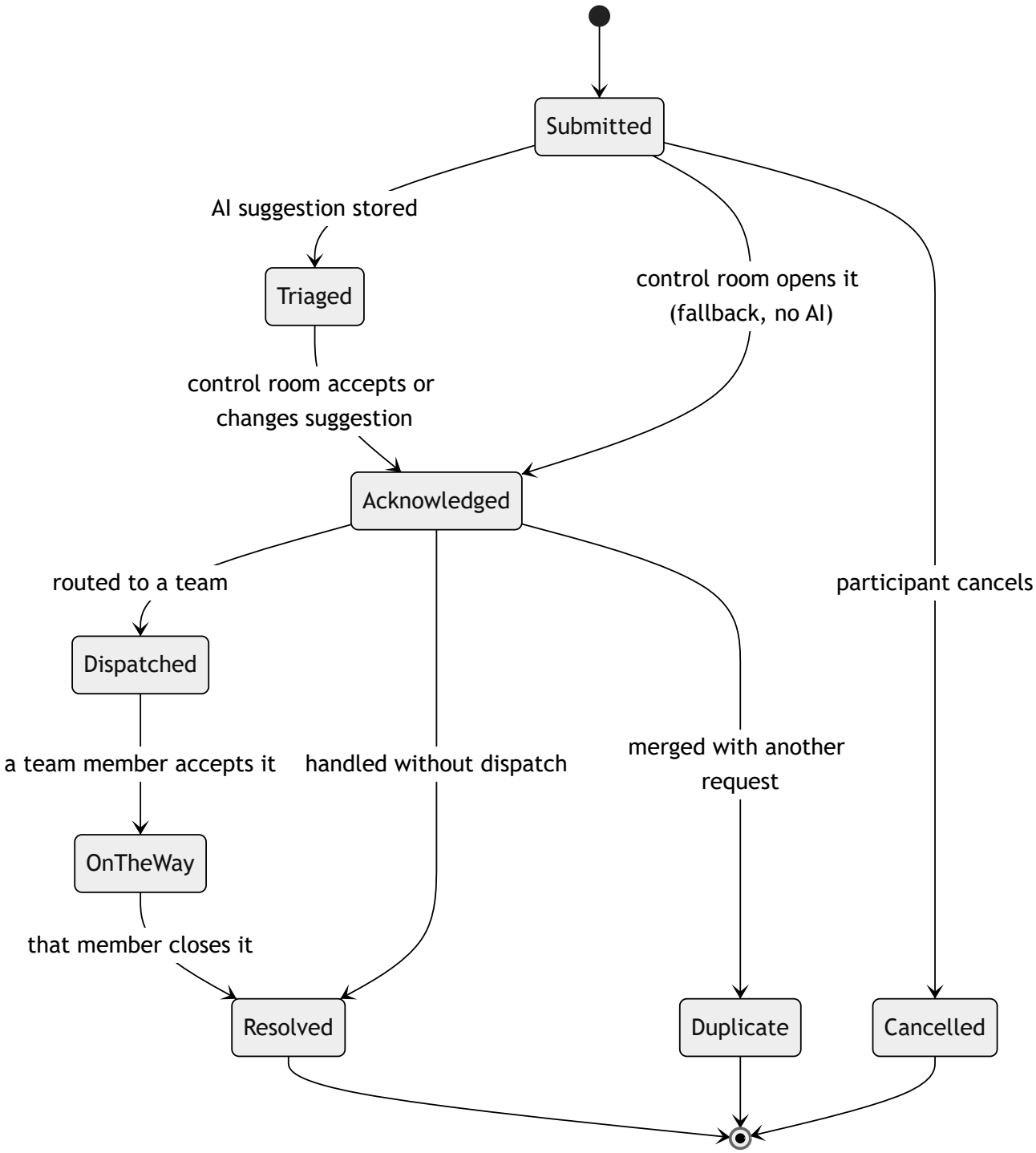
Rules from the repository:

- Location is marked on the map by the participant and shared only with the request (annex C6, SafeZone pattern). Survey Q10: 71 % would share the phone's location; product decision 4 of 30 October 2026 (report, *User Research*) asks for it first, with the map as the fallback. Phased on 1 November 2026 (#175): this journey uses the map in Sprint B; the phone's location comes from Sprint C, after ADR-007 and assessment M-1 are amended.
- The AI only **suggests**; it never lowers a priority (annex C9, NFR-22).

[PROPOSED]: the participant sees the status of their request (the market annex notes that no current channel shows the status of a request, §4 *UX* row).

W3 · Control room handles a help request

Lifecycle of a help request **[PROPOSED]**. The report says the team "*assesses its priority and routes it to the right operational team*", but does not define statuses.



Steps:

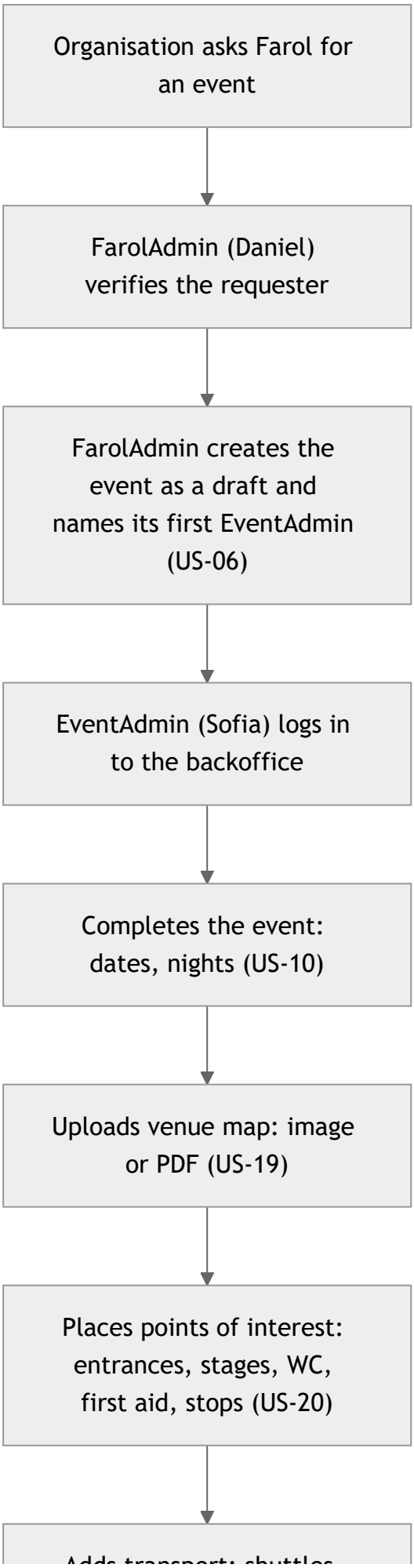
1. A new request appears on the list and the venue map of every member with `HelpRequests.Triage` (the control room, P2), with category, location, time and, if available, the AI suggestion (US-26, US-27).
2. A control-room member acknowledges it and confirms or changes the priority (US-28).
3. They route it to a team of the event with `HelpRequests.Respond`, by default the team that handles its help category (e.g. *Nursing* for `Health`; `features.md` §4.2, US-29), or handle it directly. The request appears on the phone of that team's members (P5, US-31); the first one who accepts it sets it *On the way* (US-32).
4. Every change is recorded with who and when (audit, assignment §6.6).

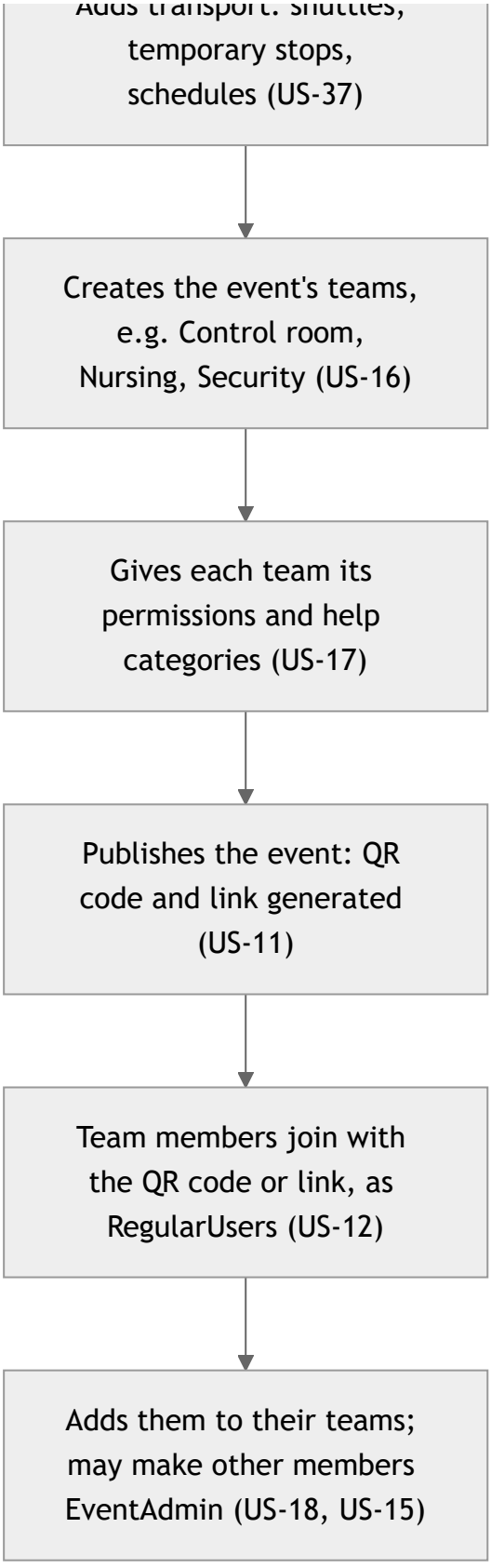
5. The team member who accepted it closes it as resolved (US-33), or the control room closes it, if it was handled from there (US-30).

> ⚠️ **GAP:** the role model is defined (features §4, to confirm with the team), and each event creates its own teams; the statuses and who closes a request are still assumptions. Needed: the interview with the security volunteer (report §4) to confirm how requests are handled today and which teams the pilot set-up should have (features §4.4).

W4 · FarolAdmin creates an event, EventAdmin sets it up

Source: report, second journey: "*creates or selects an event in the backoffice, uploads or configures the venue map, adds key points such as entrances, stages, toilets, first aid and transport stops*".


Organisation asks Farol for an event
FarolAdmin (Daniel) verifies the requester
FarolAdmin creates the event as a draft and names its first EventAdmin (US-06)
EventAdmin (Sofia) logs in to the backoffice
Completes the event: dates, nights (US-10)
Uploads venue map: image or PDF (US-19)
Places points of interest: entrances, stages, WC, first aid, stops (US-20)

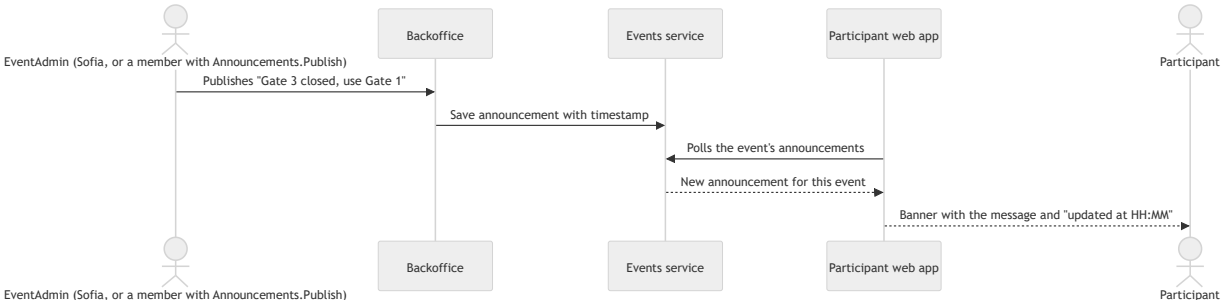


[PROPOSED]: the steps *create teams*, *set their permissions*, *add members to teams* and *publish with QR code* are not in the report but are needed for per-event permissions (report, *Product vision*) and for routing (W3). Since 22 October 2026 the teams are not fixed: each event creates its own from the permission list in `features.md` §4.2; §4.4 has a suggested set-up for the Queima. People join the event first (US-12) and are added to teams after, because only members of the event can be in its teams (US-18).

[PROPOSED] 10 October 2026: the first three steps (request, verification, creation by the FarolAdmin, persona P4) are not in the report either. Farol hosts many festivals, so an event can only be created for a verified organisation, and its EventAdmins manage only that event (F3, US-06; roles in `features.md` §4). The FarolAdmin never sees the event's help requests or reports. To confirm with the team (personas PE-5).

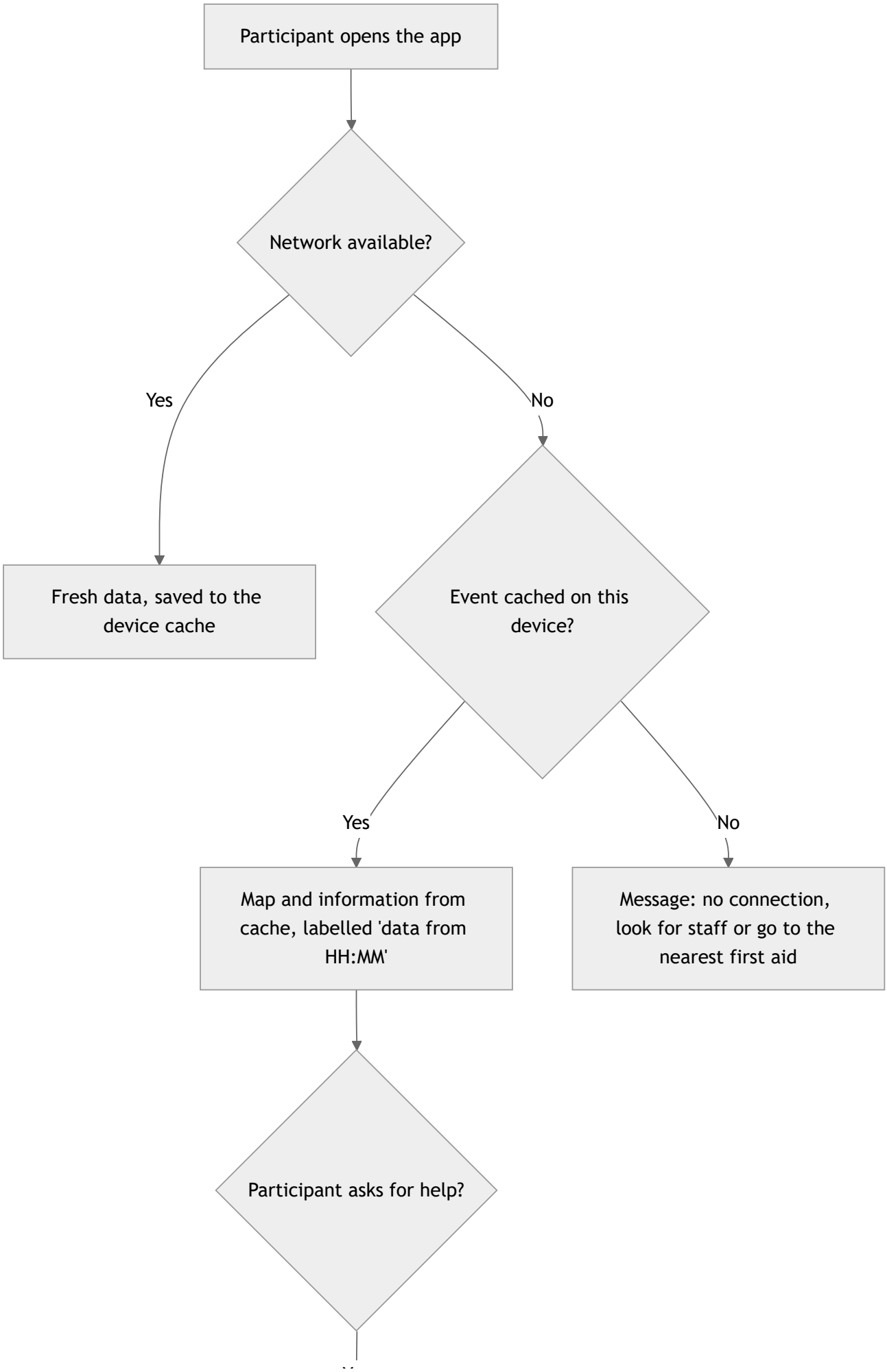
W5 · Team publishes a live update

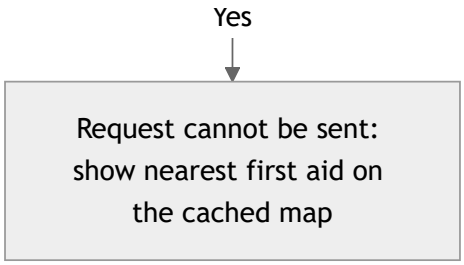
Source: report, "*updates information during the night*"; root cause 2.



W6 · Participant loses the network (F11, Could)

Source: report §5, technical risks ("*Congested mobile network at the venue: requires caching, offline mode and short messages*"); assumption A8; assignment §6.8.


Participant opens the app
Network available?
Yes
No
Fresh data, saved to the device cache
Event cached on this device?
Yes
No
Map and information from cache, labelled 'data from HH:MM'
Message: no connection, look for staff or go to the nearest first aid
Participant asks for help?

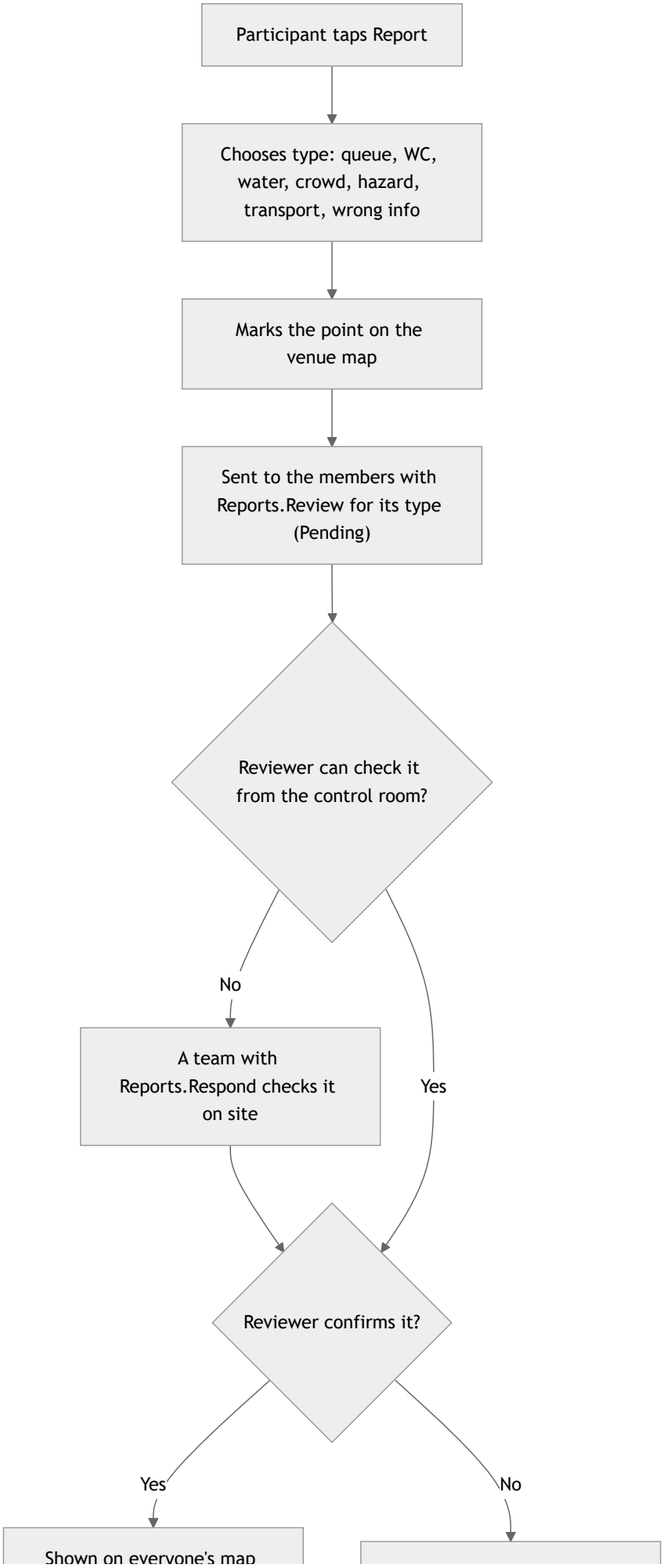


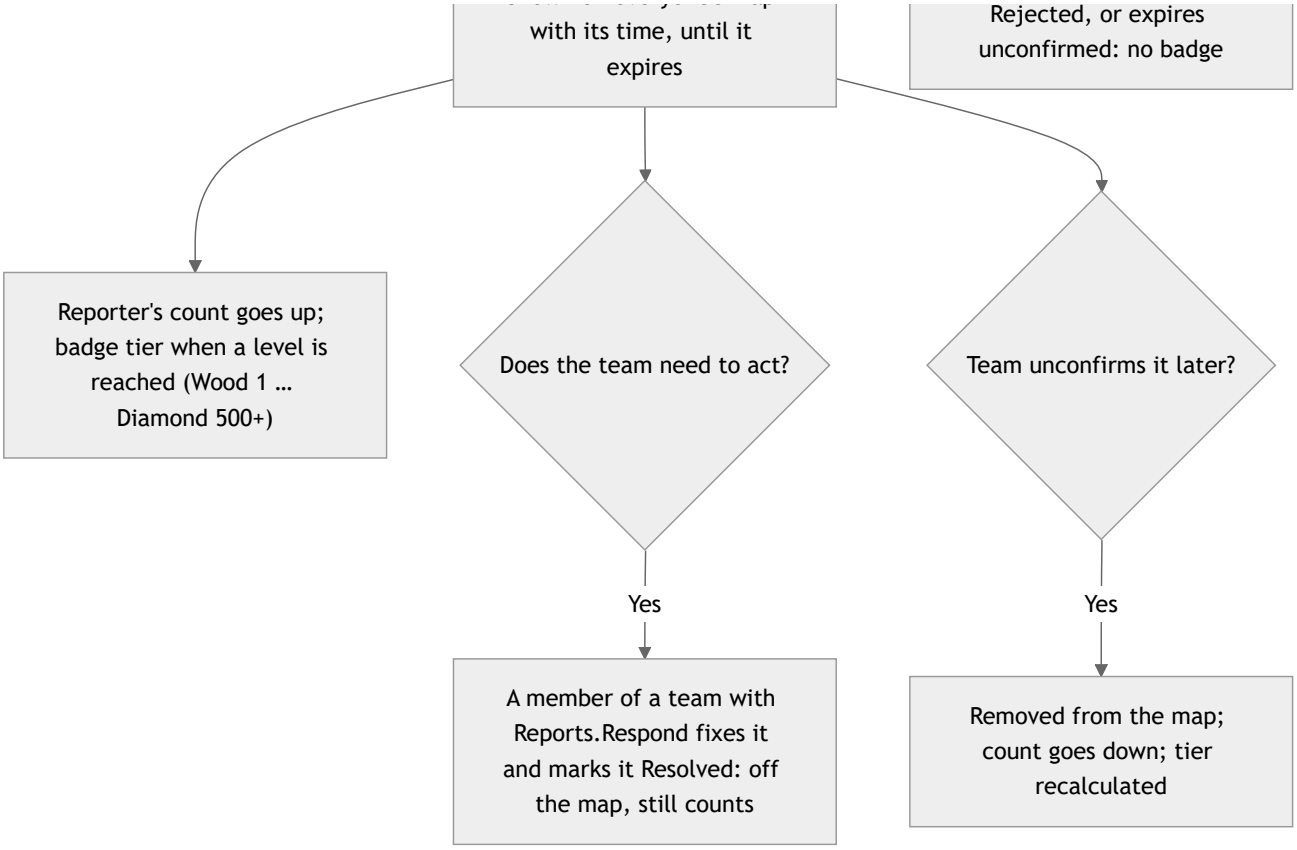
[PROPOSED]: help requests are **not** queued silently while offline, because a request that looks sent but is not is worse than an honest failure ("fail safely", assignment §6.8).

Decided (9 October 2026): network loss is **not** one of the degraded-mode scenarios of assignment §6.8, and F11 is a Could. The §6.8 scenarios are **D1 AI triage unavailable** and **D2 transport source unavailable** (`features.md` §3.1).

W7 · Participant reports a problem and earns a badge

Source: engagement mechanism decided by the team on 8 October and detailed on 9 October 2026 (`features.md` §6). The participant is logged in; badges belong to the account.

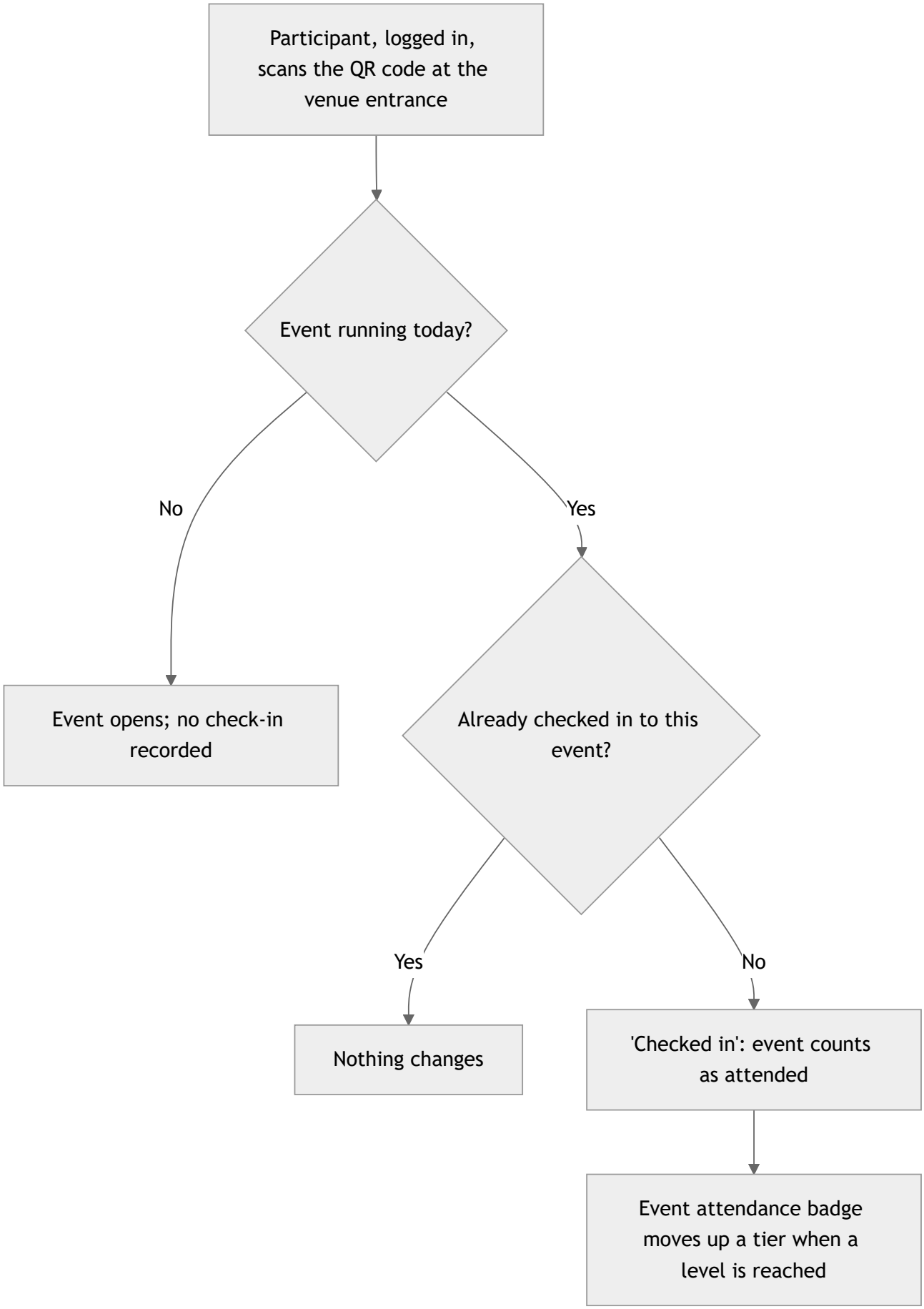

Participant taps Report
Chooses type: queue, WC, water, crowd, hazard, transport, wrong info
Marks the point on the venue map
Sent to the members with Reports.Review for its type (Pending)
Reviewer can check it from the control room?
No
A team with Reports.Respond checks it on site
Yes
Reviewer confirms it?
Yes
No
Shown on everyone's map



Urgent situations about a person are **not** reports: the app sends the participant to *Ask for help* (W2).

W8 · Participant checks in at the venue (F16, Should)

Source: attendance badges decided on 9 October 2026 (`features.md` §6.3).



Opening the event at home or from a link never counts as attendance.

5. PRODUCT BACKLOG

`Docs/Product/features.md`

Main Features and Initial Backlog

> LABDSOF 2026/27 · Sprint A (Sprint 1) · Assignment §7.1.2 (deliverables 4 *MVP definition* and 5 *Product Backlog*)
> **Product name: Farol** (decided 9 October 2026).
> **Status:** feature list and MoSCoW revised on **9 October 2026** on top of the team meeting of 8 October (minutes). Decided: login is mandatory, response teams, degraded-mode scenarios, external integration, badge tiers, attendance check-in. Still open: report types and expiry (§6.1) and the AI priority scale (AI-3, §5.2); the AI provider was decided on 27 October 2026 (guide K1; ADR-005 accepted on 31 October). **[PROPOSED] 10 October 2026:** a platform admin (persona P4) creates events and names their first admin (F3, F4), to confirm with the team. **16 October 2026:** user stories (§7) checked against the DoR; issues created on 22 October 2026 (#58 to #121, §8.3); sprint goals and initial release plan (§8) **[PROPOSED]**, to confirm with the team. **22 October 2026:** new role model, `FarolAdmin` (global), `EventAdmin` and `RegularUser` (per event), with teams and permissions defined by each event's admins (§4), replacing the fixed roles and teams of 9 October; stories split into smaller ones and renumbered US-01 to US-62 (§7.1). To confirm with the team.
> **Sources:** `problem-opportunity-report-queima.md` (*MVP definition*, §5 risks) · `market-and-competitor-analysis-product-specific.md` (§1, §2) · assignment PDF (§5.5, §5.6, §6).
> Legend: see `product-vision.md`.

1. How the product solves the problem

Root cause (report §2)	What the product does	Features
1. No single source of operational information	One platform per event where the team publishes everything participants need	F5, F9, F10
2. Communication is static	The team updates information during the night; every item shows its last-update time. Participants report what they see, and the team confirms it	F10, F11, F12, F13
3. No shared spatial reference of the venue	The venue map, uploaded by the team, is the common reference for participants and operators	F5, F6, F12
4. Incident coordination depends on radio and phone *(hypothesis)*	Help requests arrive structured, with location, category and status, and are triaged and routed to a team of the event in the backoffice	F6, F7, F8
5. Slow processes at the gate	Out of scope as a process; at most, entrance status information	F17

2. Feature list and priority

MoSCoW revised on 9 October 2026. The MVP still follows the report's scope mitigation: "*prioritise help requests, the map and transport information in the MVP*" (report §5, execution risks). *Rank* is the priority order within the Must-haves; the build order is the release plan (§8.2).

ID	Feature	Component (annex §1)	MoSCoW	Rank	Persona	Notes
F1	Login and logout	—	Must	1	P1–P5	**Login is mandatory for every user**, participants included. Mocked authentication is allowed for the MVP (assignment §6.6, footnote 1)
F2	Personal account management	—	Must	2	P1–P5	Profile, password, consent, badge visibility, delete account (deletes badges and personal data)
F3	Event management	C1	Must	3	P4, P3	Create, edit, publish and close events. Every other feature is scoped by event; isolation between events is a top technical risk (report §5). The `FarolAdmin` (P4) creates an event, for a verified organisation, and names its first `EventAdmin`; the EventAdmins edit, publish and close it (§4)
F4	Event roles, teams and permissions	C1	Must	4	P3, P4	**22 Oct:** roles per event, `EventAdmin` and `RegularUser`, plus the global `FarolAdmin`, who never sees an event's help requests, reports or participants. EventAdmins create the event's teams (e.g. *Security*, *Nursing*), set each team's permissions and add RegularUsers to them (§4)
F5	Venue map with points of interest	C3	Must	5	P1, P3	Map uploaded as an image or PDF; points placed on it (entrances, stages, WC, first aid, transport stops)
F6	Help request	C6	Must	6	P1	Category and location: the participant marks the place on the map; from Sprint C the app first asks for the phone's location, with the map as the fallback (survey, 30 October 2026; phased on 1 November 2026, #175). Location shared only with the request

ID	Feature	Component (annex §1)	MoSCoW	Rank	Persona	Notes
F7	Incident backoffice	C7	Must	7	P2, P5	Receive, prioritise, route to a team and close help requests; live list and map; full status history. Members of a team with the *respond* permission get what is routed to their team on the phone and update its status (§4.3)
F8	AI help-request triage, with non-AI fallback	C9	Must (assignment §5.6)	8	P2	Suggests category, priority and team (one of the event's teams that handles the category, §4.2); the operator decides. Kept separate from F7 on purpose (see note below). See §5
F9	Transport information with external integration	C5	Must (assignment §6.5)	9	P1, P3	**Porto Digital urban platform** (FIWARE NGSI v2): live positions of the city buses (§3), plus event shuttles and temporary stops entered by the team
F10	Live announcements, with last-update time	C4	Must	10	P1, P3	
F11	Weak-network mode: cached map and information, "data from HH:MM"	C10	Could	—	P1	Not one of the §6.8 degraded-mode scenarios (§3.1). Moved from Must to Could on 9 October 2026. **Reopened on 30 October 2026:** candidate for Should at the next planning, since battery and weak network are top barriers in the student survey
F12	Community reports on the map	—	Must (assignment §5.5)	11	P1	Waze-style reports (queues, WC out of service, …). See §6
F13	Report confirmation and unconfirmation by the responsible team	—	Must	12	P2, P5	Only the responsible team confirms a report; a confirmation can be withdrawn. See §6.2
F14	Badges for confirmed reports (all-time and per event)	—	Must	13	P1	Two of the three badge tracks. See §6.3

ID	Feature	Component (annex §1)	MoSCoW	Rank	Persona	Notes
F15	Badges for event attendance	—	Should	—	P1	Third badge track. See §6.3
F16	Event check-in at the venue	—	Should	—	P1, P3	The logged-in participant scans the event's QR code **at the venue, during the event dates**; this is what counts as "attended" for F15. See §6.3
F17	Entrance status per gate, updated manually by the team	C8	Should	—	P1, P3	**Could → Should on 30 October 2026** (#164): waiting more than 30 min to get in is the most common problem in the student survey (80 %). Manual status only; no automatic prediction
F18	Friends inside the app	—	Could	—	P1	Add and remove friends
F19	Chat between friends	—	Won't (this release)	—	P1	
F20	Link to the ticket on the ticketing partner's site	C11	Won't (this release)	—	P1	Ticketing is a partner, not part of the MVP (annex C11)

Out of scope (report, *Out of scope*; annex §1): physical capacity of entrances, heavy integrations, advanced crowd prediction, full automation, own ticketing.

Removed on 9 October 2026: the former "*participant web app opened by link or QR, without an account*". Login is now mandatory, so the participant app is simply the client application (assignment §6.1), reached after F1. The QR code still opens the event, after login.

Why F7 and F8 stay separate (proposed answer to "should they be merged?"): on the slide they are presented together as *incident handling*. In the backlog they stay apart because (1) F7 must work on its own: it **is** the fallback when the AI is down (assignment §5.6.2, scenario D1); (2) F8 has its own evaluation, acceptance criteria and Responsible AI assessment (assignment §5.6.1, Sprint 1 deliverable 6); (3) they can be built and demoed in different sprints. If the team prefers one ID, rename F8 to *F7.b*. Nothing else changes.

3. Assignment requirements covered by the features

Assignment requirement	How the product meets it	Status
§5.5 Ethical engagement design	F12–F15: community reports, team confirmation and badges	Decided (§6)
§5.6 At least one AI-assisted workflow, with evaluation and baseline or fallback	F8: help-request triage	Decided; details open (§5.2)

Assignment requirement	How the product meets it	Status
§6.1 Client application	Participant web app (PWA) and backoffice, both behind login	Decided
§6.3 At least one asynchronous workflow	Help request saved and shown to the event team at once; AI triage runs in parallel and adds a suggestion (see `architecture.md`)	Decided (ADR-004)
§6.5 Integration with at least one external system or realistic simulator	**Porto Digital urban platform**, FIWARE NGSI v2 bus API (F9; accepted by the professor, ADR-008)	Decided
§6.6 Authentication and authorisation	F1 login for every user; F2 account and consent; F4 per-event roles and team permissions (§4); audit records for help requests (`AuditEntry` in the domain model; NFR-16)	Decided; mocked authentication allowed for the MVP
§6.8 At least one degraded-mode scenario	**D1** AI unavailable and **D2** transport source unavailable (§3.1)	Decided

3.1 Degraded-mode scenarios (decided, 9 October 2026)

Two scenarios, no more. Both are demoed by breaking one dependency while the rest of the product keeps working. Assignment §6.8: "*fail safely and communicate limitations clearly to users*".

D1 · AI triage unavailable

Dependency	AI model provider used by the triage worker (F8)
Detected when	No valid suggestion within **30 s** of submission (SC3), because of a timeout (response 20 s, one attempt: the local profile, NFR-11), a provider error, an invalid or incomplete answer (unknown category, missing field), or the provider being marked down after repeated failures (circuit breaker: at least half of the last 10 calls failed or were slow, minimum 4 calls). Values: `architecture.md` §9.1
What the operator sees	The request appears in the backoffice **at once** (it never waits for the AI), with the participant's category, the label **"Not triaged"**, in order of arrival. While the provider is down (circuit breaker open), a banner says "*AI triage unavailable: requests are shown by order of arrival*"
What the operator does	Sets the priority and team by hand. Nothing else changes in F7
What the participant sees	Nothing different. Their request is sent and tracked as usual
When the AI comes back	The banner disappears after the first successful call, and new requests are triaged again. Requests already shown as "Not triaged" stay for manual triage: a request is never sent to the AI again once its 30 s deadline has passed, and a late result is discarded (AI runtime contract, ADR-005 decision 3). A suggestion **never overwrites** an operator's decision and **never lowers** a priority (NFR-22)
Recorded	Every fallback is logged with the reason (timeout, error, invalid output) and the correlation id, and counted as a metric (`farol.triage.fallback`); the circuit breaker state is on the dashboard (ADR-012)

How it is demoed	On the local machine (local-only delivery, ADR-011), the triage worker points at **WireMock** (the simulator of guide J4), which normally passes requests to the real AI provider. The presenter makes it drop the connection (or answer late, or return invalid output) and a participant submits help requests from the web app's phone pages; the services carry no fault-injection code. Step-by-step script: `architecture.md` §9.3
Test	US-36 (SC5): 100 % of requests are shown to the operator with the AI off. Reports integration test on every PR, with WireMock in place of the provider (delay over the timeout, 503, dropped connection, invalid body); also under load in Sprint C (scenario L5)

D2 · Transport source unavailable

Dependency	Porto Digital urban platform (FIWARE NGSI v2 bus API, ADR-008)
Detected when	The import fails (no answer, HTTP error), the feed is invalid or incomplete (fails validation on import), or the data is older than its freshness limit: **5 min** (decided 22 October 2026; the *y* of the meeting with the professor, P4), so late observations are dropped on import. Bus positions normally arrive about 1 min old, and a bus covers about 2 km in 5 min, so an older position misleads. The 24 h proposed earlier was for a static GTFS timetable, dropped with the professor's answer
What the participant sees	The **last valid data**, with "*Data from HH:MM*" and a warning "*Live transport information unavailable*", plus a link to the operator's website. **Event shuttles and temporary stops entered by the team are not affected**: they are internal data
What the team sees	A warning in the backoffice that the transport source is down, with the time of the last successful import
Invalid data	Never replaces valid data: the import is rejected as a whole and the previous version stays in use
Recovery	Import every 30 s (the *x* of the meeting with the professor, P4), retried with backoff (3 attempts, 1 s and 2 s apart; after repeated failures the circuit breaker waits 60 s before trying again, so a public service we do not control is not hammered); the warning disappears after the first valid import
How it is demoed	On the local machine, Transports points at **WireMock**, the simulator of the Porto Digital API (guide J4), which passes requests to the real API or replays the responses recorded on 25 October 2026 (`infra/wiremock/`). The presenter makes it return invalid data, then drop the connection, and the participant opens the transport screen. Script: `architecture.md` §9.3
Test	US-42, US-43: Transports integration test on every PR, with WireMock replaying recorded Porto Digital responses, then a 503, a timeout and an invalid payload, and a fixed clock past the freshness limit: the last valid data, its time and the stale flag are returned, shuttles still shown, invalid data never stored

4. Roles, teams and permissions (22 October 2026)

Defined on 22 October 2026, to confirm with the team. It **replaces** the fixed event roles (`Owner`, `Admin`, `Operator`, `Responder`) and the four fixed response teams decided on 9 October 2026: each event now creates its own teams and decides what each one may do. The personas do not change: P2 (operator) and P5 (responder) are now **jobs**, given by the permissions of the team a user belongs to, not roles.

4.1 Roles

Role	Scope	What it can do
FarolAdmin	The whole platform (global)	Creates events for verified organisations and names their first EventAdmin; names a new EventAdmin when an event has none; suspends and reactivates events. Sees only each event's name, dates, organisation, EventAdmins and status, **never** its help requests, reports or participants (persona P4)
EventAdmin	One event	Everything in that event: edits, publishes and closes it; creates its teams and sets their permissions; adds users to teams; makes other members EventAdmin (persona P3)
RegularUser	One event	Every user who joins the event (scanning its QR code or opening its link, US-12). Uses the participant features: asks for help, reports problems, checks in (persona P1). Gets more permissions only from the teams they are in

- Roles belong to a user's **membership of an event**, not to the account. The same person can be **EventAdmin in one event and RegularUser in another**.
- A user who is not a member of an event has no access to its data (US-14). `FarolAdmin` is the only global role, and it gives no access to any event's data.
- An event always keeps **at least one EventAdmin**: the last one cannot be removed or demoted, nor delete their account (US-05; guide D7).

4.2 Teams and permissions

An EventAdmin creates the teams of their event (e.g. *Security*, *Nursing*, *PCR*, *Logistics*, *Control room*), gives each team permissions from the fixed list below, and adds RegularUsers of the event to it.

Permission	Members of the team can…	Persona
`HelpRequests.Triage`	See every help request of the event, set its priority and status, route it to a team and close it	P2
`HelpRequests.Respond`	Receive the help requests routed to their team, accept them and mark them resolved	P5
`Reports.Review` *(for chosen report types)*	Confirm, reject and unconfirm reports of those types; send them to a team to check or fix	P2
`Reports.Respond`	Receive the reports sent to their team, answer "true / not true" and mark them resolved	P5
`Announcements.Publish`	Publish, edit and remove announcements	P3, P2
`Map.Edit`	Upload the venue map and edit its points of interest	P3
`Transport.Edit`	Add and change event shuttles and temporary stops	P3
`Entrances.Update`	Update the status of each entrance	P2, P5

- Each team also has the **help categories it handles** (`Health` , `Safety` , `LostPerson` , `Theft` , `Other`). They give the default team when a request is routed, and the teams the AI may suggest (F8).
- A user can be in **several teams** of the same event and gets the permissions of all of them.

- A change to a team's permissions or members applies at the member's **next action** and is recorded with who made it and when.
- An EventAdmin has every permission in their event without being in a team.
- The permissions are a **fixed list** in the product; EventAdmins choose from it and cannot invent new ones. This keeps authorisation testable (SC8).

4.3 Who receives what, and who acts

Professor feedback on the presentation (12 October 2026): it was not clear who receives the reports and how they are handled. With teams and permissions:

What	Goes to	Who acts
Help request	Every member with `HelpRequests.Triage` (one shared queue, the control room)	The control room sets priority and team, then routes it to a team, whose members with `HelpRequests.Respond` accept it, go to the point on the map and resolve it
Report	Members with `Reports.Review` for its type	They confirm or reject it. If it cannot be checked from the control room, or needs fixing, they send it to a team with `Reports.Respond`

- **Help requests** always pass through the control room first, because they need a decision: priority, team, whether someone must go.
- **Reports** go straight to whoever reviews their type, because the type already says who is responsible.

Both journeys as BPMN: `user-workflows.md`, *Complete journeys*. The lanes there are personas (P2, P5), which are now team permissions.

4.4 Pilot set-up for Queima das Fitas

A suggested starting point for the pilot event. The EventAdmin creates it by hand (US-16 to US-18) and can change it.

Team	Permissions	Help categories	Report types it reviews
Control room	`HelpRequests.Triage`, `Reports.Review`, `Announcements.Publish`, `Entrances.Update`	—	All
Nursing (medical)	`HelpRequests.Respond`	`Health`	—
Security	`HelpRequests.Respond`, `Reports.Respond`, `Entrances.Update`	`Safety`, `Theft`	—
PCR (risk-prevention committee)	`HelpRequests.Respond`, `Reports.Respond`	`LostPerson`, `Other`	—
Logistics	`Reports.Respond`, `Transport.Edit`	—	—

The help categories still need to be checked against the interview with the security volunteer (DM-4).

5. AI-assisted workflow: help-request triage

5.1 What is decided

Decision (8 October 2026): help-request triage. It is part of the core journey (help requests are the top MVP priority), its fallback already exists in the product (F7), and it is easy to evaluate with a synthetic dataset. The participant Q&A assistant was rejected: a wrong transport answer at night is a higher safety risk, and a chat interface would need justification (assignment §5.6).

	Help-request triage
What the AI does	Reads the participant's category and optional text, then suggests a **category, a priority and the team** (one of the event's teams that handles the category, §4.2). The operator accepts or changes the suggestion
User need	Requests arrive without category or priority (root cause 4); assumption A4
Non-AI fallback	Scenario D1 (§3.1): participant's category, order of arrival, "Not triaged"
Human control	The AI only suggests. It never lowers a priority, and it never overwrites an operator's decision (NFR-22)
Speed target	Suggestion within **30 s** of submission (SC3)
Market	Raven Controls does this for professional control rooms (annex C9); nobody does it for academic events

5.2 Still to define

Research available (22 October 2026, #53): Responsible AI assessment proposes answers to AI-1–AI-11, with an evaluation package and ADR-005, accepted as architecture on 31 October 2026 (#166). Still pending: the team review of the labels, provider/privacy review and measured model/SC3/SC5 evidence (guide K1, K2). The original questions below remain the review checklist.

This is the list the team works through before the AI work starts. Status: guide K1 and K2 (`architecture-decisions-guide.md`).

#	Item	Open question	Needed for
AI-1	**Input**	Exactly which fields go to the model: category, free text, map zone, time of night? Is the location sent, or only the zone?	Privacy, prompt design
AI-2	**Output format**	A fixed JSON schema: category (fixed list), priority (`Low` / `Medium` / `High` / `Critical`), team (one of the event's teams, §4.2), a short reason for the operator. What happens with an answer outside the schema (→ D1)	Validation, UI
AI-3	**Priority scale**	What each level means, with examples, so people and the AI apply the same rule. What is the "default priority" of each category, so that "never lowers a priority" is measurable	Evaluation, NFR-22
AI-4	**Model and provider**	Which provider and model; hosted API or local model; cost per request; what happens to the text sent (data retention by the provider, GDPR)	Cost, privacy, ADR

#	Item	Open question	Needed for
AI-5	**Prompt design**	System prompt, few-shot examples, European Portuguese informal language, how the participant's text is isolated from the instructions	Quality, prompt injection
AI-6	**Evaluation dataset**	Size (proposal: ≥ 100 cases), who writes and labels it, share of difficult cases (typos, slang, mixed languages, empty text, prompt injection, two problems in one request), how origin, scope and limits are documented (§5.6.1)	Evaluation
AI-7	**Acceptance criteria**	Confirm SC4: 0 urgent cases under-prioritised, category accuracy ≥ 80 %; add response-team accuracy? Compared with the non-AI baseline (participant's category only)	Go / no-go
AI-8	**What the operator sees**	Is the suggestion shown with a reason or a confidence? How is it marked as a suggestion, not a fact (§5.6.2)? One click to accept?	UI, trust
AI-9	**Monitoring**	Metrics: latency p95, fallback rate, % of suggestions accepted or changed by operators, cost per night	Observability (§6.7)
AI-10	**Safety and trust**	How each item of §5.6.3 is handled: inaccurate output, prompt injection, disclosure, harmful content, bias (e.g. by language), user correction, human approval	Responsible AI assessment
AI-11	**Responsible AI Opportunity Assessment**	The Sprint 1 document itself (deliverable 6): proposed workflow, expected value, risks, evaluation approach, fallback	Sprint 1 deliverable

6. Ethical engagement mechanism: community reports and badges

Decision (8 October, detailed 9 October 2026): participants report what they see on the venue map, in the style of Waze. The responsible team confirms useful reports, and participants earn **badges** for confirmed reports and for events attended. Reports keep information fresh for everyone (root cause 2) and show the team problems sooner; badges give participants a reason to report.

6.1 What can be reported

Each report is a type, a point on the venue map and a time. It is not a help request: anything urgent or about a person goes through *Ask for help*.

Report type	Example	Field team in the pilot set-up (§4.4)	Expires after
Queue at an entrance	"Gate 2: long queue (> 15 min)" · short / medium / long	Security	30 min
Crowded area	"Very crowded in front of the main stage"	Security	20 min
WC problem	Out of service, no water, …	Logistics	60 min
Water point or bar out of water	"No water at bar 3"	PCR	60 min
Hazard	Broken glass, spill, dark area, broken fence, …	Logistics	Until the team closes it

Report type	Example	Field team in the pilot set-up (§4.4)	Expires after
Transport problem	"Shuttle stop moved", "shuttle full", "shuttle not passing"	Logistics	30 min
Wrong information	A map point or announcement that is no longer true	Logistics	Until the team closes it

6.2 How a report behaves

1. A participant places a report on the map. It goes straight to the **members with `Reports.Review` for its type** (§4.2, §4.3), who see it **in the backoffice at once**. Its status is *Pending*.
2. **Only a member with `Reports.Review` for its type (or an EventAdmin) confirms or rejects it.** If it cannot be checked from the control room, they send it to a team with `Reports.Respond` to check it on site. Other participants see it only once it is confirmed. They can tap "*still there*" or "*not there anymore*": this helps the team decide, but it does not confirm the report and does not count for badges. **[PROPOSED]**
3. A confirmed report counts for the reporter's badges and appears on everyone's map with its time.
4. The team can **unconfirm** a report (e.g. confirmed by mistake, or found to be false). It disappears from the map, **no longer counts** for badges, and the reporter's badge levels are recalculated, so a badge can go down a tier.
5. A report **expires** after the time set for its type (§6.1). An expired confirmed report still counts for badges; it is only removed from the map.
6. A report never changes official information (map, announcements, transport) automatically. The team does that.
7. If the problem needs action (a broken WC, broken glass), the reviewer sends it to a team with `Reports.Respond`, and a member of that team fixes it and marks the report *Resolved*: it leaves the map and still counts for badges. A report nobody confirms is removed when it expires and earns nothing.

Report statuses **[PROPOSED]**: `Pending → Confirmed | Rejected | Expired`, then `Confirmed → Resolved | Expired | Unconfirmed`.

6.3 Badges (decided)

Badges are always linked to the participant's **account** (F1, F2). There are no anonymous badges. There are **three badge tracks**; each one moves up through the same six tiers as the count grows.

Track	What is counted	Scope	MoSCoW
Event attendance	Events attended	All events, across the years	Should (F15)
Report history	Confirmed reports	All events, across the years	Must (F14)
Event reporter	Confirmed reports	One specific event (e.g. "Queima 2027")	Must (F14)

Tier	Count (events attended or confirmed reports)
Wood	1–2

Tier	Count (events attended or confirmed reports)
Bronze	3–9
Silver	10–49
Gold	50–99
Platinum	100–499
Diamond	500+

There are no badges for help requests: emergencies must never be gamified.

How attendance is proved (decided, 9 October 2026): an event counts as attended when the participant, **logged in**, scans the event's QR code **at the venue during the event dates** (F16, check-in). It counts once per event. Opening the event at home, or from a link, does not count. The QR codes are placed at the venue entrances by the event team.

6.4 The five questions of assignment §5.5

1. **Behaviour encouraged:** participants report what they see at the venue, which keeps information accurate for everyone, and they come back to the events.
2. **Why it helps the community:** other participants avoid queues and broken services; the team learns about problems before they become incidents.
3. **Misuse prevention:**
   - only reports **confirmed by the responsible team** count for badges, so false reports earn nothing;
   - the team can unconfirm a report, and the badge count goes back down;
   - reports are rate-limited per account;
   - reports expire;
   - the team can hide reports; suspending an account from reporting is **[PROPOSED]** (no story yet);
   - reports never change official information automatically.
4. **Exclusion or exposure:**
   - no public leaderboard and no public profile; badges are private by default;
   - nothing shows other participants who made a report;
   - attendance badges show neither which nights you were there nor where you were;
   - participants who don't report lose no information.
5. **Control and opt-out:** reporting and badges are optional. In account management (F2) a participant can hide their badges or stop earning them (US-56); deleting them on their own is **[PROPOSED]** (no story yet); deleting the account deletes the badges (US-05).

7. Initial backlog: user stories

Rewritten on 9 October 2026 to match the feature list above; checked against the Definition of Ready on 16 October 2026 (professor feedback T1, notes); split into smaller stories and renumbered on 22 October 2026, with the role model of §4. One story = one issue = one PR. Acceptance criteria use *Given / When / Then*. Every

story names its persona and its dependencies; a story moves to *Ready* on the board only when none of its dependencies blocks it (§8.4). Won't-have features (F19, F20) have no stories.

7.1 DoR check and split

DoR check (16 October 2026)

DoR item	What failed	Fix
User story format	Old US-04 had no *so that*	Added (US-14)
Persona	No story named a persona ID; old US-01 and US-02 said only "user"	*Persona* line on every story (P1–P5, or the technical goal)
Testable *Given / When / Then*	Help request offline needed the cached map (F11, Could); routing assumed push notifications (guide C5 decided polling) and had no criterion to close; adding a shuttle had no criterion; transport assumed a timetable feed, but the source is not decided; "too many reports" had no number; reports had no *reject*; entrance status had nothing for the team side	Criteria rewritten (see each story). Numbers that are new are marked **[PROPOSED]**
Dependencies identified	No story listed its dependencies	*Depends on* line on every story; blockers in §8.4
One story = one PR	Several stories held two or more PRs of work	Split (below)
AI fallback	The AI suggestion did not say what happens when the AI fails	*AI fallback* line pointing to US-36 (scenario D1)
On the board	No story had an issue	Issues #58 to #121 created on 22 October 2026 from the user-story form (§8.3). All of them are on the GitHub Projects board (checked 24 October 2026); the DoR box "On the GitHub Projects board" is ticked in each story issue (24 October 2026)

How the stories were split (22 October 2026)

The 29 stories of 16 October became **62**. Each story is now **one user action, or one rule, that can be built, tested and demoed on its own in one PR of a few days**. Three rules were used:

1. **One action per story.** *Log in* and *log out* are different actions with different tests, so they are two stories (US-01, US-02). The same for publishing, joining and closing an event, or for accepting and resolving a request.
2. **Separate what is blocked from what can start.** A part that waits for an open decision becomes its own story, so the rest can be *Ready*. Example: account deletion (US-05) waits for guide D7; sign-up (US-03) does not.
3. **Separate apps and personas.** What the event team does in the backoffice and what the participant sees are different PRs, on different pages of the web app (guide F1). Example: the team adds shuttles (US-37), the participant sees them (US-39).

Old story (16 October)	New stories
US-01 Log in and log out	US-01, US-02; the QR-code criterion moved to US-12
US-02 Create and edit account	US-03, US-04
US-29 Delete account	US-05
US-25 Create an event	US-06, US-07
US-28 Transfer Owner, suspend event	US-08, US-09
US-03 Edit, publish and close an event	US-10, US-11, US-12, US-13
US-04 Isolation between events	US-14
US-05 Invite members, roles and teams	US-15, US-16, US-17, US-18 (new role model, §4)
US-06 Venue map and points	US-19, US-20, US-21
US-07 Find a type of place	US-22
US-08 Ask for help	US-23, US-24, US-25
US-09 See new requests	US-26, US-27
US-10 Prioritise, route, close	US-28, US-29, US-30
US-26 Responder	US-31, US-32, US-33
US-11 AI suggestion	US-34, US-35
US-12 Requests without AI (D1)	US-36
US-13 Shuttles	US-37, US-38, US-39
US-14 Next transport options	US-40, US-41
US-15 Out-of-date transport (D2)	US-42, US-43
US-16 Announcements	US-44, US-45
US-17 Weak-network mode	US-46
US-18 Report a problem	US-47, US-48, US-49
US-19 Confirm, reject, unconfirm	US-50, US-51
US-27 Send a report to a responder	US-52, US-53
US-20 Badges for confirmed reports	US-54, US-55, US-56
US-21 Attendance badge	US-57
US-22 Check-in	US-58
US-23 Entrance status	US-59, US-60
US-24 Friends	US-61, US-62

Why these numbers

Both confirmed: the report limit on 24 October 2026 and the responder's 5 s (NFR-02) on 26 October 2026 (NFRs §4–§5).

Number	Story	Why
5 reports in 10 minutes per account	US-48	A participant can see several problems at once (a queue, a broken WC, broken glass), so 5 allows a real burst. More than that in 10 minutes looks like spam or badge farming, which §6.4 must prevent. Reports expire after 20–60 minutes (§6.1), so 10 minutes is shorter than any expiry. It is a setting, to tune with real data. Confirmed on 24 October 2026 (#137)
≤ **5 s** until a routed request appears on a responder's phone	US-31	Same target as SC2 for the control room: the responder is on the critical path of a help request. With polling (guide C5) it needs a refresh every 2–3 s on the open screen, which one instance per service can serve at the expected load

Why these criteria were rewritten

Story	Before	After	Why
US-25 (no network)	"shows the nearest first-aid point"	Says the request was not sent, lets the participant send it again, and shows the first-aid points if the map is already loaded	Finding the nearest point offline needs the cached map, which is F11 (Could). A Must story cannot depend on a Could. The new criterion can be tested with the map in memory
US-29 (routing)	"its responders get an alert on their phone"	The request appears in the routed team's list (US-31)	Push notifications were never decided; the team decided polling (guide C5). The phone side is US-31, so US-29 stops where the control room's work ends
US-41 (transport)	"next departures" from the STCP feed	"next buses at the stops near the venue", from whatever source is chosen	The source is waiting for the professor (Porto Digital FIWARE or another). FIWARE gives buses in circulation, not timetables, so the old criterion could fail whichever source is chosen

Issue type of each story

README §1 allows four issue types. Rule used in §8.3, from the split between participant pages and event-team pages of the web app (guide F1):

- **[Front Office]**: the participant's pages of the web app, built for the phone (P1).
- **[Back Office]**: the event team's work: event management on the web (P3, P4) and the backoffice of the control room and of the field teams (P2, P5). Field-team members work on the phone, but in the backoffice field view (personas, P5), not in the participant pages.
- **[Technicians]**: technical work with no single user screen: login and logout (US-01, US-02, used by every page), isolation between events (US-14, a test in every service), the import of the transport source (US-40) and the walking skeleton.

F1 · Login and logout

US-01 · As a **user**, I want to log in, so that I can use my account and the events I belong to.
Persona: P1, P2, P3, P4, P5 · *Depends on:* walking skeleton (§8.2); identity provider (guide E1; mocked

authentication is allowed for the MVP, assignment §6.6)

- Given I have an account, when I log in with valid credentials, then I see the events I am a member of.
- Given I enter wrong credentials, when I log in, then I see an error that does not say which field was wrong.

US-02 · As a **user**, I want to log out, so that nobody else can use my session on this device.
Persona: P1, P2, P3, P4, P5 · *Depends on:* US-01

- Given I am logged in, when I log out, then my session ends and protected pages ask me to log in again.
- Given I logged out, when the old session token is used, then it is refused.

F2 · Personal account management

US-03 · As a **new user**, I want to create an account, so that I can join events on Farol.
Persona: P1, P2, P3, P5 · *Depends on:* US-01; identity provider (guide E1)

- Given I am a new user, when I sign up and accept the terms, then my account is created and I am logged in.
- Given I do not accept the terms, when I sign up, then no account is created.
- Given the e-mail is already used by an account, when I sign up, then no new account is created.

US-04 · As a **user**, I want to edit my name and password, so that my account stays correct and safe.
Persona: P1, P2, P3, P4, P5 · *Depends on:* US-03

- Given I am logged in, when I edit my name, then the change is saved.
- Given I change my password, when I log in again, then only the new password works.

US-05 · As a **user**, I want to delete my account, so that my personal data leaves Farol.
Persona: P1, P2, P3, P5 · *Depends on:* US-03, US-54; account deletion (guide D7)

- Given I delete my account, then my personal data, memberships and badges are deleted, and my past reports stay only as anonymous statistics.
- Given I deleted my account, when I try to log in with it, then I am refused.
- Given I am the last EventAdmin of an event, when I try to delete my account, then I am asked to name another EventAdmin first.

F3 · Event management

US-06 · As the **FarolAdmin**, I want to create an event for a verified organisation and name its first EventAdmin, so that only real organisers run events on Farol.
Persona: P4 · *Depends on:* US-01; role model (§4)

- Given an organisation asks for a new event, when I confirm the requester represents it and create the event with a name and dates, then it is saved as a draft that participants cannot see, and the named user becomes its EventAdmin.
- Given a user who is not the FarolAdmin, when they try to create an event, then the action is refused.

US-07 · As the **FarolAdmin**, I want to see the list of events with only their basic details, so that I can manage the platform without seeing private event data.
Persona: P4 · *Depends on:* US-06

- Given I open the list of events, then I see each event's name, dates, organisation, EventAdmins and status.
- Given I am the FarolAdmin and not a member of event X, when I try to open its help requests, reports or participants, then the request is refused.

US-08 · As the **FarolAdmin**, I want to name a new EventAdmin for an event, so that an event is never left without someone in charge.
Persona: P4 · *Depends on:* US-06

- Given the organisation asks (e.g. its only EventAdmin left), when I name a new EventAdmin, then they get the role in that event and the change is recorded with my name and the time.

US-09 · As the **FarolAdmin**, I want to suspend and reactivate an event, so that an abandoned or misused event cannot be used.
Persona: P4 · *Depends on:* US-06

- Given I suspend an event, then participants can no longer open it, and its EventAdmins see that it is suspended.
- Given I reactivate a suspended event, then it returns to the status it had before.

US-10 · As an **EventAdmin**, I want to edit my event's details, so that participants see the right name, dates and description.
Persona: P3 · *Depends on:* US-06

- Given I am an EventAdmin of a draft or published event, when I change its name, dates or description, then the change is saved and shown to participants.
- Given I am a RegularUser of the event, when I try to edit it, then the action is refused.

US-11 · As an **EventAdmin**, I want to publish my event, so that participants can find it.
Persona: P3 · *Depends on:* US-06

- Given my event is a draft, when I publish it, then participants can open it, and its QR code and link are generated.

US-12 · As a **participant**, I want to join an event by scanning its QR code or opening its link, so that I can use it.
Persona: P1 · *Depends on:* US-01, US-11

- Given I am logged in, when I scan the QR code of a published event, then I become a RegularUser of that event and it opens.
- Given I am not logged in, when I scan the QR code, then I am asked to log in and then taken to that event.
- Given the event is a draft, closed or suspended, when I scan its QR code, then I cannot join it.

US-13 · As an **EventAdmin**, I want to close my event, so that it stops receiving help requests and reports when it is over.
Persona: P3 · *Depends on:* US-11

- Given I close the event, then participants can no longer send help requests or reports to it, and they see it is closed.

US-14 · As a **member of event X**, I want no one outside my event to see its data, so that our help requests, reports and participants stay private (SC8).
Persona: P1, P2, P3, P5; technical goal SC8 (report §5, technical risks) · *Depends on:* US-01, US-06; `event_id` filtering (guide D3, decided); role model (§4)

- Given a user is a member of event X only, when they request any data from event Y, then the request is refused and the attempt is logged with the user, the event and the time.
- Given a user is EventAdmin in event X and RegularUser in event Y, when they try an EventAdmin action in event Y, then it is refused.
- Given a service stores event data, then an integration test proves that a member of event X cannot read or change event Y's data (guide D3).

F4 · Event roles, teams and permissions

US-15 · As an **EventAdmin**, I want to make a member of my event an EventAdmin, or remove that role, so that I can share the management of the event.
Persona: P3 · *Depends on:* US-12; role model (§4)

- Given a RegularUser of my event, when I make them EventAdmin, then they can manage this event only, and the change is recorded with my name and the time.
- Given an EventAdmin of my event, when I make them a RegularUser again, then they lose the EventAdmin permissions at their next action.
- Given I am the last EventAdmin, when I try to remove my own role, then the action is refused.

US-16 · As an **EventAdmin**, I want to create, rename and delete the teams of my event (e.g. *Security*, *Nursing*), so that the event's organisation is reflected in Farol.
Persona: P3 · *Depends on:* US-06; role model (§4)

- Given I create a team with a name, then it appears in my event's team list, with no members and no permissions.
- Given a team name already exists in my event, when I create another with the same name, then it is refused.
- Given I delete a team, then its members lose its permissions, and the requests routed to it go back to the control room queue.

US-17 · As an **EventAdmin**, I want to set the permissions and help categories of each team, so that each team can do exactly what its job needs.
Persona: P3 · *Depends on:* US-16

- Given a team, when I give it permissions from the list in §4.2, then its members can do those actions in this event only.
- Given I give `Reports.Review` to a team, then I choose the report types it reviews.
- Given I set the help categories a team handles, then it becomes the default team for those categories when a request is routed.

- Given I remove a permission, then the team's members lose it at their next action, and the change is recorded with my name and the time.

US-18 · As an **EventAdmin**, I want to add RegularUsers of my event to a team, and remove them, so that the right people get the team's permissions.
Persona: P3 · *Depends on:* US-12, US-16

- Given a RegularUser of my event, when I add them to the *Nursing* team, then they get the team's permissions in this event.
- Given a user who is not a member of my event, when I try to add them to a team, then it is refused.
- Given a user is in two teams, then they have the permissions of both.
- Given I remove a user from a team, then they lose its permissions at their next action.

F5 · Venue map

US-19 · As an **EventAdmin** (or a member with `Map.Edit`), I want to upload the venue map as an image or PDF, so that participants and the team share the same reference.
Persona: P3 · *Depends on:* US-06; venue map library (guide F3, ADR-007, #44)

- Given I upload a valid image or PDF, when I save, then it becomes the event's current map version.
- Given I upload a file that is not an image or PDF, when I save, then it is refused with a message and the current map stays.
- Given I upload a file larger than 20 MB, when I save, then it is refused with a message giving the limit, and the current map stays (ADR-013, L-4).

US-20 · As an **EventAdmin** (or a member with `Map.Edit`), I want to place, edit and remove points of interest on the map, so that people find entrances, stages, WC, first aid and transport stops.
Persona: P3 · *Depends on:* US-19

- Given I place a point of type *first aid* with a name, when I save, then it is stored at that position on the map.
- Given I move or remove a point, when I save, then the change is stored.

US-21 · As a **participant**, I want to open the venue map with its points of interest, so that I know where things are.
Persona: P1 · *Depends on:* US-12, US-20

- Given the event has a map, when I open it, then I see the current version with every point, its icon and its name.
- Given the team uploaded a new map version, when I open the map again, then I see the new version.

US-22 · As a **participant**, I want to filter the map by a type of place (WC, first aid, entrance, transport stop), so that I find it quickly.
Persona: P1 · *Depends on:* US-21

- Given the map has first-aid points, when I filter by *first aid*, then only those points are highlighted.
- Given the map has no point of the type I filter by, when I filter, then I see a message saying so.

F6 · Help request

US-23 · As a **participant**, I want to ask for help by choosing a category and sharing where I am, so that the team can reach me.
Persona: P1 · *Depends on:* US-14, US-21

- Given I choose a category, when I ask for help, then I mark where I am on the map, and when I submit I see a confirmation.
- From Sprint C (survey decision 4, phased on 1 November 2026, #175): given I choose a category, when I ask for help, then the app first asks for permission to use the phone's location; if I allow it, the request carries it; if I refuse or it is not available, I mark where I am on the map.
- Given I submitted a request, then my location is visible only to members with `HelpRequests.Triage`, to EventAdmins and to the team it is routed to.
- Given I have sent 5 help requests in the last 10 minutes, when I submit another, then it is refused with a message saying when I can ask again, and the first-aid points are shown on the map (ADR-013, L-2: answer 429 with `Retry-After`).

US-24 · As a **participant**, I want to follow the status of my help request, so that I know whether help is coming.
Persona: P1 · *Depends on:* US-23

- Given I submitted a request, when I open it, then I see its status (*Sent*, *Seen*, *On the way*, *Resolved* or *Closed*) and the time of the last change.
- Given its status changes, then I see the new status within 5 s while the screen is open (polling, guide C5).

US-25 · As a **participant**, I want to know when my help request was not sent, so that I never think help is coming when it is not.
Persona: P1 · *Depends on:* US-23

- Given I have no network, when I submit, then the app tells me the request was not sent, lets me send it again, and never shows it as sent.
- Given the map is already loaded, when the request fails, then the app shows where the first-aid points are.

F7 · Incident backoffice

US-26 · As a **control-room member** (`HelpRequests.Triage`), I want to see new help requests in a list as they arrive, so that I can react quickly.
Persona: P2 · *Depends on:* US-17, US-23

- Given a request is submitted for my event, then it appears in my list within 5 s without reloading the page (SC2; polling, guide C5).
- Given I do not have `HelpRequests.Triage` in this event, when I open the list, then it is refused.

US-27 · As a **control-room member**, I want to see the help requests on the venue map, so that I know where each one is.
Persona: P2 · *Depends on:* US-26

- Given a request has a location, then it is shown at that point on the venue map, with its category and priority.

US-28 · As a **control-room member**, I want to set a request's priority and status, with every change recorded, so that everyone knows how urgent it is and what happened.
Persona: P2 · *Depends on:* US-26

- Given I change the priority or status of a request, then the change is recorded with my name, the time, and the old and new values (SC9).
- Given I open a request, then I see its full change history.

US-29 · As a **control-room member**, I want to route a request to a team, so that the right people deal with it.
Persona: P2 · *Depends on:* US-17, US-28

- Given I open a request, then the team that handles its category is proposed by default (US-17).
- Given I route a request to the *Nursing* team, then it appears in that team's list with its point on the map (US-31), and the routing is recorded.
- Given a team without `HelpRequests.Respond`, then I cannot route requests to it.

US-30 · As a **control-room member**, I want to close a request, so that finished cases leave the active queue.
Persona: P2 · *Depends on:* US-28

- Given I close a request, then it leaves the active list, the participant sees it *Closed*, and the closing is recorded.

US-31 · As a **field-team member** (`HelpRequests.Respond`), I want to see on my phone the requests routed to my team, so that I know where to go.
Persona: P5 · *Depends on:* US-29

- Given a request is routed to my team, then it appears on my phone within 5 s (NFR-02), with its category and its point on the venue map.
- Given a request is routed to another team, then I do not see it.

US-32 · As a **field-team member**, I want to accept a request, so that the control room and the participant know someone is on the way.
Persona: P5 · *Depends on:* US-31

- Given I accept a request, then its status becomes *On the way* and the participant sees "Help is on the way".
- Given another member of my team already accepted it, then I see that and cannot accept it too.

US-33 · As a **field-team member**, I want to mark a request resolved, so that the control room and the participant know it is over.
Persona: P5 · *Depends on:* US-32

- Given I accepted a request, when I mark it resolved, then the participant and the control room see it *Resolved*, and the change is recorded.

F8 · AI help-request triage

US-34 · As a **control-room member**, I want each new help request to arrive with a suggested category, priority and team, so that I can route it faster.

Persona: P2 · *Depends on:* US-17, US-26; AI provider and model (guide K1); input, output schema and priority scale (§5.2 AI-1 to AI-3) · *AI fallback:* US-36 (scenario D1)

- Given a help request is submitted and the AI is available, then a suggestion is shown within 30 s, marked as a suggestion (SC3).
- Given the event's teams and their help categories (US-17), then the AI only suggests a team of this event.
- Given the AI suggests a priority lower than the category's default, then the default priority is kept (NFR-22; the per-category floor of the local AI plan).

US-35 · As a **control-room member**, I want to accept or change the AI suggestion, so that I stay in control of the decision.
Persona: P2 · *Depends on:* US-34 · *AI fallback:* US-36

- Given I accept the suggestion, then its values become the request's values and the change is recorded as mine.
- Given I already set a priority, when a suggestion arrives, then my priority is not changed.

US-36 · As a **control-room member**, I want requests to keep arriving when the AI is down, so that no one is left without help (scenario D1).
Persona: P2 · *Depends on:* US-34 · *AI fallback:* this story is the fallback

- Given the AI does not answer within 30 s, or answers with invalid data, then the request is shown with the participant's category, marked "Not triaged", in order of arrival (SC5).
- Given the AI provider is down, then the backoffice shows a banner saying triage is unavailable.
- Given a fallback happens, then it is logged with its reason (timeout, error or invalid output) and counted in the fallback-rate metric.

F9 · Transport information and external integration

US-37 · As an **EventAdmin** (or a member with `Transport.Edit`), I want to add the event's shuttles and temporary stops with their schedule, so that participants know how to get home.
Persona: P3 · *Depends on:* US-20

- Given I add a shuttle with its stops and departure times, when I save, then it is stored for the event.

US-38 · As an **EventAdmin** (or a member with `Transport.Edit`), I want to mark a shuttle as delayed or cancelled, so that participants do not wait for nothing.
Persona: P3 · *Depends on:* US-37

- Given I mark a shuttle as delayed or cancelled, then the change is stored with the time it was made.

US-39 · As a **participant**, I want to see the event's shuttles and temporary stops, so that I know how to get home.
Persona: P1 · *Depends on:* US-37

- Given the event has shuttles, when I open the transport screen, then I see each shuttle's stops and next departures.
- Given a shuttle was marked delayed or cancelled (US-38), then I see that with the time of the change.

US-40 · As the **Farol team**, we want to import the external transport source regularly and validate it, so that participants get regular public transport data (assignment §6.5).
Persona: technical goal (assignment §6.5) · *Depends on:* walking skeleton; transport data source: Porto Digital urban platform, accepted by the professor (ADR-008); proof of concept #48 as reference

- Given the source answers with valid data, when the import runs, then the data is stored with the time of the import.
- Given the source answers with invalid or incomplete data, when the import runs, then the import is rejected as a whole and the previous data stays in use.

US-41 · As a **participant**, I want to see the next regular buses (STCP) home from the venue, so that I don't walk home or drive.
Persona: P1 · *Depends on:* US-12, US-40

- Given the external transport source was imported, then I see the next buses at the stops near the venue, with the time of the data.

US-42 · As a **participant**, I want to know when the transport information may be out of date, so that I don't rely on it blindly (scenario D2).
Persona: P1 · *Depends on:* US-39, US-41; resilience values (guide J1, #43)

- Given the transport source is unavailable or its data is older than the freshness limit, then I see the last valid information with "Data from HH:MM" and a warning, and event shuttles are still shown.
- Given the source comes back, then the warning disappears after the next valid import.

US-43 · As an **EventAdmin**, I want to be warned when the transport source is down, so that the team can tell participants another way.
Persona: P3 · *Depends on:* US-40; resilience values (guide J1, #43)

- Given the source is unavailable, then the backoffice shows a warning with the time of the last successful import.
- Given the source comes back, then the warning disappears after the next valid import.

F10 · Announcements

US-44 · As an **EventAdmin** (or a member with `Announcements.Publish`), I want to publish an announcement (e.g. "Gate 3 closed, use Gate 1"), so that participants find out immediately.
Persona: P3 · *Depends on:* US-12

- Given I publish an announcement, then participants with the app open see it within 15 s without reloading, with its publication time (SC6).
- Given I do not have `Announcements.Publish`, when I try to publish, then it is refused.

US-45 · As an **EventAdmin** (or a member with `Announcements.Publish`), I want to edit or remove an announcement, so that wrong or old information disappears.
Persona: P3 · *Depends on:* US-44

- Given I edit or remove an announcement, then participants see the change within 15 s, and an edited one shows the time of the edit.

F11 · Weak-network mode (Could)

US-46 · As a **participant**, I want the map and the last information to work when the network drops, so that I'm not left without information.
Persona: P1 · *Depends on:* US-21, US-44

- Given I opened the event once with network, when I lose connection, then I still see the map and information with "Data from HH:MM".

F12 · Community reports

US-47 · As a **participant**, I want to report a problem on the map (e.g. a long queue at a gate), so that the team knows about it.
Persona: P1 · *Depends on:* US-17, US-21; report types and expiry times (§6.1)

- Given I choose a report type and a point on the map, when I submit, then the members with `Reports.Review` for that type see it in the backoffice straight away, with status *Pending*.
- Given my report is pending, then other participants do not see it, and I see it as *Pending*.

US-48 · As a **participant**, I want a clear message when I report too often, so that I know when I can report again.
Persona: P1 · *Depends on:* US-47

- Given I have sent 5 reports in the last 10 minutes, when I send another, then it is refused with a message saying when I can report again.

US-49 · As a **participant**, I want to see the confirmed reports on the map, so that I avoid queues and broken services.
Persona: P1 · *Depends on:* US-50

- Given a report was confirmed, then it appears on every participant's map with its type and time, and without the reporter's name.
- Given a report reaches the expiry time of its type (§6.1), then it leaves the map.

F13 · Report confirmation

US-50 · As a **reviewer** (`Reports.Review`), I want to confirm or reject a report, so that only true information is shown and rewarded.
Persona: P2 · *Depends on:* US-47

- Given a pending report of a type I review, when I confirm it, then it is shown on participants' maps (US-49) and counts as a confirmed report of the reporter.
- Given a pending report, when I reject it, then it never appears on participants' maps and does not count.
- Given a report of a type I do not review, then I cannot confirm or reject it (an EventAdmin can).

US-51 · As a **reviewer**, I want to unconfirm a report, so that a mistake or a false report stops being shown and rewarded.
Persona: P2 · *Depends on:* US-50

- Given a confirmed report, when I unconfirm it, then it disappears from the map, no longer counts, and the change is recorded with my name and the time.

US-52 · As a **reviewer**, I want to send a report to a field team to check it on site, so that reports I cannot check from the control room still get an answer.
Persona: P2, P5 · *Depends on:* US-50, US-31

- Given a pending report, when I send it to a team with `Reports.Respond`, then its members get it on their phone with its point on the map.
- Given a member answers "true" or "not true", then the answer comes back to me, and I confirm or reject the report.

US-53 · As a **field-team member** (`Reports.Respond`), I want to fix the problem of a confirmed report and mark it resolved, so that it leaves the map.
Persona: P5 · *Depends on:* US-52

- Given a confirmed report was sent to my team to fix, when I mark it resolved, then it leaves the map and still counts for the reporter's badges.

F14 · Badges for confirmed reports

US-54 · As a **participant**, I want a *Report history* badge for all my confirmed reports, so that my contribution over the years is recognised.
Persona: P1 · *Depends on:* US-50, US-51

- Given my confirmed-report count reaches 1, 3, 10, 50, 100 or 500, then my *Report history* badge moves to Wood, Bronze, Silver, Gold, Platinum or Diamond.
- Given one of my confirmed reports is unconfirmed, then my count goes down and the tier is recalculated, so the badge can go down a tier.

US-55 · As a **participant**, I want an *Event reporter* badge for each event, so that my contribution to that event is recognised.
Persona: P1 · *Depends on:* US-54

- Given my confirmed reports in event X reach a tier threshold, then my *Event reporter* badge for event X moves to that tier.
- Given one of my confirmed reports in event X is unconfirmed, then that badge is recalculated.

US-56 · As a **participant**, I want to hide my badges or stop earning them, so that taking part stays my choice (§6.4).
Persona: P1 · *Depends on:* US-54

- Given I hide my badges in my account, then no one else can see them.
- Given I turn badges off, then no new badges are earned until I turn them on again.

F15 · Badges for event attendance (Should)

US-57 · As a **participant**, I want an *Event attendance* badge for the events I attend, so that coming back is recognised.

Persona: P1 · *Depends on:* US-54, US-58

- Given I checked in to an event (US-58), then the event counts as attended, once per event.
- Given my attended-event count reaches a tier threshold, then my *Event attendance* badge moves to that tier.

F16 · Event check-in (Should)

US-58 · As a **participant**, I want to check in at the venue by scanning the event's QR code, so that my attendance is recorded for my badges.
Persona: P1 · *Depends on:* US-12

- Given I am logged in and the event is running, when I scan the venue QR code, then I see "Checked in" and the event counts as attended.
- Given I already checked in to this event, when I scan again, then nothing changes (one check-in per event).
- Given the event has not started or has ended, when I scan the QR code, then the event opens but no check-in is recorded.

F17 · Entrance status (Should)

US-59 · As a **team member** with `Entrances.Update`, I want to set the status of each entrance, so that participants choose a better gate.
Persona: P2, P5 · *Depends on:* US-20

- Given I set Gate 2 to "long queue", then the status is saved with the time of the update.

US-60 · As a **participant**, I want to see the status of each entrance, so that I choose a gate with a shorter queue.
Persona: P1 · *Depends on:* US-21, US-59

- Given the team marked Gate 2 as "long queue", then I see it on the map with the time of the update.

F18 · Friends (Could)

US-61 · As a **participant**, I want to send and accept friend requests, so that I can find people I know who use Farol.
Persona: P1 · *Depends on:* US-03

- Given I send a friend request and it is accepted, then each of us appears in the other's friend list.
- Given a friend request is declined, then neither of us appears in the other's list.

US-62 · As a **participant**, I want to remove a friend, so that I control who is in my list.
Persona: P1 · *Depends on:* US-61

- Given I remove a friend, then we no longer appear in each other's list.

8. Sprint goals and initial release plan [PROPOSED]

Assignment §7.1.2, item 5, and §7.2–§7.3. Proposed on 16 October 2026 and updated on 22 October 2026 with the smaller stories, to confirm with the team at the next planning meeting. Order follows the MoSCoW priority in §2 and the dependencies in §7; the rank is not the build order (§8.5). The reasons are in §8.5.

8.1 Sprint goals

Sprint	Dates	Goal
B · Build and operate the vertical slice	2 Nov – 6 Dec 2026	**A logged-in participant joins a published event and asks for help on its venue map; the control room sees the request at once with an AI suggestion, or without one when the AI is down, and routes it to a team whose member accepts and resolves it.** The event's teams and permissions are set by its EventAdmin. Built and tested by the pipeline on every merge and run locally with Docker Compose (ADR-011), with logs and metrics, and bus data read from the external source. Answers the Sprint B investor question: *does the product deliver observable user value, and can the team deploy, monitor and operate it?*
C · Evaluate, harden and present	7 Dec 2026 – 3 Jan 2027	**The MVP is complete: accounts, transport with event shuttles and the D2 warning, live announcements, community reports reviewed by the right team, and badges.** The AI triage is evaluated against the non-AI baseline, and both degraded modes (D1, D2) are demoed. Answers: *is this a credible product and a maintainable system?*

8.2 Initial release plan

Every merge to `main` is built and tested (guide H4). Delivery is local: a *release* is a git tag (guide H7), built and run on a laptop for the review (`local-delivery.md`, guide H5).

Release	When	What it contains	Assignment
R0 · Walking skeleton	Sprint A, before Sprint B starts on 2 Nov 2026 (#58)	One Reports service with synthetic help-request create/read in memory, diagnostic health, HTTP tests, Dockerfile, Compose and CI smoke on every PR (#166; run instructions). Keycloak, persistence/outbox and the full application are Sprint B. Not a user story: a task issue	§7.1.2 item 10 (Sprint A)
R1 · Vertical slice	Sprint B review (6 Dec 2026)	27 stories: US-01, 02, 06, 11, 12, 14, 16, 17, 18, 19, 20, 21, 23, 24, 26 to 36, 40, 41. The help-request journey end to end (W2 + W3), with teams and permissions, AI triage, scenario D1 and the first external integration	§7.2.2 items 1–13
R2 · MVP	Sprint C review (3 Jan 2027)	Every remaining Must (28 stories): US-03, 04, 05, 07, 08, 09, 10, 13, 15, 22, 25, 37, 38, 39, 42, 43, 44, 45, 47 to 56. Should US-58, US-57, US-59 and US-60 if there is time; then Could US-46, US-61 and US-62 (milestone *Sprint C* since 1 November 2026, #175)	§7.3.2 items 1–2, 8
Later (product roadmap)	After the course	Whatever Sprint C cuts. Won't: F19, F20	§7.3.2 item 13

Cut line for Sprint C. If Sprint C runs short, drop in this order: US-46, US-61 and US-62; then US-57 and US-58; then US-59 and US-60; then US-52 and US-53; then US-22; then US-08, US-09, US-10 and US-45 (reasons in

§8.5).

8.3 Stories on the board

Each story is a GitHub issue created from the user-story form, titled `[<Type>] US-XX: <Title>` (README §1). Type rule in §7.1. Created on 22 October 2026 with the label `user story` (the walking skeleton: `task`) and the milestone of their sprint (`Sprint A`, `Sprint B`, `Sprint C`; Could stories have none). Each issue's *Dependencies* links the issues it needs. The old feature issues #2 to #16 (Sprint A feature discovery) were archived from the board on 24 October 2026 (#138) and closed as *not planned* on 31 October 2026: each one is covered by the stories below or is out of scope (#6, #8, #10, #14). #11 (Problem and Opportunity Report) was closed as delivered on 24 October 2026.

Board: GitHub Projects, *LABDSOF* (project 1 of `tomasoliveira21`; private, open to the team). Every issue in the table below is on it, with the other open issues of the repository (auto-add workflow). How its fields are used, decided and applied on 24 October 2026 (#138); the board is the source of truth for the values (they are not repeated in this table):

Field	Values	Rule
Status	Backlog → Ready → In progress → In review → Done	*Ready* only when the story meets the DoR and none of its dependencies blocks it (§8.4); a dependency on an unblocked story that is not built yet only orders the work. *Done* follows the DoD; the board workflows move closed issues and merged PRs to *Done*
Priority	P0, P1, P2	Must in the release of the next sprint → **P0** (now R1, Sprint B; the walking skeleton #58, R0, is also P0); every other Must → **P1**; Should and Could → **P2**. Re-applied at the start of each sprint
Size	XS, S, M, L, XL	From the number of acceptance criteria: 1 → XS, 2 → S, 3 → M, 4 → L, 5 or more → XL. One step up when the story depends on an open or external decision or system (E1, role model, D3, F3, K1, ADR-008, J1, §6.1, D7). One step up for US-21, US-23 and US-26 (first use of the map and of polling, §8.5) and US-46 (offline cache). An XL story is split before it is *Ready*
Sprint	Milestones *Sprint A*, *Sprint B*, *Sprint C*	The issue's milestone (this table's *Sprint* column); board views are grouped or filtered by *Milestone*. No *Iteration* field, so the sprint is kept in one place (decided on 24 October 2026 instead of the Iteration field of `definition-of-ready-and-done.md`)
Estimate, *Start date*, *Target date*	—	Not used for now

Result of the rules on 24 October 2026: **P0** 28, **P1** 28, **P2** 7; **XS** 16, **S** 22, **M** 16, **L** 9, **XL** 0. On 30 October 2026, US-23 (#71) went from L to XL (5 criteria after the survey decision on location), so it is split before it is *Ready*, and US-59 and US-60 (#118, #119) got the *Sprint C* milestone: **L** 8, **XL** 1. Only #58 was *Ready* on 24 October. On 31 October 2026, with R0 merged (#167) and #58 closed, the stories with no open blocker of §8.4, directly or through *Depends on*, are US-01, US-02 and US-40 (Sprint B) and US-03, US-04, US-43, US-61 and US-62; every other story still waits for the role model, the venue map library (F3), the priority scale (AI-3) or the report types (§6.1). The board holds the current *Status* values.

Story	Issue	Type	Sprint
Walking skeleton (task)	#58	Technicians	A
US-01 · Log in	#59	Technicians	B
US-02 · Log out	#60	Technicians	B
US-03 · Sign up	#86	Front Office	C
US-04 · Edit name and password	#87	Front Office	C
US-05 · Delete my account	#88	Front Office	C
US-06 · Create an event and name its first EventAdmin	#61	Back Office	B
US-07 · List events with basic details only	#89	Back Office	C
US-08 · Name a new EventAdmin	#90	Back Office	C
US-09 · Suspend and reactivate an event	#91	Back Office	C
US-10 · Edit event details	#92	Back Office	C
US-11 · Publish an event	#62	Back Office	B
US-12 · Join an event by QR code or link	#63	Front Office	B
US-13 · Close an event	#93	Back Office	C
US-14 · Keep each event's data private	#64	Technicians	B
US-15 · Make or remove an EventAdmin	#94	Back Office	C
US-16 · Create, rename and delete teams	#65	Back Office	B
US-17 · Set a team's permissions and help categories	#66	Back Office	B
US-18 · Add and remove team members	#67	Back Office	B
US-19 · Upload the venue map	#68	Back Office	B
US-20 · Place points of interest	#69	Back Office	B
US-21 · Open the venue map	#70	Front Office	B
US-22 · Filter the map by type of place	#95	Front Office	C
US-23 · Ask for help on the map	#71	Front Office	B
US-24 · Follow my help request	#72	Front Office	B
US-25 · Help request not sent	#97	Front Office	C
US-26 · Live list of help requests	#73	Back Office	B
US-27 · Help requests on the map	#74	Back Office	B
US-28 · Set priority and status, with history	#75	Back Office	B
US-29 · Route a request to a team	#76	Back Office	B
US-30 · Close a request	#77	Back Office	B

Story	Issue	Type	Sprint
US-31 · Field team sees routed requests	#78	Back Office	B
US-32 · Accept a request	#79	Back Office	B
US-33 · Mark a request resolved	#80	Back Office	B
US-34 · AI triage suggestion	#81	Back Office	B
US-35 · Accept or change the AI suggestion	#82	Back Office	B
US-36 · Requests without AI (scenario D1)	#83	Back Office	B
US-37 · Add shuttles and temporary stops	#98	Back Office	C
US-38 · Mark a shuttle delayed or cancelled	#99	Back Office	C
US-39 · See the event's shuttles	#100	Front Office	C
US-40 · Import the external transport source	#84	Technicians	B
US-41 · See the next buses home	#85	Front Office	B
US-42 · Out-of-date transport warning (D2)	#101	Front Office	C
US-43 · Transport source down warning (D2)	#102	Back Office	C
US-44 · Publish an announcement	#103	Back Office	C
US-45 · Edit or remove an announcement	#104	Back Office	C
US-46 · Weak-network mode (Could)	#117	Front Office	—
US-47 · Report a problem on the map	#105	Front Office	C
US-48 · Report rate limit	#106	Front Office	C
US-49 · See confirmed reports on the map	#107	Front Office	C
US-50 · Confirm or reject a report	#108	Back Office	C
US-51 · Unconfirm a report	#109	Back Office	C
US-52 · Send a report to be checked on site	#110	Back Office	C
US-53 · Fix a report and mark it resolved	#111	Back Office	C
US-54 · *Report history* badge	#112	Front Office	C
US-55 · *Event reporter* badge	#113	Front Office	C
US-56 · Hide or stop earning badges	#114	Front Office	C
US-57 · *Event attendance* badge (Should)	#115	Front Office	C
US-58 · Check in at the venue (Should)	#116	Front Office	C
US-59 · Set entrance status (Should)	#118	Back Office	C
US-60 · See entrance status (Should)	#119	Front Office	C
US-61 · Send and accept friend requests (Could)	#120	Front Office	—

Story	Issue	Type	Sprint
US-62 · Remove a friend (Could)	#121	Front Office	—

8.4 What still blocks *Ready*

These dependencies were open on 22 October 2026; the struck-through ones were closed by 31 October 2026, with the source. A story that needs one stays in *Backlog*, not *Ready*, until it is closed. All are expected before Sprint B starts on 2 November.

Open dependency	Blocks
~~Walking skeleton (R0, #58), in Sprint A; needs the deployment platform (guide G1, #53)~~ Closed on 31 October 2026: local delivery (guide G1, ADR-011); R0 merged in #167 and #58 closed	~~Every story (first to be built)~~
~~Identity provider (guide E1)~~ Closed on 22 October 2026: Keycloak (ADR-006)	~~US-01, US-03~~
Role, team and permission model (§4): team confirmation, then the domain model. ~~Where permissions are checked (guide E2)~~ closed on 22 October 2026: in each service's use cases, against local copies (ADR-006). Replaces personas PE-5 and PE-6	US-06, US-14, US-15, US-16
Venue map library (guide F3, ADR-007, #44)	US-19
~~AI provider and model (guide K1); AI-1, AI-2~~ closed: Ollama with `qwen3:4b` (ADR-005, accepted on 31 October 2026). Still open: AI-3, the priority scale (API outline O7)	US-34
~~Resilience values (guide J1, #43)~~ Closed on 25 October 2026 (guide J1, #43, PR #52; `architecture.md` §9.1)	~~US-42, US-43~~
Report types and expiry times (§6.1): team decision	US-47
~~Account deletion (guide D7)~~ Closed on 27 October 2026 (guide D7, #135; ADR-006)	~~US-05~~

Stories that depend on a blocked story wait for it too (see each story's *Depends on*).

8.5 Why this plan

Why this Sprint B goal

Assignment §7.2.1 asks Sprint B for "*an operational end-to-end version of the product's most important user journey*". That journey is the help request:

- the report's scope mitigation puts help requests first ("*prioritise help requests, the map and transport information in the MVP*", report §5), and F6–F8 are the first features after the foundations (rank 6–8, §2);
- W2 + W3 is journey 1 in `user-workflows.md`, and it is already drawn end to end in BPMN;
- it holds the riskiest parts of the product: the AI workflow (§5.6), the asynchronous workflow (§6.3), isolation between events (report §5) and, since 22 October, permissions set per team. Building them in Sprint B leaves Sprint C for evaluating them.

The goal also covers every Sprint B deliverable of §7.2.2:

§7.2.2 deliverable	Covered by
1 · Functional client application	US-12, US-21, US-23 (phone pages); US-06, US-11, US-16 to US-18 (laptop pages)
2 · Authentication and authorisation	US-01, US-02, US-14, US-16 to US-18
3 · Persistent data storage	Every story (PostgreSQL, one database per service, guide D1–D2)
4 · Documented API contracts	OpenAPI written first for each story (contract-first, guide C7)
5 · At least two backend components	Auth & Users, Events, Reports and Transports (guide A2)
6 · One asynchronous workflow	Help request shown in the backoffice at once; AI triage adds a suggestion in the background (US-23, US-34; §3, row §6.3)
7 · Initial external-system integration	US-40, US-41
8 · First version of the AI capability	US-34, US-35
9 · Non-AI baseline or fallback	US-36
10 · Automated tests	DoD, Level 2, on every story
11 · CI/CD pipeline	R0, walking skeleton
12 · Structured logs, health checks, initial metrics	R0 (health check); DoD Level 2 (logs with correlation ID); US-36 (fallback-rate metric)
13 · Demonstration of a failure condition	Scenario D1 (US-36)
14 · Updated backlog and architecture documentation	This section, updated at each planning

Why these stories in Sprint B, and whether it is realistic

Every Sprint B story is a step of the journey or something it needs first:

- **Account and event:** US-01 and US-02 → US-06 → US-11 → US-12.
- **Teams and permissions:** US-16 → US-17 → US-18. Without them nobody has `HelpRequests.Triage` or `HelpRequests.Respond`, so no one could see or act on a request.
- **Map:** US-19 → US-20 → US-21, where the participant marks their location.
- **Help request:** US-23, US-24 → control room US-26 to US-30 → field team US-31 to US-33.
- **AI and its fallback:** US-34 to US-36.
- **External integration:** US-40, US-41 (deliverable 7).
- **Isolation:** US-14, a top technical risk and SC8.

Nothing in Sprint B is there for convenience, so Sprint B has no cut line.

Capacity: 27 stories in 5 weeks (the walking skeleton is done in Sprint A), for 4 people: about 7 stories per person, under 1.5 per person per week. After the split each story is one action and one PR of one to three days. The largest are US-17 (permission checks), US-21 and US-23 (first use of the map), US-26 (first use of polling) and US-34 (AI).

Risks and what happens if they occur:

Risk	Plan
A blocker in §8.4 is not closed by 2 November	The stories that need it stay in *Backlog*; the team starts with the Sprint B stories that had no open blocker on 31 October 2026 (US-01, US-02, US-40; §8.3)
The team does not confirm the role model (§4)	US-16 to US-18 are replaced by teams created by a seed script with the pilot set-up (§4.4); the other stories do not change, because they check permissions, not team names

Why US-40 and US-41 are in Sprint B, and US-42 and US-43 in Sprint C

Sprint B must already have an *initial external-system integration* (§7.2.2, item 7), so the import (US-40) and its screen (US-41) are in Sprint B. Sprint B needs only **one** failure condition (§7.2.2, item 13), and D1 (US-36) covers it. The *resilience and degraded-mode demonstration* is a Sprint C deliverable (§7.3.2, item 8), where D2 (US-42, US-43) is the second scenario. US-42 also needs the event shuttles (US-39), so that they stay on screen when the source is down.

Why sign-up (US-03 to US-05) is in Sprint C

- Mocked authentication is allowed for the MVP (assignment §6.6, footnote 1), so seeded test accounts are enough for the help-request journey; log in and log out (US-01, US-02) are in Sprint B.
- Sign-up is not a step of that journey.
- The terms and consent text follow the Security & Privacy assessment §8 (guide E5; written on 25 October 2026, to review with the team).
- US-03 to US-05 all handle personal data, so they are built and tested together.

Why the FarolAdmin's other stories (US-07 to US-09) are in Sprint C

Only creating the event (US-06) is on the journey. Listing events, naming a new EventAdmin and suspending an event are the FarolAdmin's day-to-day and exception work. The FarolAdmin is an internal user (P4), who can do them by hand while they are not built.

Why this cut order in Sprint C

Sprint C has 28 Must stories in four weeks, including Christmas, and it also holds the evaluation, accessibility, security, performance and presentation deliverables (§7.3.2). If the stories do not fit, they are dropped in this order:

1. **US-57 and US-58**: Should (MoSCoW). The engagement mechanism (§5.5) is already complete with the two Must badge tracks (US-54, US-55).
2. **US-52 and US-53**: reports can still be confirmed and rejected from the control room (US-50).
3. **US-22**: a convenience. The map already shows every point with its icon (US-21).
4. **US-08, US-09, US-10 and US-45**: the FarolAdmin can do US-08 and US-09 by hand; the name and dates of an event are set when it is created (US-06); a wrong announcement can be corrected by publishing a new one (US-44).

Never cut:

- **US-03, US-04 and US-05**: personal data (consent and the right to erasure), checked in the security review (§7.3.2, item 6).
- **US-25**: a participant must never think a request was sent when it was not.
- **US-37 to US-39, US-42 and US-43**: scenario D2 (§7.3.2, item 8).
- **US-44**: the answer to root cause 2, static communication (§1).
- **US-47 to US-51 and US-54 to US-56**: the engagement mechanism, which the assignment requires (§5.5), with its opt-out (§6.4).

Why the walking skeleton comes first

It is a Sprint A deliverable (§7.1.2, item 10), so it stays in Sprint A (#58) and is done before Sprint B starts. DoD Level 2 asks for a green pipeline on `main`, with the image build and the smoke test, so no story can be *Done* before the skeleton exists. One week is enough for one endpoint, one test, a Dockerfile and the pipeline, because the CI/CD choices are decided (guide H1–H5) and delivery is local (guide G1, ADR-011). R0 was merged in #167 on 31 October 2026. **Source:** professor meeting of 22 October 2026.

Responsible AI Opportunity Assessment and provider investigation

> **Research snapshot of 22 October 2026 (#53)**, kept as evidence and as the record of the alternatives; it is not rewritten. **Decisions in force:** ADR-005, the local AI decisions, NFR-11 and NFR-20. Where this document differs, they win: for example, local Ollama with `qwen3:4b`, one 20 s attempt and no hosted fallback.

22 October 2026 · #53 · K1–K2 / AI-1–AI-11 · PROPOSED. Research artefacts for the academic MVP; no clinical validation, production deployment or measured model performance.

Current decision update (26 October 2026): Local-only delivery is authorised because of cloud costs. Local Ollama `qwen3:4b` Q4_K_M is selected for team review. The complete decision fixes runtime/cost, dynamic event-team contract, safeguards, dataset review and evaluation procedure. The historical cloud/API assessment below is retained as an alternative; it is not the current default. No inference test is claimed.

Compatibility after merging the 22 October role model: the fixed Medical/Security/PCR names in this evaluation are a frozen synthetic event fixture, not a global production team list. Current `features.md` §4 allows event-defined teams with permissions and handled categories. Implementation must supply/validate the current event's eligible teams, abstain when none is eligible, and evaluate another event with different names/permissions. The existing draft labels/baseline remain research evidence and do not prove that cross-event team selection.

Opportunity, boundaries and human control

F8 assists the incident operator with category, operational priority and response-team suggestions for help requests. The expected value is more consistent routing in informal Portuguese, not automated diagnosis, emergency advice or autonomous dispatch. Existing F7 works immediately without AI: participant category, order of arrival, **Not triaged**. The operator approves/changes every suggestion. The system must never overwrite an operator decision or lower an existing priority (domain invariant I4).

A successful suggestion can help the operator, but a confident wrong answer can delay assistance. Keep the original request visible, mark the result **AI suggestion**, show a short reason and allow correction. Do not show numerical confidence unless it has been calibrated on separate evidence. No treatment advice, dispatch tool, internet access or privileged database access is granted to the model.

K1: provider and Ollama feasibility

Candidate	Technical/economic assessment	Limits and go/no-go evidence
Hosted Mistral Small 4, dated ID `mistral-small-2603`	First candidate for the low-volume **synthetic** cloud demo: no dedicated inference VM; supports structured output	Must meet the same quality/latency tests as local inference; account availability, EU processing and retention/training controls need verification
Ollama + `qwen3:4b` Q4_K_M	Feasible to run locally or on a dedicated cloud CPU/GPU host; good comparison candidate without API-token charges	Published model file is about 2.5 GB, not total RAM; prototype with 8 GiB host RAM, 4 CPU cores, context 4096 and concurrency 1, then measure. No claim this meets 30 s
Ollama on cloud GPU	Technically possible with compatible GPU/runtime; isolates inference from the five services	Quota and hourly GPU/model-storage/network cost can exceed student credits; do not provision for a low-volume demo without evidence that CPU/hosted options fail
Azure-hosted inference	Keeps procurement near the proposed Azure platform	Subscription model availability, quota, region and pricing are unverified; not assumed included in student credits

Sources: Mistral's model card and pricing; Ollama's model entry, Docker support and runtime FAQ. Ollama is a serving runtime, not a model. The proposed 8 GiB sizing is an experiment budget, not an official minimum. KV cache/context, concurrency, quantisation and runtime overhead change memory demand. Keep Ollama off the database/broker host to avoid resource contention and use a persistent model cache. Never expose its inference port publicly.

Local Ollama supports JSON-schema output; that page says Ollama Cloud currently does not. Mistral provides custom structured output. Schema enforcement does not establish semantic correctness. Freeze model version/digest, prompt/schema/dataset hashes and settings for any comparison; mutable aliases alone are not reproducibility evidence.

Cost example, not measured usage: the listed Mistral Small 4 prices on the research date are $0.15 per million input tokens and $0.60 per million output tokens (standard pricing). At an assumed 800 input + 120 output tokens, one call costs $0.000192; 10,000 calls cost $1.92 before retries, regional/tier premiums, taxes and changes in token counts. A 100-case comparison repeated three times is $0.0576 under those assumptions. Count every billed token, including reasoning if applicable; set a proposed $5 demo investigation cap and an application-side stop/disable switch. This does not authorise a purchase or API execution.

Current recommendation: local Ollama for the cost-constrained academic delivery, with no paid fallback and synthetic inputs. Measure quality, full-stack memory and timing before declaring it release-ready. The hosted preference in the candidate table belongs to the original cloud scenario; it does not override the current local selection and acceptance gates.

Input, output and proposed priority rubric

Send only participant `category` and optional sanitised text (max 500 characters), current priority floor and trusted event-specific eligible team IDs/categories. The fixed names below describe only the frozen evaluation

fixture. Do not send account/event IDs, precise coordinates, names, phone numbers, credentials, tickets or the complete request record. A coarse zone is not needed for categorisation and is excluded initially. An opaque local correlation ID is kept outside the provider payload. Free text can still contain sensitive information after redaction: minimisation is not anonymisation.

System prompt separates trusted rules from a JSON user-data message. No tools or retrieval. Output schema fixes five categories, four priorities, three help-response teams, short reason and abstention. Unknown keys, invalid enums, excessive strings, timeout, refusal or semantic violations cause fallback. In this frozen pilot fixture, `Logistics` is not a help-request destination; current production eligibility must follow event-defined permissions/categories, not the name. Fixture default routing: Health→Medical, Safety/Theft→Security, LostPerson/Other→PCR; **Other→PCR needs team validation**. A missing or unauthorised team in an event forces manual routing. The fixed-name fixture schema must be adapted to that event-derived allowlist before production integration.

Priority	Proposed operational meaning; synthetic examples
Critical	Immediate serious danger described: person not responding, active severe assault, crushing or fire threatening people. Operational review flag, not diagnosis
High	Ongoing credible threat or someone needing prompt staff assistance: harassment, escalating crowd pressure, vulnerable person lost
Medium	Assistance needed without an explicit immediate danger; health complaint requiring staff review, theft already occurred
Low	Unclear/non-urgent support or administrative request; missing context causes abstention/manual review

Proposed deterministic baseline floors: Health=Medium, Safety=High, LostPerson=Medium, Theft=Medium, Other=Low. Effective stored priority is `max(existing priority, baseline floor, accepted suggestion)`; compare the **raw suggestion before clamping** in the evaluator so guards cannot hide model under-prioritisation. These operational labels require review by the team and a suitably experienced responder; no dataset label is medical advice.

Save the request and its outbox event before inference; show it to the operator immediately. The 30 s deadline includes queue time, inference, validation and storage. For the selected local profile use connect 2 s, response 20 s, one attempt, no retry and the remaining overall deadline; the full admission/circuit policy is specified for review in the local decision. Ignore late suggestions if an operator acted; concurrent updates must enforce that atomically. Never wait for AI before acknowledging a durable request.

K2: Responsible AI risks and controls

This assessment uses the risk-identification/measurement/management approach of the NIST AI RMF, applied to the repository's operator-assistance workflow.

Risk	Proposed control	Evidence before use / residual risk
Wrong category/team, missed urgent case	Operator approval, conservative priority floor, bounded teams/schema, raw-output quality gate	Zero urgent under-prioritisations on the held-out set; cannot prove real-world safety from 100 synthetic cases

Risk	Proposed control	Evidence before use / residual risk
Prompt injection	Untrusted JSON input, no tools, fixed schema, allowlisted labels, adversarial examples	Injection tests; semantic manipulation remains possible despite valid JSON
Sensitive text disclosure	Field minimisation/redaction, no prompt logs, provider gate; local inference alternative	Security review of data flow, lawful basis, access, processors and assessment need before any real data
Bias in language/style	PT-PT slang/typos, English/mixed-language cases and paired paraphrases	Report per-slice failures/counts; small synthetic slices cannot establish fairness
Harmful or diagnostic output	Category/priority/team only, short operator-facing reason; escape rendered text, no advice	Human review of reasons, not just JSON validation; reason may repeat sensitive text
Automation bias	Explicit AI label, original input visible, easy correction; no fabricated confidence	Review accepted/changed suggestions and operator feedback
Outage, slow model, quota/cost exhaustion	Immediate F7 visibility, 30 s end-to-end bound, fallback, monitoring and disable switch	AI-off/slow/malformed tests (SC5); runtime evidence still pending
Reproducibility/leakage	Versioned artefacts and documented label review for the academic demo; fresh blinded holdout as a future improvement	Record hashes and split use; public-in-repo test labels are not a blinded benchmark
Drift or provider model replacement	Dated model ID where possible, canary re-evaluation on model/prompt changes	Re-run full quality and latency tests; provider availability may change

Mistral documents default EU hosting with possible feature-dependent transfers. Its privacy policy describes ordinary abuse-monitoring retention, and ZDR documentation describes eligibility/request requirements for supported stateless endpoints. EU hosting, training opt-out and zero retention are different controls. Do not assume ZDR is automatic or that a European vendor makes every use lawful. The security assessment must check the actual paid plan, training setting, DPA/subprocessors and endpoint before real data; until then use synthetic inputs only.

Evaluation plan and current evidence

The evaluation package contains 100 explicitly synthetic authored cases, 20 development / 80 test, including empty text, ambiguous requests, slang/typos, English/mixed text, multiple problems and prompt injection. The labels are draft operational judgements created for this investigation, **not independently validated expert ground truth**. For the minimum demo, a second teammate reviews/adjudicates the proposed labels and records that review. Independent blinded labels/fresh holdout are optional future improvements; the existing test split remains visible and cannot be presented as blinded.

Compare the participant-category/default-team baseline with raw model outputs. Proposed gates: zero urgent under-prioritisation or urgent abstention/timeout; category/team accuracy ≥80% on non-abstention reference cases; valid on-time suggestions for ≥95% of those cases; correct abstention on every reference abstention case; end-to-end SC3 ≤30 s for accepted suggestions; SC5 100% visibility with AI disabled. Report all cases and slice counts, errors, coverage, latency and cost; do not score missing/invalid predictions as successful fallback.

Coverage alignment (26 October 2026, #145): ADR-005 defines the common ≥95% within 30 s proposal and its denominators. Offline expected abstentions are evaluated separately; runtime failures/abstentions remain in the one-hour operational denominator. This changes the specification, not measured model results.

For the minimum local demo, run the selected Ollama model once on the frozen test split, review reasons and record actual latency/memory. Show one success and one AI-off fallback. Repeated candidate comparisons and concurrency studies are optional; the current decision supersedes the broader research experiment. This research supplies an offline scorer and baseline, not deployed latency instrumentation or inference results. No model/provider was called, so model accuracy, tokens, cloud capacity, safety and SC3/SC5 are **not measured** here.

The original features AI-1–AI-11 questions are answered as **proposals** by the input/schema/rubric/provider/prompt/dataset/gates/UI/monitoring/risk sections above. Team acceptance and implementation follow separately.

Which gate decides what (added 31 October 2026, #162): an offline run decides only the **quality gate** (accuracy, urgent cases, abstentions, floor and category/team checks). A direct model-call benchmark omits `elapsed_ms` (evaluation README, step 4), so the scorer reports no on-time coverage and its timing gate cannot pass; on-time coverage (≥ 95 % within 30 s) is measured on the instrumented one-hour cohort of the local decision, not offline.

Baseline, known limitations and how users are informed (added 31 October 2026, #162)

This section is added to the snapshot so that deliverable 6 states its numbers and its limits in one place (assignment §5.6.1, §5.6.2, §5.6.3). It adds no new decision.

Non-AI baseline, reproduced. The baseline report applies the no-AI policy (participant category, default team, category priority floor) to the 80 test cases (72 non-abstention references):

Measure	Baseline	Gate (ADR-005 decision 4; evaluation README)
Category accuracy	0.708 (51 of 72)	≥ 0.80
Team accuracy	0.806 (58 of 72)	≥ 0.80
Urgent cases missed (under-prioritised, abstained or missing)	24 of 31	0
Correct expected abstentions	0 of 8	all

Of the gates in this table, the baseline passes only team accuracy; it meets suggestion coverage, the priority floor and consistency trivially, because it always suggests. **Where the AI must beat it** is the existing gate on urgent cases: zero misses against the baseline's 24 of 31, plus category accuracy ≥ 0.80 and every expected abstention. No new gate is added here. The value the AI is expected to bring is therefore earlier attention to urgent requests whose participant category understates them; it is not measured until the model runs (guide K2).

Known limitations:

- **Synthetic, team-authored labels.** The 100 cases and their labels were written by the team; they are not independent expert ground truth, and the test split is visible in the repository, so it is not a blinded benchmark.
- **Small slices.** The English, mixed-language, slang, injection, abstention and ambiguous slices have 3 to 8 test cases each, and multiple problems only one, too few to establish fairness or robustness.
- **No calibrated confidence.** The model gives no score that has been checked against outcomes, so none is shown.
- **One issue per suggestion.** The prompt asks for the most urgent described issue only (system prompt); a request describing several problems gets one suggestion.
- **Latency not measured.** Time on the demo laptop's CPU is unknown until the model runs (ADR-005).
- **Fixed team names.** The evaluation uses one frozen event's teams; other events' teams are an integration check.
- **Provenance of retrieved information:** not applicable. The model has no tools and no retrieval; it sees only the minimised fields listed above.

How users are informed. The operator always sees the original request next to the result, which is labelled **AI suggestion** with a short reason, never as a verified fact, and can change it; no confidence number is shown (§ *Opportunity, boundaries and human control* above). When the AI is unavailable, requests show "*Not triaged*" and a banner says so (`features.md` §5). Participants are told in the sign-up terms that an AI may suggest a priority and that a person decides (security and privacy assessment §8).

Local AI: provider, Responsible AI and evaluation decision

Issue: #153, continuation of #149 / merged PR #152 · **Tracker:** #147 · **Updated:** 26 October 2026

Architecture accepted; execution/evaluation evidence pending. Status reconciled on 31 October (#166) against guide/NFR decisions recorded on 26–27 October. Select local Ollama with `qwen3:4b` Q4_K_M for the academic delivery. This is an architectural selection, not a measured quality claim. The professor's cost clarification authorises local delivery; it does not certify this model. Application implementation stays with #81, #82 and #83.

K1: selected platform and cost

Run Ollama in the optional Compose `ai` profile on the demonstration machine, on a private network at `http://ai:11434` (the Compose service `ai`, reached by Reports through `wiremock`, `local-delivery.md` §1), with a persistent model volume and no published host port. Reports owns the asynchronous triage worker; the browser never calls Ollama. Use synthetic inputs until the privacy review. Disable cloud features with `OLLAMA_NO_CLOUD=1`; do not configure API credentials or a hosted fallback. Download the model before the demo and test offline operation. No inference VM or API-token subscription is needed; existing hardware, electricity, storage and initial download still have costs.

Use one Ollama runtime, one model and one request at a time. No hosted-provider comparison, GPU server or machine purchase is required for this local demo. Model download size is not runtime RAM: load the model before presentation and measure usage on the actual laptop with the application. The observed machine has 16 GiB physical RAM and about 4 GiB allocated to Docker; no full-stack capacity result is claimed. Run the optional AI and observability scenarios sequentially if needed. R0 can run with AI off, but that alone does not demonstrate working inference.

Hosted Mistral and cloud CPU/GPU remain the dated alternatives in the assessment. They add account, network, privacy and variable-cost dependencies and are excluded from the current default. A later change needs new cost/privacy approval and an ADR update. Failure of the local quality gates means retaining manual operation and reporting a failed candidate, not silently selecting a paid service or relaxing a target.

Runtime and resilience contract

Setting	Selected local profile
API	`POST /api/chat`, `stream: false`, schema supplied through `format`, `think: false`; no tools or retrieval
Generation	Temperature 0, seed 42, context 4096, `num_predict: 256`, `keep_alive: "10m"`; freeze these in each run manifest
Capacity	`OLLAMA_NUM_PARALLEL=1`, `OLLAMA_MAX_LOADED_MODELS=1`, `OLLAMA_MAX_QUEUE=1`; a bounded application admission queue, with expired work discarded before inference

Setting	Selected local profile
Deadline	30 s from durable submission to stored valid suggestion, including outbox/queue, HTTP, validation and storage; never reset this clock on retry or redelivery
Admission	At most 5 s waiting before starting inference; start only with at least 23 s of the original budget remaining. Otherwise mark unavailable and leave manual triage visible
HTTP	Connect timeout 2 s; response timeout 20 s; one attempt, no retry. Total HTTP call also capped at 22 s and by the remaining overall deadline
Circuit	Last 10 calls, minimum 4; open at >=50% failures or slow calls (>16 s), remain open 30 s, allow one probe; queue-expired requests remain coverage misses
Completion	Recheck deadline, event eligibility and operator version atomically before saving; discard stale/late results. Fallback never overwrites an operator decision

This selects J1's already identified local 20 s / single-attempt alternative for review. The hosted 10 s / two-attempt profile remains a researched alternative, not the local default. The 16 s slow threshold is 80% of the local response timeout; actual timing and circuit configuration must be verified in implementation. No retry storm or blocking AI call is allowed on request submission. A separate warm-up uses synthetic text before measurement; report cold start separately.

Sources checked for this recommendation: Ollama chat API, structured outputs, runtime configuration and model entry. Pin the actual full model digest and Ollama image digest when downloading; a mutable tag or a short web digest is insufficient. No model has been downloaded or run by this documentation change.

Event-specific production contract

The existing prompt/schema/dataset are a frozen single-event research fixture. Preserve their hashes/baseline; Medical/Security/PCR are **not** global product roles. The application contract is:

1. Reports obtains a trusted snapshot of this event's teams, handled categories and `HelpRequests.Respond` permissions from its local projection of domain events. Do not perform cross-service REST calls or trust team names in participant text. If required configuration is missing/stale, abstain and retain manual routing.
2. Supply only participant category, redacted optional text (maximum 500 characters), current priority floor, and opaque eligible team IDs with handled categories. IDs identify teams only within the supplied allowlist; no event/account/request ID, coordinates, names or contact details enter the prompt. Keep the correlation ID outside it.
3. Return exactly `category`, `priority`, `teamId`, `reason` and `abstain`, with no extra fields. Categories/priorities follow the fixture enums. `reason` has 1–240 characters. Generate `teamId`'s allowlist from the trusted snapshot, permitting null for abstention; if no team is eligible, bypass inference and use manual triage.
4. Independently validate that a non-abstaining team belongs to this event, can respond to the suggested category, and still has permission at persistence time. Reject unknown, cross-event, revoked, malformed or category-incompatible choices. Do not repair invalid output into a scored success.
5. If several teams qualify, use an explicitly configured category default when available; otherwise abstain for operator choice. An abstention has `teamId: null`. The fixture's old `team` output stays only in the frozen

evaluator; an explicit adapter/new version is required for integration, never silently reinterpret its scores.

6. Retain the original text/category and raw proposal, label it **AI suggestion**, and require operator approval before dispatch. Store effective priority as the maximum of current priority, baseline floor and accepted suggestion; score raw predictions before this protection.

Required routing scenarios: two events with different team IDs/names; identical display names across events; cross-event ID; revoked permission; incompatible category; no eligible team; multiple teams without a default; stale projection; prompt injection inventing a team; operator edit during inference. Expected invalid/missing-configuration outcome is abstention/manual routing, never unauthorised dispatch. These are implementation acceptance cases, not tests claimed to have run.

K2: Responsible AI and dataset answers

Concern	Decision and acceptance evidence
Purpose and people	Operational assistance to a festival incident operator; no diagnosis, treatment advice, autonomous dispatch or access to tools/privileged data
Input and prompt	Minimise/redact fields above; trusted system rules and JSON user data separated. Treat incident text as untrusted, including instructions embedded in it
Rubric	Critical: immediate serious danger; High: ongoing credible threat/vulnerable person lost; Medium: assistance without explicit immediate danger; Low: non-urgent support. Missing information means abstention, not invented facts
Priority floors	Health=Medium, Safety=High, LostPerson=Medium, Theft=Medium, Other=Low; preserve any higher existing priority. Operational proposals, not clinical guidance
Human control	Original request visible immediately, AI label and short reason, operator can reject/change; no numerical confidence without calibration and no overwrite after operator action
Safety	Zero urgent under-prioritisations, urgent missing/invalid/late/abstained outputs; >=80% category/team accuracy, >=95% coverage; human review of every reason in the holdout
Bias and abuse	Report PT-PT slang/typos, English/mixed language, ambiguous/multiple issues and injection slices with counts; small synthetic sets do not prove fairness or safety
Privacy	Local-only inference; no prompt/response bodies, identity or coordinates in telemetry. Synthetic-only research. The security assessment reviews real-data lawful basis, access, retention and deletion before real use
Retention	Synthetic evaluation artefacts versioned; telemetry follows the observability plan (rotated logs, 14-day maximum exports; no persistent trace store required). Incident records follow the separately reviewed product retention policy, not an invented AI retention period
Failure and accountability	AI-off/slow/malformed tests must retain 100% request visibility; record model/config version and sanitised failure metadata; operator feedback triggers review
Changes	Re-run evaluation when model, prompt, schema, rubric or event routing changes; no automatic update of mutable model tags

The dataset is the existing **100 authored synthetic cases: 20 development / 80 test**, with stable IDs and explicit provenance. It is for repeatable regression and comparison with the non-AI baseline, not training or proof of real-world accuracy. Do not add invented expert approval. For the academic demo, ask one teammate to

review the proposed labels/rubric, prioritising every urgent/abstention case, and record reviewer/date and corrections truthfully. Review the remaining labels before using aggregate accuracy as accepted quality evidence. Keep the 20/80 split; do not require another dataset or add 20 mandatory cases. Blinded independent labelling and fresh holdout collection are future strengthening, not demo prerequisites. No review is claimed to have occurred.

Reproducible evaluation and release gate

1. Validate the existing package using the commands in its README. Keep `cases.jsonl`, the prompt/schema and committed baseline unchanged by this decision.
2. Freeze code commit, dataset/split hashes, prompt/schema hashes, full model/runtime digests, settings above, OS/CPU/RAM/Docker allocation and experiment time. Prompt tuning uses the 20 development cases only; do not tune against test results. The visible repository test file is not blinded evidence.
3. Run the frozen 80-case test split once sequentially using input fields only, with the existing fixture prompt/schema. Record every selected ID including errors as `output: null`; never replace failed predictions with reference labels or the deterministic baseline. Store raw outputs and sanitised reasons for review.
4. Score that run using `evaluate.py --predictions /path/to/run-1.jsonl --split test --out /path/to/report-1.json`. Record provider-only time separately as `provider_ms`; omit `elapsed_ms` until actual submission-to-persistence instrumentation exists. A direct Ollama benchmark cannot certify SC3.
5. For the basic demo, preload the model, show one successful synthetic suggestion and then stop Ollama to show manual fallback/request visibility. Record actual elapsed time and container memory. Three repeated evaluations, cold-start studies and 5/10-concurrent-load experiments are optional extensions; never claim they ran.
6. Evaluate the reviewed fixture and a small manual dynamic-team checklist (valid team, wrong event, no eligible team); report counts, all errors, expected abstentions and reason review. Release gates are >=80% category/team accuracy, >=95% total coverage on non-abstention references, all reference abstentions correct, no priority-floor violations and zero urgent misses. Failures stay in denominators. This offline run has no `elapsed_ms` (step 4), so the scorer cannot compute valid on-time coverage: that half of the gate is decided by the instrumented one-hour cohort of step 7.
7. Measure the integrated rolling one-hour cohort: all pre-defined AI-eligible requests form the denominator; finalise after their 30 s deadlines. Require >=95% valid suggestions stored by 30,000 ms, inclusive. Errors and abstentions remain operational misses. Check SC5 separately: 100% durable request visibility with AI disabled. A zero denominator means no data.

Run artefacts: `manifest.json`, `predictions-run-1.jsonl`, scorer reports, routing-case report, human reason-review/adjudication record, and observed latency/memory. Record failures and limitations alongside successes; no raw real incidents or credentials in Git. Suggested output directory is a dated `evaluation/runs/YYYY-MM-DD-local-qwen3/`, created only when a real run exists.

What is closed and what remains

K1 has a selected local platform/model, cost position, sizing experiment, runtime settings, deadline and fallback. K2 has purpose, data provenance/split, input/output and event routing, human safeguards, risk controls and an executable evaluation procedure using the existing scorer. **These documentary questions are**

answered for review. Label review, one actual model evaluation, observed capacity and integrated SC3/SC5 evidence remain execution/review work; no benchmark, real-data privacy clearance or implemented application is claimed. ADR-005 records architectural acceptance; evaluation/release evidence remains pending.

Triage evaluation package

#53 · **22 October 2026 · research tooling only.** Every case is synthetic, authored in this investigation; none came from a real incident, user account or clinical dataset. Labels are unreviewed operational proposals. No model was called.

Role-model compatibility: Medical/Security/PCR are fixture team names for this synthetic evaluation only. Current events create their own teams and assign permissions/categories (`features.md` §4). A production integration must accept the current event's eligible teams and reject cross-event or unauthorised choices. This frozen scorer is not that production contract and must not replace it. Add a separate multi-event routing evaluation before implementation acceptance.

Dataset and rubric

cases.jsonl contains 100 distinct inputs with stable IDs: 20 development and 80 test. Development examples may inform the prompt; test cases may not. The split is deterministic (every fifth ID is development), not a random or blinded real-world sample. Labels remain visible in the repository, so do not train/tune on the test file or present it as an independent benchmark. A teammate reviews/adjudicates labels before accepted quality claims. For the basic academic demonstration, keep the existing split; a fresh blinded holdout is a future improvement, not a required new dataset.

Each row records input category/text/current-priority, expected category, **minimum** acceptable priority, expected team, abstention, split and slice tags. Operational priorities/routing and their limits are in the assessment. The `urgent` tag is descriptive; scoring derives urgent status from a non-abstention reference with minimum High/Critical. Never claim a lower raw prediction passed merely because a later guard clamped it.

Slices include plain language, informal Portuguese/typos, English, mixed language, wrong participant category, missing/ambiguous text, multiple concerns and prompt injection. Some slices are small and overlap. They test examples rather than estimate prevalence, fairness or clinical safety. They contain no real identities. Every reference category is represented, but the set is not a population-balanced sample.

Reproduce the offline checks

From the repository root, Python 3.10+ with only the standard library:

```
python3 Docs/Research/architecture-investigation/evaluation/evaluate.py --validate
python3 Docs/Research/architecture-investigation/evaluation/evaluate.py --self-test
python3 Docs/Research/architecture-investigation/evaluation/evaluate.py --baseline --split test --out D
```

The baseline preserves participant category and uses deterministic category defaults from the proposal. It is a comparison for the **no-AI routing policy**; it is not a model and does not invent latency measurements. The committed baseline report records counts, denominator-inclusive accuracy, coverage, urgent misses, slice

results and dataset/prompt/schema hashes. The self-test uses expected-label fixtures **only to verify the scorer**; those fixtures are never reported as model predictions.

The scorer checks the precise simple object schema shipped here (required keys, types, enums, lengths, no extras). It is not a general-purpose JSON Schema implementation. Missing/invalid output is failure; duplicate/unknown IDs abort scoring. Valid JSON with inconsistent category/team is also a semantic failure. Free-text reason safety requires independent human review and is not certified by the scorer.

Selected local provider run

1. Freeze exact model/version or digest, runtime version, prompt/schema/dataset hashes, temperature/settings, region/hardware and request limits in a run manifest. Use development cases only to change prompts; freeze before running test. For the minimum local demonstration, run the selected model once on the test split and report every case. Repeated/cross-provider comparisons are optional extensions.
2. Pass the fixed system prompt, schema and each row's **input only**. Never send reference labels, split or tags to the model. No credentials or external API calls are required by this package. The selected provider is local Ollama `qwen3:4b` Q4_K_M with no paid API; use the fixed settings, dataset review and routing acceptance contract. Actual runs remain execution work using synthetic data.
3. Save one JSONL wrapper per selected case: `{"id":"T001","elapsed_ms":1234,"output":{"category":"Health","priority":"Critical","team":"Medical","reason":"Requer revisão imediata pela equipa.","abstain":false}}`. A timeout/refusal/transport error is represented by `output:null` with observed elapsed time, not a fabricated valid fallback prediction. Only include IDs from the selected split.
4. `elapsed_ms` must represent submission-to-stored-valid-suggestion time including queue and persistence for SC3. A direct model-call benchmark can score quality but must omit `elapsed_ms` here and record `provider_ms` separately in its manifest; it does not establish deployed SC3. Scoring an asserted elapsed value cannot verify the measurement instrument.
5. Run `python3 Docs/Research/architecture-investigation/evaluation/evaluate.py --predictions /path/to/test-predictions.jsonl --split test --out /path/to/run-report.json`.
6. Compare with the baseline. Review reasons, all urgent misses, language/injection slices, observed latency/memory and any actual usage. A provider benchmark, labels, privacy review and an AI-off end-to-end test remain pending.

Proposed gates

On non-abstention reference cases, category/team accuracy ≥80% and suggestion coverage ≥95%. Zero urgent under-prioritisation, including missing/invalid/abstained urgent outputs. Correct abstention on all abstention-reference cases. No raw priority-floor or category/team violations. Reported timing additionally requires every case timed, no urgent case past 30 s and ≥95% valid on-time coverage. A direct model-call run omits `elapsed_ms` (step 4), so it decides only the quality gate; on-time coverage comes from the instrumented one-hour cohort of the local decision. Ratios count missing/invalid predictions in the denominator. Empty slices return null, never perfect scores.

Alignment (26 October 2026, #145): the ≥95% proposal is shared with ADR-005 and observability. Offline denominators are the non-abstention reference cases; the operational denominator is all AI-eligible

submissions in a one-hour cohort, with failures/abstentions retained and the final 30 s deadlines allowed to expire before finalising. Report both populations separately. Valid on-time output must also respect category/team consistency and the raw priority floor; event-specific team eligibility remains an integration check, because this fixture has fixed team names. The self-test covers 19/20 (95%, pass) versus 18/20 (90%, fail) and the inclusive 30,000 ms boundary. The baseline has no inference or latency samples and is not evidence that the target has been met.

These offline gates cannot establish production readiness: reviewed labels, review of reasons, real end-to-end SC3 timing, SC5 visibility with AI off and the provider/privacy assessment are separate checks.

Docs/Technical/architecture.md

High-Level System Architecture

> LABDSOF 2026/27 · Sprint A · Assignment §6 (Common Technical Requirements) and §7.1.2 (deliverable 7, *Technical Design*)
> **Status (31 October 2026, #162):** reflects the team decisions recorded in `architecture-decisions-guide.md` (Portuguese). Items marked **[PROPOSED]** are not decided yet; the owner is in that document. The Sprint A skeleton implements only a small part of this architecture (§13). The reference diagrams are drawn in draw.io and listed in §14 (ADR-009); the Mermaid diagrams below are working sketches.
> Legend: see `product-vision.md`.

> **Research update (22 October 2026, #53):** The investigation supplies recommendations for deployment, observability and AI, including C1 hosting and D2 server topology. ADR-005 and ADR-012 are **Accepted as architectural choices** (status reconciled on 31 October against guide/NFR decisions from 26–27 October); implementation/model/telemetry evidence remains separate. ADR-011 now records the professor-authorised local-only delivery due to costs, as relayed to the team; the cloud evaluation is retained. Implementation and measured evidence remain pending; the diagrams were aligned on 26–27 October (#133) and 31 October 2026 (#171, #162; §14).

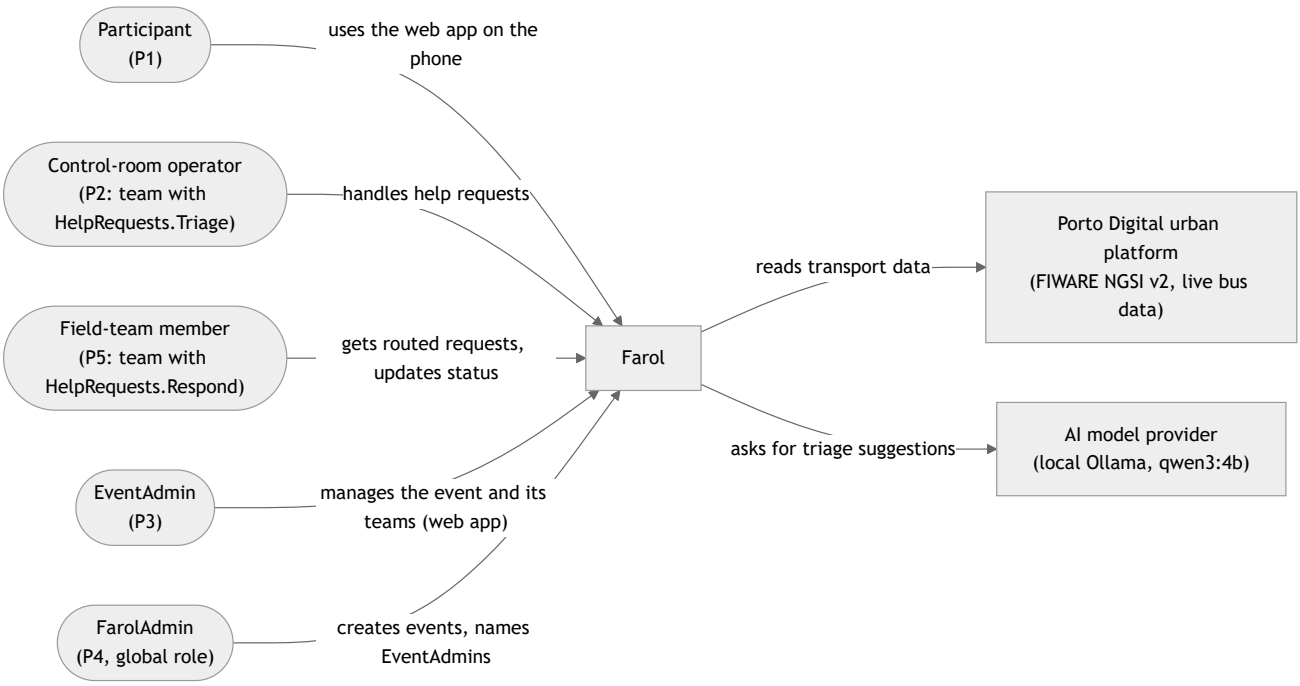
1. Architecture drivers

What shapes the design, and where it comes from:

#	Driver	Source
D1	**Isolation between events**: one team must never see or change another event's data	Report, *Per-event permissions*; §5 technical risks
D2	**Read peaks**: tens of thousands of participant requests within minutes (e.g. end of a concert)	Report §5 technical risks
D3	**Weak or congested network** at the venue: short messages, data with its update time	Report §5; assumption A8; annex C10
D4	**Freshness**: every piece of information shows when it was last updated	Report §5; annex C4
D5	**Sensitive data**: location and help requests (possibly health) need consent, minimisation, short retention	Report §5 regulatory risks
D6	**AI with fallback**: the main workflow must keep working when the AI is unavailable	Assignment §5.6.2; annex C9
D7	**At least two independently deployable backend components**, or a justified modular monolith	Assignment §6.2
D8	**At least one asynchronous workflow** and **one external integration** (or realistic simulator)	Assignment §6.3, §6.5
D9	**Small team, three sprints**	Report §5 execution risks

The measurable quality targets that follow from these drivers, the success criteria and assignment §6 are listed in one place in `non-functional-requirements.md`.

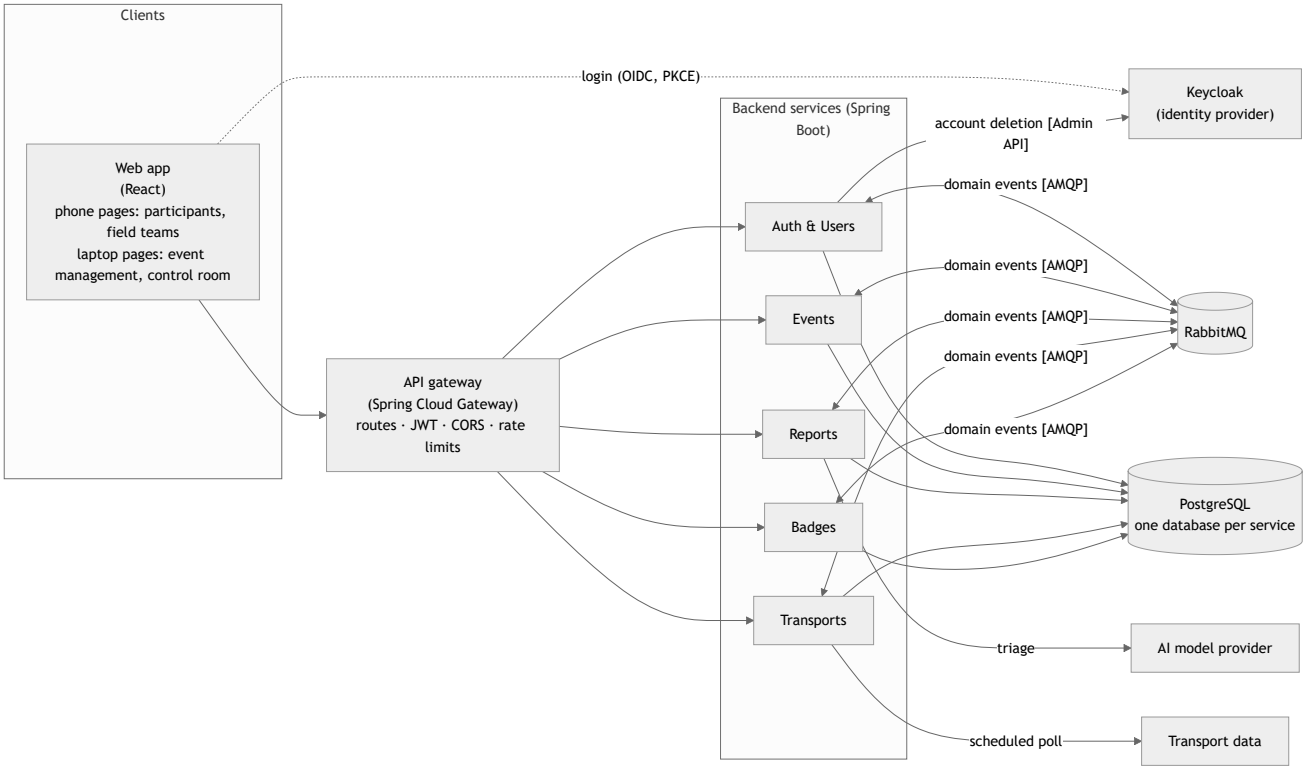
2. System context



The identity provider, Keycloak, is self-hosted inside Farol (ADR-006), so it is not an external system at this level (guide B2); it appears from logical level 2.

3. Container view

The five services are deployed independently from Sprint B (§11). The Sprint A skeleton runs only Reports (#166, §13).



Not drawn: every container sends logs, metrics and health checks to the observability stack (assignment §6.7; Prometheus/Grafana, Actuator and Docker logs; ADR-012, local observability decisions). Clients get live updates by **polling** (no push in the MVP). Keycloak keeps its own database on the PostgreSQL server; every service validates the token against Keycloak's keys (ADR-006).

4. Responsibilities and data ownership

Service	Responsibility (features)	Owns (see `domain-model.md`)
Auth & Users	Authentication, authorisation, accounts, the global `FarolAdmin` role, memberships of events with their role (`EventAdmin`, `RegularUser`) and team assignments, friends (F1, F2, roles and team members of F4, F18)	UserAccount, EngagementPreferences, GlobalRoleGrant, EventMembership, TeamAssignment; friendships; consent records (sign-up terms, assessment §8.4; help-request consent, §8.3); its own audit history
Events	Events, venue map and points of interest, teams with their permissions and help categories, announcements, check-in, entries (F3, teams of F4, F5, F10, F16, F17)	Event, VenueMapVersion, PointOfInterest, Team, TeamPermission, Announcement, AttendanceCheckIn; its own audit history

Service	Responsibility (features)	Owns (see `domain-model.md`)
Reports	Community reports and their confirmation, help requests, incident backoffice, AI triage with fallback (F6, F7, F8, F12, F13)	HelpRequest, TriageSuggestion, CommunityReport, FieldTask, AuditEntry (each service owns its own audit history, `domain-model.md`)
Transports	External transport integration and event shuttles (F9)	TransportOption, TransportDeparture, BusVehicle, BusObservation
Badges	Badges (F14, F15)	BadgeProgress

Friendships (F18, a *Could*) are not in the domain model yet.

Where roles and permissions live (`features.md` §4, to confirm with the team):

- **Auth & Users** holds who is who: the `FarolAdmin` grant, each user's membership of each event with its role, and which members are in which team. It publishes a domain event when any of these changes.
- **Events** holds the teams: their names, permissions (fixed list, with the report types for `Reports.Review`) and help categories. It publishes a domain event when a team or its permissions change.
- A member's permissions in an event are the union of their teams' permissions; an `EventAdmin` has all of them. Each service that checks a permission keeps a local copy of the memberships, team assignments and team permissions it needs (rules below). The token carries only the identity; every service and the gateway validate it, and the permission is checked in each use case (decided on 22 October 2026: guide E2, ADR-006).

Rules:

- Each service owns its data and its database; no service reads another service's database.
- Services talk **only through events**. Each service keeps a local copy of what it needs from another (e.g. Reports keeps the teams, their permissions and help categories from Events; every service keeps the memberships and team assignments of its events from Auth & Users). The cost is eventual consistency.
- Each service is its own Maven project in `backend/<service>/` and depends only on the others' contracts (OpenAPI and events). Inside, it follows a **hexagonal layout**: `domain`, `application`, `adapters/in`, `adapters/out`.
- APIs are **contract-first**: hand-written OpenAPI in `backend/<service>/contracts/openapi.yaml`, paths under `/api/v1`, server and client code generated (`spec-driven-development.md`).

5. Communication

From → To	Style	Why
Apps → services	Synchronous (HTTP/REST)	Request–response by nature
Apps (live lists, request status, announcements)	**Polling**	Enough for the MVP; no push infrastructure
Service → service	**Asynchronous only** (RabbitMQ, Spring AMQP, *topic* exchanges)	A service that is down does not stop the others

From → To	Style	Why
Transports → transport data	Scheduled poll	The external source may be slow or down; never called inside a user's request

The endpoints of each service and the events each one publishes and consumes are in the API outline and event catalogue.

6. The asynchronous workflow: help-request triage

Required by assignment §6.3 and §7.1.2 (*Asynchronous workflow*). End-to-end flow: `user-workflows.md`, W2.

Concern (assignment §6.3)	How it is handled
Lost messages	Reports saves the request and the outgoing event in the same transaction (**outbox table**); a polling job publishes pending events to RabbitMQ
Duplicate events	Consumers are idempotent: each keeps a table of processed `message_id`s
Retries	**4 attempts** in the consumer (Spring Boot 4: `spring.rabbitmq.listener.simple.retry.max-retries=3`), exponential backoff **1 s → 2 s → 4 s** (at most 10 s), so about 7 s before the **dead-letter exchange**. Messages that cannot be converted or fail validation go straight to the DLQ (`AmqpRejectAndDontRequeueException`). A failing AI call is **not** a message failure: the triage worker records the fallback (§9) and acknowledges the message, so the AI policy (one attempt locally; hosted retry alternative in J1) and message retries never multiply (otherwise the old hosted policy permits 2 × 4 = 8 calls and more than a minute, far beyond SC3). **Why 4 attempts over about 7 s:** the failures a retry can fix (a database connection lost, the broker restarting) clear within seconds; errors that last longer are bugs or poison messages that no retry fixes, so more attempts would only delay the DLQ. Exponential backoff gives a recovering dependency room, and the 10 s cap bounds how long a consumer thread is blocked (stateless retry runs on the listener thread)
Dead letters	One DLQ per queue (`<queue>.dlq`). `RepublishMessageRecoverer` adds the exception and the original routing key as headers. DLQ messages are **never lost and never reprocessed automatically**: a DLQ that is not empty shows on the DLQ panel of the dashboard and is an acceptance check, not one of the three core alerts (assignment §6.7; ADR-012, NFR-34); once the cause is fixed, they are moved back to the original queue by hand (safe, because consumers are idempotent); messages that will never be valid are logged and deleted. No DLQ message can hide a help request from the operator: the backoffice reads Reports' own database, and an untriaged request is simply "Not triaged". **Why by hand:** a message that failed four times usually failed because of a bug or bad data, and replaying it automatically would loop; a person replays it once the cause is fixed. The recoverer keeps the reason, which the default `RejectAndDontRequeueRecoverer` loses. No time-to-live on the DLQ: nothing disappears unnoticed, and the volume is small
Outbox publisher	Polls every 1 s, batches of 100, with publisher confirms. If the broker is down, rows stay pending and are sent on a later run; the age of the oldest pending row is a metric, alerted when it stays above 60 s for 2 min (decided 24 October 2026). **Why:** 1 s keeps the delay to consumers small compared with the 30 s of SC3, at the cost of one indexed query per second per service; 60 s is 60 missed runs, which means the broker is down, not slow, and the 2 min window keeps a short peak from raising the alert
Failures	If no suggestion arrives in time, the request stays in the operator's list with the participant's category, in order of arrival (non-AI fallback), marked "not triaged"

Concern (assignment §6.3)	How it is handled
Eventual consistency	The backoffice shows "awaiting triage" until the suggestion arrives; the operator can act before it does
Clean-up after account deletion	**[PROPOSED]** (#162): published outbox rows and DLQ messages can hold personal data, such as a display name in `UserProfileChanged`. Each service deletes published outbox rows after a retention window to be set with NFR-19 (longer than any redelivery); processed `message_id` tables hold no personal data and are pruned only for size; and DLQ messages that name a deleted user are purged on `UserDeleted`

7. External integration

Integration	Purpose	Handling of unavailable, late, incomplete or invalid data (assignment §6.5)
Transport data: Porto Digital urban platform (FIWARE NGSI v2, live bus positions; ADR-008)	Transport for F9	Calls with a timeout; data validated on import; data older than its freshness limit counts as late. On any of these, the last valid data is kept and shown with its time and a warning (US-42, US-43); event shuttles entered manually are unaffected. A **simulator or recorded real responses** under the team's control (guide J4) is used for automated tests, to cause failures and invalid data in the demo, and as a fallback on presentation day
AI model provider [ACCEPTED architecture; evaluation pending] (local Ollama + qwen3:4b Q4_K_M, ADR-005)	Triage suggestions (F8)	Timeout (connect 2 s, response 20 s), one attempt, circuit breaker, then fallback (§9.1); the suggestion is never shown as a fact (assignment §5.6.2)

8. Security, authentication and authorisation

Scope: identity provider, token validation, service-to-service authentication, secrets, Security & Privacy assessment. Decided by the team on 22 October 2026 (record): ADR-006 (identity) and ADR-013 (gateway).

Concern	Decided
Accounts	**Login mandatory for every user**, participants included (9 October 2026, DM-1)
Identity provider	**Keycloak**, self-hosted, Authorization Code + PKCE for the web app. The token carries **identity only**; the account is created in Auth & Users at the first `GET /me` (guide E1, ADR-006)
Authorisation	Checked **per event** (ADR-006; `event_id` on every event-scoped table, ADR-003; NFR-14): the user's role in that event (`EventAdmin` or `RegularUser`) and the permissions of their teams there (§4). `FarolAdmin` is the only global role and gives no access to any event's data. Accounts, roles and memberships live in Auth & Users, teams and their permissions in Events. **Every service validates the JWT** (the gateway too) and checks the permission **in the use case**, against its local copies of memberships and teams; changes reach it as events within ≤ 5 s (guide E2, ADR-006)
Isolation between events	`event_id` on every event-scoped table, filtered by the application; each service has an integration test proving that a member of event X cannot read or change event Y. A non-member gets 404

Concern	Decided
Audit	Every change to a help request recorded as an `AuditEntry` (domain model; NFR-16, SC9)
Rate limiting	**Per account, never per IP**, at the gateway: 120 requests/min; **5 help requests per 10 min**. Reports per account (US-48) are a Reports business rule (guide C6, ADR-013)
Service-to-service and secrets	No HTTP between services; one RabbitMQ user per service. Secrets from environment variables (`.env` locally, GitHub Actions secrets in CI); gitleaks on every PR, planned for Sprint B (guide E3, E4; ADR-010 point 9)

Security and Privacy Assessment (Sprint A deliverable 9, guide E5): `security-and-privacy-assessment.md`, written on 25 October 2026, to review with the team. It holds the initial threat model, the sensitive data, the access-control requirements, the data-minimisation measures, retention and deletion, consent and the local demo environment, plus the findings that change the design before Sprint B (§10 there).

9. Degraded modes

Decided (9 October 2026): the §6.8 scenarios are **AI unavailable (D1)** and **transport source unavailable (D2)** (`features.md` §3.1). **Decided (22 October 2026):** the resilience patterns and values (§9.1), how D1 and D2 are triggered, observed and tested (§9.2), and the demo script (§9.3), all run on the local machine of the authorised local-only delivery (ADR-011). Guide items J1 and J2. **WireMock** is the simulator of guide J4: proposed in J2, confirmed in J4 on 26 October 2026 (#155), after replaying recorded Porto Digital responses through it (`infra/wiremock/`).

Unavailable	What the user sees
AI provider	Requests go to the operator untriaged, with the participant's category, in order of arrival
Transport source	Last known data with its time and a warning
Another service	The other services keep working (events only between services, local copies)
RabbitMQ	Requests are still saved (outbox); triage and copies between services catch up when the broker returns

9.1 Resilience patterns and values (J1)

Library: Resilience4j 2.4 (`resilience4j-spring-boot4`, which supports Spring Boot 4 since March 2026), used only in the outgoing adapters (`adapters/out`). It is the only mature option with a circuit breaker for Spring Boot 4, it is configured in `application.yml`, and it publishes Micrometer metrics and its state in the health check, which the dashboard of ADR-012 needs (assignment §6.7). Alternatives considered: Spring Framework 7 core retry (no circuit breaker; Spring AMQP uses it for message retries, §6), Spring Cloud Circuit Breaker (an extra layer over Resilience4j, tied to the Spring Cloud release train) and Failsafe (no Spring Boot auto-configuration or metrics).

Timeouts are set on the HTTP client (`RestClient`), not with a Resilience4j `TimeLimiter`, because the calls are blocking and a client timeout also frees the connection. Retry wraps the circuit breaker, so every attempt counts for the circuit, and nothing is retried while the circuit is open. External calls never run inside a user's request: the AI is called by the triage worker and the transport source by a scheduled import.

External call	Timeout	Retry	Circuit breaker	When it fails
Local AI (Reports, triage worker; K1: Ollama with `qwen3:4b`)	connect 2 s, response 20 s; entire call ≤ 22 s and within the remaining overall deadline	One attempt; no retry	last 10, minimum 4; ≥ 50 % failures or slow (> 16 s) calls; open 30 s; one probe	D1 fallback; at most 5 s admission wait, start only with ≥ 23 s remaining. Original submission-to-storage deadline remains 30 s. **J1 timeout/circuit policy accepted on 27 October 2026** (not a model-performance or telemetry implementation approval), following the complete contract (#153; NFR-11)
Hosted AI alternative (Reports, triage worker)	connect 2 s, response 10 s	2 attempts, 2 s apart; only on timeout, connection error, 5xx and 429; never on other 4xx or invalid output	last 10 calls, at least 4; opens at ≥ 50 % failed or slow (> 8 s) calls; open 30 s; 1 trial call	D1 fallback. Worst case ≈ 22 s, inside the 30 s of SC3. Not used while K1 is local; a hosted provider needs a new cost and privacy approval and an ADR-005 update
Porto Digital (Transports, scheduled import every 30 s: the *x* of the meeting with the professor on 22 October, P4)	connect 3 s, response 5 s	3 attempts, 1 s and 2 s apart	last 6 calls, at least 3; opens at ≥ 50 % failed calls; open 60 s; 1 trial call	D2. Only buses within 2 km of the venue, with an NGSI geo-query (`georel=near;maxDistance:2000`): 10 KB instead of 190–300 KB for the whole city (measured on 22 October)
Identity provider: Keycloak, self-hosted in the local Compose (E1, ADR-006; JWKS keys, every service)	connect 2 s, response 3 s	none	none	Keys fetched from the Keycloak container (`keycloak:8080`) only when a token arrives, cached for 5 min and refreshed when an unknown `kid` arrives (ADR-006); if the refresh fails, the cached keys stay in use. **Confirmed on 26 October 2026**

Why these values:

- **Local AI, 20 s response and one attempt:** a small model on the demo laptop's CPU is slower than a hosted one, so it gets a longer response time; two attempts of 20 s would not fit in the 30 s of SC3, so there is only one. The admission limit (at most 5 s waiting, start only with ≥ 23 s left) keeps a late call from passing the 30 s, and slow calls (> 16 s, 80 % of the timeout) open the circuit. The real time is measured on the demo machine (plan).
- **Hosted AI alternative, 10 s response and 2 attempts (not the selected local profile):** the most that fits in the 30 s of SC3: 10 s + 2 s + 10 s = 22 s, leaving about 8 s for the outbox (≤ 1 s), the queue and saving the suggestion. A third attempt would not fit (34 s). A hosted model answers a short classification prompt with a small JSON output in a few seconds, so 10 s is a generous margin. Connect 2 s: a reachable provider accepts a connection in well under a second; waiting longer only delays the fallback.

- **Hosted AI alternative only, retry only transient failures** (the local profile has no retry): a timeout, a dropped connection, a 5xx or a 429 can succeed a moment later. Other 4xx (wrong key, malformed request) fail the same way every time. Invalid output is not retried either: the same prompt tends to fail the same way, the fallback is safe, and it keeps the worst case short.
- **Local AI, a small circuit (10 calls, at least 4, 50 %, open 30 s):** at night only a few help requests arrive per minute. With a large window, many requests would each wait up to 22 s before the circuit opened; with this one, it can open as soon as 4 calls are recorded (one attempt per request, so 4 requests), when at least 2 of them failed, or at least 2 were slow (separate failure-rate and slow-call-rate thresholds, both 50 %; `slowCallRateThreshold` must be set, because its default is 100 %). Below 4 recorded calls Resilience4j computes no failure rate, so the circuit stays closed. 50 % tolerates an isolated invalid answer. Slow calls (> 16 s, 80 % of the 20 s timeout) count as failures, so a model that is degrading opens the circuit before it starts failing. (The hosted alternative uses the same window with 2 attempts per request and slow > 8 s.) 30 s open is short enough that triage returns quickly after the provider recovers (at most one or two requests miss it), and long enough to stop paying 22 s per request during an outage. One trial call is enough evidence at this traffic level.
- **Porto Digital, every 30 s:** bus positions arrive about 1 min after they are observed (measured on 16 and 22 October). Importing more often brings no newer data; importing less often makes it older.
- **Porto Digital, 5 s response, 3 attempts:** the API answered in 0.4–0.8 s (six times the margin). The worst case (3 × 5 s + 3 s = 18 s) is shorter than the 30 s interval, so two imports never overlap.
- **Porto Digital, circuit open 60 s:** one failed import (3 failed attempts) opens it. During an outage, calls drop from 6 to 1 per minute, so we do not hammer a public service we do not control, and recovery shows within about 90 s.
- **Porto Digital, geo-query:** the participant only needs buses near the venue; it cuts the payload 20–30 times and the validation work with it.
- **Identity provider:** tokens are validated locally with the cached keys; the call happens only when the keys change, so it needs no retry or circuit, but it must be short because it runs inside a user's request.

9.2 How D1 and D2 are triggered, observed and tested (J2)

	D1 · AI unavailable	D2 · Transport source unavailable
Detected	No valid suggestion: timeout, connection error, provider error or invalid output (fails the JSON schema)	Import fails after its retries, the data fails validation, or the data is older than the freshness limit
Degraded state	Circuit breaker open or half-open → banner "*AI triage unavailable*". A single fallback only labels that request, so one bad answer does not alarm the operator	Last import failed, or data older than the limit → participant warning. Backoffice warning from the first failed import, so the team knows before participants do
Freshness limit	—	**5 min** for bus positions (the *y* of the meeting with the professor on 22 October, P4); older observations are dropped on import, which also covers *late* data (assignment §6.5). Why: positions normally arrive about 1 min old, and a bus in the city covers about 2 km in 5 min, so an older position misleads. It replaces the 24 h proposed for a static GTFS timetable, dropped with the professor's answer (ADR-008)

	D1 · AI unavailable	D2 · Transport source unavailable
Recovery	Circuit closes after a successful trial call; the banner disappears. Only requests submitted after recovery are triaged. A request that missed its 30 s deadline stays "Not triaged" for manual triage: the deadline is never reset and late results are discarded (AI plan, ADR-005 decision 3), and nothing ever overwrites an operator's decision (NFR-22)	First valid import clears the warnings; invalid data never replaces valid data
Observed in the UI	Request in the backoffice **at once**, with the participant's category and the label *"Not triaged"*; banner	*"Data from HH:MM"*, warning *"Live transport information unavailable"*, event shuttles still shown; backoffice warning with the time of the last good import
Observed in operations	Metrics: `resilience4j.circuitbreaker.state{name=ai-provider}`, `farol.triage.fallback{reason}`. Log line per fallback with the reason and the correlation id. Shown on the local dashboard of ADR-012 (triage/fallback panel)	Metrics: `resilience4j.circuitbreaker.state{name=transport-source}`, `farol.transport.import{result}`, `farol.transport.data.age` (seconds). Log line per failed import. Shown on the local dashboard of ADR-012 (transport freshness panel)
Automated test	**US-36** (SC5), Reports integration test, on every PR. WireMock replaces the provider and answers with a delay over the timeout, a 503, a dropped connection and an invalid body; 20 requests per case. Pass: 100 % shown to the operator as "Not triaged" within 30 s, fallback reason recorded, circuit opens and WireMock receives no more calls; after recovery, a new request gets a suggestion and an operator's priority is kept	**US-42, US-43**, Transports integration test, on every PR. WireMock replays recorded Porto Digital responses: one valid import, then a 503, a timeout and an invalid payload; a fixed `Clock` moves past the freshness limit. Pass: the API returns the last valid data with `dataAsOf` and `stale=true`, shuttles still returned, invalid data never stored; after a valid import, `stale=false`

Simulator and demo trigger (guide J4, confirmed by the team on 26 October 2026, #155, NFR-31): WireMock in front of each external system. In the tests and in the local demo run, the AI provider URL and the Porto Digital URL (both configuration, ADR-004) point at a **WireMock** container, one more service in the local Compose file:

- **Normal operation:** WireMock *proxies* every request to the real system (the local Ollama container for the AI, K1), so the demo starts with real AI suggestions and live buses.
- **Recorded responses:** WireMock replays real Porto Digital responses (*record and playback*): 300 recorded on 25 October 2026, one a minute, kept in `infra/wiremock/` with the script that turns them into stubs. Each stub holds only the buses within 2 km of the venue, as the geo-query would; every position's time is rendered relative to *now*, so the data passes the 5 min freshness limit with each bus's real delay; and each snapshot answers two imports, so the buses move at their real speed. The automated tests replay them, and on presentation day they replace the real API if it is down.
- **Failure:** the presenter adds, through WireMock's admin API, a higher-priority stub that drops the connection, answers later than the timeout or returns invalid data, and removes it to recover.
- **Without WireMock:** in a local run that does not need failures, the URLs point straight at the real systems. WireMock is never part of a service image.

Why: the services carry no fault-injection code, the professor's request for a simulator is met with the same tool the tests already use (Testcontainers supports it), and it works with HTTPS, because WireMock is itself the HTTPS client of the real system. Alternatives considered: a fault switch inside each service (failure code shipped in every image), our own simulator in Spring Boot (more code to keep close to the NGSI v2 format), Toxiproxy (network faults only, no recording or invalid data, and it breaks the certificate check of HTTPS providers), and stopping a container (needs a restart and shows only one kind of failure). Backup if WireMock fails during the demo: point the URL at an unreachable address and restart the service container. The demo runs on the local machine (guide G4, ADR-011); the Compose set-up is covered by guide G5 and G6.

9.3 Demo script (D1 and D2, about 5 minutes)

Before the demo: Farol running on the local machine with `docker compose` (ADR-011); WireMock in proxy mode in front of the AI provider and Porto Digital; one published event with a venue map, points of interest and an event shuttle; three windows side by side: **participant** (the web app's phone pages, on a phone or a phone-sized browser window, logged in, event open), **operator** (backoffice, laptop pages), **dashboard** (circuit breaker state, fallbacks, transport data age: the local Grafana dashboard of ADR-012; until it exists, the Actuator `health` and `metrics` endpoints). Each failure is one prepared `curl` command to WireMock's admin API.

D1 · AI unavailable

1. Participant sends a help request (*Health*). Operator: it appears at once, and a suggestion arrives within 30 s (SC3; latency on the demo laptop is not yet measured, ADR-005), marked as a suggestion. *Normal behaviour.*
2. Presenter adds the AI *connection dropped* stub.
3. Participant sends a second request. Operator: it appears **at once** with the participant's category and the label "*Not triaged*". Participant: the usual confirmation; nothing looks different.
4. Participant sends a third and a fourth request. Both appear at once as "*Not triaged*". After the fourth, the circuit has 4 recorded calls with 3 failed (≥ 50 %, minimum 4 calls, §9.1) and opens: the banner "*AI triage unavailable: requests are shown by order of arrival*" appears. Dashboard: circuit `OPEN`, fallback counter +3 with the reason `ProviderError` (API outline §4.1).
5. Operator sets the priority and team of the second request by hand and dispatches it. *F7 works without the AI.*
6. Presenter removes the stub, waits 30 s, and the participant sends a fifth request. It gets a suggestion, the circuit closes and the banner disappears. The third and fourth requests stay "*Not triaged*" (their deadline has passed; §9.2 *Recovery*); the second keeps the operator's values.
7. Optional: repeat with the *slow answer* stub to show that even when the AI hangs, the request still appears at once.

D2 · Transport source unavailable

1. Participant opens *Transport*: buses near the venue and the event shuttle, with a recent "*Data from HH:MM*".
2. Presenter adds the Porto Digital *invalid data* stub. Within one import (≤ 30 s): the import is rejected and the buses on screen do not change; the time stops moving and the warning "*Live transport information unavailable*" appears, with a link to the operator's site; the **event shuttle is still shown**. Backoffice: warning with the time of the last good import. *Invalid data never replaces valid data.*
3. Presenter swaps it for the *connection dropped* stub. Dashboard: failed imports, circuit `OPEN` after one import, data age growing. The participant's screen stays as in step 2.

4. Presenter removes the stub. Within about 90 s (circuit open time plus one import), the warnings disappear and the time updates.

Invalid data comes first because an invalid answer is still a successful HTTP call: shown after the circuit has opened, it would only reach Transports once the circuit lets a trial call through.

10. Deployment view

Current scope: local-only deployment/execution. As relayed to the team, the professor authorised proceeding locally **because of cloud-hosting costs**, provided the team explains that reason in the final report/presentation. See the clarification record and ADR-011. The cloud assessment and dated costs remain documented evidence and future alternatives, not a requirement to provision cloud resources.

Local delivery procedure (decided 23 October 2026, #129): `local-delivery.md`. One laptop runs one `infra/compose.yaml`: five independently replaceable backend containers, the API gateway (ADR-013), Keycloak (ADR-006), one RabbitMQ with a persistent volume, **one PostgreSQL instance with six separate databases/users** (five services and Keycloak), a `web` container serving the React app, and WireMock in front of the external systems; AI and observability are optional profiles. The gateway, Keycloak and the web app are the only containers reachable from the network; admin pages are published on `127.0.0.1` only. HTTPS for the phones, which the PKCE login needs off `localhost`, is open (security assessment §9). Only `docker compose` commands are used; a manual smoke-test checklist runs after every start, update or rollback. Phones open the web app at the laptop's LAN address; a second laptop is kept as a backup. Minimum RAM and recovery times are still to be measured: the in-memory Sprint A skeleton cannot measure them (plan).

Preserved requirements and limits:

- **One instance per service** initially; independent images and service replacement still demonstrate the microservice boundaries.
- **Sprint A minimum (#166):** one Reports image and CI build/test/container smoke on every PR. The full pipeline below is the Sprint B target, not an implemented R0 claim. **Multi-stage Dockerfiles**, Maven, Java 21 and Spring Boot 4.x. **CI/CD (ADR-010):** GitHub Actions checks every service in every run. Pull requests run the fast part (build, unit and Testcontainers integration tests, Checkstyle, ESLint, gitleaks); pushes to `main`, release tags and manual runs also build the images, scan them with Trivy and run the smoke test with Compose on the runner. Documentation-only and draft PRs skip the code jobs to save Actions minutes. No registry and no credentials: nothing is published.
- The earlier automatic-deploy-to-cloud-production promise is superseded for this delivery scope. Delivery is building locally from the release tag and running the smoke test; CI never reaches a personal machine. Cloud credentials/ACR/provisioning are not prerequisites.
- **Versioning (guide H7):** one **SemVer** version for the whole monorepo, set by an annotated git tag `vX.Y.Z` on `main`; our seven images (the five services, the API gateway of ADR-013 and the web app) get the same version without the `v` (`farol-<service>:0.2.0`), other builds are `dev`. Third-party images (Keycloak, PostgreSQL, RabbitMQ, WireMock, the observability stack) keep their own exact version, pinned in the Compose file (ADR-011) and changed only in a PR. One MINOR release at the end of each sprint, one per release of `features.md` §8.2 (`v0.1.0` Sprint A = R0, `v0.2.0` Sprint B = R1, `v1.0.0` Sprint C = R2), PATCH only to fix a release already demonstrated, tagged on `main` (no release branches, so a PATCH also ships whatever is already on `main`). Git tags only: no registry and no GitHub Releases; images are built from the tag, rolling back means building the previous tag, and release notes go in the `CHANGELOG.md`.

- **Flyway** migrations remain compatible with the previous application image (*expand and contract*); rolling back code does not undo database migrations.
- Local metrics and logs (no traces are stored, ADR-012) and degraded-mode/integration tests still provide evidence. No cloud HA, managed scaling or cloud recovery result is claimed.
- Persistent local volumes are not a backup or a guarantee against host/disk loss. The six databases are dumped with `pg_dump` before every demo and update; queued messages and disk loss are not covered.

10.1 Peak load and scaling (J3)

Expected peak (design target, not a measurement):

Step	Value	Source
Entries per night, average	≈ 31,000 (250,000 over 8 nights)	Evidence E1 (problem report)
Busiest night	40,000 (average + 30 %)	Assumption: the nights with the biggest headliners and weekend nights fill above the average; no official capacity per night was found (to check with the organisers, FAP). The margin keeps the design on the safe side
People with the app that night	60 % → 24,000	Event-app adoption of 55–65 % with promotion (problem report; vendor figures), and the survey target V1 (≥ 60 % would use it). The top of the range on purpose: for a load test, more users is the safe error
Open it in the 10 min after the last concert	50 % → **12,000 open apps** (the web app open in a phone browser)	Assumption: everyone leaves at once and *how do I get home* (F9) is the main question of the night; report §5: *"tens of thousands of requests within minutes"*
Requests per open app	1 poll every 10 s + about 5 screen loads	The 10 s polling interval follows SC6 (announcement on screen in ≤ 15 s: one 10 s interval plus the server time; SC6 raised from 10 s on 24 October 2026) and C5 (polling, no push)
Read peak	≈ 1,200 req/s of polling + ≈ 100 req/s of screen loads ≈ **1,300 req/s for 10 min**	Mostly announcements (Events) and transport (Transports). Check: 780,000 requests in 10 min, consistent with the report's "tens of thousands within minutes"
Help requests	**1 per second** for 10 min	600 requests in 10 min is far more than the PCR (90 people) could handle; it is set high on purpose so the asynchronous path is never the bottleneck

Scaling with one instance per service (C5), in this order:

1. **Less work per request:** conditional requests (ETag from the item's last-update time, so a 304 needs no data read); the venue-map image under a versioned URL with a long cache lifetime, so it is downloaded once; polling every 10 s with random jitter, only while the page is visible (Page Visibility API), and slower on `429` or `503` with `Retry-After`. Rate limiting stays with the gateway decision (C6).
2. **Isolation:** help requests live in Reports, away from the read peak (Events and Transports). A read peak can slow the map, but not a help request (SC2).
3. **Vertical scaling:** more CPU and memory for the most loaded container (its limits in the Compose file), connection pool sized to match. On one local machine this is bounded by the machine itself.

4. **More instances, when C5 changes: Events first** (it receives about 90 % of the peak: announcement polling, map and points of interest), then Transports. Both are stateless. Locally this is a second container of the same image behind the gateway (C6), e.g. `docker compose up --scale events=2`, on the same machine. Before that, the outbox publisher and the scheduled imports must run on a single instance (`FOR UPDATE SKIP LOCKED` or ShedLock), or every instance would publish and import the same data.

Why this order: steps 1 and 2 cost no infrastructure and protect what matters most, the help request; step 1 alone removes most of the work, because most polls find nothing new. Scaling is the last resort, and C5 still limits it; any change to C5 must be motivated by the load-test results (guide A4: evolution driven by evidence). Managed cloud scaling (more instances on several hosts) is not part of the local-only delivery; it stays an evaluated alternative in ADR-011 and the deployment research, and nothing here claims it was demonstrated.

Load test (Sprint C performance evidence, assignment §7.3.2 item 7): k6. Scripts in JavaScript; open-model scenarios (requests arrive at a set rate, as they do at the end of a concert, instead of a fixed number of users who wait for each answer, which hides slowdowns); pass criteria written as *thresholds* that fail the run; one binary (or its official Docker image) that runs next to the local Compose stack. It runs by hand in Sprint C, against the full system started with `docker compose` on the local machine (guide G1 and G4, ADR-011), not on every PR, because it needs the whole system and takes about 20 min; a public CI runner cannot reach a team member's machine. The k6 scripts, the Compose file used and the k6 summary (JSON) of each run are committed as the evidence. The AI provider is replaced by WireMock with 2–5 s latency, so the test measures Farol, costs nothing and does not hit the provider's rate limits; the transport import is not affected by user load. Alternatives considered: Gatling (Java DSL, but a heavier JVM and Maven setup) and JMeter (XML plans that are hard to review in a PR).

Local run. k6 generates the load from a second computer on the same network when one is available; otherwise from the same machine, with CPU and memory limits on the Farol containers so k6 does not take their resources, and its own CPU use recorded. The results describe that machine (its CPU, memory and the Compose limits are written down with them), not a cloud environment. If one machine cannot generate or serve the L2 rate, L4 records the limit it reached and the report states it; that limit, not an assumed cloud capacity, is the evidence for the scaling plan.

Why these pass criteria: **p95 < 500 ms** on the server leaves most of SC1's 5 s for a weak venue network (driver D3); **errors < 1 %** because a failed poll is retried 10 s later, but more than 1 in 100 would be visible to users; **0 lost help requests** is the product's core promise (SC5); the L4 goal of **1.5 ×** the peak covers errors in the assumptions above, such as higher adoption.

Scenario	Load	Pass criteria
L1 Smoke	1 virtual user, 1 min, every endpoint	0 errors
L2 End of concert	0 → 1,300 req/s in 3 min, held 10 min. Mix: 85 % announcement polling, 8 % transport, 5 % map and points of interest, 2 % other	errors < 1 %; p95 < 500 ms, p99 < 1.5 s; no container restart; an announcement published during the run reaches the pollers in ≤ 15 s (SC6)
L3 Help during the peak	L2 plus 1 help request/s; an operator polls the backoffice list every 2 s	600 of 600 requests shown (0 lost); time to the backoffice p95 ≤ 5 s (SC2); suggestion p95 ≤ 30 s (SC3)
L4 Stress	From 200 req/s, +200 req/s every 2 min, until p95 > 1 s or errors > 5 %	No pass or fail: records the limit of each service on one instance on the local machine, the evidence for the scaling plan. Goal ≥ 1.5 × the peak (≈ 2,000 req/s)

Scenario	Load	Pass criteria
L5 D1 under load (Should)	L3 with the WireMock *connection dropped* stub on the AI provider	100 % of requests shown as "Not triaged" (SC5)

11. Architectural style and its evolution

Decided: microservices from the start (ADR-001). They meet assignment §6.2 directly, isolate failures (a read peak does not stop help requests) and give small, closed contracts per service.

Evolution (decided 21 October 2026, guide A4; professor's answer): the architecture is defined from the start: the five services are independently deployable from Sprint B, each with its own `main`, Dockerfile and database, which meets the Sprint B requirement of two independently deployable backend components (§6.2); there is **no modular-monolith phase**. There is no imposed path: any change is **motivated by evidence** (measurements, tests, failures), not by the calendar, and is recorded as a new or superseding ADR. For example, scaling a service for the peak (J3) would be recorded with the evidence that motivated it. Those ADRs are the *architecture evolution narrative* of §7.3.2 (item 11). This replaces the proposal of 15 October (modular monolith in Sprints A–B, split in Sprint C). **Note (31 October 2026):** the Sprint A skeleton runs only Reports (#166, ADR-010 point 9); ADR-001 decision 3 was amended for it on 1 November 2026 (#175, record): the first service in Sprint A, all five from Sprint B.

12. Technology choices

Decision	Choice
Backend	**Java 21 + Spring Boot 4.x**, Maven
Frontend	**One React web app** for every user; each page is built for the phone (participants, field teams) or for a laptop (event management, control room). The React Native app was removed on 22 October 2026 (ADR-002, #125)
Database	**PostgreSQL**, one database per service; Flyway
Message broker	**RabbitMQ** with Spring AMQP
Live updates	**Polling**
Resilience	**Resilience4j 2.4** (timeouts, retry, circuit breaker); message retries in Spring AMQP
Simulator of external systems (tests and demo)	**WireMock** (guide J2, J4; confirmed on 26 October 2026, tested with 3.13.2). Recorded Porto Digital responses and their stubs in `infra/wiremock/`
Load testing	**k6**
API	Contract-first **OpenAPI**, `/api/v1`
CI/CD	**GitHub Actions**, monorepo, public repository since 31 October 2026; no registry (ADR-010). Today: one `CI` job (Maven build and tests, image, Compose smoke test) on every PR, `main`, tags and manual runs, plus SonarQube Cloud (#167, #169). Sprint B target: fast checks on PRs, images + Trivy + Compose smoke test on `main` and tags

Decision	Choice
Versioning	**SemVer**, one version for the monorepo, git tag `vX.Y.Z` per sprint; our seven images `farol-<service>:X.Y.Z`, third-party images pinned; no registry (§10)
Architecture views	**C4** (all four levels, level 3 for every service) and **4+1** views without scenarios, drawn in **draw.io**
Deployment	**Local-only scope authorised by the professor, as relayed to the team; cloud omitted due to costs.** One Compose file, one PostgreSQL instance with six databases (five services and Keycloak). ADR-011, `local-delivery.md`; implemented in Sprint B (the Sprint A skeleton runs only Reports, §13)
AI provider and model	**[ACCEPTED architecture; evaluation pending]** Local Ollama + qwen3:4b Q4_K_M selected; hosted assessment retained. Runtime/routing/evaluation contract in local AI decisions, gates in ADR-005
Observability	**[ACCEPTED architecture; implementation pending]** Local Micrometer/Actuator + Prometheus/Grafana (two containers), Grafana-managed alerts and correlated structured Docker logs. No separate trace/collector/alert container. Azure tooling retained only as an alternative. ADR-012
Identity	**Keycloak** (OpenID Connect), self-hosted; Spring Security resource server in every service; `oidc-client-ts` in the web app. ADR-006
API gateway	**Spring Cloud Gateway (Server WebMvc)**, CORS and Bucket4j rate limits per account. ADR-013
Venue map	**[PROPOSED]** The image with an **SVG** overlay for the markers, inside a `react-zoom-pan-pinch` zoom/pan container; normalised `MapPosition` coordinates; the same component on phone and laptop pages (to confirm with a spike on a real iPhone and Android phone before the map is built). PDF maps are rendered to a PNG on upload by the Events service (**Apache PDFBox**). See ADR-007, waits for review

13. Architecture Decision Records

Location and format: `Docs/Technical/adr/NNNN-title.md`, Michael Nygard format plus *Alternatives considered*.

ADR	Decision	Status
ADR-001	Architectural style and evolution (§11)	Accepted
ADR-002	Backend and frontend technology	Accepted
ADR-003	Database and data ownership	Accepted
ADR-004	Service communication: message broker, delivery guarantees, communication only through events	Accepted
ADR-005	AI triage provider and evaluation	Accepted architecture; evaluation/privacy gates pending
ADR-006	Identity and access control	Accepted
ADR-007	Venue map representation	Proposed; rewritten for the web app on 23 October 2026, waits for review

ADR	Decision	Status
ADR-008	Transport data source	Accepted
ADR-009	Architecture documentation (C4, 4+1, draw.io)	Accepted
ADR-010	CI/CD strategy, including versioning (§10, guide H4–H8)	Accepted
ADR-011	Local deployment and cloud alternatives	Local-only delivery authorised; procedure decided 23 October 2026 (`local-delivery.md`); implementation pending
ADR-012	Observability	Accepted architecture; instrumentation and SLO validation pending
ADR-013	API gateway, CORS and rate limiting	Accepted

> **Sprint A skeleton (#58/#166):** one Reports synthetic help-request create/read demo in memory, diagnostic health, HTTP tests, Dockerfile, Compose and CI; run instructions and Sprint B boundary, branch execution evidence. The authenticated multi-service application is Sprint B. #58 was closed on 31 October 2026 after #167; #148 stays open for telemetry implementation.

14. Architecture diagrams

4+1 views without scenarios, drawn in draw.io (ADR-009). Each `.drawio.svg` file renders here and opens in draw.io for editing. The logical view also includes the domain model.

View	Levels
Logical	1 · 2 · 3 · 3 web · 4 reports · 4 transports
Process: help request and triage	1 · 2 · 3 · 4
Process: transport import	1 · 2 · 3 · 4
Implementation (development)	1 · 2 · 3 · 3 frontend · 4 reports · 4 transports
Physical (local delivery)	1 · 2 · 3
Logical → implementation	1 · 2 · 3 · 3 web · 4 reports · 4 transports
Logical → physical	1 · 2 · 3

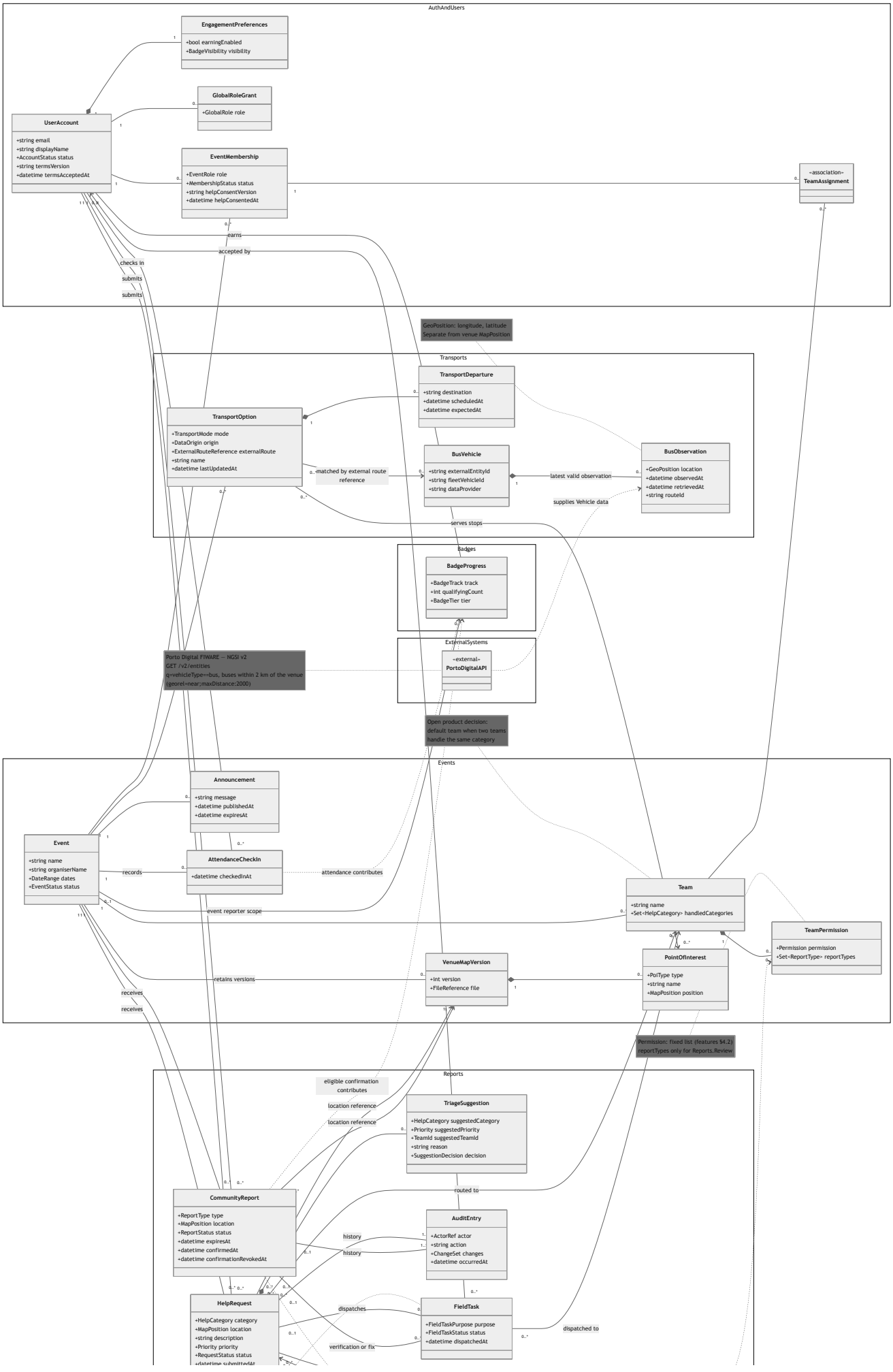
Updated on 26 October 2026 (#133): one web app and no mobile app, the API gateway and Keycloak in the logical, implementation and physical views, and the physical view of the local delivery: one machine, one container per service and one PostgreSQL server with six databases (five services and Keycloak). WireMock appears only in the physical view; logically, Reports and Transports talk to the AI provider and Porto Digital.

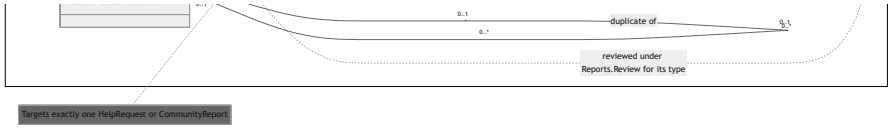
Updated on 27 October 2026 (#133): the web app is opened at level 3 (*3 web*: the level-3 diagram with the web app's components, or `frontend/` 's folders, instead of one box), replacing the level-4 web diagrams; logical

level 3 has one Person; the AI provider is the local Ollama model (qwen3:4b, guide K1), with a hosted provider dashed as the alternative in the physical view; Prometheus/Grafana and event contracts (`contracts/events/` , one JSON Schema per event) without `[PENDING]` ; every diagram centred. The only marker left is the `frontend/` folder layout.

Farol — Domain Model

> Conceptual model organised into five services: **Auth & Users**, **Events**, **Reports**, **Transports** and **Badges**. Service boundaries, attributes and cardinalities are proposed for review.
> **22 October 2026:** roles, teams and permissions follow the role model in `features.md` §4 (to confirm with the team). The draw.io export `domain-model.svg` was redrawn on 31 October 2026 (#170) to match the diagram below: event-defined `Team`, `TeamPermission` and `TeamAssignment`, `GlobalRoleGrant` / `FarolAdmin` and per-event `EventMembership` replace the fixed `ResponseTeam` types. This is documentary alignment only: no new team vote and no implementation. Open point still shown in the diagram: which team is the default when two teams handle the same category. The Mermaid diagram and this file remain the source; the SVG opens and edits in draw.io. **31 October 2026 (#162):** the diagram below gained the help-request text, the AI reason, the e-mail, the consent fields and an optional first membership (API outline D3); `domain-model.svg` does not show them yet, nor the 2 km geo-query of the Porto Digital note (1 November 2026, #175), and must be updated in draw.io.


AuthAndUsers
EngagementPreferences
+bool earningEnabled
+BadgeVisibility visibility
GlobalRoleGrant
+GlobalRole role
UserAccount
+string email
+string displayName
+AccountStatus status
+string termsVersion
+datetime termsAcceptedAt
EventMembership
+EventRole role
+MembershipStatus status
+string helpConsentVersion
+datetime helpConsentedAt
«association»
TeamAssignment
earns
accepted by
checks in
submits
submits
GeoPosition: longitude, latitude
Separate from venue MapPosition
Transports
TransportDeparture
+string destination
+datetime scheduledAt
+datetime expectedAt
TransportOption
+TransportMode mode
-DataOrigin origin
-ExternalRouteReference externalRoute
+string name
+datetime lastUpdatedAt
matched by external route reference
BusVehicle
+string externalEntityId
+string fleetVehicleId
+string dataProvider
latest valid observation
BusObservation
+GeoPosition location
+datetime observedAt
+datetime retrievedAt
+string routeId
supplies Vehicle data
serves stops
Badges
BadgeProgress
+BadgeTrack track
+int qualifyingCount
+BadgeTier tier
ExternalSystems
«external»
PortoDigitalAPI
Porto Digital FIWARE — NGSI v2
GET /v2/entities
q=vehicleType==bus, buses within 2 km of the venue
(georel=near;maxDistance:2000)
Open product decision:
default team when two teams
handle the same category
Events
Announcement
+string message
+datetime publishedAt
+datetime expiresAt
Event
+string name
+string organiserName
+DateRange dates
+EventStatus status
AttendanceCheckIn
+datetime checkedInAt
records
attendance contributes
event reporter scope
Team
+string name
+Set<HelpCategory> handledCategories
TeamPermission
+Permission permission
+Set<ReportType> reportTypes
PointOfInterest
+PoiType type
+string name
+MapPosition position
VenueMapVersion
+int version
+FileReference file
retains versions
receives
receives
Permission: fixed list (features §4.2)
reportTypes only for Reports.Review
Reports
eligible confirmation contributes
location reference
location reference
TriageSuggestion
+HelpCategory suggestedCategory
+Priority suggestedPriority
+TeamId suggestedTeamId
+string reason
+SuggestionDecision decision
routed to
CommunityReport
+ReportType type
+MapPosition location
+ReportStatus status
+datetime expiresAt
+datetime confirmedAt
+datetime confirmationRevokedAt
history
history
AuditEntry
+ActorRef actor
+string action
+ChangeSet changes
+datetime occurredAt
HelpRequest
+HelpCategory category
+MapPosition location
+string description
+Priority priority
+RequestStatus status
+datetime submittedAt
dispatches
verification or fix
FieldTask
+FieldTaskPurpose purpose
+FieldTaskStatus status
+datetime dispatchedAt
dispatched to



Entities and concepts

Service / area	Entity or concept	Purpose and main rule
Auth & Users	**UserAccount**	One account across events for participants and event teams. Holds the e-mail (login name, also kept by Keycloak; assessment §3) and the display name only (NFR-19, assessment M-6). **[PROPOSED]** (assessment §8.4): the version and time of the accepted terms. What a user can do in an event comes from their membership of that event and the teams they are in, not from simply having an account.
Auth & Users	**EngagementPreferences**	The account's badge earning and visibility preferences. Badges are private by default; earning is optional.
Auth & Users	**GlobalRoleGrant**	Grants the only platform-wide role, `FarolAdmin` (persona P4): creates events for verified organisations, names EventAdmins, suspends and reactivates events. It does not grant access to any event's help requests, reports or participants.
Auth & Users	**EventMembership**	Links an account to one event with a role (`EventAdmin` or `RegularUser`) and a membership status. Created as `RegularUser` when the user joins the event (US-12), or as `EventAdmin` when the FarolAdmin names them (US-06, US-08). The same account can be EventAdmin in one event and RegularUser in another. Once the first EventAdmin is named, the event keeps at least one active EventAdmin: the last one cannot be removed or demoted. A new event has no membership until then (API outline D3 = B). **[PROPOSED]** (assessment §8.3): the version and time of the help-request consent given when joining.
Auth & Users	**TeamAssignment**	Puts a member of an event in one of that event's teams. Only members of the event can be assigned (US-18). A member can be in several teams and gets the permissions of all of them.
Events	**Event**	A festival edition with its own organiser, dates, status and operational data. Defines the isolation boundary for event-specific information.
Events	**Team**	A team created by an EventAdmin for one event (e.g. *Control room*, *Nursing*, *Security*), with a name unique in the event and the help categories it handles. The handled categories give the default team when a request is routed and the teams the AI may suggest. Deleting a team returns the requests routed to it to the control-room queue (US-16). Which team is the default when two teams handle the same category is still open.
Events	**TeamPermission**	One permission given to a team, from the fixed list in features §4.2: `HelpRequests.Triage` , `HelpRequests.Respond` , `Reports.Review` , `Reports.Respond` , `Announcements.Publish` , `Map.Edit` , `Transport.Edit` , `Entrances.Update` . For `Reports.Review` it also lists the report types the team reviews. A change applies at the member's next action and is audited.

Service / area	Entity or concept	Purpose and main rule
Events	**VenueMapVersion**	A version of the uploaded venue image/PDF. Historical versions remain available for the requests and reports that refer to them. Uploaded by an EventAdmin or a member with `Map.Edit`.
Events	**PointOfInterest**	A named place on a map version, such as an entrance, stage, WC, first-aid point or transport stop. Edited by an EventAdmin or a member with `Map.Edit`.
Events	**Announcement**	Official event information published by an EventAdmin or a member with `Announcements.Publish`, with publication time and optional expiry.
Events	**AttendanceCheckIn**	Records an authenticated participant's attendance at the venue during the event dates. Counts once per account/event; included as a Should-have feature.
Reports	**HelpRequest**	A participant's request for assistance, with category, map location, an optional text of at most 500 characters (assessment M-2), priority and status. Goes to the shared queue of the members with `HelpRequests.Triage` (the control room), who route it to a team with `HelpRequests.Respond`; it never earns badges.
Reports	**TriageSuggestion**	Optional AI recommendation attached to a help request, with a short reason shown to the operator (ADR-005 decision 2). The suggested team is one of the event's teams that handles the category. A member with `HelpRequests.Triage` accepts or changes it; it never overwrites a human decision or blocks the request.
Reports	**CommunityReport**	A report about a place or service, routed by type to the members with `Reports.Review` for that type. Visible to other participants only while confirmed and active.
Reports	**FieldTask**	Work dispatched to a team to assist with a request (team with `HelpRequests.Respond`), or to verify or fix a report (team with `Reports.Respond`). Targets exactly one request or report; only one member of that team can accept it.
Cross-cutting; shown in Reports	**AuditEntry**	Records the actor, time and previous/new values of a change. Each service owns its own audit history; the diagram shows request/report history as the operational example.
Transports	**TransportOption**	A transport service relevant to an event: manual shuttle or an external service reference. Can serve several stops and match several observed buses.
Transports	**TransportDeparture**	A scheduled or expected departure associated with a transport option. Requires manual data or another validated source; the bus-vehicle endpoint does not supply departure times.
Transports	**BusVehicle**	A bus identified by its external entity ID and provider. Shared external transport data, rather than an event-owned vehicle.
Transports	**BusObservation**	The latest valid bus position, route reference and observation/retrieval timestamps. A fresh retrieval does not imply a fresh observation.

Service / area	Entity or concept	Purpose and main rule
Badges	**BadgeProgress**	Derived account progress for ReportHistory, EventReporter or EventAttendance. Confirmed reports count after expiry/resolution; withdrawing confirmation removes their credit.
External system	**PortoDigitalAPI**	Porto Digital FIWARE bus API, supplying NGSI v2 vehicle observations. Provides bus context, not timetables or predicted arrivals.

How to read the model

- **Scope:** event-specific references must resolve within the same event. Accounts, account-wide badge progress and external bus observations can span events. Associations across services express conceptual references, not shared database tables.
- **Roles and permissions:** a member's permissions in an event are those of the teams they are in (TeamAssignment → Team → TeamPermission). An `EventAdmin` has every permission in their event without being in a team; a `RegularUser` in no team has only the participant features. `FarolAdmin` is outside every event and has no event permission. Memberships and team assignments live in Auth & Users, teams and their permissions in Events.
- **Locations:** `MapPosition` uses normalised coordinates on a specific venue-map version. `GeoPosition` uses geographic longitude/latitude for buses. They are different coordinate systems.
- **Cardinalities:** `1` means exactly one; `0..1` means optional; `0..*` means zero or more; `1..*` means one or more. The optional Event association on BadgeProgress applies only to the EventReporter track.
- **Notation:** a filled diamond denotes composition; a plain association connects concepts; a dashed arrow expresses a dependency or contribution. Namespace boxes group concepts by service; ExternalSystems contains the external provider.

Farol — API Outline and Event Catalogue

> Sprint A deliverable 7 (assignment §7.1.2; guide C8). Issue #123.
> What each service exposes over REST and which domain events it publishes and consumes. This is an **outline**: the complete `openapi.yaml` of each service and the formal event schemas are written per story, contract-first (guide C7, `spec-driven-development.md`).
> Stories follow the numbering in `features.md` §7 (US-01 to US-62). Entities follow `domain-model.md`. Roles and permissions follow `features.md` §4.
> Items marked **[WAITS: On]** follow the current proposal and wait for another owner's decision On. The outline's own decisions and those still waiting are in §10.
> Aligned on 26 October 2026 with ADR-006 (identity and access), ADR-013 (gateway, rate limits) and the Security and Privacy Assessment (AC-9, M-5).
> 27 October 2026: account deletion (guide D7, #135) in §1.4, §2 and §7.4.

1. Scope and conventions

1.1 Scope

- **Must** and **Should** features (F1–F10, F12–F16). The **Could** features (F11, F17, F18) are listed by name in §9.
- Per endpoint: method, path, purpose, the role or permission required, and the story. Request and response fields only for the core flow (help request, triage, field tasks).
- No service calls another service's REST API: services talk only through events (guide C4, ADR-004).
- Not in this outline: the endpoints of the Sprint A walking skeleton in Reports (`GET /api/v1/status` and the synthetic `/api/v1/demo/help-requests`, only with the `demo` profile). They are diagnostic, not part of the product API (`openapi.yaml`, run instructions, #166).

1.2 Paths

- Every path starts with `/api/v1` (guide C7).
- Paths are **per resource**, with no service prefix: `/api/v1/events/{eventId}/help-requests`. Each path belongs to exactly one service; the gateway routes by path, following the route table of ADR-013 (guide C6). A new path must fit that table, or the table changes in the same PR.
- Anything that belongs to one event has `{eventId}` in its path, so every service can check membership and filter by event (guide D3, US-14).
- Resource names are plural and kebab-case, and match the entity names in `domain-model.md`.

1.3 Methods and state changes

- `GET` reads, `POST` creates, `PATCH` changes fields, `DELETE` removes.
- A change of state is an action: **`POST /{id}/<action>`** (e.g. `/publish`, `/close`, `/confirm`, `/accept`). Each action is one use case and, where another service needs it, one domain event.

1.4 Identity

- Login, logout and sign-up happen in Keycloak; Auth & Users has no `/login`, `/logout` or sign-up endpoint (ADR-006, guide E1).
- The account is created **just in time**, at the user's first authenticated `GET /me`; Auth & Users then publishes `UserProfileChanged` (§7.4). Never for a deleted `sub` (tombstone, §2): a token issued before the deletion gets `401`.
- `/me` stands for the account of the token's user. A user's `userId` is the token's `sub`.

1.5 Authorisation

- Checked **per event**: the user's role in that event (`EventAdmin` or `RegularUser`) and the permissions of their teams there. `FarolAdmin` is global and gives no access to any event's data.
- Each service decides from its own local copies: memberships and team assignments from Auth & Users (`MembershipChanged`), teams and their permissions from Events (`TeamChanged`). The token carries **only the identity** (`sub`, `email`, `name`, `aud = farol-api`); every service validates it and checks the permission in its use case (ADR-006, guide E2). A disabled or deleted account is refused at once (`UserStatusChanged`, `UserDeleted`).
- The tables use: **Authenticated** (any logged-in user), **Member** (any member of the event), **EventAdmin**, **FarolAdmin**, or a permission such as `HelpRequests.Triage`. An `EventAdmin` has every permission of their event.

1.6 Live lists and pagination

- Live lists (backoffice, request status, announcements, reports on the map) are polled. They take `updatedSince` (ISO-8601 timestamp) and return only what changed, plus the `serverTime` to send next time.
- Other lists take `limit` (default 50, max 200) and `cursor`.

1.7 Errors

- **Problem Details** (RFC 9457), `application/problem+json`.
- **Rate limits** (ADR-013), per account, never per IP: 120 calls per minute on every path and **5 help requests per 10 minutes**, at the gateway; 5 reports per 10 minutes per event, in Reports (US-48). Login and sign-up are protected by Keycloak's brute-force detection.
- `400` invalid input, `401` not logged in, `403` no permission, `404` not found **or not a member of the event** (so as not to reveal that the event exists), `409` state conflict (e.g. a field task already accepted), `422` breaks a business rule (e.g. removing the last EventAdmin), `413` file too large (map upload over 20 MB), `429` too many requests, with `Retry-After`, `503` service unavailable, with `Retry-After`.

2. Auth & Users

Stories: US-01 to US-05, US-08, US-12, US-15, US-18.

Owns accounts, the `FarolAdmin` grant, memberships of events and team assignments. Keeps local copies of the events (status, name) and their teams (`EventChanged` , `TeamChanged`).

Method	Path	Purpose	Requires	Story
—	*identity provider*	Sign up, log in and log out happen in Keycloak; the web app sends its token on every call (§1.4). Sign-up asks for the terms on Keycloak's form	—	US-01, US-02, US-03
`GET`	`/me`	My account: name, e-mail and global roles (`FarolAdmin`), so the web app knows what to show. The first call creates the account (§1.4)	Authenticated	US-01, US-03
`GET`	`/me/memberships`	The events I am a member of, with my role, my teams and each event's name and status	Authenticated	US-01
`PATCH`	`/me`	Change my display name. The password changes in Keycloak	Authenticated	US-04
`DELETE`	`/me`	Delete my account; `422` if I am the last EventAdmin of an event or the last `FarolAdmin` . `202` while the deletion is pending (flow below)	Authenticated	US-05
`GET`	`/accounts?email=`	Find the account to name as EventAdmin: exact match only, no partial search and no list; answers only the account id and display name; audited (assessment AC-9)	FarolAdmin	US-06, US-08
`POST`	`/events/{eventId}/members`	Name a user EventAdmin of an event (`{ userId, role: EventAdmin }`), the first one or a new one; recorded with who and when	FarolAdmin	US-06, US-08

Method	Path	Purpose	Requires	Story
`POST`	`/events/{eventId}/members/me`	Join the event as RegularUser (QR code or link); `404` if the event is not published in the local copy	Authenticated	US-12
`GET`	`/events/{eventId}/members`	List the event's members with role and teams; filters `role`, `teamId`	EventAdmin	US-15, US-18
`PATCH`	`/events/{eventId}/members/{userId}`	Make a member EventAdmin or RegularUser (`{ role }`); `422` for the last EventAdmin; recorded with who and when	EventAdmin	US-15
`PUT`	`/events/{eventId}/members/{userId}/teams/{teamId}`	Add a member to a team; `404` if the user is not a member or the team is not in the local copy	EventAdmin	US-18
`DELETE`	`/events/{eventId}/members/{userId}/teams/{teamId}`	Remove a member from a team	EventAdmin	US-18

- The first EventAdmin of a new event (US-06) is named with the same `POST /events/{eventId}/members`, right after the event is created in Events (D3). The event reaches Auth & Users through `EventChanged`; until then the call returns `404` and the web app retries. If it never succeeds, the event shows no EventAdmin in the FarolAdmin's list (US-07), who names one there (US-08).
- E-mails are not verified (ADR-006), so before naming an account EventAdmin the FarolAdmin confirms with the organisation, outside Farol, that it is theirs (assessment AC-9).
- **Account deletion** (ADR-006, guide D7, #135). In the MVP: US-05 is Must and the GDPR gives the right to erasure.
  i. Refused with `422` while the user is the last EventAdmin of an event (`features.md` §4.1; they name another first, US-15, or the FarolAdmin does, US-08), or the last `FarolAdmin`. Checked in the same transaction that changes memberships, so an event is never left without an EventAdmin.
  ii. Otherwise Auth & Users marks the account *pending deletion*, refuses it, and publishes `UserStatusChanged` (`Disabled`), so every service refuses it at once.
  iii. Deletes the user in Keycloak (Admin API, client credentials), which also ends its sessions. If Keycloak is down, answers `202`, and a scheduled job retries.
  iv. In one transaction: deletes the account, the `FarolAdmin` grant, memberships, team assignments, friendships (both sides) and consent records; keeps a **tombstone** (only the `sub` and `deletedAt`); writes `UserDeleted` to the outbox. The audit records only that a deletion happened, and when (assessment AC-10).
  v. Each consumer deletes or anonymises what it keeps (§7.4) and keeps its own tombstone.

- **Tombstone:** Keycloak's `sub` is a UUID that is never reused and, once the account is gone, points to no one. It refuses a token issued before the deletion (still valid for up to 5 min) and makes late events about that `sub` (e.g. a retried `MembershipChanged`) be ignored, so no deleted copy comes back.
- Publishes `MembershipChanged`, `GlobalRoleChanged`, `UserProfileChanged`, `UserStatusChanged` and `UserDeleted` (§7).

3. Events

Stories: US-06, US-07, US-09 to US-11, US-13, US-16, US-17, US-19 to US-22, US-44, US-45, US-58.

Owns events, teams with their permissions and help categories, map versions and points of interest, announcements and check-ins. Keeps local copies of the memberships and team assignments (`MembershipChanged`) and of the `FarolAdmin` grants (`GlobalRoleChanged`).

Events

Method	Path	Purpose	Requires	Story
`POST`	`/events`	Create an event as a draft (`{ name, organiserName, dates }`); the FarolAdmin then names its first EventAdmin in Auth & Users (§2)	FarolAdmin	US-06
`GET`	`/events`	List every event with its name, dates, organisation, EventAdmins and status only	FarolAdmin	US-07
`GET`	`/events/{eventId}`	The event's details, status and, once published, its join link (the QR code encodes it)	Member, EventAdmin	US-10, US-11, US-12
`PATCH`	`/events/{eventId}`	Edit the name, dates or description of a draft or published event	EventAdmin	US-10
`POST`	`/events/{eventId}/publish`	Publish a draft; generates the join link	EventAdmin	US-11
`POST`	`/events/{eventId}/close`	Close the event: no more help requests or reports	EventAdmin	US-13
`POST`	`/events/{eventId}/suspend`	Suspend the event: participants can no longer open it	FarolAdmin	US-09
`POST`	`/events/{eventId}/reactivate`	Return a suspended event to its previous status	FarolAdmin	US-09

Teams and permissions

Method	Path	Purpose	Requires	Story
`GET`	`/events/{eventId}/teams`	The event's teams with their permissions and help categories	EventAdmin, `HelpRequests.Triage`, `Reports.Review`	US-16, US-29, US-52
`POST`	`/events/{eventId}/teams`	Create a team (`{ name }`), with no members and no permissions; `409` if the name is taken	EventAdmin	US-16
`PATCH`	`/events/{eventId}/teams/{teamId}`	Rename a team or set the help categories it handles (`{ name, handledCategories }`)	EventAdmin	US-16, US-17
`PUT`	`/events/{eventId}/teams/{teamId}/permissions`	Replace the team's permissions (`[{ permission, reportTypes }]` , from the fixed list in `features.md` §4.2); recorded with who and when	EventAdmin	US-17
`DELETE`	`/events/{eventId}/teams/{teamId}`	Delete a team; Reports returns its routed requests to the control-room queue when it consumes `TeamChanged`	EventAdmin	US-16

Venue map

Method	Path	Purpose	Requires	Story
`POST`	`/events/{eventId}/map-versions`	Upload a map (multipart, image or single-page PDF); a PDF is rendered to PNG (ADR-007); `400` for any other file and `413` above 20 MB (ADR-013 L-4), and the current map stays	`Map.Edit`	US-19
`GET`	`/events/{eventId}/map-versions/current`	The current map version with every point of interest (type, name, position)	Member	US-21, US-22

Method	Path	Purpose	Requires	Story
`GET`	`/events/{eventId}/map-versions/{version}`	An older version, for the requests and reports that refer to it	Member	US-27
`GET`	`/events/{eventId}/map-versions/{version}/image`	The map's PNG	Member	US-21
`POST`	`/events/{eventId}/map-versions/{version}/points-of-interest`	Place a point (`{ type, name, position }`)	`Map.Edit` (`Transport.Edit` for a transport stop, §5)	US-20, US-37
`PATCH`	`/events/{eventId}/map-versions/{version}/points-of-interest/{poiId}`	Move or rename a point	`Map.Edit`	US-20
`DELETE`	`/events/{eventId}/map-versions/{version}/points-of-interest/{poiId}`	Remove a point	`Map.Edit`	US-20

- Filtering by type of place (US-22) happens in the app, over the points the current version already returns.
- A new map version starts with a copy of the previous version's points, which the team then moves.
- The files stay in a volume of the Events service and their references in its database (ADR-011, local deployment).

Announcements and check-in

Method	Path	Purpose	Requires	Story
`GET`	`/events/{eventId}/announcements?updatedSince=`	Live list of announcements; removed ones come back once as `removed: true`, so the web app drops them (polling every 10 s; SC6 ≤ 15 s)	Member	US-44, US-45
`POST`	`/events/{eventId}/announcements`	Publish an announcement (`{ message, expiresAt }`)	`Announcements.Publish`	US-44
`PATCH`	`/events/{eventId}/announcements/{announcementId}`	Edit it; it shows the time of the edit	`Announcements.Publish`	US-45
`DELETE`	`/events/{eventId}/announcements/{announcementId}`	Remove it	`Announcements.Publish`	US-45

Method	Path	Purpose	Requires	Story
`POST`	`/events/{eventId}/check-ins/me`	Check in with the venue QR code (`{ venueCode }`); a second check-in changes nothing; `422` if the event is not running	Member	US-58

- The venue QR code is not the join link: it carries a code shown only at the venue, so nobody checks in from home.
- Publishes `EventChanged` , `TeamChanged` , `VenueMapVersionPublished` , `PointOfInterestChanged` and `CheckInRecorded` (§7).

4. Reports

Stories: US-23 to US-36, US-47 to US-53.

Owns help requests, triage suggestions, community reports, field tasks and their audit history. Keeps local copies of the memberships and team assignments (`MembershipChanged`), the teams with their permissions, help categories and report types (`TeamChanged`), the event status (`EventChanged`) and the map versions (`VenueMapVersionPublished`). The AI triage runs inside Reports, through a queue (§7).

Help requests

Method	Path	Purpose	Requires	Story
`POST`	`/events/{eventId}/help-requests`	Ask for help with a category and a point on the map (§4.1); `422` if the event is closed or suspended. Sending the same `clientRequestId` again returns the request already saved, so the app can retry safely. After 5 in 10 minutes the gateway answers `429` with `Retry-After` (ADR-013 L-2)	Member	US-23, US-25
`GET`	`/events/{eventId}/help-requests?updatedSince=`	Live list. Each caller sees only what they may: their own requests (`mine=true`), all of the event's (`HelpRequests.Triage`), or those routed to their teams (`HelpRequests.Respond`). Filters `status` , `teamId`	Member	US-24, US-26, US-27, US-31

Method	Path	Purpose	Requires	Story
`GET`	`/events/{eventId}/help-requests/{requestId}`	One request. The participant sees only its status and the time of the last change; the control room also sees the triage suggestion and the default team	Owner, `HelpRequests.Triage`, routed team	US-24, US-29, US-34
`PATCH`	`/events/{eventId}/help-requests/{requestId}`	Set the priority (`{ priority }`); recorded with who, when, old and new value	`HelpRequests.Triage`	US-28
`POST`	`/events/{eventId}/help-requests/{requestId}/acknowledge`	Mark it *Seen*	`HelpRequests.Triage`	US-28
`POST`	`/events/{eventId}/help-requests/{requestId}/route`	Route it to a team (`{ teamId }`); creates a field task; `422` if the team has no `HelpRequests.Respond`	`HelpRequests.Triage`	US-29
`POST`	`/events/{eventId}/help-requests/{requestId}/accept-suggestion`	Take the AI's category, priority and team as the request's values; recorded as the member's decision	`HelpRequests.Triage`	US-35
`POST`	`/events/{eventId}/help-requests/{requestId}/close`	Close it: it leaves the active list	`HelpRequests.Triage`	US-30
`GET`	`/events/{eventId}/help-requests/{requestId}/history`	Full change history (who, when, old and new values)	`HelpRequests.Triage`	US-28
`GET`	`/events/{eventId}/triage-status`	Whether the AI triage is available, for the banner of scenario D1	`HelpRequests.Triage`	US-36

Community reports

Method	Path	Purpose	Requires	Story
`POST`	`/events/{eventId}/reports`	Report a problem (`{ type, location, clientRequestId }`), saved as *Pending*; `429` with `Retry-After` when the participant reports too often	Member	US-47, US-48
`GET`	`/events/{eventId}/reports?updatedSince=`	Live list. Participants see the confirmed, active reports without the reporter, and their own; reviewers see every report of the types they review	Member	US-47, US-49, US-50

Method	Path	Purpose	Requires	Story
`POST`	`/events/{eventId}/reports/{reportId}/confirm`	Confirm it: it goes on the map and counts for badges	`Reports.Review` for its type	US-50
`POST`	`/events/{eventId}/reports/{reportId}/reject`	Reject it	`Reports.Review` for its type	US-50
`POST`	`/events/{eventId}/reports/{reportId}/unconfirm`	Withdraw the confirmation: it leaves the map and stops counting; recorded with who and when	`Reports.Review` for its type	US-51
`POST`	`/events/{eventId}/reports/{reportId}/dispatch`	Send it to a team to check or fix (`{ teamId, purpose: Verify | Fix }`); creates a field task; `422` if the team has no `Reports.Respond`	`Reports.Review` for its type	US-52, US-53

- Reports expire on their own, after the time of their type (`features.md` §6.1); there is no endpoint for it.

Field tasks

Method	Path	Purpose	Requires	Story
`GET`	`/events/{eventId}/field-tasks?updatedSince=`	Live list of the tasks of my teams, each with its request or report and its point on the map	`HelpRequests.Respond` , `Reports.Respond`	US-31, US-52
`POST`	`/events/{eventId}/field-tasks/{taskId}/accept`	Accept a task; the help request becomes *On the way*; `409` if another member already accepted it	Member of the task's team	US-32
`POST`	`/events/{eventId}/field-tasks/{taskId}/answer`	Answer a *Verify* task (`{ found: true | false }`); the answer goes back to the reviewer	Member who accepted it	US-52
`POST`	`/events/{eventId}/field-tasks/{taskId}/resolve`	Mark the help request, or the report of a *Fix* task, resolved	Member who accepted it	US-33, US-53

- Publishes `HelpRequestSubmitted` (for its own triage consumer), `ReportConfirmed` and `ReportUnconfirmed` (§7).

4.1 Core flow: request and response fields

Ask for help: `POST /events/{eventId}/help-requests`

```
{
  "clientRequestId": "6f1c…",          // generated by the app; makes a retry safe (US-25)
  "category": "Health",                // Health | Safety | LostPerson | Theft | Other
  "location": { "mapVersion": 3, "x": 0.42, "y": 0.77 },  // normalised position on that map version
  "description": "Friend fainted near the bar"            // optional
}
```

A help request, as the control room sees it: `GET /events/{eventId}/help-requests/{requestId}`

```
{
  "id": "hr-81…",
  "eventId": "ev-12…",
  "category": "Health",
  "location": { "mapVersion": 3, "x": 0.42, "y": 0.77 },
  "description": "Friend fainted near the bar",
  "priority": "High",                  // scale: features §5.2, AI-3 [WAITS: O7]
  "status": "Submitted",               // Submitted | Seen | OnTheWay | Resolved | Closed
  "submittedAt": "2027-05-08T23:14:05Z",
  "updatedAt": "2027-05-08T23:14:05Z",
  "defaultTeamId": "tm-03…",           // the team that handles the category (US-29)
  "routedTeamId": null,
  "triage": {
    "state": "Suggested",              // Pending | Suggested | NotTriaged | Accepted | Changed
    "suggestedCategory": "Health",
    "suggestedPriority": "High",
    "suggestedTeamId": "tm-03…",
    "suggestedReason": "Fainting reported; needs first aid",  // short plain text from the AI, escaped
    "notTriagedReason": null           // Timeout | ProviderError | InvalidOutput | Abstained | Unavail
  }
}
```

- The participant gets only `id`, `category`, `status` and `updatedAt`, which the app shows as *Sent*, *Seen*, *On the way*, *Resolved* or *Closed* (US-24).
- `Unavailable`: the AI was not called because the circuit was open or the request could not start in time (AI plan, *Admission*). `NoEligibleTeam`: no team of the event can respond to the category, so inference is skipped (AI plan, item 3). Added on 31 October 2026 (#162).
- The request is saved and shown at once, with `triage.state: Pending`. If no valid suggestion arrives within 30 s it becomes `NotTriaged` (scenario D1, US-36). A suggestion never overwrites a value a member already set, and never lowers the default priority (NFR-22).

A field task: `GET /events/{eventId}/field-tasks?updatedSince=`

```
{
  "items": [
    {
      "id": "ft-55…",
      "purpose": "Assist",            // Assist (help request) | Verify | Fix (report)
      "status": "Open",               // Open | Accepted | Answered | Resolved | Cancelled
      "teamId": "tm-03…",
      "target": { "kind": "HelpRequest", "id": "hr-81…", "category": "Health",
                  "location": { "mapVersion": 3, "x": 0.42, "y": 0.77 } },
      "acceptedBy": null,
      "dispatchedAt": "2027-05-08T23:15:10Z",
      "updatedAt": "2027-05-08T23:15:10Z"
    }
  ],
  "serverTime": "2027-05-08T23:15:12Z"
}
```

- A field task targets exactly one help request or report. `Cancelled` covers a team deleted while the task was open: the request goes back to the control-room queue (US-16).

5. Transports

Stories: US-37 to US-43.

Owns the event's transport options (shuttles and external bus routes), their departures, and the bus observations imported from Porto Digital (ADR-008). Keeps local copies of the memberships and team assignments (`MembershipChanged`), the teams' permissions (`TeamChanged`), the event status (`EventChanged`) and the transport stops (`PointOfInterestChanged`).

Method	Path	Purpose	Requires	Story
`GET`	`/events/{eventId}/transport-options`	The event's shuttles with their stops, next departures and status (delayed or cancelled, with the time of the change), and the external bus routes	Member	US-39
`POST`	`/events/{eventId}/transport-options`	Add a shuttle (`{ mode: Shuttle, name, stopPoiIds, departures }`) or an external bus route (`{ mode: ExternalBus, externalRoute, stopPoiIds }`)	`Transport.Edit`	US-37, US-41
`PATCH`	`/events/{eventId}/transport-options/{optionId}`	Change its name, stops or departures	`Transport.Edit`	US-37
`DELETE`	`/events/{eventId}/transport-options/{optionId}`	Remove it	`Transport.Edit`	US-37
`POST`	`/events/{eventId}/transport-options/{optionId}/delay`	Mark a shuttle delayed (`{ delayMinutes, note }`); `0` clears it; stored with the time of the change	`Transport.Edit`	US-38

Method	Path	Purpose	Requires	Story
`POST`	`/events/{eventId}/transport-options/{optionId}/cancel`	Mark a shuttle cancelled	`Transport.Edit`	US-38
`GET`	`/events/{eventId}/bus-observations`	Latest bus positions on the event's external routes, with `dataAsOf` (time of the data) and `stale: true` when the source is down or the data is older than the freshness limit	Member	US-40, US-41, US-42
`GET`	`/events/{eventId}/transport-source-status`	Whether the Porto Digital source is available, and the time of the last valid import, for the backoffice warning	EventAdmin, `Transport.Edit`	US-40, US-43

- The import from Porto Digital (US-40) is a scheduled job inside Transports, with a timeout and validation; it is never called inside a user's request, and there is no endpoint for it. An invalid import is rejected as a whole and the previous data stays.
- Shuttles are internal data: they are shown even when the source is down (scenario D2).
- Porto Digital gives bus positions, not timetables (domain model, `TransportDeparture`). US-41 ("next regular buses") can only show the buses on their way; departure times come from the shuttles entered by the team. **[WAITS: O6]**
- Stops are points of interest of type *transport stop* in Events, so a member with `Transport.Edit` can place them (US-37): Events accepts `Transport.Edit` for that type of point.
- Publishes nothing.

6. Badges

Stories: US-54 to US-57.

Owns each account's badge progress and its badge preferences. Keeps what it needs from the confirmed reports (`ReportConfirmed` , `ReportUnconfirmed`), the check-ins (`CheckInRecorded`) and the events' names (`EventChanged`).

Method	Path	Purpose	Requires	Story
`GET`	`/me/badges`	My badges: *Report history*, *Event attendance* and one *Event reporter* per event, each with its count and tier	Authenticated	US-54, US-55, US-57
`GET`	`/me/badges/preferences`	Whether I earn badges and whether others can see them	Authenticated	US-56
`PUT`	`/me/badges/preferences`	Change them (`{ earningEnabled, visibility }`); while earning is off, no new badges are earned	Authenticated	US-56

- The counts and tiers are computed from the events above; there is no endpoint to change them. Unconfirming a report lowers the count and recalculates the tier (US-54, US-55).

- The preferences move from Auth & Users (`EngagementPreferences` in the domain model) to Badges, so that Badges decides on its own. **[WAITS: O5]** If they stay in Auth & Users, Auth & Users publishes `EngagementPreferencesChanged` and the two `/me/badges/preferences` endpoints move to `/me/engagement-preferences`.
- There is no endpoint for another user's badges: no Must or Should story shows them (D9). It comes with the story that needs it, e.g. friends (F18), and then respects `visibility`.
- Publishes nothing.

7. Event catalogue

7.1 Envelope

Every message carries:

Field	Meaning
`messageId`	Unique id; consumers keep the processed ids to ignore duplicates (`architecture.md` §6)
`type`	Event name, e.g. `HelpRequestSubmitted`
`schemaVersion`	Version of the payload schema
`occurredAt`	When it happened (ISO-8601, UTC)
`producer`	The service that published it
`eventId`	The Farol event it belongs to, when there is one
`actorUserId`	The `sub` of the user whose action caused it, for audit (ADR-006); empty when the system caused it (a scheduled job, an expiry)
`payload`	The event's data (§7.3)

7.2 Topology (RabbitMQ)

- **One topic exchange per producer:** `farol.<service>` (e.g. `farol.reports`).
- **Routing key:** `<aggregate>.<past-tense-verb>` (e.g. `help-request.submitted`).
- **Event name:** PascalCase, past tense (e.g. `HelpRequestSubmitted`).
- **One queue per consumer and producer:** `<consumer>.<producer>-events` (e.g. `badges.reports-events`), with its dead-letter queue.

7.3 Payload rule

- **Full state** (event-carried state transfer): the payload has the whole current state the consumers need, with a `version`.
- The consumer upserts its local copy and ignores any version older than the one it has.
- **One exception:** `HelpRequestSubmitted` carries only `requestId`. Its producer and consumer are both Reports, so the triage worker reads the request from Reports' own database, and no help-request text or location sits in RabbitMQ, a dead-letter queue or a message trace (assessment M-5, T11).

7.4 Events

Every payload except `HelpRequestSubmitted` also carries `version` (§7.3). `eventId` is in the envelope.

Event · routing key	Producer	Consumers (why)	Payload
`MembershipChanged` · `membership.changed`	Auth & Users	**Events**, **Reports**, **Transports**: decide who may call each endpoint of an event (role and teams, §1.5) and refuse non-members (US-14); Events also lists each event's EventAdmins (US-07)	`userId`, `role` (`EventAdmin` | `RegularUser`), `status` (`Active` | `Removed`), `teamIds`
`GlobalRoleChanged` · `global-role.changed`	Auth & Users	**Events**: only the FarolAdmin creates, lists, suspends and reactivates events (US-06, US-07, US-09)	`userId`, `role` (`FarolAdmin`), `granted`
`UserProfileChanged` · `user-profile.changed`	Auth & Users	**Reports**: the name of who accepted a field task, in the backoffice; **Events**: the names of the EventAdmins (US-07). Without it, both show only the `userId`. Also published when the account is created (§1.4)	`userId`, `displayName`
`UserStatusChanged` · `user-status.changed`	Auth & Users	**Events**, **Reports**, **Transports**, **Badges**: refuse a disabled account at once, without waiting for its token to expire (ADR-006; assessment AC-6). Also published with `Disabled` when the user asks to delete the account (§2)	`userId`, `status` (`Active` | `Disabled`)
`UserDeleted` · `user.deleted`	Auth & Users	Every consumer keeps a tombstone of the `userId`, refuses it and ignores later events about it (§2), then: **Reports**: closes the user's open help requests (system action, empty `actorUserId`) and cancels their open field tasks; removes the requester from their help requests and the reporter from their reports, and deletes their text and map point, so only category or type, priority, status, times and team stay, as statistics; in audit entries, `acceptedBy` and field tasks, keeps the opaque `userId` and deletes the copied display name (the backoffice shows *Deleted user*); deletes the membership copies. **Events**: deletes the user's check-ins and the copies of memberships, `FarolAdmin` grant and display name; audit entries keep the opaque `userId`. **Transports**: deletes the membership copies. **Badges**: deletes progress, preferences and the copies of reports and check-ins. Published by Auth & Users after Keycloak and its own data are deleted (§2; guide D7)	`userId`, `deletedAt`

Event · routing key	Producer	Consumers (why)	Payload
`EventChanged` · `event.changed`	Events	**Auth & Users**: knows the event exists before its first EventAdmin is named (US-06) and accepts joins only to published events (US-12); **Reports**: refuses help requests and reports in a closed or suspended event (US-13, US-09); **Transports**: refuses a suspended event; **Badges**: the event's name on the *Event reporter* badge (US-55)	`name`, `organiserName`, `dates`, `status` (`Draft` | `Published` | `Closed` | `Suspended`)
`TeamChanged` · `team.changed`	Events	**Auth & Users**: a member can only be put in a team of the event (US-18); **Reports**: who triages, reviews and responds, the default team for a category (US-29), the teams the AI may suggest (US-34), and the requests to return to the queue when a team is deleted (US-16); **Transports**: who has `Transport.Edit` (US-37)	`teamId`, `name`, `permissions` (`[{ permission, reportTypes }]`), `handledCategories`, `deleted`
`VenueMapVersionPublished` · `venue-map-version.published`	Events	**Reports**: a help request or report must point to an existing map version of the event (US-23, US-47)	`mapVersion`, `publishedAt`
`PointOfInterestChanged` · `point-of-interest.changed`	Events	**Transports**: a shuttle's stops must be transport-stop points of the event, shown with their names (US-37, US-39)	`poiId`, `mapVersion`, `type`, `name`, `position`, `deleted`
`CheckInRecorded` · `check-in.recorded`	Events	**Badges**: counts the event as attended, once (US-57)	`userId`, `checkedInAt`
`HelpRequestSubmitted` · `help-request.submitted`	Reports	**Reports** (its own triage consumer): reads the request from its own database, asks the AI for a suggestion and saves it (US-34, §4.1). Published through the outbox, so a saved request is never lost (`architecture.md` §6)	`requestId` only (§7.3)
`ReportConfirmed` · `report.confirmed`	Reports	**Badges**: adds the report to the reporter's counts and recalculates the tiers (US-54, US-55)	`reportId`, `reporterUserId`, `type`, `confirmed: true`, `confirmedAt`
`ReportUnconfirmed` · `report.unconfirmed`	Reports	**Badges**: removes it from the counts and recalculates the tiers, which can go down (US-54, US-55)	`reportId`, `reporterUserId`, `type`, `confirmed: false`, `revokedAt`

- Transports and Badges publish nothing. The status of a help request reaches the participant by polling (§1.6), not by an event.
- `ReportConfirmed` and `ReportUnconfirmed` carry the same full state, so Badges keeps one row per report and upserts it; a report confirmed, unconfirmed and confirmed again counts once. After its reporter deletes the account, a report is published with an empty `reporterUserId`, and Badges ignores it.
- If the badge preferences stay in Auth & Users (§6), Auth & Users also publishes `EngagementPreferencesChanged` to Badges.

7.5 Queues

Queue	Bound to (routing keys)
`auth-users.events-events`	`farol.events` : `event.changed` , `team.changed`
`events.auth-users-events`	`farol.auth-users` : `membership.changed` , `global-role.changed` , `user-profile.changed` , `user-status.changed` , `user.deleted`
`reports.auth-users-events`	`farol.auth-users` : `membership.changed` , `user-profile.changed` , `user-status.changed` , `user.deleted`
`reports.events-events`	`farol.events` : `event.changed` , `team.changed` , `venue-map-version.published`
`reports.reports-events`	`farol.reports` : `help-request.submitted` (triage consumer)
`transports.auth-users-events`	`farol.auth-users` : `membership.changed` , `user-status.changed` , `user.deleted`
`transports.events-events`	`farol.events` : `event.changed` , `team.changed` , `point-of-interest.changed`
`badges.auth-users-events`	`farol.auth-users` : `user-status.changed` , `user.deleted`
`badges.events-events`	`farol.events` : `event.changed` , `check-in.recorded`
`badges.reports-events`	`farol.reports` : `report.confirmed` , `report.unconfirmed`

Each queue has its dead-letter queue (`<queue>.dlq`); retries and backoff follow `architecture.md` §6.

8. Coverage

Every Must and Should story (US-01 to US-58, except the Could US-46) has at least one endpoint or event. `…/` stands for `/api/v1/events/{eventId}/` ; the other paths start with `/api/v1` .

Story	Endpoints	Events
US-01	Keycloak (§1.4), `GET /me` , `GET /me/memberships`	—
US-02	Keycloak (§1.4)	—
US-03	Keycloak sign-up (§1.4), `GET /me`	`UserProfileChanged`
US-04	`PATCH /me`	—
US-05	`DELETE /me`	`UserDeleted`
US-06	`GET /accounts` , `POST …/members` , `POST /events`	`GlobalRoleChanged` , `EventChanged`

Story	Endpoints	Events
US-07	`GET /events`	`MembershipChanged`, `GlobalRoleChanged`, `UserProfileChanged`
US-08	`GET /accounts`, `POST …/members`	—
US-09	`POST …/suspend`, `POST …/reactivate`	`GlobalRoleChanged`, `EventChanged`
US-10	`GET /events/{eventId}`, `PATCH /events/{eventId}`	—
US-11	`GET /events/{eventId}`, `POST …/publish`	—
US-12	`POST …/members/me`, `GET /events/{eventId}`	`EventChanged`
US-13	`POST …/close`	`EventChanged`
US-14	—	`MembershipChanged`
US-15	`GET …/members`, `PATCH …/members/{userId}`	—
US-16	`GET …/teams`, `POST …/teams`, `PATCH …/teams/{teamId}`, `DELETE …/teams/{teamId}`	`TeamChanged`
US-17	`PATCH …/teams/{teamId}`, `PUT …/teams/{teamId}/permissions`	—
US-18	`GET …/members`, `PUT …/members/{userId}/teams/{teamId}`, `DELETE …/members/{userId}/teams/{teamId}`	`TeamChanged`
US-19	`POST …/map-versions`	—
US-20	`POST …/map-versions/{version}/points-of-interest`, `PATCH …/map-versions/{version}/points-of-interest/{poiId}`, `DELETE …/map-versions/{version}/points-of-interest/{poiId}`	—
US-21	`GET …/map-versions/current`, `GET …/map-versions/{version}/image`	—
US-22	`GET …/map-versions/current`	—
US-23	`POST …/help-requests`	`VenueMapVersionPublished`
US-24	`GET …/help-requests`, `GET …/help-requests/{requestId}`	—

Story	Endpoints	Events
US-25	`POST …/help-requests`	—
US-26	`GET …/help-requests`	—
US-27	`GET …/map-versions/{version}`, `GET …/help-requests`	—
US-28	`PATCH …/help-requests/{requestId}`, `POST …/help-requests/{requestId}/acknowledge`, `GET …/help-requests/{requestId}/history`	—
US-29	`GET …/teams`, `GET …/help-requests/{requestId}`, `POST …/help-requests/{requestId}/route`	`TeamChanged`
US-30	`POST …/help-requests/{requestId}/close`	—
US-31	`GET …/help-requests`, `GET …/field-tasks`	—
US-32	`POST …/field-tasks/{taskId}/accept`	—
US-33	`POST …/field-tasks/{taskId}/resolve`	—
US-34	`GET …/help-requests/{requestId}`	`TeamChanged`, `HelpRequestSubmitted`
US-35	`POST …/help-requests/{requestId}/accept-suggestion`	—
US-36	`GET …/triage-status`	—
US-37	`POST …/map-versions/{version}/points-of-interest`, `POST …/transport-options`, `PATCH …/transport-options/{optionId}`, `DELETE …/transport-options/{optionId}`	`TeamChanged`, `PointOfInterestChanged`
US-38	`POST …/transport-options/{optionId}/delay`, `POST …/transport-options/{optionId}/cancel`	—
US-39	`GET …/transport-options`	`PointOfInterestChanged`
US-40	`GET …/bus-observations`, `GET …/transport-source-status`	—
US-41	`POST …/transport-options`, `GET …/bus-observations`	—
US-42	`GET …/bus-observations`	—

Story	Endpoints	Events
US-43	`GET …/transport-source-status`	—
US-44	`GET …/announcements`, `POST …/announcements`	—
US-45	`GET …/announcements`, `PATCH …/announcements/{announcementId}`, `DELETE …/announcements/{announcementId}`	—
US-47	`POST …/reports`, `GET …/reports`	`VenueMapVersionPublished`
US-48	`POST …/reports`	—
US-49	`GET …/reports`	—
US-50	`GET …/reports`, `POST …/reports/{reportId}/confirm`, `POST …/reports/{reportId}/reject`	—
US-51	`POST …/reports/{reportId}/unconfirm`	—
US-52	`GET …/teams`, `POST …/reports/{reportId}/dispatch`, `GET …/field-tasks`, `POST …/field-tasks/{taskId}/answer`	—
US-53	`POST …/reports/{reportId}/dispatch`, `POST …/field-tasks/{taskId}/resolve`	—
US-54	`GET /me/badges`	`ReportConfirmed`, `ReportUnconfirmed`
US-55	`GET /me/badges`	`EventChanged`, `ReportConfirmed`, `ReportUnconfirmed`
US-56	`GET /me/badges/preferences`, `PUT /me/badges/preferences`	—
US-57	`GET /me/badges`	`CheckInRecorded`
US-58	`POST …/check-ins/me`	—

- **US-14** (isolation) is also covered by the conventions: `{eventId}` in every event path (§1.2), the per-event check from local copies (§1.5) and `404` for non-members (§1.7). The integration test per service is in guide D3.
- **US-25** (request not sent) is mostly in the app; the API only makes the resend safe with `clientRequestId`.
- **US-40** (import) is a scheduled job inside Transports with no endpoint of its own; its result is what the two endpoints return.

No service calls another's REST API (guide C4): every row in §7.4 is a message on RabbitMQ, every consumer in §7.5 reads from its own queue, and each "Requires" column is checked against a local copy. The outbound

HTTP calls go only to external or infrastructure systems, never to another Farol service: Transports → Porto Digital, from a scheduled job (§5); Reports → the AI provider, from the triage worker; Auth & Users → the Keycloak Admin API, on account deletion (§2); and every service → Keycloak's keys (JWKS), when they change (`architecture.md` §9.1).

9. Future (Could features)

Not outlined; listed by name only.

- F11 · Weak-network mode (US-46)
- F17 · Entrance status (US-59, US-60)
- F18 · Friends (US-61, US-62)

10. Decisions to take

10.1 Outline decisions

Decided on 23 October 2026. The outline above already follows them.

#	Question	Options	Recommended	Where	Answer
D1	How are the lists that are not live paginated?	**A.** `limit` (default 50, max 200) and an opaque `cursor` · **B.** `page` and `size`	A: stable when rows are added while paging	§1.6	**A**
D2	What does a user who is not a member of the event get?	**A.** `404`, as if the event did not exist · **B.** `403`	A: does not reveal that the event exists (US-14)	§1.7	**A**
D3	How is the first EventAdmin of a new event created?	**A.** Auth & Users creates the membership when it consumes `EventChanged` with `firstEventAdminUserId` (one call by the FarolAdmin) · **B.** The FarolAdmin creates the event, then calls `POST /events/{eventId}/members`	A: the event is never left without an EventAdmin if the second call fails	§2, §7.4	**B**: one flow for the first and later EventAdmins; an event left without one is fixed through US-08
D4	Do Reports and Events show people's names, or only their `userId`?	**A.** Names, through `UserProfileChanged` · **B.** Only the `userId`	A: the control room needs to see who is on the way, and the FarolAdmin who the EventAdmins are (US-07)	§7.4	**A**
D5	What points does a new map version start with?	**A.** A copy of the previous version's points · **B.** None	A: a corrected map rarely moves every point	§3	**A**

#	Question	Options	Recommended	Where	Answer
D6	Where are the map files kept?	**A.** A volume of the Events service, with the references in its database · **B.** Inside its database	A: keeps the database small; one volume in the local deployment (ADR-011)	§3	**A**
D7	What does the venue QR code for check-in carry?	**A.** A venue code, different from the join link · **B.** The join link	A: nobody checks in from home (US-58, `features.md` §6.3)	§3	**A**
D8	Who places the temporary transport stops?	**A.** Events, which accepts `Transport.Edit` for transport-stop points · **B.** Transports, which keeps its own stops outside the map	A: stops stay on the venue map with the other points (domain model)	§3, §5	**A**
D9	Is there an endpoint for another user's badges?	**A.** No: remove `GET /users/{userId}/badges` until a story needs it · **B.** Yes, `404` while hidden	A: no Must or Should story shows another user's badges, so hiding them (US-56) needs nothing	§6	**A**
D10	How does a help request become *Seen*?	**A.** A member with `HelpRequests.Triage` calls `/acknowledge` · **B.** Automatically, when one of them opens it	A: a `GET` changes nothing, and *Seen* is recorded with who saw it	§4	**A**
D11	How does a field team answer a report it was sent to check?	**A.** A field task with purpose `Verify` and `/answer` (`found: true | false`) · **B.** A separate resource for report checks	A: one list and one flow for every field task	§4	**A**
D12	Can the app resend a help request or a report safely?	**A.** Yes, with a `clientRequestId` that returns what was already saved · **B.** No idempotency key	A: a resend after a lost answer never creates a duplicate (US-25)	§4	**A**

10.2 Decisions of other owners

The outline follows the current proposal and marks it **[WAITS: On]**. When it is decided, the outline is updated in the documentation review. O1 to O4 are closed.

#	Decision	What the outline assumes	Where
O1	Gateway routing (guide C6)	✅ **Closed** (22 October 2026): the route table of ADR-013 follows the outline's paths	§1.2
O2	Sign-up and login (guide E1)	✅ **Closed** (22 October 2026, ADR-006): sign-up, login and logout in Keycloak; `POST /accounts` removed, the account is created at the first `GET /me`	§1.4, §2

#	Decision	What the outline assumes	Where
O3	What the token carries (guide E2)	✅ **Closed** (22 October 2026, ADR-006): only the identity; roles and teams come from local copies (`MembershipChanged` , `TeamChanged`)	§1.5
O4	Account lookup by e-mail (privacy)	✅ **Closed** (26 October 2026): `GET /accounts?email=` for the FarolAdmin, with the rules of the assessment's AC-9	§2
O5	Where the badge preferences live (domain model)	In Badges (`/me/badges/preferences`); if they stay in Auth & Users, it publishes `EngagementPreferencesChanged`	§6
O6	US-41 and the Porto Digital data (backlog)	Shows the buses on the event's routes, not timetables; the story still says STCP	§5
O7	Priority scale (`features.md` §5.2, AI-3)	Named levels such as `High` ; the values come from AI-3	§4.1

Local delivery and demo

> **Decided 23 October 2026** (#129; guide G4–G7, H5 and H6). Implements ADR-011 points 4–6. The Sprint B slice applies this procedure; the full stack below is the **Sprint B target**. Sprint A now has a separate minimum described in §0: one Reports synthetic in-memory demo service; it does not claim the full stack is implemented.
> **Aligned on 26 October 2026** with ADR-006 (Keycloak), ADR-013 (API gateway) and the Security and Privacy Assessment §9 (ports). HTTPS for the phones is open (§8).

Delivery is local only, because cloud hosting costs were not justified (clarification record). Everything runs on one laptop with Docker. The phones open the same React web app in their browser.

0. Sprint A: run the executable minimum first

The 31 October review (#166) limits deliverable 10 to a small one-service slice, HTTP tests, Dockerfile, Compose and CI. Follow the R0 run instructions and actual branch evidence. The current `infra/compose.yaml` starts Reports only with profile `demo`, bound to `127.0.0.1:8080`: Swagger creates and reads synthetic help requests with fixed identity and in-memory storage. `GET /api/v1/status` remains diagnostic. No secret file is required. #58 was closed on 31 October 2026 after #167; #148 remains open until actual observability exists.

The procedures in §§1–8 are retained for **Sprint B**, when the named services/configuration exist. They must not be followed as if Keycloak, a web UI, databases and the broker are already part of R0. Authentication and domain guarantees apply when business endpoints are introduced; the demo uses only fixed synthetic identity/event values and is absent without the demo profile.

1. What runs

One Compose file, `infra/compose.yaml`. Images are built from the repository: there is no registry (guide H7).

Compose service	What it is	Profile	Port published
`auth-users`, `events`, `reports`, `transports`, `badges`	The five Spring Boot services, one container each, image `farol-<service>:${VERSION}`	default	none
`gateway`	The API gateway, image `farol-gateway:${VERSION}` (ADR-013): the only entry point of the API, with CORS, JWT validation and the rate limits	default	`0.0.0.0:8080`
`keycloak`	The identity provider (ADR-006), pinned version, realm imported from `infra/keycloak/farol-realm.json`	default	`0.0.0.0:8180`

Compose service	What it is	Profile	Port published
`web`	Serves the React app, image `farol-web:${VERSION}`. The app calls the gateway and Keycloak directly	default	`0.0.0.0:${WEB_PORT}` (3000, guide F2)
`postgres`	**One** PostgreSQL instance with **six databases and six users**: one per service and one for Keycloak. No user can read another's database (ADR-003)	default	none
`rabbitmq`	The broker, with a fixed node name, a named volume and one user per service (ADR-006)	default	`127.0.0.1:15672` (management UI)
`wiremock`	WireMock in front of Porto Digital and of the AI provider (the local `ai` container, guide K1): proxy, recorded responses and failure stubs (guide J2, J4). Its root is `infra/wiremock/`, mounted at `/home/wiremock`, with the recorded Porto Digital stubs. Transports and Reports point at it by default	default	`127.0.0.1:8090` (admin API)
`ai`	Local model: Ollama with `qwen3:4b` (guide K1, AI plan). Reports reaches it through `wiremock`	`ai`	none
`prometheus`, `grafana`	Observability stack (ADR-012, accepted architecture)	`observability`	`127.0.0.1` only

- **Only the gateway, Keycloak and the web app are published to the network** (`0.0.0.0`, ADR-013). Three admin pages are also published, but **only on** `127.0.0.1`, so only the demo laptop opens them: RabbitMQ's management UI, WireMock's admin API (the presenter uses it to trigger D1 and D2, §8) and Grafana. Nothing else is published: the services, PostgreSQL and RabbitMQ's AMQP port are reachable only inside Compose (assessment §9).
- **One PostgreSQL instance, not six.** Separate databases with separate users keep the ownership rule. More containers would only add memory. Supersedes the "five PostgreSQL containers" suggestion in the deployment research (G6).
- An init script in `infra/postgres/` creates the six databases and users from the `.env` values on the first start.
- Third-party images (Keycloak, PostgreSQL, RabbitMQ, WireMock, observability) are pinned to an exact version in the Compose file and change only in a PR (guide H7).
- Health checks: `postgres` (`pg_isready`), `rabbitmq` (`rabbitmq-diagnostics ping`), `keycloak` (`/health/ready` on its management port), the gateway and each service (`/actuator/health`). Services and Keycloak wait for their database to be healthy (`depends_on: condition: service_healthy`). They do **not** wait for RabbitMQ or Keycloak: they reconnect on their own, fetch Keycloak's keys when the first token arrives (ADR-006), and a help request is still saved to the outbox while the broker is down (§9 of `architecture.md`).
- Named volumes for PostgreSQL and RabbitMQ. A volume is not a backup (§6).

2. Configuration and secrets

- `infra/.env.example` is versioned and lists every variable with a harmless example value. `infra/.env` is in `.gitignore`. Each person copies the example once: `cp infra/.env.example infra/.env`.

- Variables:
  - `VERSION`, `WEB_PORT` and `PUBLIC_HOST`: the address the phones use, the laptop's LAN IP for the demo, `localhost` while developing. The Keycloak hostname (`KC_HOSTNAME`), the token issuer the services check (`FAROL_ISSUER_URI`), the gateway's allowed origin and the web app's API and login addresses are built from it.
  - The PostgreSQL superuser and the six database users and passwords; the RabbitMQ admin and the five service users.
  - Keycloak's admin user, the `farol-auth-users` client secret and `FAROL_BOOTSTRAP_FAROL_ADMIN_EMAIL` (ADR-006).
  - `TRANSPORT_SOURCE_URL` (WireMock by default), the AI provider settings (guide K1) and `SPRING_PROFILES_ACTIVE`.
- No real secret goes into the repository, the images or the CHANGELOG. Passwords are generated per machine, never the image defaults. The realm export holds `${…}` placeholders, not secrets (assessment finding 8). K1 chose local Ollama, which needs no API key (guide K1, ADR-005); a hosted provider, if ever approved, would keep its key only in `infra/.env`.

3. Demo data

- Demo data comes from **Flyway migrations kept only for the demo**, in `src/main/resources/db/demo/` of each service. They run only with the Spring profile `demo` (`spring.flyway.locations` adds `classpath:db/demo`).
- Demo accounts come from the Keycloak realm import (ADR-006); their Farol accounts are created at their first login.
- The data is synthetic: invented events, users, teams and requests, with no real personal data.
- To start from a clean state, delete the volumes and start again (§5). The realm import and the migrations recreate everything.

4. Starting

Requirements: Docker Desktop (or Docker Engine with the Compose plugin) and git. The minimum RAM is measured with the walking skeleton and written here.

```
git checkout v0.1.0                  # the release to show (guide H7); use main while developing
cp infra/.env.example infra/.env     # first time only; set VERSION=0.1.0 and PUBLIC_HOST
docker compose -f infra/compose.yaml build
docker compose -f infra/compose.yaml up -d
docker compose -f infra/compose.yaml --profile ai --profile observability up -d   # optional components
```

Then run the smoke test (§7).

5. Updating, rolling back and resetting

Action	Commands
Update everything to a new release	`git checkout vX.Y.Z`, set `VERSION=X.Y.Z` in `.env`, `docker compose -f infra/compose.yaml build`, then `up -d`
Replace a single service	`docker compose -f infra/compose.yaml build <service>` then `docker compose -f infra/compose.yaml up -d --no-deps <service>`. The other containers keep running. This is how independent deployment is shown
Roll back	Check out the previous tag and build it again, as for an update (guide H7). Flyway migrations are not undone: they must be compatible with the previous image (*expand and contract*). After rolling back, check that the consumers and the outbox are empty or draining (RabbitMQ management UI)
Reset the demo data	`docker compose -f infra/compose.yaml down -v` then `up -d`. Deletes the volumes; the realm import and the demo migrations recreate the data
Stop	`docker compose -f infra/compose.yaml down` (keeps the volumes)

Run the smoke test after every update or rollback.

6. Backup and recovery (G7)

- **Before every demo and before every update**, take a dump of the six databases, Keycloak's included, so the accounts stay consistent with Auth & Users: `docker compose -f infra/compose.yaml exec -T postgres pg_dump -U <superuser> -Fc <database> > backups/<database>.dump` (one per database; `backups/` is in `.gitignore`).
- Restore one database: `docker compose -f infra/compose.yaml exec -T postgres pg_restore -U <superuser> --clean -d <database> < backups/<database>.dump`, then restart that service.
- What this covers: going back to a known demo state, and moving that state to the backup laptop. What it does **not** cover: messages in RabbitMQ queues (no backup of queued messages), and the loss of the laptop's disk between dumps. The RPO for disk loss is **not guaranteed**. Retention is decided (NFR-19, assessment §7); only backup in production stays out of scope (guide D5, 🦉).
- The time to roll back and to restore (RTO) is measured once the Sprint B slice has databases, and written here; the in-memory Sprint A skeleton cannot measure it (§9, #166). Until then, no recovery figure is claimed.

7. Smoke test

A checklist of commands, run by hand after starting, updating or rolling back. It passes only if every line passes.

#	Check	Command	Pass
1	Every container is healthy. The health checks call `/actuator/health` and Keycloak's `/health/ready`	`docker compose -f infra/compose.yaml ps`	Every container `running (healthy)`

#	Check	Command	Pass
2	The web app opens	`curl -fsI http://localhost:${WEB_PORT}/`, and open `http://<LAN IP>:${WEB_PORT}` on a phone	HTTP 200; the page loads on the phone
3	The gateway routes and validates tokens	`curl -s -o /dev/null -w '%{http_code}' http://localhost:8080/api/v1/me`	`401`: the call reached the gateway and was refused without a token
4	Login works	Log in with a demo account on the laptop (`localhost`); on a phone once HTTPS is decided (§8)	Keycloak's page opens; back in the app, the account's name is shown (`GET /me`)
5	One round trip through RabbitMQ	One service publishes an event and another consumes it. The exact command is written here in Sprint B: the Sprint A skeleton has no broker (#166)	The consumer shows the change; the queue is empty afterwards

`ci.yml` exists since #167 (ADR-010 point 9). In Sprint B the same checks, without the phone and the login, are added to it, to run in CI with Compose on every push to `main`, every tag and manual runs (Sprint B). Today `ci.yml` (#167) runs the Sprint A skeleton's smoke test instead (status, synthetic create and read, Swagger) on every pull request, push to `main`, tag and manual run.

8. The demo

Machine. One team member's laptop runs the demo. A second laptop is ready as a backup: same tag, same `.env`, the latest dumps restored, smoke test passed the day before. The laptop has full-disk encryption and a screen lock: it holds every database (assessment §9).

Phones. The phones and the laptop join the same network, preferably a phone hotspot or a router of our own: venue Wi-Fi often blocks traffic between clients. The phones open `http://<laptop LAN IP>:${WEB_PORT}`. Allow ports 8080, 8180 and `${WEB_PORT}` in the laptop's firewall (on Windows, Docker Desktop asks the first time). Show the URL as a QR code on the screen.

- **Open: HTTPS for the phones** (assessment §9, finding 2). Over plain HTTP anyone on the network can read the tokens, and the browser blocks the Web Crypto call that the PKCE login needs on a page that is not HTTPS or `localhost`, so **login fails on a phone**. The assessment proposes a certificate from a local authority (e.g. mkcert) on the gateway, Keycloak and the web app, trusted once on each demo phone, or the phone pages in a laptop browser's device mode. This procedure is updated when it is decided. Until then, the phone pages are shown in device mode on `localhost`.
- Turning off PKCE, token validation, CORS or Keycloak's HTTPS requirement to make the demo work is **not allowed** (assignment §11). Every security control stays on; D1 and D2 break a dependency, not a control.

Degraded modes (scenarios D1 and D2, `features.md` §3.1; the demo script and the values are in guide J2, `architecture.md` §9.3):

Show	How
D1 · AI unavailable	Add an AI failure stub (dropped connection, slow answer) in WireMock through its admin API, and remove it to recover (`architecture.md` §9.3). Also: `docker compose -f infra/compose.yaml stop ai`, or start without the `ai` profile
D2 · Transport source unavailable, late or invalid	The demo always goes through `wiremock`. Add a failure stub (no answer, late, invalid data) through its admin API, or `docker compose -f infra/compose.yaml stop wiremock`
Broker down	`docker compose -f infra/compose.yaml stop rabbitmq` : requests are still saved (outbox) and catch up after `start rabbitmq`
Another service down	`docker compose -f infra/compose.yaml stop badges` : the other services keep working

9. What is still open

- The minimum RAM of the demo laptop, and the measured RTO for rollback and restore (guide G7): the in-memory Sprint A skeleton cannot measure them (#166), so they are measured with the Sprint B slice.
- The exact round-trip command in the smoke test: Sprint B slice.
- HTTPS for the phones, which the PKCE login needs (§8): assessment §9, finding 2.
- Closed: the AI container is Ollama with `qwen3:4b` (guide K1, ADR-005); the degraded-mode script and x = 30 s, y = 5 min (guide J1, J2; `architecture.md` §9). Still open: the model measurements on the demo laptop (guide K1).

Docs/Technical/adr/0001-architectural-style.md

ADR-001 · Architectural style and evolution

Status

Accepted: style decided by the team on 15 October 2026; evolution decided after the professor's answer to the email of 15 October (received by 21 October 2026). Source: `architecture-decisions-guide.md` A1, A2, A3 and A4.

Deployment-scope clarification (recorded 22 October 2026): as relayed to the team, the professor authorised local-only execution due to cost (record, ADR-011). Independent deployment in this ADR can be demonstrated with separate local service containers; no cloud deployment is required for the authorised delivery scope. The five-service architecture is retained.

Sprint A delivery (recorded 31 October 2026): Sprint A delivered one service, the R0 walking skeleton of Reports (#166, ADR-010 point 9). The five services, their databases and the broker are deployed from Sprint B, the sprint in which the assignment requires two independently deployable components (§7.2.2). Decision 3 was amended on 1 November 2026 to match (record, #175).

Context

- The assignment asks for at least two independently deployable backend components, or a modular monolith with convincing evidence that services would add unnecessary complexity (§6.2, architecture driver D7).
- A read peak (tens of thousands of participants at the end of a concert, driver D2) must not stop help requests.
- Each service is specified by a small contract (OpenAPI and events), the base of Spec-Driven Development.
- The report asks for an *architecture evolution narrative* (§7.3.2, item 11). The team proposed a modular monolith in Sprints A and B, split into services in Sprint C, and asked the professor whether the architecture should change between sprints or be defined from the start.
- **Professor's answer:** there is no imposed path. The architecture is defined from the start, and any evolution must be motivated by evidence, not by the calendar. Sprint B already requires two independently deployable backend components, or a properly justified modular monolith (§6.2).

Decision

1. **Microservices** (A1), five services (A2): **Auth & Users**, **Events**, **Reports**, **Transports** and **Badges**.
2. Each service is its own **Maven project** in `backend/<service>/`, in the same repository; no service depends on another's code, only on the contracts (A3).
3. **Microservices from the start** (A4; amended on 1 November 2026, record): the architecture is five services from the first sprint, with no modular-monolith phase. Sprint A delivers the first one, the R0 skeleton of Reports (#166); from Sprint B every service is built, deployed and owns its database (assignment §7.2.2).

4. Any change to this structure is **motivated by evidence** (measurements, tests, failures), not by the calendar. Each change is a new ADR or one that supersedes an earlier one; together they are the evolution narrative.

Consequences

- §6.2 is met once more than one service runs (Sprint B; Sprint A delivered one, see *Status*); no evidence against services is needed.
- A failure or read peak in one service does not stop the others (communication only through events, ADR-004). Locally all services share one machine and one PostgreSQL server, which stay a common point of failure (ADR-011, consequences).
- More to run from Sprint B: five deployables, five databases (ADR-003) and the broker, with their pipeline jobs. Sprint A runs one service (R0, see *Status*).
- The ArchUnit boundary test planned for the monolith phase is no longer needed: separate Maven projects cannot import each other's classes.
- The evolution narrative depends on recording, in each ADR, the evidence that motivated the change.

Rationale

- Services meet §6.2 by design, so no evidence against services has to be gathered later.
- The professor asked for the architecture to be defined from the start and changed only on evidence; a split planned for Sprint C would be a change driven by the calendar.
- Separate services keep a read peak from stopping help requests (driver D2), and separate Maven projects enforce the boundaries without an extra test.

Alternatives considered

Option	Why not chosen
Modular monolith in Sprints A and B, split into services in Sprint C	A change driven by the calendar, not by evidence; the professor asked for the architecture to be defined from the start. A split in Sprint C would also add risk at the end of the project
Modular monolith as the final state	Needs convincing evidence that services add unnecessary complexity (§6.2); a read peak would affect help requests
Monolith without enforced module boundaries	Ruled out by the team (guide §0)

ADR-002 · Backend and frontend technology

Status

Accepted: decided by the team on 11 October 2026 (Spring Boot, React) and 15 October 2026 (versions, Maven, React Native); updated on 21 October 2026 (services deployed independently from the start, guide A4) and on 22 October 2026 (the React Native app is removed; one React web app, meeting with the professor P9, #125). Source: `architecture-decisions-guide.md` §0, F1 and H3.

Context

Farol needs a backend made of five services, deployed independently from Sprint B (guide A2, A4; Sprint A delivered one service, ADR-001 decision 3), a client for event management (P3, on a laptop) and for participants and responders, who use it on their phones at the venue. The assignment asks the team to justify its technology choices (§7.1.2, item 8). The team is small and has three sprints (architecture driver D9), so learning a new stack would take time away from the product.

Decision

- **Backend: Java 21 (LTS)** and **Spring Boot 4.x**, built with **Maven** (guide §0, H3). Each service is its own Maven project in `backend/<service>/` (guide A3), with its own `main`, Dockerfile and deployment.
- **Frontend: one React web app** for every user (guide §0, F1). Each page is built for one use: **on the phone** (participants, field teams) or **on a laptop** (event management, control room). It runs in the browser; there is no app to install.

Why: Spring Boot and React are the stacks the team already knows and uses. One web app means one frontend codebase, nothing to install for participants (personas, P1: "*No installation*"), and no app-store distribution to the professors.

Consequences

- No time spent learning a new language or framework; the team can start building features in the walking skeleton.
- Spring has ready-made support for the other decisions: Spring AMQP for RabbitMQ (ADR-004), Spring Data JPA and Flyway for PostgreSQL (ADR-003), and `openapi-generator` server stubs for the contract-first APIs (guide C7).
- One frontend to build and maintain. Phone pages and laptop pages share the components, the generated API client and the login.
- A JVM service uses more memory and starts more slowly than, for example, a Node.js or Go service. With five services running from Sprint B (no monolith phase) this raises hosting cost from the start; acceptable while each service runs as a single instance (guide C5).

- What a browser on a phone cannot do well: reliable push notifications and background location. The product needs neither (polling, guide C5; the location is marked on the map, US-23). Working offline (US-46, Could) needs a service worker, decided with that story.

Alternatives considered

Option	Why not chosen
.NET (ASP.NET Core)	Comparable to Spring Boot in features, but the team is less used to it
Node.js (e.g. NestJS)	One language across backend and frontend, but less familiar to the team for backend work, and weaker typing and tooling for a domain with strict rules (per-event isolation, audit)
Python (e.g. FastAPI)	Good for AI work, but the team has less experience building services with it
Angular or Vue for the web app	No advantage over React, which the team already knows
React web app for event management plus a **React Native** mobile app for the rest	Decided on 15 October 2026 and dropped on 22 October 2026 (#125): two frontends, a setup (Expo or bare) and a way to deliver the app to the professors without the app stores, for features that a web app also covers
Native Android and iOS apps	Two codebases for a team of four; not realistic in three sprints

Open points

- Venue map in the web app: guide F3 / ADR-007, rewritten for the web, waits for review.

ADR-003 · Database and data ownership

Status

Accepted: decided by the team on 15 October 2026. Updated on 21 October 2026: no modular-monolith phase (guide A4). Updated on 27 October 2026: account deletion (guide D7, #135). Source: `architecture-decisions-guide.md` D1, D2, D3, D6 and D7.

Context

- The assignment asks for **one** database technology unless more are justified (§6.4).
- Farol is split into five services (guide A2), deployed independently from Sprint B (guide A4, ADR-001 decision 3; see the note of 31 October 2026), that talk only through events (guide C4). A service that reads another service's tables would break that boundary.
- **One team must never see or change another event's data** (architecture driver D1). The report asks for automated tests that prove it.
- The venue map is an uploaded image with coordinates on it, not a geographic map (annex C3), so no spatial queries are needed.
- With local delivery (guide H5, ADR-011), one service can be replaced or rolled back on its own (`docker compose up -d --no-deps`, `local-delivery.md`), so a new version of a service can start while the old schema is still in use, and an old version can start on a newer schema.

Decision

1. **PostgreSQL** is the only database technology (D1). No PostGIS: the map is an image.
2. **One database per service**, which only that service reads and writes (D2). Data that a service needs from another arrives through events and is kept as a local copy (ADR-004). This applies from the first sprint that runs more than one service: there is no shared-database monolith phase (guide A4). The Sprint A skeleton has one service and no database (R0, #166); the databases arrive in Sprint B (note of 31 October 2026).
3. **Per-event isolation:** every table that holds data of an event has an `event_id` column, and the application filters by it on every query (D3). **Each service has an integration test** that proves a member of event X cannot read or change data of event Y.
4. **Flyway** runs the migrations when the service starts (D6). Every migration stays compatible with the previous version of the service (*expand and contract*): add first, switch the code, remove in a later release.
5. **Account deletion** (D7): each service deletes or anonymises its own data about the user, local copies included, when it consumes `UserDeleted`, and keeps a tombstone with only the opaque `sub`, so the account is refused and late events do not bring a copy back. Audit entries keep the opaque `sub` without a name. What each service does is in the API outline §7.4.

Consequences

- One technology to learn, run and back up. The same PostgreSQL runs locally and in the tests (Testcontainers); the cloud is an alternative only (ADR-011).
- Services cannot join across databases. Cross-service views (e.g. the event's teams and their permissions inside Reports) come from local copies and are **eventually consistent**.
- Isolation depends on the application filter being present in every query; a missing filter is a data leak. The per-service isolation test is the safety net, so it must cover every new event-scoped table.
- *Expand and contract* makes some changes (renames, type changes) take two releases.
- Five databases (from Sprint B; see the note of 31 October 2026) cost more than one. Locally they share **one PostgreSQL server** with Keycloak's database, six databases with one user each (guide G5, `local-delivery.md`).

Rationale

- The data is relational and the outbox needs transactions, so one relational technology covers every service (§6.4).
- A database per service keeps the boundaries of guide C4: no service can couple to another through its tables.
- An `event_id` filter with a mandatory test gives per-event isolation without the set-up cost of row-level security or a database per event.
- *Expand and contract* lets one service be replaced or rolled back while the others keep running.

Alternatives considered

Option	Why not chosen
MongoDB or another document store	The data is relational (events, memberships, roles, help requests with audit) and needs transactions for the outbox (ADR-004)
PostgreSQL + PostGIS	The map is an image with its own coordinates (annex C3); no geographic queries are needed
One shared database for all services	Simpler to run, but services would couple through tables, breaking guide C4 and independent deployment (assignment §6.2)
Row-level security in PostgreSQL for event isolation	Enforced by the database, but harder to set up and test with a pooled connection per service; the application filter plus a mandatory test is enough for the MVP
One database or schema per event	Strong isolation, but events are created at run time; migrations and operations would multiply with every event
Liquibase	Equivalent to Flyway; Flyway's plain SQL migrations are simpler to review

Open points

- Number of PostgreSQL servers and where they run: ✅ closed on 26 October 2026. One PostgreSQL server on the local machine, with six databases and six users (five services and Keycloak); no user can read another's database (guide G5, ADR-011, `local-delivery.md`).
- ~~Account deletion (anonymise or delete per service): guide D7.~~ Closed on 27 October 2026 (decision 5, #135).
- Cache for read peaks (D4): future improvement, outside the MVP. Retention and local backup (D5) were decided on 26 October 2026: help-request content deleted 30 days after the event ends, Keycloak login events kept 30 days (NFR-19); a `pg_dump` of the six databases before every demo and every update, kept at most 30 days, queued messages not covered (NFR-13, `local-delivery.md` §6). Production backup stays a future improvement (guide D5).

ADR-004 · Service communication

Status

Accepted: decided by the team on 15 October 2026. Updated on 21 October 2026: services deployed independently from the start (guide A4) and the transport source (guide A2, J4). Updated on 22 October 2026: retry values before the dead-letter queue (guide C3) and resilience values for the Porto Digital poll (guide J1). Source: `architecture-decisions-guide.md` C1, C2, C3, C4, C5 and C7.

Context

- The assignment asks for at least one asynchronous workflow that handles retries, duplicate events, failures and eventual consistency (§6.3). In Farol this is the help-request triage (`architecture.md` §6).
- Five services (guide A2) are deployed on their own from Sprint B (guide A4, ADR-001 decision 3; see the note of 31 October 2026 in *Consequences*); Sprint B already requires at least two (assignment §6.2). A service that is down must not stop the others: a help request must still be saved when the AI or another service fails (architecture driver D6).
- Each service owns its database (ADR-003), so it cannot read another service's data directly.
- Transports reads an external source, the Porto Digital API, which can be unavailable, slow or return invalid data (guide A2, J4).
- The apps need fresh data (backoffice list, request status, announcements), but the team is small (driver D9).

Decision

1. **Broker: RabbitMQ** (C1), used through **Spring AMQP** (C2). *Topic* exchanges for publish-subscribe; a **dead-letter exchange** for messages that keep failing.
2. **Only events between services** (C4). No service calls another service's REST API. Each service keeps a local copy of what it needs from another (e.g. Reports keeps the teams, their permissions and help categories from Events; every service keeps the memberships and team assignments of its events from Auth & Users).
3. **Delivery guarantees** (C3):
   - **Outbox table** in each service: the business change and the outgoing event are saved in the same transaction; a polling job publishes pending rows to RabbitMQ. An event is never lost and never sent for a change that was rolled back.
   - **At-least-once delivery, idempotent consumers:** each consumer records the `message_id`s it has processed and ignores duplicates.
   - After **4 attempts** with exponential backoff (1 s, 2 s, 4 s), the message goes to the **dead-letter queue** of its queue, with the failure reason in its headers. DLQ messages are never reprocessed automatically: they show on the DLQ panel of the dashboard (ADR-012, NFR-34) and are moved back by hand once the cause is fixed (`architecture.md` §6).
4. **Apps → services: synchronous REST, contract-first** (C7). The OpenAPI file is written by hand in `backend/<service>/contracts/openapi.yaml`, paths start with `/api/v1`, and `openapi-generator` produces

the Spring interfaces and DTOs and the app clients (`spec-driven-development.md`).

5. **External sources are read by a scheduled poll** in the owning service, never inside a user's request: Transports polls the Porto Digital API with a timeout, validates the data and keeps the last valid copy (`architecture.md` §7). Its HTTP client points to a configurable base URL, so the same code runs against the real API or the simulator / recorded responses (guide J4).
6. **Polling for live data** (C5): the apps poll for the backoffice list, request status and announcements. One instance per service for now.

Consequences

- Once more than one service runs, the broker, the outbox and eventual consistency are real between deployed services, not simulated inside one process. Note of 31 October 2026: the Sprint A skeleton has one service and no broker (R0, #166, ADR-010 point 9); this starts in Sprint B.
- A service can be down without blocking the others; its events wait in the outbox or in its queue and are processed when it comes back.
- **Eventual consistency:** local copies lag behind the source by the time it takes an event to arrive. The UI must handle it (e.g. "awaiting triage").
- Every service needs an outbox table, a publishing job and a processed-messages table. This is repeated code; a shared library is not allowed (guide A3: services share only contracts), so each service keeps its own small copy.
- Contracts become the only coupling between services and apps; a breaking change in an OpenAPI file or an event must be versioned.
- Polling adds load during read peaks and delays updates by up to the polling interval. Acceptable for the MVP; push (WebSocket or Server-Sent Events) can replace it later.
- RabbitMQ is one more component to run in every environment. Testcontainers covers it in the tests.
- Tests and the degraded-mode demo (D2) run Transports against the simulator, never the real API, so they are repeatable and failures can be injected on demand.

Rationale

- Events only, with local copies, keep a service that is down from breaking the others (driver D6), which synchronous REST between services would not.
- The outbox with idempotent consumers is the simplest way to lose no event and send none for a rolled-back change (§6.3), without running Debezium.
- RabbitMQ covers publish-subscribe and dead-lettering with less to run than Kafka, and runs the same way locally and in the tests.
- Polling is enough for the MVP's freshness and keeps the small team away from connection handling (driver D9).

Alternatives considered

Option	Why not chosen
Apache Kafka	Built for high-volume event streams and replay, which Farol does not need; heavier to run locally and in the cloud
Redis Streams	Would add Redis only for messaging; no built-in dead-letter handling
Cloud queue (e.g. Azure Service Bus, Amazon SQS)	Ties the code to one cloud platform (none is used: delivery is local, guide G1) and cannot run the same way locally and in tests
Synchronous REST between services	Simpler to follow, but couples availability: a service that is down breaks its callers (driver D6)
Publishing to the broker inside the business transaction, without an outbox	Simpler, but an event can be lost or sent for a change that was rolled back (assignment §6.3, lost messages)
Debezium (change data capture) to publish the outbox	No polling job, but one more component to run; a polling job is enough for the MVP volume
WebSocket or Server-Sent Events for live data	Better freshness, but needs connection handling and scaling work; polling is enough for the MVP
Code-first OpenAPI (generated from the controllers)	Faster to start, but the contract would follow the code; contract-first is the base of Spec-Driven Development

Open points

- Resolved on 22 October 2026: retry values (guide C3) and Porto Digital poll values (guide J1: every 30 s, freshness limit 5 min); see `architecture.md` §6 and §9. The DLQ is watched on the dashboard of ADR-012 (accepted on 31 October 2026), not by an alert.
- Resolved on 26 October 2026: the simulator tool is **WireMock**, confirmed in guide J4 (#155); see ADR-008, decision 4.
- Resolved: RabbitMQ runs locally in a container (guide C1 and G1; the cloud is an alternative, ADR-011); API gateway, rate limiting and CORS (guide C6, ADR-013, 22 October 2026); API outline and event catalogue (guide C8, `api-outline.md`, #123).

ADR-005 · AI triage provider and evaluation

Status

Accepted — architectural decision; runtime/evaluation evidence pending. Status reconciled on 31 October 2026 (#166) with the existing guide K1/J1 and NFRs §5: team-confirmed targets on 26 October and the 27 October local Ollama/Qwen3, no-cloud-fallback and single-attempt profile. This records those existing decisions, not a new team vote or model-quality approval. K2 label/reason/routing evaluation, privacy gates before real data and measured SC3/SC5 remain outstanding; acceptance of the architecture does not waive them. Covers K1–K2 / AI-1–AI-11.

Delivery-scope update: the professor authorised local-only application execution due to cost (clarification). Local Ollama `qwen3:4b` Q4_K_M is now the selected local architecture, specified in the complete local AI decision; hosted inference remains a researched alternative. The authorisation itself does not approve a model. Fixed team names in the synthetic schema are an event fixture, not the production role model: derive eligible teams from the event's configured permissions/categories and validate isolation.

Context

F8 suggests category, priority and response team; an operator decides. F7 must work without AI. SC3 allows 30 s from submission to stored suggestion and SC5 requires all requests visible with AI off. NFR-22 forbids lowering priorities or overwriting operator decisions. The complete Responsible AI assessment records sources, costs, input/output, risks and evidence gaps.

Decision

1. Select local Ollama `qwen3:4b` Q4_K_M, private Compose AI profile, cloud features disabled and no paid API fallback. Use the fixed runtime settings, single 20 s attempt, event-specific contract and single-model, single-run demo evaluation in the implementation decision. Retain hosted `mistral-small-2603` and cloud GPU as assessed alternatives; no provisioning is authorised.
2. Minimise input to participant category, sanitised optional text and current priority floor. Fixed categories/priorities, event-derived eligible team IDs, a short operator-facing reason and abstention; no tools, identifiers, exact location or autonomous dispatch.
3. Enforce schema/semantic validation, immediate F7 visibility and a total 30 s inference budget. Late/error/abstained results become fallback. Protect operator updates and clamp stored priorities atomically; measure raw predictions separately so safety guards cannot conceal errors.
4. Require zero urgent under-prioritisations on the reviewed holdout, proposed category/team accuracy ≥80%, valid on-time coverage ≥95% within 30 s (the operational target uses a one-hour window), reason review and an AI-off integration test. The supplied labels require documented team review; a passing synthetic set cannot establish clinical safety.
5. Synthetic data only until the security assessment confirms actual endpoint, processing/retention/training controls and privacy assessment. Freeze versions/settings and log metadata rather than request bodies.

Coverage definition (aligned 26 October 2026, #145)

The **confirmed common target is at least 95% valid suggestions stored within 30 s end to end**. This replaces the earlier 90% evaluation proposal and agrees with observability I2 and NFR-03 in PR #139. The team confirmation recorded in NFRs §5 supports the architectural acceptance above; target alignment itself does not establish measured compliance.

- **Valid and on time:** schema and semantic checks pass, the result is not an abstention, the team is eligible for that event, the raw priority respects its floor, and the suggestion is stored no later than 30,000 ms after submission. Queue/outbox wait, all inference attempts, validation and persistence consume that same deadline. Valid suggestions at exactly 30,000 ms count; later ones do not.
- **Operational SLI:** valid on-time suggestions / **all AI-eligible requests submitted in a one-hour window**, including provider/circuit errors, timeouts, refusals, malformed output and abstentions. Those failures remain misses; a working non-AI fallback is not an AI suggestion. Eligibility is determined by the workflow before the provider result, not by dropping difficult/failed requests afterwards.
- **Window accounting:** evaluate a submission cohort only after every request's 30 s deadline has expired (a one-hour cohort closes and is finalised 30 s later). The dashboard can use rolling one-hour cohorts with the same deadline lag. Report counts and degraded periods alongside the complete ratio, not instead of it; zero eligible requests means **no data**, not 100%.
- **Offline evaluation:** use the same ≥95% threshold on the frozen holdout's **non-abstention reference cases**. Expected abstentions are checked separately and are not silently added to the operational numerator. Count missing, invalid, refused, late and unexpectedly abstained outputs as misses. Total suggestion coverage, before timing, also has a ≥95% gate. The fixtures cannot establish production event eligibility; that requires the event-specific integration test.
- **Independent safeguards:** zero urgent under-prioritisations or missing/abstained/late urgent suggestions; ≥80% category/team accuracy; correct expected abstentions and human review of reasons. SC5 still requires 100% request visibility with AI unavailable. A 95% AI target never permits losing 5% of help requests.

The dataset run and a one-hour operational cohort have different populations and are reported separately. Direct provider latency without queue/storage timing cannot pass the end-to-end timing gate. The offline scorer applies the 95% proposal; no model or runtime measurement is supplied by this alignment.

Consequences

- **Selected local model.** No help-request text leaves the demo machine (NFR-20) and there is no API-token cost. Why `qwen3:4b` Q4_K_M: it can run on a laptop CPU without token charges (published model file about 2.5 GB; prototype with 8 GiB host RAM, 4 CPU cores, context 4096, concurrency 1) (Responsible AI assessment, provider table). Latency on the demo machine's CPU and full-stack memory remain unmeasured; download size is not runtime RAM.
- **One request at a time.** Ollama runs with `OLLAMA_NUM_PARALLEL=1` and a queue of 1 (plan, capacity row), so a burst of help requests waits in the application's admission queue, and requests that cannot start in time become fallback (D1).
- **Hosted alternative, not selected:** token cost would avoid an always-on inference host but adds network, account, privacy and quota dependencies.

- Evaluation and monitoring become release evidence; release eligibility stays conditional until those measurements exist; the selected documentary recommendation is explicit.

Alternatives considered

- Cloud Ollama as default: greater infrastructure/memory cost and unknown CPU latency; retain as a candidate if measurements justify it.
- Hosted model selected solely by headline accuracy/price: insufficient evidence for Portuguese triage or sensitive-data handling.
- Only deterministic routing: remains the fallback/baseline; cannot provide the required AI-assisted workflow by itself.
- Autonomous classification/dispatch: rejected by the existing operator-control decision and NFR-22.

Acceptance evidence pending

Documented label review and one local model evaluation; deployed SC3/SC5 evidence; privacy review for real data; runtime/evaluation review. Paid-provider clearance/spend applies only if a hosted alternative is reconsidered. Research artefacts and offline baseline are linked in the evaluation package.

ADR-006 · Identity and access control

Status

Accepted: decided by the team in the meeting of 22 October 2026 (record). Source: `architecture-decisions-guide.md` E1–E4.

Context

- Login is mandatory for every user, participants included (F1; DM-1). Farol is one React web app, used on phones and laptops (guide F1).
- Auth & Users owns authentication, authorisation, accounts, memberships of events, team assignments and the `FarolAdmin` grant (guide A2). Teams and their permissions live in Events (role model of 22 October 2026, `features.md` §4).
- Services talk only through events and keep local copies of what they need (ADR-004, guide C4).
- Delivery is local only; Azure is the studied cloud alternative (ADR-011, guide G1).
- Mocked authentication is allowed for the MVP, but security controls must not be switched off for the demo (assignment §6.6, §11).
- The top risk is a member of one event reading or changing another event's data (SC8, US-14).

Decision

1. **Keycloak** is the OpenID Connect provider, self-hosted in a container with its own PostgreSQL database. In Sprint B its realm import with test accounts is the mocked authentication.
2. **Tokens carry identity only** (`sub` , `email` , `name` , `aud = farol-api`). Roles per event (`EventAdmin` , `RegularUser`), team assignments and the global `FarolAdmin` role live in Auth & Users; teams and their permissions live in Events. They reach the other services as events, and each service keeps a local copy.
3. **Every service validates the JWT** (Spring Security resource server: signature, issuer, audience, expiry); the API gateway validates it too (ADR-013).
4. **Permissions are checked in the application layer** of each service, against its local copies, with an `event_id` filter on every query. An EventAdmin has every permission of the event; a RegularUser has the permissions of their teams. A non-member gets 404, a member without the permission 403. No copy means no access.
5. **`FarolAdmin`** is the only global role. It is never accepted for an event's operational data.
6. Accounts are created **just in time**, at the first authenticated `GET /api/v1/me` . Account deletion goes through the Keycloak Admin API with client credentials, then a deletion event to every service.
7. The web app uses **Authorization Code + PKCE** (public client), tokens in memory, access tokens of 5 min and refresh tokens rotated on every use. Session idle is 12 h, or 30 days with *Remember me*.
8. **Service-to-service:** no HTTP between services; one RabbitMQ user per service, limited to its own exchanges and queues.

9. **Secrets** come from environment variables: `.env` locally, GitHub Actions secrets in CI, Azure Key Vault if the cloud alternative is resumed. gitleaks runs as a GitHub Actions job on every PR, planned for Sprint B (ADR-010 point 9).

Keycloak setup

Item	Setting
Realm	`farol`, exported to `infra/keycloak/farol-realm.json` (test users only, never imported outside development and the demo)
Clients	`farol-web`: public, Authorization Code + PKCE, redirect URIs on the web app's origins · `farol-auth-users`: confidential, service account limited to managing users (account deletion)
Audience	A mapper adds `aud = farol-api` to `farol-web`; every service requires it
Login	Email as username; Portuguese and English pages; generic error on wrong credentials (US-01)
Brute-force detection	On, per account (not per IP: carrier-grade NAT at the venue)
Password policy	Minimum length 10, not equal to the email
Sign-up	Off in Sprint B (seeded accounts); on in Sprint C for US-03, with the terms accepted on the form
Email verification, reset by email	Out of the MVP: no SMTP server. Known limitation
Token lifetimes	Access token 5 min; refresh tokens rotated. Session idle 12 h (a night shift on a control-room laptop); 30 days with *Remember me* (P1 on their own phone)
Database	Its own `keycloak` database on the PostgreSQL server
Address	`localhost:8180` locally, its own hostname on Azure. **Not** behind the API gateway: the browser talks to it during login. `KC_HOSTNAME` set to that public URL
Start	`--import-realm`; `start-dev` while developing, `start` (production mode, HTTP on `localhost`) for the demo
First FarolAdmin	`FAROL_BOOTSTRAP_FAROL_ADMIN_EMAIL` grants the role at provisioning when the email matches; no screen for it in the MVP

Each service sets the public issuer and the internal key address separately, so it starts even when Keycloak is down and fetches keys only when a token arrives (keys cached, refreshed on an unknown `kid`, #43):

```
spring.security.oauth2.resourceserver.jwt:
  issuer-uri: ${FAROL_ISSUER_URI}   # e.g. http://localhost:8180/realms/farol (checked against iss)
  jwk-set-uri: ${FAROL_JWKS_URI}    # e.g. http://keycloak:8080/realms/farol/protocol/openid-connect/ce
  audiences: farol-api
```

Where each permission is checked

From `features.md` §4.

Capability	Requires	Checked by
Join a published event (QR code or link)	Authenticated	Auth & Users (US-12)
Ask for help, report a problem, see own requests and reports, check in, see own badges	Member (any role)	Reports, Events, Badges
See every help request; set priority and status, route, close	`HelpRequests.Triage`	Reports
Accept and resolve the requests routed to own team	`HelpRequests.Respond`	Reports
Confirm, reject and unconfirm reports of a type; send them to a team	`Reports.Review` (for that type)	Reports
Answer and resolve the reports sent to own team	`Reports.Respond`	Reports
Publish, edit and remove announcements	`Announcements.Publish`	Events
Upload the map, edit points of interest	`Map.Edit`	Events
Add and change shuttles and temporary stops	`Transport.Edit`	Transports
Update entrance status (Could, US-59)	`Entrances.Update`	Events
Edit, publish and close the event; create teams and set their permissions	EventAdmin	Events
Make members EventAdmin; add and remove team members	EventAdmin	Auth & Users
Create, list, suspend and reactivate events	FarolAdmin	Events
Name the first or a new EventAdmin	FarolAdmin	Auth & Users
See any event's help requests, reports or participants	❌ never for FarolAdmin	Every service

Changes for the API outline and D7

The API outline (#123, merged on 26 October 2026 in #124) and guide D7 are updated to match this decision (meeting of 22 October 2026):

- **O2:** login, logout and sign-up happen in Keycloak; `POST /accounts` goes, because the account is created at the first `GET /me`.
- **O3:** the token carries only the identity, as the outline already assumes.
- **O1:** closed by the gateway's route table (ADR-013).
- **Event catalogue:** keep `MembershipChanged` (full state, with team ids), `TeamChanged` and `GlobalRoleChanged`; publish `UserProfileChanged` at just-in-time provisioning too; **add** an account-disabled event, so services deny at once; **define** `UserDeleted` with the flow below; **add** the acting user's `sub` to the envelope, for audit.
- **D7, account deletion:** refused while the user is the last EventAdmin of an event. Otherwise Auth & Users deletes the user in Keycloak (Admin API, client credentials), deletes its own data, then publishes `UserDeleted`. If Keycloak is down, the deletion stays pending and is retried.

Consequences

- One more container and database to run on the local machine; its memory is part of the local sizing (ADR-011).
- The identity provider can be replaced by configuration (e.g. Entra External ID on Azure), because no domain data lives in it.
- Role and permission changes are eventually consistent: target **≤ 5 s**, measured in an integration test. This is how "applies at the member's next action" (features §4.2) is met.
- Every service that checks permissions consumes membership and team events and keeps copies of them.
- Account deletion spans Keycloak and Auth & Users; a pending deletion is retried when Keycloak is down.
- Email verification and password reset by email wait for an SMTP server. Keycloak's sign-up form reveals whether an email is taken; accepted for the MVP.
- Tokens in browser memory are exposed to XSS; a backend-for-frontend is a later hardening.

Addendum · 31 October 2026 (#170)

Documentary alignment only: no new decision and no new team vote. `domain-model.svg` was redrawn to show this ADR's split: memberships, team assignments and the `FarolAdmin` grant (`GlobalRoleGrant`) in Auth & Users; teams and their permissions (`Team`, `TeamPermission`) in Events. The question of which team is the default when two teams handle the same category stays open (see `domain-model.md`).

Alternatives considered

Option	Why not chosen
Mock only (seeded users, JWTs signed with a fixed key)	No real sign-up, brute-force protection or session handling; looks like a disabled security control (assignment §11)
Spring Authorization Server inside Auth & Users	Login, sign-up, reset pages and brute-force protection written by us; at least one extra week of security-critical code
Managed service (Entra External ID, Amazon Cognito, Auth0)	Needs a cloud tenant even for the local delivery; harder local development and testing. Stays the replacement if the Azure alternative is resumed
Roles and permissions as token claims	Every team change becomes a Keycloak Admin API call; tokens grow with events and teams; stale until refresh; domain split across two systems
Validation only at the gateway, with a trusted `X-User-Id` header	Anyone who reaches a service directly can forge the header
Token exchange at the gateway (phantom token, RFC 8693)	High effort for no MVP benefit
Mutual TLS between all services	There is no service-to-service HTTP to protect

ADR-007 · Venue map representation

Status

Proposed: investigated in issue #44 for the React Native app and approved on 22 October 2026. The same day the React Native app was removed and the product became **one React web app** (meeting with the professor, 22 October 2026, P9; ADR-002; #125), so on 23 October 2026 the rendering part (*Decision* 1 and 4) was rewritten for the browser. It waits for review. The rest (normalised coordinates, PDF converted on upload, file storage) does not change. Source: `architecture-decisions-guide.md` F3; related: D1 (PostgreSQL, no PostGIS), A2 (the Events service owns the venue map), F1 and G1. User stories (numbering of 22 October 2026): US-19 to US-23, US-27, US-46, US-47 (`features.md`).

Context

The venue map (F5, Must) is the shared spatial reference of the product: points of interest (US-20 to US-22), help requests (US-23), community reports (US-47) and the control room's view (US-27) are all placed on it.

- **One React web app** (F1). Participants and field teams open its phone pages in the browser of their phone (iPhone or Android); event management and the control room use its desktop pages on a laptop. The map must work with **touch on phones** and with **mouse and wheel on a laptop**.
- The map is **an image uploaded by an EventAdmin** (or a member with `Map.Edit`), as an image file or a PDF (US-19). There is **no geographic data**: D1 decided against PostGIS, and the domain model stores positions as `MapPosition`, **normalised coordinates** (x and y between 0 and 1) on a specific `VenueMapVersion`. Bus positions (`GeoPosition`) are a separate coordinate system and are not drawn on this map.
- The map needs **pan and zoom**, markers with an **icon per type**, **filtering by type** (US-22: "*only those points are highlighted*"), and **tapping to mark a location** (US-23, US-47).
- Expected load per map: tens to a few hundred points of interest, plus active reports and requests.
- The map should keep working offline once it has been opened (US-46, Could), so a single file that can be cached is preferred.

What was compared

Criterion	Why it matters here
Touch on phone browsers (Safari on iOS, Chrome on Android) and mouse on a laptop	Same map on phone pages and desktop pages
Pan and zoom quality	Participants read a dense map on a small screen, often in a crowd
Markers and filtering on top of the image	US-20 to US-23, US-47
Performance	A few hundred markers while zooming, on mid-range phones

Criterion	Why it matters here
Maintenance	The app must still build at the end of the project (Sprint C)
Testability	The Definition of Done asks for tests

Decision

1. **Rendering: the image plus an SVG overlay, inside `react-zoom-pan-pinch`.** The map image is an `<img>`, and the markers are elements of an inline `<svg>` of the same size on top of it. Both sit inside the `TransformWrapper` / `TransformComponent` of `react-zoom-pan-pinch`. The same map component is used by the phone pages (view, filter, mark a location) and by the desktop pages (place and edit points, control room).
   - **Why SVG:** the browser draws it with no dependency. Markers are ordinary React components, so filtering, icons and tests are plain React code, and each marker can have an accessible label.
   - **Why `react-zoom-pan-pinch`:** it zooms **any content**, not only an image, which is what a map with markers needs. It gives pinch and pan on touch, wheel zoom and drag with a mouse, and double-tap or double-click out of the box. It is MIT-licensed and maintained (version 4.2.0, September 2026).
   - **Coordinates:** each point is drawn at `x · width`, `y · height` of the displayed image, and its size is divided by the current zoom so icons stay readable. A tap or click is converted back to normalised coordinates before it is sent to the API, so positions do not depend on the screen size, the device or the image resolution.
   - **Filtering by type (US-22):** React state. The selected types are highlighted and the rest are dimmed.
2. **PDF maps become an image on upload, in the backend.** The Events service renders the PDF with **Apache PDFBox 3.x** (Apache 2.0, Java, so it fits the Spring Boot backend) into a PNG.
   - Only **single-page** PDFs are accepted, with a file-size limit, because the input is untrusted.
   - The rendered image is at most **4096 px on the long side**, and larger uploaded images are scaled down to the same limit. A larger image only costs mobile data and memory on the phone.
   - The `VenueMapVersion` keeps the **original file** and the **rendered image**. Clients only ever download the rendered image.
   - **Why on upload, not in the browser:** see *Alternatives considered*. Converting once means every page, on a phone or a laptop, shows the same raster, which normalised coordinates depend on.
3. Where the files are stored is **not** decided here. It was decided in the API outline (decision D6 = A, 23 October 2026): a volume of the Events service, with the references in its database (`api-outline.md` §10.1; local deployment, ADR-011).
4. **Check before the map is built.** Before US-21 and US-22 are implemented, a **spike** runs the map component with a few hundred markers on a **real iPhone (Safari)** and a **real Android phone (Chrome)**: pinch, pan, tap to mark a location, and that the page itself does not zoom instead of the map. **If the spike fails**, the pinch and pan layer is written directly with Pointer Events (about one component). The rest of this decision, the SVG overlay and the coordinate model, stays the same.

Consequences

- **Nothing to install and one codebase:** the map runs in any current phone or laptop browser, and the phone pages and the desktop pages share the same component.
- **One coordinate model end to end:** the backend and the web app use normalised `MapPosition` on a `VenueMapVersion`; no geo projection is needed.
- **All pages see the same raster.** Offline use (US-46, Could) needs the PNG of the map version cached by a service worker; that is decided with US-46.
- **Testable:** markers, filters and the tap-to-coordinate conversion are React components and functions, tested like the rest of the web app; the PDF conversion is a backend unit test.
- **We write the map behaviour ourselves:** marker scaling, hit areas and tap-to-coordinate conversion are our code and need component tests. There is no built-in clustering; if a map ever has too many points, clustering or a move to a canvas must be added.
- **The Events service parses untrusted PDFs.** PDFBox runs with size and page limits, and a failed conversion rejects the upload with a clear error instead of saving a broken map version.
- **Library risk:** one dependency for pan and zoom. If it stops being maintained or fails the spike, the fallback in *Decision* 4 replaces it without touching the markers.

Alternatives considered

Option	Why not chosen
Leaflet `CRS.Simple` (1.9.4, BSD-2-Clause)	Mature map behaviour (markers, popups, clustering) for free. Rejected because it is a geographic map library used for a flat image, it keeps its own layer model outside React, and its React wrapper `react-leaflet` 5.0.0 uses the Hippocratic licence, which is not an OSI open-source licence. Leaflet 2.0 is still in alpha. **Second fallback** if we need clustering later.
Pinch and pan written by hand with Pointer Events	No dependency, but the gesture maths (two-finger pinch centre, bounds, wheel zoom on a laptop) becomes our code to test. Kept as the fallback in *Decision* 4.
Canvas drawing (e.g. a canvas library)	Best performance for thousands of markers, but markers are no longer DOM elements, so tests and accessibility get harder. Not needed for a few hundred points.
PDF converted in the browser (e.g. pdf.js)	Every page would convert again on every device and could produce a different raster; offline caching gets harder.
React Native app with `react-native-svg` and `react-native-zoom-toolkit`	The version of this ADR approved on 22 October 2026. Dropped with the React Native app (ADR-002, #125).

Open points

- **Phone location (1 November 2026, record, #175):** product decision 4 of 30 October 2026 (phone location first, map as the fallback) is phased. Sprint B uses map points only. Before the phone's location arrives in Sprint C, this ADR must say how a phone position (latitude, longitude, accuracy) is placed on a map version with no geographic data, e.g. two or three reference points set by the EventAdmin; assessment M-1 changes with it.

- Phone spike on a real iPhone and a real Android phone (*Decision* 4): before US-21 and US-22; owner to be set when the stories are planned.
- ~~Where the original and rendered files are stored: guide G1.~~ Closed: a volume of the Events service (`api-outline.md` §10.1, decision D6).
- ~~Upload file-size limit: set when US-19 is specified.~~ Closed: 20 MB at the gateway for the map upload route (ADR-013, limit L-4).

Sources (checked 23 October 2026)

- npm registry: `react-zoom-pan-pinch` 4.2.0 (MIT), `leaflet` 1.9.4 (BSD-2-Clause), `react-leaflet` 5.0.0 (Hippocratic-2.1).
- Maven Central: Apache PDFBox 3.0.7.

ADR-008 · Transport data source

Status

Accepted: proposed by the team on 15 October 2026 and accepted by the professor (answer received by 21 October 2026), who also asked for a simulator or recorded responses under the team's control. Updated on 26 October 2026: WireMock confirmed as that simulator (guide J4, #155). Source: `architecture-decisions-guide.md` A2 and B2.

Context

- The assignment asks for integration with at least one external system or a realistic simulator, handling unavailable, late, incomplete or invalid data (§6.5, architecture driver D8).
- F9 shows participants how to get to and from the venue; transport source unavailable is one of the two degraded-mode scenarios (D2, `features.md` §3.1).
- The **Porto Digital urban platform** publishes open data through a FIWARE broker (NGSI v2), including the buses in circulation in Porto: `GET /v2/entities?q=vehicleType==bus` (`domain-model.md`).

Decision

1. The **Transports** service imports bus positions from the **Porto Digital urban platform** (FIWARE NGSI v2) with a scheduled job **every 30 s** (the *x* of the meeting with the professor on 22 October 2026, P4) and stores the result; users only read the stored data, never the API. It asks only for the buses within 2 km of the venue (NGSI geo-query). Timeout, retry and circuit-breaker values: guide J1, `architecture.md` §9.1.
2. The import handles the three failure cases: **unavailable** (timeout, HTTP error), **late** (observation older than the freshness limit of **5 min**, the *y* of the same meeting: past it, the app shows a warning) and **invalid or incomplete** data. Invalid data never replaces the last valid data, and each value keeps its observation and retrieval time.
3. Event shuttles and temporary stops are entered by the event team and do not depend on the external source.
4. The team keeps a **simulator of the API, or recorded real responses**, under its control (guide J4). Tool: **WireMock** (proposed in guide J2, #43; confirmed in guide J4 on 26 October 2026, #155), run as a container next to the local system, which proxies to the real API, replays recorded real responses and returns failures on demand. The responses recorded on 25 October 2026 and the stubs made from them are in `infra/wiremock/`. The source URL is configuration, so Transports can point at either. It is used for repeatable automated tests, to cause failures, delays and invalid data in the degraded-mode demo, and so the presentation does not depend on the real API. The real integration stays the Porto Digital API.

Consequences

- Real, live data from an open API, with a controlled replacement for tests and the demo.

- One more component to maintain (the simulator or the recordings), and it must stay close to the real NGSI v2 format.
- The source gives **bus positions only**, not timetables or predicted arrivals (domain model, `TransportDeparture`); departures come from manual data or another validated source.
- When the source is down, participants see the last valid data with its time and a warning (`architecture.md` §7 and §9).
- Positions are geographic coordinates, separate from the venue-map coordinates (`GeoPosition` vs `MapPosition`).

Rationale

- The Porto Digital API gives real, live bus positions from an open source, which is what participants need to leave the venue (F9); the GTFS feed gives static timetables only.
- The professor asked for the real integration and also for a simulator under the team's control; WireMock serves both the repeatable tests and the degraded-mode demo, and the presentation does not depend on the real API.
- A scheduled import with a freshness limit keeps users away from the external API and lets the app show the last valid data with its time when the source fails (§6.5).

Alternatives considered

Option	Why not chosen
STCP GTFS feed	Static timetables only; public availability was not confirmed
Weather API (e.g. IPMA)	Plan B only; less useful to participants than transport data
Simulator only, no real API	Fully under our control, but the professor asked for the real integration; the simulator is kept for tests and the demo (decision 4)

Open points

- Import interval (x = 30 s) and freshness limit (y = 5 min) decided on 22 October 2026 (guide J1 and J2), as asked in the meeting with the professor (P4).
- Resolved on 26 October 2026: the simulator tool is **WireMock** (decision 4, #155).

ADR-009 · Architecture documentation

Status

Accepted: decided by the team on 11 and 15 October 2026; the diagram layout was reviewed on 21 October 2026; the diagrams were updated on 26 October 2026 for the decisions since then (one web app, API gateway, Keycloak, local delivery; #133). Source: `architecture-decisions-guide.md` B1 to B7.

Context

- The Sprint A *Technical Design* deliverable asks for the high-level architecture, the main views and the decisions behind them (assignment §7.1.2).
- The architecture is microservices from the start (ADR-001): one web app, the API gateway, Keycloak, five services, RabbitMQ, one database per service and two external systems. One picture cannot show all of it at a readable size.
- The diagrams must stay editable by the whole team, live in the repository next to the code and be reviewed in pull requests like any other change.
- The asynchronous workflows (help-request triage, transport import) are easier to follow as sequences than as boxes (assignment §6.3).
- Delivery is local only, on one machine with Docker Compose (ADR-011, guide G1), so the physical view shows that machine.

Decision

1. **Notation: C4 inside the 4+1 views, without scenarios** (B6). The user workflows W1–W8 (`user-workflows.md`) already play the role of the scenarios.
   - **Logical view:** C4 boxes and the domain model. Level 1 is the system context, level 2 groups Farol into frontend, backend and infrastructure, level 3 shows the containers (web app, API gateway, Keycloak, five services, RabbitMQ, one database per service and Keycloak's) and level 4 the components inside one container.
   - **Process view:** dynamic diagrams of the help request with asynchronous triage and of the transport import, levels 1 to 4.
   - **Implementation (development) view:** repository, top-level folders, projects (one Maven project per service) and the hexagonal packages (`domain` , `application` , `adapters/in` , `adapters/out` , B5).
   - **Physical view:** where each part runs on the local machine of the local delivery (ADR-011), and the browser that uses it.
2. **Four zoom levels per view, three in the physical view.** The physical view stops at level 3, which shows one container per service and one PostgreSQL server with six databases, one per service and one for Keycloak (professor, 22 October 2026). Level 4 has **one diagram per service** (Reports, Transports). The web app is opened at level 3 instead (*3 web*: the level-3 diagram with the web app's components, or `frontend/` 's folders, in place of its box), so it is seen next to the gateway and Keycloak it calls. Every service follows the

same hexagonal layout, so Reports and Transports are the examples; the level-4 diagrams of Auth & Users, Events and Badges are written with each service.

3. **View mappings:** for each level, one diagram puts the logical diagram next to the implementation diagram and another puts it next to the physical diagram (levels 1 to 3), with dashed arrows from each logical element to its counterpart.
4. **Tool: draw.io** (B1), drawn with the draw.io MCP or by hand. Each diagram is one `.drawio.svg` file: an SVG that renders on GitHub and in Markdown and also embeds the editable draw.io source. There are no separate `.drawio` and image files to keep in sync.
5. **Layout rules:** one title per diagram (`[<View>, level N] Farol`), C4 colours, no legend, every arrow labelled with its purpose and, where useful, its protocol. Persons are a single **User** at logical levels 1 to 3 (updated on 27 October 2026, #133).
6. **Location:** `Docs/Technical/diagrams/<view>-view-level-<N>[-<part>].drawio.svg` and `mapping-logical-<view>-level-<N>[-<part>].drawio.svg` , linked from `architecture.md` §14. Service-specific diagrams written later go in `backend/<service>/docs/` (B4).
7. **Decisions:** ADRs in `Docs/Technical/adr/NNNN-title.md` , Michael Nygard format plus *Alternatives considered* (B7).

Consequences

- Each level answers one question at a readable size; the mappings show how the views fit together without merging them into one diagram.
- One file per diagram: renders in the browser, opens in draw.io, and its diff in a pull request is still text (though hard to read).
- More files to keep consistent: 32 diagrams today (logical 6, process 8, implementation 6, physical 3, mappings 9). A change to a logical diagram also needs the two mapping diagrams of that level.
- The Mermaid diagrams in `architecture.md` become working sketches; the draw.io diagrams are the reference.
- **[PENDING]** markers in the diagrams must be removed when the decision they wait for is taken. Since 27 October 2026 only the `frontend/` folder layout is left.

Rationale

- One picture cannot show the whole architecture at a readable size, so the views are split by zoom level, with mappings to show how they fit together.
- draw.io keeps the layout under control and is editable by the whole team; a `.drawio.svg` file renders on GitHub and holds its own source, so there is no image to keep in sync.
- The workflows W1–W8 already play the role of the scenarios, so the 4+1 views are used without a scenarios view.

Alternatives considered

Option	Why not chosen
Mermaid C4 or C4-PlantUML (diagrams as code)	Easy diffs, but little control over layout; large diagrams become unreadable (B1)
Structurizr (one model, many views)	Keeps the views consistent automatically, but one more tool and DSL for the team to learn
Separate `.drawio` source plus exported PNG	Two files per diagram that can drift apart; `.drawio.svg` holds both
4+1 with a scenarios view	The workflows W1–W8 already cover the scenarios (B6)
One level-4 diagram for all containers	Tried first; too dense, and it mixed apps and services that share nothing at that level

Open points

- Level-4 diagrams for Auth & Users, Events and Badges: written with each service (B4).
- Physical view: ✅ closed on 26 October 2026 with the local delivery (ADR-011, `local-delivery.md`).
- Identity provider in logical level 1: ✅ closed; Keycloak is self-hosted (ADR-006), so it is a container from level 2, not an external system.

ADR-010 · CI/CD strategy

Status

Accepted: decided by the team on 23 October 2026 (#131). Source: `architecture-decisions-guide.md` H1–H8. It brings together the versioning decision (H7, #127) and the local delivery procedure (`local-delivery.md`, #129, ADR-011). The walking skeleton is to implement it (R0, #58); the minimum Sprint A scope was narrowed by the 31 October review (#166); the full pipeline remains the Sprint B target.

Context

- The assignment asks for an automated pipeline that compiles, tests, runs static analysis and dependency or security checks, builds container images and deploys to a suitable environment, with rapid and visible feedback when a change breaks something (§8.5). Builds must be reproducible, secrets handled securely and rollback documented (§8.4).
- Five Spring Boot services and the API gateway (ADR-013, `backend/gateway/`), each its own Maven project in `backend/<name>/` with its own Dockerfile (ADR-001, ADR-002), and one React web app in `frontend/`, all planned in one monorepo (originally private, now public; guide H2, H8 and point 10). The initial workflow was `.github/workflows/conventions.yml` (branch name, PR title, linked issue on every PR); R0 now also has executable CI (point 9).
- Delivery is **local only**, because cloud hosting costs were not justified (ADR-011). The earlier "deploy to a cloud production environment on every merge" (guide H4, H5) no longer applies. Delivery means building from the release tag on a laptop with `docker compose` and running the smoke test by hand (`local-delivery.md`). CI never reaches anyone's machine.
- Versioning (guide H7): one SemVer version for the monorepo, an annotated tag `vX.Y.Z` on `main`, images `farol-<service>:X.Y.Z` (other builds `dev`), **no registry** and no GitHub Releases. Rolling back means building the previous tag.
- The Security and Privacy Assessment found no dependency or image scanning (finding 12, threat T18) and left the answer to this ADR.
- The 23 October pipeline was designed around a private repository and limited Actions minutes. On 31 October the repository was made **public** and SonarQube Cloud analysis was requested (#168). The earlier blanket claim that Sonar requires payment for private repositories was incorrect: its current Free offering also permits private projects up to 50k lines. See the activation guide; plan eligibility is confirmed in the owner account.

Decision

1. **Tool and layout (H1, H2, H3, H8). GitHub Actions** in the monorepo (now public; point 10). **Maven** with **Java 21** and **Spring Boot 4.x** for every service; every image is built from a **multi-stage Dockerfile** in its own folder (`backend/<name>/Dockerfile`, `frontend/Dockerfile`).

2. **Two workflows.**

- `conventions.yml`, unchanged, on every PR. Dependabot PRs are exempt, because Dependabot cannot follow the branch, title and issue conventions.
- `ci.yml` with the jobs below. Its required check is one final job, `CI`, that passes only if every other job passed or was skipped. Branch protection then needs only two required checks, **Conventions** and **CI**, and adding a service does not change them.

3. **Jobs and when they run (H4).** The whole repository is always checked: there are no per-service path filters, every service runs in every run that runs code. The one exception is a pull request or push that only changes documentation (every file under `Docs/` or ending in `.md`): it skips the code jobs.

Job	What it does	PR	Push to `main`	Tag `vX.Y.Z`, manual run
`changes`	Decides whether the change is documentation only	✓	✓	not needed: always everything
`secrets`	**gitleaks** over the PR's commits; fails on any secret	✓ (also docs)		
`backend` (matrix, one entry per service and the gateway)	`mvn -B verify`: compile, unit tests, integration tests with **Testcontainers** (PostgreSQL, RabbitMQ), **Checkstyle** (Google rules, fails the build)	✓	✓	✓
`web`	`npm ci`, **ESLint**, tests, `npm run build`	✓	✓	✓
`delivery`	Builds the seven images (five services, gateway, web app) with `docker compose -f infra/compose.yaml build` (`VERSION` is `X.Y.Z` on a tag, `dev` otherwise), scans them with **Trivy**, starts them with `up -d --wait` from `infra/.env.example`, runs the smoke test of `local-delivery.md` §7 without the phone and the login (all containers healthy, web app answers HTTP 200, the gateway answers `401` without a token, one RabbitMQ round trip), then `down -v`		✓	✓
`CI`	Required check: fails if any job above failed or was cancelled	✓	✓	✓

- **Pull requests get the fast part:** compile, tests, static analysis and secrets. **`main`, tags and manual runs get everything**, including images, the image scan and the smoke test. A Dockerfile or Compose error is found on `main` right after the merge; whoever merged fixes it forward straight away (`main` stays deployable).
- **Draft PRs** skip `backend`, `web` and `delivery`; they run when the PR is marked ready for review.
- **Manual run** (`workflow_dispatch`) runs everything on any branch. It is how the full pipeline is shown live in a sprint review.
- Runs of the same PR cancel the older run (`concurrency`). Every job has a `timeout-minutes`. Maven and npm downloads are cached (`setup-java`, `setup-node`).

4. **Tags (H7).** A tag runs the same full pipeline. It also checks that the tagged commit is on `main`. The images are built and tested but **not published or kept**: there is no registry and no image artefact. Delivering a release, and rolling back, mean building that tag locally (`local-delivery.md` §4, §5).

5. **Delivery (H5).** CD stops at a tested, tagged commit. The release is delivered by building that tag locally with `docker compose` and running the smoke test by hand (`local-delivery.md`). The pipeline never connects to a team member's machine. Cloud deployment is **not part of the current scope** (ADR-011); the pipeline's `delivery` job runs the same Compose file and smoke test on the CI runner instead.

6. **Secrets and permissions (H6).** No registry or cloud deployment credentials are needed. #168 **adds `SONAR_TOKEN` as a repository secret solely for Sonar analysis**, with organization/project identifiers in repository variables; fork/Dependabot code never receives it. Workflows run with `permissions: contents: read` (`conventions.yml` with none). The CI smoke test uses the harmless values of `infra/.env.example`. Any secret added later (for example, a hosted AI key, guide K1) is a GitHub Actions secret, never a file in the repository.

7. **Dependency and security checks.**

   - **Dependabot** (`.github/dependabot.yml`): security alerts and security update PRs on; version update PRs **monthly**, one grouped PR per ecosystem (Maven per service, npm, Docker, GitHub Actions), so they do not eat the Actions minutes.
   - **Trivy** in `delivery`: fails on a CRITICAL vulnerability that has a fix available; the others are reported in the job log.
   - **gitleaks** in `secrets`, on every PR.
   - **Dependabot and Trivy answer finding 12 of the security assessment** (T18, no dependency or image scanning): Dependabot covers the dependencies of every ecosystem, Trivy the images, including the pinned third-party ones (Keycloak, PostgreSQL, RabbitMQ). The finding is then marked as answered in the assessment.

8. **Branch protection (H8, README §6).** On `main`, the required checks are **Conventions** and **CI**, plus the review rules of README §5 and §6. **Active since 30 October 2026** through a repository ruleset: pull request with one approval, Copilot review, no force push (checked on 31 October 2026: `protected: true`). **No check is required yet. To do:** add `CI` (point 9; `ci.yml` exists since #167) and `conventions`, the name GitHub shows for the job of `conventions.yml` (README §6; `team-and-process.md` §4, agreed on 8 October 2026).

9. **Walking skeleton R0 — narrowed 31 October 2026 (#166).** Implement one Reports service, a synthetic help-request create/read demo plus diagnostic health, real HTTP tests, multi-stage Dockerfile and Compose. The actual `.github/workflows/ci.yml` has one required job named `CI`: Maven verify, container build/start, JSON smoke check and cleanup on **every PR**, main push, tag and manual run, including drafts/documentation changes. No path filters, cloud/registry access or deployment to a personal machine. The broader jobs in points 2–7 (matrix/web, Testcontainers, Checkstyle/ESLint, gitleaks, Trivy, Dependabot and release-tag ancestry/version handling) are the accepted **Sprint B target**, not implemented capabilities of the minimum. Two independently deployable backend components are a Sprint B requirement, not a Sprint A blocker. The first reviewed release tag remains `v0.1.0`; no release/tag is created by #166.

10. **SonarQube Cloud addition (#168, 31 October 2026).** Add an independent `SonarQube Cloud` job for same-repository PRs into main, main pushes and manual main runs. It builds/tests Reports with JaCoCo XML,

then runs the pinned Maven scanner and waits for the quality gate; missing configuration or a failed gate fails the job. Fork/Dependabot PRs and tags/feature-branch manual runs skip this job. No documentation/draft filters apply to R0. This implements Java analysis for the current Reports module only, not the full matrix/security pipeline above. The owner has created the Sonar project, both GitHub identifier variables are configured and `SONAR_TOKEN` is stored as a repository secret. Run 41202153038 attempt 2 completed authenticated PR #169 analysis with quality gate PASSED on 31 October. Setup, scope and verification. This is a user-requested implementation update, not a new team vote.

Consequences

- The R0 correction intentionally builds/tests and runs one container on every PR to provide visible evidence. The documentation/draft skip policy above applies when the full Sprint B pipeline is implemented, not to this minimum.
- A code PR costs roughly one runner-minute per service plus the web app and gitleaks; a full run on `main` adds the image builds and the smoke test. These are estimates. **Measured on R0 (31 October 2026, eleven successful runs of `ci.yml` from 12:59 to 17:55 UTC on PRs and `main`):** a whole run took 68–90 s. In runs 41203497613, 41208418720, 41222205000 and 41226836648, the `CI` job (Maven verify, image build, Compose start and smoke test of one service) took 61–75 s and the `SonarQube Cloud` job, in parallel, 59–71 s. Usage is checked at each sprint review; if it gets close to the limit, the next cut is to run `delivery` only on tags and manual runs.
- Image, Trivy and Compose failures appear after the merge, not before. That is the price of saving minutes: the team fixes `main` straight away instead of starting new work.
- The smoke test runs on a clean runner every time something reaches `main`, so `local-delivery.md` stays executable. The manual smoke test on the demo laptop is still needed: CI does not cover the phones, the LAN or the laptop's resources.
- Nothing is published, so a release is only as reproducible as its build. Base images are pinned in the Dockerfiles and Compose file (ADR-011); a rebuilt tag can still pick up newer OS packages.
- The documentation-only rule is narrow on purpose (`Docs/` and `*.md` only): a change to a workflow, a Dockerfile, the Compose file or any code runs the code jobs. Tags and manual runs never skip anything.
- No paid GitHub feature is required by this implementation. SonarQube Cloud adds Java static analysis and measured coverage (#168); Checkstyle/ESLint remain planned style checks, and gitleaks/Trivy/Dependabot remain separate planned security checks.

Alternatives considered

Option	Why not chosen
Everything on every PR (the original H4)	Simplest, catches image and Compose errors before the merge, but the full run on every push of every PR would use up the Actions minutes of a private repository. *Note of 31 October 2026:* the repository is public since 31 October (point 10), and the R0 `ci.yml` already runs its whole job on every PR (point 9); this reason is to be revisited when the Sprint B pipeline is built

Option	Why not chosen
A long-lived `dev` branch: light checks on PRs into `dev`, everything on `dev` → `main`	Saves the same minutes, but breaks GitHub Flow (README §1), needs merge commits instead of squash for `dev` → `main`, and finds errors later, mixed with other people's work
One workflow per service, with path filters	Fewer minutes per PR, but a change to a shared contract would not test the services that use it, and the required checks would change with every service (H4)
Publishing images to GitHub Container Registry on each tag	Lets the demo laptop pull instead of build, but needs a registry, credentials and retention; guide H7 chose git tags only
Keeping the images as workflow artefacts	Around 1–2 GB per run in Actions storage, and still a second way to deliver besides building the tag
Deploying to the cloud from the pipeline	Out of scope because of cost (ADR-011)
A self-hosted runner on a team laptop, to deploy there	The pipeline would reach a personal machine and depend on it being on; rejected in ADR-011
SonarQube Cloud	Originally excluded under an incorrect private-project pricing assumption; now selected (#168), subject to account setup and a real quality-gate result. Other security checks retain their own scope.
SpotBugs next to Checkstyle	More findings, more false positives and more minutes; can be added if Checkstyle proves too weak
OWASP Dependency-Check	Slow (downloads the NVD database) and overlaps with Dependabot and Trivy

ADR-011 · Local deployment and cloud alternatives

Status

Local-only delivery authorised by the professor, as relayed to the team on 22 October 2026. Source: recorded clarification, #53. This records the authorised delivery scope. **The local delivery procedure was decided on 23 October 2026** (#129; guide G4–G7 and H5) and is specified in `local-delivery.md`; the walking skeleton applies it. No completed deployment, model benchmark or cloud resource provisioning is claimed. The pipeline is in ADR-010.

This supersedes this ADR's initial proposal to validate Azure as the current deployment target. The complete cloud investigation and cost model remain available as evidence and a future alternative.

Context

Five services are independently deployable from Sprint B; Sprint A runs one, the R0 skeleton of Reports (ADR-001 decision 3, amended on 1 November 2026), each with its own database (ADR-003), communicating via RabbitMQ (ADR-004). Hosting the application, broker, databases and telemetry incurs costs not covered by an assumption of free student hosting. As relayed to the team (recorded 22 October 2026), the professor agreed the project may proceed locally, provided the team explains that cloud deployment was omitted because of cost.

Decision and proposed implementation

1. **Current scope: deploy and demonstrate locally.** No cloud platform, subscription, remote VM or managed-service purchase is required. Preserve the cost evaluation; include the cost reason and the professor's authorisation in the final report/presentation.
2. Keep the five backend services independently built and replaceable as containers on one local machine. The topology is five backend containers, the API gateway (ADR-013), Keycloak (ADR-006), RabbitMQ, **one PostgreSQL instance with six separate databases/users** (five services and Keycloak), a `web` container serving the React app and WireMock in front of the external systems (J2, J4): eleven containers. Local inference and observability are added behind Compose profiles. Phones open the web app in their browser over the LAN. Only the gateway, Keycloak and the web app reach the network; admin pages are published on `127.0.0.1` only. How phones log in, since the PKCE login needs HTTPS off `localhost`, is open (security assessment §9). Exact resource sizing remains to be measured.
3. Preserve service/database ownership and denied cross-service access. Sharing a PostgreSQL server or physical host does not permit shared tables or code dependencies. Separate PostgreSQL containers were considered and not taken: they add memory without adding ownership guarantees.
4. One versioned `infra/compose.yaml` with optional profiles, pinned base images, `infra/.env` created from a versioned `infra/.env.example` (no secrets in the repository), health checks and named volumes for PostgreSQL/RabbitMQ. Services wait for their database but not for the broker. Synthetic demo data comes

from demo-only Flyway migrations. Missing AI or broker must not block a durable request save/outbox entry. These remain implementation acceptance checks, not delivered application behaviour.

5. CI still builds and tests the images. Delivery is building locally from the release tag (guide H7) and running a manual smoke-test checklist (all containers healthy, web app opens, the gateway refuses a call without a token, login works, one RabbitMQ round trip), using only `docker compose` commands: no script or Makefile. This replaces the old cloud-production-deploy-on-every-merge expectation. CI never reaches a personal machine; it runs the same Compose file and smoke test (without the phone and the login) on its own runner on every push to `main`, every tag and manual runs (ADR-010).
6. Roll back by building the previous tag, for one service or all of them, without undoing Flyway migrations. Check background consumers, outbox and queue drain. Before every demo and update, `pg_dump` the six databases to return to a known state or move it to the backup laptop. Local volumes are not backups; queued messages and host/disk loss are not covered (RPO not guaranteed; production backup, D5, stays deferred). RTO is measured with the walking skeleton (guide G7); the in-memory Sprint A skeleton cannot measure it (#166), so when it is measured is open.

Consequences

- Avoids recurring cloud-hosting expenditure for the academic delivery. Existing local equipment, electricity and any optional API usage are not asserted to have zero cost.
- A shared local machine is a common failure/resource boundary. No cloud HA, managed scaling, public reachability or cloud recovery is demonstrated.
- Local independent service replacement, integration, degraded modes and observability still need evidence; the cloud exemption does not waive those concerns.
- The report/presentation must state the cost rationale and what was actually run, using the retained dated evaluation as support.

Alternatives considered

- Azure Container Apps + PostgreSQL + RabbitMQ VM: researched with costs, but not required/provisioned for this delivery after the professor's authorisation.
- ECS/Fargate + RDS + Amazon MQ: viable future alternative, still has hosting and operational costs.
- Vercel-only deployment: does not cover continuously running workers and the persistent broker by itself; unnecessary for local delivery.
- All containers on one cloud VM: a lower-complexity cloud alternative, but still incurs VM/storage cost; not required now.
- One physical machine per database/service: stronger host isolation at higher resource/operational cost; independent deployment does not require it.

Evidence still pending

Compose/skeleton implementation following `local-delivery.md`, minimum RAM of the demo laptop, measured RTO, repeatable local startup, independent update/restart, authorised smoke tests, event isolation, AI/broker failure tests and observed recovery/resource use. AI choice/privacy and detailed observability

proposals remain separately reviewable. No cloud benchmark is necessary to claim local delivery; do not claim one was performed.

Walking-skeleton documentary completion (26 October 2026, #58)

The course-slide-based R0 and local decision detail apply the accepted local procedure. R0 sends one synthetic minimal request through the authenticated web/gateway/Reports path, persists it and displays stored state; a real broker round trip completes the technical slice without AI. *Superseded for Sprint A on 31 October 2026 (plan, #166, ADR-010 point 9): R0 is one Reports service in memory; this slice is the Sprint B target.* The document adds environment separation, local resource experiment budgets, explicit smoke acceptance and timed recovery targets. It does not replace the Compose-only/manual local runbook with a custom startup script or claim measured RAM/RTO.

ADR-012 · Observability

Status

Accepted — architectural decision; implementation/evidence pending. Status reconciled on 31 October 2026 (#166) with guide I1–I3 and NFRs §5: team-confirmed local stack/SLOs on 26 October and the 27 October simplification to Prometheus/Grafana, correlated Docker logs and three Grafana alerts. This records the existing agreement, not a new vote. The local specification is accepted as the selected architecture; #148 is reopened for actual instrumentation/dashboard/alert work. No live dashboard, implemented metric or measured SLO is claimed.

Delivery-scope update: use the local observability option for the authorised local-only delivery. Azure Monitor/log ingestion proposals remain documented cloud alternatives, not current provisioning requirements. Targets and evidence still need validation.

Context

The asynchronous triage spans queueing, inference and persistence. Fallback must preserve operator visibility, while RabbitMQ, the AI and transport source can fail. Assignment §6.7 requires observable evidence. The research specification defines SLIs/denominators, dashboard panels, metric contracts, alerts, sampling and privacy controls.

Decision

1. Micrometer/Actuator health/metrics and structured logs with a correlation ID carried through HTTP and AMQP. Assignment §6.7 permits correlation or trace identifiers; no OpenTelemetry agent or distributed trace backend is required for this demo. Record submission-to-stored-suggestion timestamps, not only provider duration.
2. **Minimum local stack: two containers, Prometheus and Grafana.** Grafana-managed alerts use its built-in alert handling; no separate Alertmanager, Collector, Tempo, Jaeger or Loki. Inspect structured logs through Docker Compose. Scrape private service/RabbitMQ endpoints. Cloud tooling remains a researched future alternative.
3. Bounded labels and structured redacted logs; no personal request/prompt content, tokens, exact coordinates or high-cardinality identifiers in metric labels.
4. The versioned Farol local operations dashboard covers API success/latency, durable-request visibility, triage/fallback, outbox/queues/DLQ, broker/resources, transport freshness. Use three core Grafana alerts: service/telemetry down, outbox stalled and useful triage below target; inspect other signals on the dashboard/in acceptance checks.
5. Use product SC3/SC5 as mandatory acceptance evidence. The useful-triage target is **≥95% valid suggestions stored within 30 s over a one-hour submission cohort**, following the canonical ADR-005 coverage definition (aligned 26 October 2026, #145). Retain failures/abstentions in the operational

denominator and finalise after the cohort deadlines; SC5 visibility remains 100%. Confirmed targets still require runtime/load/degraded-mode evidence. No-data windows must not appear healthy.

Consequences

- Correlation across asynchronous boundaries using existing logs, plus two local metrics/dashboard containers.
- No dedicated trace store or log search UI; operators follow one correlation ID in Compose logs.
- Demo data is synthetic; redaction and bounded labels still apply.

Alternatives considered

- Local stack with an OpenTelemetry Collector, Tempo and a separate Alertmanager (the earlier proposal of #151), dropped in the 27 October simplification (#153): assignment §6.7 accepts correlation *or* trace identifiers, so correlation IDs in the logs are enough and a distributed tracing platform is unnecessary for the demo; Grafana-managed alert rules replace the separate Alertmanager (plan). It remains an optional future extension.
- Fully self-hosted Prometheus/Loki/Tempo/Grafana in cloud: portable but adds stores/patching/recovery for this team; optional future extension.
- Console logs only: insufficient latency/queue visibility and no measurable error budget.
- Provider-call latency only: hides outbox/queue/storage delays and cannot establish SC3.

Acceptance evidence pending

Correlated request log entries, scraped metrics/dashboard, no-sensitive-data inspection, one trigger/resolve alert drill and actual SC3/SC5 evidence. No cloud-ingestion quote or trace-store evidence is required for local delivery. The telemetry and the failure/load evidence are integrated with the CI/CD and resilience work.

I1–I3 completion record (26 October 2026, #148)

The decision specification fixes ports, 15 s scrape/evaluation, 1-day/512 MiB metric storage, rotated correlated logs, cohort accounting, four dashboard groups and three core Grafana alerts. Outbox alert is **>60 s for 2 min**, superseding the earlier 30 s research proposal and matching the NFR/resilience value. Configuration, compatible pinned versions and observed correlated logs/alerts/SLOs are implementation evidence, not open software-selection decisions.

ADR-013 · API gateway, CORS and rate limiting

Status

Accepted: decided by the team in the meeting of 22 October 2026 (record). Source: `architecture-decisions-guide.md` C6. Numbered 013 because ADR-011 (deployment) and ADR-012 (observability) were already taken.

Context

- One React web app (guide F1) calls five services deployed independently from Sprint B (guide A2, A4; Sprint A delivered one service, ADR-001 decision 3). Any later change of shape must be driven by evidence, and should not change the public API.
- Every request is authenticated (ADR-006). Reports must be rate-limited per account (US-48); help requests needed a decision.
- At the venue, thousands of phones share a few public IP addresses (carrier-grade NAT), so limits per IP would block crowds.
- The web app must be told to slow down at the read peak (429 or 503 with `Retry-After`, #43).
- Services talk only through events (ADR-004); the gateway must not compose answers from several services.
- Delivery is local only; Azure is the studied cloud alternative (ADR-011).

Decision

1. A **Spring Cloud Gateway (Server WebMvc)** application, `backend/gateway/`, is the **only public entry point** for the API. Spring Cloud 2025.1 with Spring Boot 4, virtual threads, no database.
2. **Routing by resource path** (`/api/v1/events/{eventId}/help-requests`), each path owned by exactly one service. The paths come from the API outline (guide C8); a test checks that each path reaches its service.
3. **CORS only at the gateway**, for the web app's origins. Never `*`, no credentials.
4. The gateway **validates the JWT** (services validate again) and checks no event roles or permissions: it holds no business logic.
5. **Rate limits per account** (the token's `sub`), never per IP, with Bucket4j. A limit answers 429 with `Retry-After` in `application/problem+json` .
6. **Reports per account** (5 per 10 min, US-48) are a **business rule in the Reports service**, not gateway configuration.
7. The gateway creates the correlation ID and returns it as `X-Correlation-Id` ; there is no distributed tracing (ADR-012, decision 1). It counts 429s per policy.
8. **Hosting:** local `docker compose` , with only the gateway (`localhost:8080`), Keycloak and the web app published. If the cloud alternative is resumed, **Azure Container Apps**: gateway, Keycloak and web app with external ingress, services with internal ingress.

Route table

Paths from the API outline (#123, merged on 26 October 2026 in #124). The outline owns the paths; the table follows it, most specific first.

Order	Path	Service
1	`/api/v1/me/badges/**`	Badges
2	`/api/v1/me/**`, `/api/v1/accounts/**`, `/api/v1/events/*/members/**`	Auth & Users
3	`/api/v1/events/*/help-requests/**`, `/api/v1/events/*/reports/**`, `/api/v1/events/*/field-tasks/**`, `/api/v1/events/*/triage-status`	Reports
4	`/api/v1/events/*/transport-options/**`, `/api/v1/events/*/bus-observations/**`, `/api/v1/events/*/transport-source-status`	Transports
5	`/api/v1/events/**` (event, teams, map versions, points of interest, announcements, check-in)	Events
—	anything else	404 at the gateway

Each route's target comes from an environment variable (e.g. `FAROL_ROUTE_REPORTS_URI=http://reports:8080`). If evidence later moves a resource to another service, only that target changes.

Rate limits

#	What	Key	Limit	Where
L-1	Every `/api/v1/**` call	account	120 requests/min	Gateway
L-2	`POST …/help-requests`	account	**5 per 10 min**	Gateway
L-3	`POST …/reports`	account + event	5 per 10 min (US-48)	Reports service
L-4	Venue map upload	—	body ≤ 20 MB (default 1 MB elsewhere)	Gateway, that route only
L-5	Login, sign-up	account	Keycloak brute-force detection	Keycloak

Counters are kept in memory (Bucket4j with Caffeine) while there is one gateway instance. The values are settings, tuned with the k6 results in Sprint C.

CORS and edge settings

- **Allowed origins:** the web container's origin and `http://localhost:5173` (Vite dev server), from configuration.

- **Headers:** allow `Authorization`, `Content-Type`, `If-None-Match`; expose `ETag`, `Retry-After`, `X-Correlation-Id`, `X-RateLimit-Remaining`.
- **Timeouts:** connect 2 s, response 10 s (longer on the upload route). An upstream that is down gives 503 with `Retry-After`; no retries at the gateway.
- `X-Forwarded-*` trusted only from the entry point in front; client identity headers are never passed on. Spring Security default response headers. The actuator is on a management port that is not published.

Consequences

- One more container, and a single point of failure while there is one instance (guide C5). It is stateless apart from the rate counters and restarts in seconds.
- The public API does not change when a resource moves to another service: only the route table changes.
- The route table must follow the API outline; the ownership test catches drift.
- Rate counters are lost on restart; if the gateway is scaled out, they move to Bucket4j's PostgreSQL backend (no Redis).
- A participant who sends more than 5 help requests in 10 minutes is refused. The page must say when they can ask again and show the first-aid points (backlog, US-23). Accepted over no limit plus the operator's *Duplicate* status.
- The gateway is part of the k6 load scenarios (#43); they show whether it or a service saturates first.

Alternatives considered

Option	Why not chosen
No gateway	CORS and limits repeated in five services; clients see every service's address
Managed cloud gateway (Azure API Management, AWS API Gateway)	Cannot run in the local delivery or be tested in CI; per-user limits depend on product and tier
Generic reverse proxy (NGINX, Traefik, Envoy, Caddy)	Per-account limits need JWT handling that is missing or a plugin; new configuration languages for the team
Spring Cloud Gateway WebFlux	Reactive model new to the team; higher ceiling not needed if the k6 scenarios confirm WebMvc
Kong Gateway open source	Uncertain availability of open-source images after 3.9
Service prefixes in paths (`/api/v1/reports/...`)	Clients would see the service split; moving a resource would change its URL
No limit on help requests	Floods reach the control room; the operator's *Duplicate* status alone was judged not enough

Security and Privacy Assessment

> LABDSOF 2026/27 · Sprint A · Assignment §7.1.2 (deliverable 9: *initial threat model, sensitive data, access-control requirements, data-minimisation measures, retention considerations*), §6.4, §6.6 and §11
> **Guide item:** E5. **Status (25 October 2026):** first version, **to review with the team**. **Updated 29 October 2026** (#159): retention and backups (§7) follow the values the team decided for NFR-13 and NFR-19. **Retention and consent confirmed by the team on 26 October 2026** (NFRs §5). **Updated 31 October 2026** (#162): ADR-005 and ADR-012 accepted, K1 chose a local model, and the Sprint A skeleton is assessed in §9.1. Controls that are already decided link to their ADR or NFR; every new control or number is marked **[PROPOSED]**. Apart from the Sprint A skeleton (§9.1), nothing is implemented yet, so this assesses the design, and §11 names the test that will prove each control. The threat model is updated with the implementation in Sprint C (assignment §7.3.2, item 6).
> **Sources:** `architecture.md`, `domain-model.md`, ADR-003, ADR-004, ADR-005, ADR-006, ADR-007, ADR-011, ADR-012, ADR-013, `features.md`, the non-functional requirements, `local-delivery.md`, the local observability plan, the API outline (#123, merged on 26 October 2026 in #124) and the Responsible AI assessment.

Summary

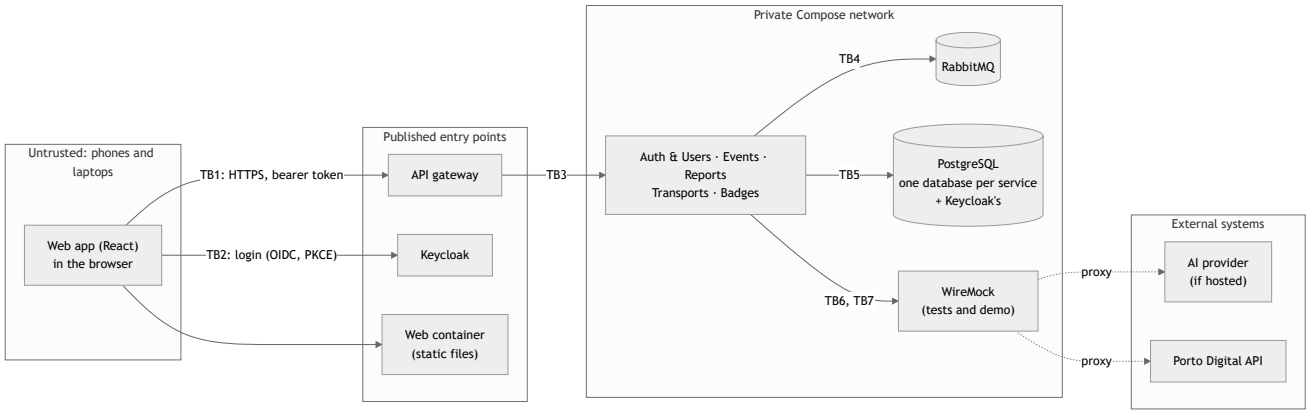
- **The most sensitive data is the help request:** a point on the venue map and an optional text that can describe someone's health, an assault or another person. It stays in Reports, is seen only by the people who handle it, never reaches logs or the message broker, and is minimised before any AI call.
- **The top risk is one event, or one participant, reading another's data** (SC8). The cross-event controls are decided (ADR-003, ADR-006). This assessment adds the check *inside* an event, with its test (T6).
- **Three findings need a change before the features that depend on them are built:** the FarolAdmin bootstrap trusts an unverified e-mail (T4), phones cannot log in to the local demo safely over plain HTTP (§9), and the triage message carries the help-request text through RabbitMQ (T11).
- **Retention, decided (NFR-13, NFR-19):** help-request content is deleted 30 days after the event closes, and only anonymous statistics stay. Container logs rotate at 10 MiB × 2 files, metrics are kept 1 day and no traces are stored. The six databases are dumped before every demo and every update, and each dump is kept at most 30 days.
- **The MVP uses only test accounts and synthetic data** (assignment §11). A real deployment would also need a DPIA and data-processing agreements with the organisers and with any hosted AI provider (§12).

1. Scope and method

Assessed	The MVP as designed on 25 October 2026: one React web app, the API gateway, Keycloak, the five services, RabbitMQ, PostgreSQL (one database per service), WireMock for tests and the demo, the AI provider and the Porto Digital API. Local-only delivery on one machine (ADR-011)

Not assessed	Physical safety at the venue; ticketing (F20) and chat (F19), which are out of the release; the cloud alternative, except where §9 says what would change
Threat model	STRIDE (spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege) on each trust boundary of §2
Privacy	GDPR principles: purpose limitation, data minimisation, storage limitation, special categories of data (art. 9) and data-subject rights. This is the team's reading for the design, not legal advice
Risk	**High:** could expose help requests or give someone admin rights. **Medium:** limited exposure, or needs a second failure. **Low:** little harm or very unlikely
People whose data Farol holds	Participants (P1), event team members and volunteers (P2, P3, P5), the FarolAdmin (P4), and **third parties** described in a help request (e.g. "my friend fainted")

2. Trust boundaries



#	Boundary	What crosses it	Main threats (§4)
TB1	Browser → gateway	Every API call, with the access token	Forged or stolen tokens, other people's data, floods, injected text (T2, T5–T7, T10, T15)
TB2	Browser → Keycloak	Passwords, sign-up, sessions	Credential guessing, account takeover, scripted sign-ups (T1, T4, T14)
TB3	Gateway → services	Calls already checked by the gateway	Bypassing the gateway, forged identity headers (T3)
TB4	Services ↔ RabbitMQ	Domain events (memberships, teams, help requests)	Forged events, personal data at rest in queues (T8, T11)
TB5	Services → PostgreSQL	All stored data	A service reading another's database (AC-8)
TB6	Reports → AI provider	Category and sanitised text of a help request	Disclosure to a third party, prompt injection (T12, T21)
TB7	Transports → Porto Digital	Bus positions coming in; nothing personal goes out	Invalid or hostile data (T20)

#	Boundary	What crosses it	Main threats (§4)
TB8	Operators → admin surfaces	Keycloak admin console, RabbitMQ management, WireMock admin API, Grafana, Actuator, PostgreSQL	Full control of the system (T16, §9)

3. Sensitive data

Every item Farol stores, where it lives and who can see it. Retention is in §7.

Data	Stored in	Who can see it	Class	Why Farol needs it
Password hash, sessions, brute-force counters	Keycloak's database	Nobody in Farol; Keycloak admin only	**Secret**	Login (F1)
E-mail	Keycloak; Auth & Users	The user; the FarolAdmin when looking up an account (AC-9)	Personal	Login name; naming an EventAdmin (US-06, US-08)
Display name	Auth & Users; copies in Reports and Events (`UserProfileChanged`)	The event team, for the members who act (who is on the way); the FarolAdmin, for EventAdmins	Personal	Accountability in the backoffice (US-07, US-32)
`sub` (account id)	Every service, logs	The team (developers)	Pseudonymous	Links records to an account without a name
Memberships, roles, team assignments	Auth & Users; copies in every service that checks permissions	EventAdmins of that event; the member	Personal	Access control (ADR-006)
Help request: category, map point, optional text, times, status	Reports	The requester (status only); `HelpRequests.Triage`; EventAdmins; the routed team	**Special category possible**	Core journey (F6, F7)
Triage suggestion and its reason	Reports	`HelpRequests.Triage`	Same as the request	AI assistance (F8)
Audit entries: actor, time, old and new values	Each service	`HelpRequests.Triage` (history), EventAdmins	Personal	Accountability (assignment §6.6, SC9)

Data	Stored in	Who can see it	Class	Why Farol needs it
Community report: type, map point, reporter, times	Reports; Badges keeps report id, type and reporter	Reviewers of its type; other participants only when confirmed, **without the reporter** (US-49)	Personal (where a person was, when)	F12, F13, badges (F14)
Check-in: account, event, time	Events; the count in Badges	The user	Personal (presence at an event)	Attendance badge (F15, F16)
Badge progress and preferences	Badges	The user; others never (outline D9)	Personal	F14, F15
Consent records (§8.4)	Auth & Users	The user	Personal	Proof of consent (GDPR art. 7(1))
Login and admin events (IP address, account, result)	Keycloak	Keycloak admin	Personal	Account protection
Container logs and metrics (no traces are stored)	Docker logs, and sanitised exports of them; Prometheus	The team	Pseudonymous: `sub` and correlation id only (ADR-012)	Operations (assignment §6.7)
Database dumps	`backups/` on the demo machine and the backup laptop; never in git	The team	Same as the databases they copy, Keycloak's included	Recovery (NFR-13)
Rate-limit counters	Gateway memory	Nobody	Pseudonymous, minutes	ADR-013
Friends (Could, US-61)	Auth & Users	The two users	Personal (social graph)	F18, after the MVP
Event, venue map, points of interest, announcements, teams, shuttles	Events, Transports	Members of the event	Not personal	Operational information
Uploaded map files	Events volume	`Map.Edit`; clients get only the rendered image (ADR-007)	Not personal, but the original file can hold metadata (author, GPS in a photo)	F5
Bus positions	Transports	Members of the event	Not personal: public open data about vehicles, latest position only	F9

Data	Stored in	Who can see it	Class	Why Farol needs it
User research	Google Forms (outside the repository)	The team	Anonymous: no name or e-mail collected (survey script, `setCollectEmail(false)`)	Problem validation

Why help requests are the most sensitive data. The category `Health` and the optional text can reveal health data (GDPR art. 9) about the requester or about someone else who cannot be asked ("friend fainted near the bar"); `Safety` and `Theft` can describe an aggressor; and the map point says where a person was at a given minute. The problem report already flags this (report §5, regulatory risks; architecture driver D5). Everything in §5 to §8 is strictest for this data.

4. Initial threat model

Status: *Decided* links to an ADR or story; **[PROPOSED]** is new and waits for the team. Tests for each threat are in §11.

#	Threat	STRIDE	Risk	Controls
T1	Guessed or reused password of an event team member, then reading help requests	S	High	*Decided:* Keycloak brute-force detection per account, never per IP (venue NAT); password of at least 10 characters, not equal to the e-mail; generic login error (ADR-006, US-01). **[PROPOSED]:** the bootstrap FarolAdmin account must set up a one-time-password app at first login (Keycloak *Configure OTP* required action in the realm export). Multi-factor login for event teams stays a known limitation (§12)
T2	Forged, expired or wrong-audience token	S	High	*Decided:* the gateway and every service check signature, issuer, audience `farol-api` and expiry (ADR-006)
T3	Calling a service directly, around the gateway, or sending forged identity headers	S, E	Medium	*Decided:* only the gateway, Keycloak and the web container are published; services validate the token themselves; client identity headers are never passed on (ADR-013)

#	Threat	STRIDE	Risk	Controls
T4	**Becoming FarolAdmin by signing up with the bootstrap e-mail.** ADR-006 grants `FarolAdmin` when a new account's e-mail equals `FAROL_BOOTSTRAP_FAROL_ADMIN_EMAIL`, but e-mails are not verified (no SMTP) and sign-up opens in Sprint C (US-03): whoever signs up first with that e-mail gets the role	E	**High**	**[PROPOSED]:** grant the role only when the token says `email_verified: true`. The realm import creates the bootstrap account already verified; a self-registered account is never verified while there is no SMTP. Amends ADR-006
T5	Reading or changing another event's data by changing `{eventId}`	I, T	High	*Decided:* `event_id` filter on every query; membership checked in the use case against the local copy; non-members get 404; an isolation test in every service (ADR-003, ADR-006, US-14)
T6	**Reading another participant's help request or pending report in the same event** by changing its id (broken object-level authorisation)	I	High	The API outline restricts `GET …/help-requests/{id}` to the requester, `HelpRequests.Triage` and the routed team, but no story tests it. **[PROPOSED]:** every query by id filters by `eventId` and by who may see the object (AC-3); US-14's isolation test gets a within-event case: a RegularUser of event X opens another participant's request of event X and gets 404
T7	Setting fields the caller must not set (priority, status, team, reporter, actor) in a request body	T, E	Medium	*Decided:* contract-first DTOs (guide C7). **[PROPOSED]:** request DTOs hold only the fields the use case accepts; actor and reporter always come from the token's `sub`; unknown fields answer 400
T8	A forged domain event, e.g. a `MembershipChanged` that makes someone EventAdmin	S, E	Medium	*Decided:* broker only on the private network; one RabbitMQ user per service (ADR-006, E3). **[PROPOSED]:** each user can write only to its own exchange `farol.<service>` and read only its own queues (AC-8); the default `guest` user is removed

#	Threat	STRIDE	Risk	Controls
T9	Hostile venue-map upload: decompression or PDF bomb, script inside an SVG or PDF, a file disguised by its extension	T, D	Medium	*Decided:* at most 20 MB at the gateway (ADR-013, L-4); single-page PDFs rendered to PNG of at most 4096 px; clients download only the rendered image (ADR-007). **[PROPOSED]:** type read from the file content (PNG, JPEG or PDF; never SVG); every upload re-encoded to PNG, which drops EXIF and PDF metadata and any active content; PDFBox with a memory limit and a render timeout; served with a fixed `Content-Type` and `X-Content-Type-Options: nosniff`
T10	Stored cross-site scripting through text other people see (help-request text, team, point and announcement names, the AI's reason, route names from Porto Digital), which could steal the tokens held in memory	T, I, E	High	*Decided:* tokens in memory only, refresh tokens rotated (ADR-006). **[PROPOSED]:** all such text is rendered as plain text (React escaping; no `dangerouslySetInnerHTML`; no HTML or Markdown in user text); a Content Security Policy on the web container: `default-src 'self'`, `connect-src` only the gateway and Keycloak, `object-src 'none'`, `frame-ancestors 'none'`; no third-party scripts, fonts or analytics
T11	**Personal data in logs, traces, queues or test recordings.** The outline's `HelpRequestSubmitted` carries the location and text, so they sit in RabbitMQ, in a dead-letter queue if the message fails, and in message traces	I	**High**	*Decided:* no request text, prompts, exact locations, names or tokens in logs or metric labels (ADR-012). **[PROPOSED]:** the triage message carries only `requestId`, and the triage worker, which is part of Reports, reads the request from Reports' own database (M-5); dead-letter handling logs the message id, type and reason, never the body; WireMock records Porto Digital only, never the AI provider

#	Threat	STRIDE	Risk	Controls
T12	Help-request text sent to a hosted AI provider that keeps it or trains on it	I	Low in the MVP: K1 chose the local model and there is no hosted fallback (NFR-20). High with real data if the hosted alternative is resumed	*Decided* (ADR-005, accepted 31 October 2026): local Ollama, no text leaves the demo machine; the AI gets only the category and sanitised text, no ids or location. If the hosted alternative is resumed: synthetic data until the provider is reviewed, and **[PROPOSED]** the redaction of M-4 and the provider checklist of §8.5
T13	A lost or shared phone of a field-team member with an open session (sessions last up to 30 days with *Remember me*)	S, I	Medium	*Decided:* access token of 5 min, refresh tokens rotated, logout (ADR-006, US-02). **[PROPOSED]:** actions that need an operational permission require a login in the last 12 h (AC-7); a routed team sees a request only while its field task is open (AC-3)
T14	Account enumeration, and the wrong person named EventAdmin: an account with the organiser's e-mail may not belong to the organiser, because e-mails are not verified	S, I	Medium	*Decided:* Keycloak's sign-up form reveals whether an e-mail is taken; accepted for the MVP (ADR-006). **[PROPOSED]:** the lookup rules of AC-9, which answer outline O4
T15	Floods: help requests, polling or uploads from one account; scripted sign-ups once sign-up opens	D	Medium	*Decided:* limits L-1 to L-5 per account, Keycloak brute-force detection (ADR-013). **[PROPOSED]:** calls with no valid token get a per-IP limit at the gateway; only broken clients or attackers send them, so crowds behind one address are not affected. Scripted sign-ups without e-mail verification: known limitation (§12)
T16	Exposed admin surfaces: Keycloak admin console, RabbitMQ management, WireMock admin API (fake bus data or fake AI suggestions), Grafana, Actuator, PostgreSQL	E, T	High	*Decided:* Actuator on a management port that is not published (ADR-013). **[PROPOSED]:** the port rules of §9

#	Threat	STRIDE	Risk	Controls
T17	Secrets in the repository: `.env`, the realm export (client secret of `farol-auth-users`, test passwords), an AI API key	I, E	Medium	*Decided:* `.env` outside git, GitHub Actions secrets in CI (ADR-006, E4); gitleaks on every PR is planned for Sprint B (ADR-010 points 7 and 9) and is not in `ci.yml` yet. **[PROPOSED]:** the realm export uses `${…}` placeholders for client secrets, filled from the environment at import; a `.env.example` lists names, never values; test passwords exist only for seeded test accounts
T18	A vulnerable dependency or base image	T, E	Medium	**[PROPOSED]** for the pipeline (ADR-010): dependency updates and checks (e.g. Dependabot) and a container-image scan (e.g. Trivy) on every PR, failing on critical findings (assignment §8.5, *dependency or security checks*)
T19	A team member denies having changed a priority, closed a request or unconfirmed a report	R	Medium	*Decided:* audit entry with actor, time, old and new values (SC9, US-28, US-51); the acting user's `sub` in the event envelope (ADR-006). **[PROPOSED]:** the application only appends audit entries; it has no code path that updates or deletes them, except the retention clean-up (§7)
T20	Invalid or hostile data from Porto Digital: oversized payloads, strings shown in the UI	T, D	Low	*Decided:* timeouts, validation, whole import rejected, geo-query that limits the size (ADR-008, J1, US-40). Strings rendered as text (T10)
T21	Prompt injection through the help-request text	T	Medium	Covered by the Responsible AI assessment: untrusted JSON input, no tools, fixed output schema, the operator decides (ADR-005). Privacy side: the model's reason can repeat the text, so it is shown only to the control room

5. Access-control requirements

The permission model is decided in ADR-006 (*Where each permission is checked*) and `features.md` §4. These are the rules every service must meet; each one is testable.

#	Requirement	Status
AC-1	**Authentication:** every `/api/v1` call needs a valid access token. There is no public API endpoint; login and sign-up happen in Keycloak	*Decided* (ADR-006, ADR-013)

#	Requirement	Status
AC-2	**Event scope:** a call about an event needs a membership of that event, checked in the use case against the service's local copy. A non-member gets 404; no copy means no access	*Decided* (ADR-006, US-14)
AC-3	**Object scope inside an event:** see the table below	**[PROPOSED]**
AC-4	**Permissions:** each use case checks one permission from the fixed list. An EventAdmin has every permission of their own event. `FarolAdmin` is never accepted for an event's help requests, reports or participants	*Decided* (ADR-006, US-07)
AC-5	**Changes to roles, teams and permissions:** only EventAdmins of the event, who can also make another member of the event EventAdmin or remove that role (US-15); the FarolAdmin names an event's first EventAdmin, or a new one (US-06, US-08); an event always keeps one EventAdmin; every change audited; applied in ≤ 5 s	*Decided* (ADR-006, US-06, US-08, US-15 to US-18)
AC-6	**Revocation:** a disabled or deleted account is refused at once by every service (account-disabled event, `UserDeleted`), and its sessions end in Keycloak	*Decided* (ADR-006)
AC-7	**Recent login for operational actions:** calls that need `HelpRequests.*` or `Reports.*` are refused when the token's `auth_time` is older than 12 h, with `401` and `WWW-Authenticate: Bearer error="insufficient_user_authentication"` (RFC 9470); the web app then sends the user to log in again. A participant on their own phone keeps the 30-day session	**[PROPOSED]**
AC-8	**Service-to-service:** no HTTP between services. RabbitMQ: each service's user can configure and write only its own exchange (`farol.<service>`, and its dead-letter exchange) and queues named `<service>.…`, and read only its own queues and the exchanges it binds them to. PostgreSQL: one login per service, owner of its own database only, `CONNECT` revoked on the others, no superuser. Keycloak: the `farol-auth-users` service account has only the user-management roles that account deletion needs	*Decided:* the principle (ADR-006, E3). **[PROPOSED]:** these grants
AC-9	**Account lookup by e-mail** (answers outline O4): FarolAdmin only; exact match, no partial search and no list of accounts; answers only the account id and display name; audited; under the normal rate limit. Because e-mails are not verified, the FarolAdmin confirms with the organisation, outside Farol, that the account is theirs before naming them EventAdmin (US-06 already asks to confirm that the requester represents the organisation)	**[PROPOSED]**
AC-10	**Audit records** (assignment §6.6): changes to help requests and reports (actor, time, old and new values); role, team, permission and membership changes; EventAdmins named, by an EventAdmin (US-15) or by the FarolAdmin (US-08); events suspended by the FarolAdmin; refused cross-event attempts (user, event, time; US-14); account deletions (only that one happened, and when). Logins in Keycloak's event log	*Decided:* help requests, reports, roles (SC9, ADR-006). **[PROPOSED]:** the rest
AC-11	**Infrastructure admin access:** see §9	**[PROPOSED]**

AC-3, who may see and change each object:

Object	Who may read it	Who may change it
Help request	The requester: only id, category, status and time of the last change (US-24). `HelpRequests.Triage` and EventAdmins: all of it. Members of the routed team: category and map point, **while the field task is open [PROPOSED]**	`HelpRequests.Triage`: priority, status, team, close. The member who accepted the field task: accept and resolve

Object	Who may read it	Who may change it
Help-request history	`HelpRequests.Triage`, EventAdmins	Nobody: append only (T19)
Community report	The reporter: their own, in any status. Other participants: confirmed and active, without the reporter (US-49). `Reports.Review` for its type: all of it, **without the reporter's name [PROPOSED]**. EventAdmins: also the reporter, to act on misuse (features §6.4)	`Reports.Review` for its type: confirm, reject, unconfirm, dispatch. The member who accepted the field task: answer, resolve
Check-in, badges	The user only; no endpoint for another user's badges (outline D9)	The user: badge preferences (US-56)
Account	The user; the FarolAdmin, through AC-9 only	The user (US-04, US-05)
Memberships and team assignments	EventAdmins of that event; the member, for their own	EventAdmins of that event, including making a member EventAdmin (US-15); the FarolAdmin, only to name the first or a new EventAdmin (US-06, US-08)

6. Data-minimisation measures

#	Measure	Status
M-1	**Location is a point the participant marks on the venue map**, normalised to one map version; the app never asks the phone for GPS. It is less precise on purpose and works inside a crowd	*Decided* (F6, user-workflows W2). **Phased on 1 November 2026** (record, #175): product decision 4 of 30 October 2026 (#164, report *User Research*) asks for the phone's location first, with the map as the fallback. Sprint B keeps the map point, so this measure holds. The phone's location comes from Sprint C, after ADR-007 places a phone position on a map version and this measure and the API location field are amended for it (consent, shared only with the request)
M-2	The help-request text is **optional, at most 500 characters**, with a hint above it: "*Describe what is happening. Do not write names or phone numbers.*"	**[PROPOSED]** (500, as in the Responsible AI assessment)
M-3	The requester sees only the status of their request; its location goes only to the people who handle it (AC-3)	*Decided* (US-23, US-24)
M-4	The AI gets only the category, the sanitised text and the priority floor: no account or event id, no location, no names (ADR-005). **[PROPOSED]:** before sending, phone numbers, e-mail addresses and document numbers are replaced with a placeholder. Redaction is not anonymisation: the text can still say something sensitive	*Decided* (ADR-005, accepted 31 October 2026); redaction **[PROPOSED]**

#	Measure	Status
M-5	**The triage message carries only `requestId`.** Producer and consumer are both Reports, so the worker can read the request from its own database: no personal text in RabbitMQ, in dead-letter queues or in message traces. An exception to the outline's full-state rule for one internal queue	*Decided* (API outline §7.3, #123, 26 October 2026)
M-6	**The account holds an e-mail and a display name only.** No phone number, birth date, student number, photo or university. The display name can be a nickname	*Decided* 26 October 2026 (NFR-19, NFRs §5)
M-7	Names are copied only where they are shown: `userId` and display name, to Reports and Events (`UserProfileChanged`); Badges and Transports get no names	*Decided* in the API outline (#123, its decision D4; merged on 26 October 2026)
M-8	Other participants never see who made a report; Badges receives the reporter, type and status of a report, not its location	*Decided* (US-49; `ReportConfirmed` in the API outline, #123, merged in #124)
M-9	A check-in is one time per account and event: no location, no list of nights; badges never show which nights or where (features §6.4)	*Decided* (US-58)
M-10	Logs and metrics hold no request text, locations, names or tokens; only `sub` and the correlation id identify a request (ADR-012). **[PROPOSED]:** no client IP addresses in application logs; Keycloak's login events keep them for account protection (§7)	*Decided* (ADR-012, accepted 31 October 2026); IPs **[PROPOSED]**
M-11	Map uploads are re-encoded to PNG, so the photo's GPS position and the PDF's author are dropped; participants receive only that image	*Proposed in ADR-007*; re-encoding **[PROPOSED]** (T9)
M-12	Bus data: only buses within 2 km of the venue, and only the latest position of each; no history of vehicle positions	*Decided* (J1, domain model)
M-13	**Tests and demo use synthetic data only:** seeded accounts with fictitious names and e-mails under the reserved `.test` domain; help-request texts written for the purpose; WireMock records Porto Digital, never the AI provider	*Decided:* synthetic data (assignment §11, ADR-005). **[PROPOSED]:** the rest
M-14	No analytics, trackers or third-party scripts in the web app; the Content Security Policy of T10 enforces it	**[PROPOSED]**

7. Retention and deletion

Assignment §6.4 asks for retention and deletion, and the privacy notice (§8) must say how long data is kept. The team decided the periods below on 26 October 2026, as NFR-19 and NFR-13 (non-functional requirements §5;

guide D5 and G7). Telemetry follows the local observability plan, simplified on 27 October 2026 (#153), and backups follow `local-delivery.md` §6. Until the clean-up jobs exist (Sprint C, one scheduled job in Reports and one in Events), the periods of help requests, reports and check-ins are a promise that is not enforced yet (§12).

Data	Kept	Then	Why
Help request: text, map point, triage suggestion and reason, field tasks, audit entries	Until the event closes, plus **30 days**	Deleted. Category, priority, status, times and team stay as statistics, without the requester or the actors	Long enough for the organiser to deal with what follows a festival night (a complaint, a lost-person case, an insurance or police question); short for data that can be about health (problem report: "*short retention*")
Community report	On the map until it expires (20–60 min, features §6.1); in Reports until the event closes, plus 30 days	Deleted from Reports. Badges keeps report id, type and reporter until the account is deleted	The all-time *Report history* badge needs the count across years
Check-in	In Events until the event closes, plus 30 days	Deleted; the attendance count stays in Badges	Same as above
Account, memberships, team assignments, badges, consent records	While the account exists	Deleted when the user deletes it (US-05)	The user's own history of events and badges
Keycloak login and admin events	**30 days** (realm setting)	Expire	Enough to investigate a brute-force attempt during an 8-night festival
Container logs	Rotated at **10 MiB × 2 files** per container (Docker)	Overwritten	Enough to follow one correlation id through a demo; there is no separate log store
Sanitised log exports	At most **14 days**	Deleted	Only as evidence of a demo or a drill; sanitised, so they hold no request text, locations, names or tokens (M-10)
Metrics	**1 day** or **512 MiB**, whichever comes first (Prometheus)	Deleted	Dashboards and alerts need only the recent window; labels hold no personal data (M-10)
Traces	**Not stored**	—	Correlation ids in the logs replace distributed tracing (ADR-012)
Dead-letter messages	Until they are replayed or judged invalid (architecture §6)	Deleted	With M-5 they hold no help-request text
Rate-limit counters	In memory, at most 10 min	Gone	Only the current window matters

Data	Kept	Then	Why
Database dumps	A dump of the six databases, Keycloak's included, before every demo and every update; each dump kept at most **30 days**	Deleted	So account deletions and the periods above reach the dumps within 30 days. Volumes are not backups (ADR-011), and queued RabbitMQ messages are not dumped
User-research data	Survey answers are anonymous and stay outside the repository	Deleted when the course ends	Only the published summaries are needed

Restoring a dump. A restored dump brings back everything deleted after it was taken, accounts deleted with US-05 included, and Keycloak's dump brings back their login. The MVP holds only synthetic data (M-13), so this is accepted as a known limitation (§12). A real deployment would need a list of deleted accounts kept outside the dumps and applied again after every restore.

Account deletion (US-05). *Decided* (guide D7, #135, 27 October 2026), from the proposal of this section; flow in the API outline §2. The account is refused at once (`UserStatusChanged` , `Disabled`); Auth & Users deletes the user in Keycloak, then its own data, then publishes `UserDeleted` . Every service keeps a tombstone with only the opaque `sub` , so the account stays refused and late events do not bring a copy back. What each service does with it:

Service	On `UserDeleted`
Auth & Users	Deletes the account, the `FarolAdmin` grant, memberships, team assignments, friendships and consent records. Refuses the deletion (`422`) while the user is the last EventAdmin of an event or the last `FarolAdmin`
Reports	Closes the user's open help requests. Removes the requester from their help requests and the reporter from their reports, and deletes their text and map point **at once**, not after the 30 days above: only category or type, priority, status, times and team stay, as statistics. Deletes the copied display name and membership copies. Audit entries, `acceptedBy` and field tasks keep the opaque `sub` , which no longer points to anyone once the account is gone
Events	Deletes the user's check-ins and the copies of memberships, `FarolAdmin` grant and display name; audit entries keep the opaque `sub`
Badges	Deletes progress, preferences and the copies of reports and check-ins
Transports	Deletes the membership copies (corrected: it keeps them to check `Transport.Edit` , outline §7.5)

8. Consent, transparency and the sign-up terms

Assignment §6.6 asks for *account and consent management*. Sign-up and account deletion (US-03 to US-05) are in Sprint C, and their terms follow this section (`features.md` §8.5).

8.1 Who is responsible, in a real deployment

For the course, Farol runs only on the team's machine with test accounts, so no one's real data is processed. In a real deployment, the **organiser of each event** decides why that event's data is processed (help requests,

reports, check-ins, its teams) and would be its **controller**; the **Farol operator** would process it on the organiser's behalf (**processor**, with a data-processing agreement), and would be the controller of the accounts and badges, which span events.

8.2 Lawful basis

Processing	Basis (GDPR)
Account, login, memberships	Contract (art. 6(1)(b)): needed to use Farol
Help request	Contract for the request itself. Where it reveals health: **explicit consent** (art. 9(2)(a)), asked as in §8.3. For a third party who cannot be asked (a friend who fainted): vital interests (art. 9(2)(c))
AI suggestion on a help request	Same basis as the request. A person always decides, so it is not a decision based solely on automated processing (art. 22)
Community reports	Contract: a feature the user chooses to use
Badges and check-in	Consent: optional, and the user can stop or hide them at any time (US-56)
Security logs (logins, refused access, audit)	Legitimate interest (art. 6(1)(f)): protecting accounts and the events' data

8.3 Consent for help requests

Decided 26 October 2026: the consent is asked when joining an event (NFR-19, NFRs §5). The wording and the screen below are **[PROPOSED]**.

- Asked **when the participant joins an event** (US-12), so it is never seen for the first time in an emergency. One short screen: the control room and the team sent to help see the category, the map point and the text; the text can describe health; an AI may suggest a priority and a person decides; how long it is kept (§7); "*Farol is not an emergency service.*" No emergency number is named (decision of 12 October 2026, T5; NFR-19).
- Optional when joining. If it was declined, *Ask for help* shows the same screen again. If it is declined again, the page says that Farol is not an emergency service and shows **the first-aid points on the map**, so nobody is left without a way to get help.
- Recorded with its time and the version of the text; it can be withdrawn in the account (F2) and applies to later requests.

8.4 What the sign-up terms must say (US-03)

- Who runs Farol, who runs each event, and how to contact them about personal data.
- What is collected and why, who sees it and how long it is kept: a short version of §3 and §7.
- That help requests are not an emergency service.
- That an AI may suggest a priority for help requests, that a person decides, and, if a hosted provider is used, which one and where it processes the text.
- The user's rights: access, correction (US-04), deletion (US-05), withdrawing a consent, objecting; and the right to complain to the CNPD.

- **Not bundled into the terms:** the help-request consent (§8.3) and badges (US-56) are separate, optional choices.
- **[PROPOSED] records:** Auth & Users stores the version and time of the accepted terms, and each membership the version and time of the help-request consent.

8.5 Before a hosted AI provider receives real text

The gate of ADR-005, item 5, written as a checklist **[PROPOSED]**. All of it must be true; until then only synthetic text goes to a hosted provider.

- [ ] A data-processing agreement is signed with the provider, and its subprocessors are listed.
- [ ] The text is processed in the EU/EEA, or under an adequacy decision or standard contractual clauses.
- [ ] Inputs are not used for training, and the provider keeps them at most 30 days, or not at all (zero retention), on the plan actually used.
- [ ] The endpoint and the model version are fixed and recorded.
- [ ] The provider is named in the privacy notice (§8.4).

A local model (Ollama) avoids the transfer altogether; this is one reason K1 chose it (27 October 2026, NFR-20), so this checklist applies only if the hosted alternative is resumed.

9. The local demo environment

The delivery runs on one machine (ADR-011), and the demo uses the web app's phone pages. That brings its own risks.

How phones reach the demo. ADR-013 publishes the gateway, Keycloak and the web app on `localhost`. A phone must reach the machine's address on a Wi-Fi network instead, and two things then fail over plain HTTP: anyone on that network can read the tokens, and the browser makes the Web Crypto digest that PKCE needs available only on HTTPS pages or `localhost` (secure contexts), so login breaks. **[PROPOSED]:** a certificate for the demo machine's name from a local certificate authority (e.g. mkcert), with HTTPS on the gateway, Keycloak and the web container, and that authority installed on the demo phones; or the phone pages shown in a laptop browser's device mode. **Not acceptable** (assignment §11): turning off PKCE, token validation, CORS or Keycloak's HTTPS requirement to make the demo work.

Ports [PROPOSED]:

Reachable from	What
The Wi-Fi network (phones)	Gateway, Keycloak's login pages, web container: HTTPS only
The demo machine only (bound to `127.0.0.1`)	Keycloak admin console, RabbitMQ management, Grafana, WireMock admin API
Nothing outside Compose	The five services, PostgreSQL, RabbitMQ's AMQP port, Actuator management ports

Also [PROPOSED]:

- Keycloak, RabbitMQ, PostgreSQL and Grafana admin passwords come from `.env`, are generated per machine, and are never the image defaults.
- The demo machine and the backup laptop have full-disk encryption (BitLocker or FileVault) and a screen lock: they hold every database and its dumps (§7). Dumps are never committed or copied anywhere else.
- Every security control stays on during the demo: rate limits, token validation, isolation, CORS (assignment §11). Scenarios D1 and D2 break a dependency, not a control.

9.1 Sprint A skeleton (R0)

The Sprint A walking skeleton (plan, #166) is one Reports service that creates and reads synthetic help requests in memory. It is the only running code, and it does not follow the controls above yet:

- **Limits:** synthetic data only; the routes exist only with the `demo` profile; the port is published on `127.0.0.1:8080` only; there is no authentication, and every request uses a fixed identity. The assignment allows mocked authentication for the MVP (§6.6, footnote). AC-1 and AC-3 apply from the Sprint B slice, when Keycloak and the gateway arrive (skeleton plan §5).
- **Controls already in place:** unknown JSON fields are rejected with 400 (`fail-on-unknown-properties`, part of T7); the image runs as a non-root user (`10001`) from base images pinned by digest; the container is `read_only`, with `cap_drop: ALL`, `no-new-privileges`, memory and process limits, and rotated logs (§7) (`backend/reports/Dockerfile`, `infra/compose.yaml`).
- **Deviation:** the text is required (`@NotBlank`), while M-2 makes it optional.
- **Open gap:** when validation rejects a `description` (blank or over 500 characters), Spring's `DefaultHandlerExceptionResolver` writes the rejected value in full to the log at WARN, as seen in a local `mvn verify` on 31 October 2026. This breaks M-10 and NFR-21; the fix and the log test of §11 (T11, M-10) are due with the Sprint B slice.

10. Findings and proposed actions

The owners of each decision are in the decisions guide; none of this is agreed until the team reviews it.

#	Finding	Proposal	Decided in	Needed by
1	FarolAdmin bootstrap trusts an unverified e-mail (T4)	Grant only with `email_verified: true`; amend ADR-006	ADR-006	Sprint B slice (Keycloak, PostgreSQL and web, skeleton plan §5), and before sign-up opens (US-03)
2	Phones and plain HTTP: tokens exposed and PKCE login broken (§9)	Local HTTPS with mkcert, or device mode	Guide G4, with Compose (G5–G6)	First demo on phones, Sprint B
3	Help-request text and location travel through RabbitMQ (T11)	Triage message with `requestId` only (M-5)	API outline and event catalogue: ✅ decided in #123 (26 October 2026)	Before US-34

#	Finding	Proposal	Decided in	Needed by
4	Object access inside an event is not tested (T6)	Within-event case in US-14	Backlog (US-14)	Sprint B planning
5	WireMock admin API and recordings (T11, T16)	Admin API only on the demo machine; never record the AI provider	Guide J4	Decided: admin API on `127.0.0.1:8090` (`local-delivery.md` §1); WireMock enters Compose in Sprint B. Done: only Porto Digital recorded, with no personal data (`infra/wiremock/`, #155)
6	30-day sessions on field-team phones (T13)	AC-7: login in the last 12 h for operational actions	ADR-006	US-17, Sprint B
7	Account lookup by e-mail, outline O4 (T14)	AC-9	API outline O4	Now
8	Client secrets in the realm export (T17)	`${…}` placeholders	ADR-006	Sprint B slice (Keycloak, PostgreSQL and web, skeleton plan §5)
9	RabbitMQ and PostgreSQL grants not specified (T8)	AC-8	This assessment, with Compose (G5–G6)	Sprint B slice (Keycloak, PostgreSQL and web, skeleton plan §5)
10	Upload hardening (T9)	Content type, re-encoding, PDFBox limits	ADR-007	US-19
11	Content Security Policy, no third-party scripts (T10, M-14)	Headers on the web container	The web container	Sprint B slice (Keycloak, PostgreSQL and web, skeleton plan §5)
12	No dependency or image scanning (T18)	Checks in CI	ADR-010	Sprint B
13	Retention periods and clean-up jobs (§7)	The periods of §7	Team (guide D5): ✅ decided as NFR-19 and NFR-13 (26 October 2026; telemetry simplified 27 October 2026)	Clean-up jobs in Sprint C
14	Consent records and the help-request consent (§8.3, §8.4)	New fields; criteria in US-03 and US-12	Domain model; stories	US-12, Sprint B
15	Reviewers see who made a report (AC-3)	Reporter visible to EventAdmins only	Team	Before US-50

11. How the controls are verified

The security tests that the strategy in assignment §8.3 asks for.

Threat or rule	Test	When
T2	Tokens with the wrong issuer or audience, expired, unsigned (`alg: none`) or signed with another key get 401, in every service and in the gateway	Every PR
T5, T6, AC-2, AC-3	Per-service isolation test (US-14), with the cross-event and the within-event cases	Every PR
AC-4	The FarolAdmin is refused an event's help requests, reports and participants (US-07)	Every PR
T4	A self-registered account with the bootstrap e-mail does not get `FarolAdmin`	Every PR (Auth & Users)
T8, AC-8	The `reports` RabbitMQ user publishing to `farol.auth-users` is refused; the `reports` database login cannot connect to the `events` database	Every PR (Testcontainers)
T9	Uploads that are too large, of the wrong type, disguised by their extension, or multi-page PDFs are refused; a JPEG's EXIF is gone from the stored image	Every PR (Events)
T11, M-10	Logs captured during the help-request integration test contain neither the text nor the map point	Every PR (Reports)
T15	The 121st call in a minute and the 6th help request in 10 minutes get 429 with `Retry-After` (L-1, L-2)	Every PR (gateway)
T17	gitleaks	Every PR (E4), from Sprint B (ADR-010 point 9)
T18	Dependency check and image scan	Every PR
§9	From another computer on the network, only the gateway, Keycloak and the web container answer, and only over HTTPS	Before each demo
Whole system	An OWASP ZAP baseline scan against the local stack; its results update §4	Sprint C (assignment §7.3.2, item 6)

12. Known limitations

- **E-mails are not verified and passwords cannot be reset by e-mail** (no SMTP, ADR-006). An e-mail does not prove who owns an account (T4, AC-9), and scripted sign-ups are possible once sign-up opens. A local mail catcher (e.g. Mailpit) would let the demo show verification without a real mail server: **[PROPOSED]** option for Sprint C.
- **Tokens in browser memory:** a cross-site scripting bug would expose them (T10). A backend-for-frontend is a later hardening (ADR-006).
- **Multi-factor login only for the FarolAdmin** (T1).

- **Dumps only before demos and updates** (NFR-13): if the disk is lost, anything saved after the last dump is lost too, and queued messages are not dumped. A restored dump brings back data deleted after it was taken, accounts included (§7).
- **One machine holds everything:** physical access to it is access to every database (§9).
- **Retention not enforced until the clean-up jobs exist** (§7).
- **No export of a user's data** beyond what the screens show; the right of access is answered by hand.
- **A real deployment would need more:** a data-protection impact assessment (GDPR art. 35), likely required because Farol processes health-related data and the location of many people; data-processing agreements with each organiser and any hosted AI provider; a published privacy notice; and a contact for data requests. This document is the starting point for that assessment.
- **This assesses a design.** Sprint C updates it with what was built and the results of §11.

Docs/Technical/plans/local-walking-skeleton.md

Sprint A walking skeleton and Sprint B handoff

Issue: #58 · **Correction/implementation:** #166 · **Updated:** 31 October 2026

1. Sprint A boundary: one service, not the full application

A review on 31 October identified deliverable 10 as a blocker and asked for the smallest executable cut before Sprint B. This supersedes the earlier multi-service/authenticated request slice as an **R0 requirement**. That complete design remains the Sprint B target, not missing work to hide behind a documentation-only closure.

R0 is **Swagger/client → create a synthetic help request in Reports → store it in memory → read it by ID**. `POST /api/v1/demo/help-requests` accepts category/description and returns a generated ID, `status: Submitted` and `triage.state: NotTriaged`; `GET /api/v1/demo/help-requests/{id}` returns the stored request. `/api/v1/status` remains the technical health check. This replaces the status-only demonstration as that review asked, while retaining one service/container.

The identity is fixed on the server to `demo-participant`, event `demo-event`, and every response marks `demo: true`. Demo routes exist only with Spring profile `demo`, explicitly enabled in Compose, and are published on loopback only. They are a synthetic prototype, not production endpoints with authentication disabled. No frontend app, database, Keycloak, gateway, broker/outbox, AI or dashboard is required. Data disappears on restart; never submit real incidents or expect dispatch.

Course references supplied earlier: *Product_Management_Slides.pdf* pp.23–25 and *Mapping_Prioritization_Metrics_Slides.pdf* pp.4–5. The skeleton is a small executable slice evolved through later increments; documentation is not a substitute for the first running increment. The current deliberately narrower cut follows the 31 October review, not a claim that the lecture examples prescribe this exact endpoint.

2. Concrete contract, code and test

Artefact	Implemented slice
Service	`backend/reports`, Java 21, Spring Boot 4.0.9, independent Maven build
REST contract	OpenAPI: synthetic create/read routes (201/200), validation (400) and missing requests (404); `/api/v1/status` remains the diagnostic (200)
Automated test	Random-port HTTP tests cover status/UI/contract, create→read stored synthetic data, invalid input/spoofed identity, missing requests and absence of demo routes without the demo profile
Image	`backend/reports/Dockerfile`: builds/tests the JAR in Maven, then runs it as a non-root user in a JRE image
Deployment	`infra/compose.yaml`: one Reports container with profile `demo`, loopback port 8080, diagnostic health check, bounded memory and rotated container logs

Artefact	Implemented slice
CI	`.github/workflows/ci.yml`: build/test plus container startup and JSON smoke check on **every PR**, main push, release tag and manual run; no docs/draft skip in this minimum
Ignore rules	`.gitignore`: `infra/.env`, `__pycache__/`, `backups/`, `target/` and local editor output

The diagnostic and demo contracts are hand-written before implementation. This isolated demo is not the US-23/US-24 production API; generated business interfaces/clients follow C7 when those stories and their auth/persistence/location/idempotency contracts are implemented. Empty domain/application classes and scaffolds for the other four services are not Sprint A prerequisites. R0 makes `description` required; the product makes it optional (assessment M-2, API outline §4.1), which the Sprint B slice aligns.

3. Run and demonstrate from the repository root

Requirements: Docker Engine/Desktop with Compose v2; Java 21 and Maven 3.6.3+ only for the independent host build. Docker builds need no host JDK. No secrets, cloud account or database is needed.

```
mvn --batch-mode --no-transfer-progress -f backend/reports/pom.xml verify
docker compose -p farol-skeleton -f infra/compose.yaml up --build -d --wait --wait-timeout 120
curl --fail http://127.0.0.1:8080/api/v1/status
docker compose -p farol-skeleton -f infra/compose.yaml ps
docker compose -p farol-skeleton -f infra/compose.yaml logs --tail=50 reports
docker compose -p farol-skeleton -f infra/compose.yaml down
```

Open **Swagger UI**, then under **Demo help requests**:

1. Expand `POST /api/v1/demo/help-requests`, choose **Try it out**, retain the synthetic example (category/description), then **Execute**.
2. Expect **201**, a generated `id`, `status: Submitted` and `triage.state: NotTriaged` (shown as **Not triaged**). The event/identity are fixed demo values; callers cannot set them.
3. Copy `id` to `GET /api/v1/demo/help-requests/{requestId}`, then execute: expect **200** with the same stored record.
4. Restart Reports to clear the temporary in-memory data; reading the old ID now returns **404**. `/api/v1/status` still responds 200.

Input categories are Health/Safety/LostPerson/Theft/Other; description is non-blank, at most 500 characters. Invalid/unknown fields return 400. To run the JAR outside Compose, explicitly activate `--spring.profiles.active=demo`; otherwise the demo routes return 404. No credentials are needed for this fixed synthetic identity. The UI loads the existing handwritten contract at `/openapi.yaml`, packaged by Maven from `backend/reports/contracts/openapi.yaml`; there is no second maintained copy or generated competing specification. Springdoc 3.0.3 follows its Spring Boot 4.0 compatibility line. Swagger assets are served locally; no external validator is configured.

Expected HTTP response: `{"service":"reports","status":"ok"}`. To avoid a port already in use, run Compose with `REPORTS_PORT=18080` and call that loopback port. Optional local overrides: copy `infra/.env.example` to

`infra/.env` and pass `--env-file infra/.env`; no credentials are required. An unavailable/conflicting port must fail startup, not be mistaken for a successful demonstration of another process.

The health check means this diagnostic HTTP endpoint responds; it does not certify a database, auth provider, broker or SLO. A named Compose project limits cleanup to this demo's resources. No volume/backups exist for the in-memory demo; a restart deliberately clears synthetic records. This does not satisfy production durability, auth, per-event isolation, idempotency or SC3/SC5.

4. Review evidence and honest issue status

Execution record records the actual commands/results on the #166 branch. CI must also pass on the PR. Reopen #58 and #148 because merged Markdown-only #150/#151/#154 did not implement their runtime acceptance. The corrective PR closes **only #166**; #58 stays open until runnable evidence is reviewed, and #148 stays open until telemetry/dashboard/alert evidence exists. **Update (31 October 2026):** #58 was closed at 13:07 UTC, after #167 merged; #148 is still open. Do not mark board items Done just because their documentation merged.

5. Sprint B handoff

The full local delivery runbook and ADR-011 remain the target: five service images, gateway, Keycloak, one PostgreSQL with six owned databases/users, RabbitMQ and WireMock; React UI plus optional AI/observability profiles. Add them incrementally to the existing Compose file instead of adding non-existent dependencies now. The gateway takes host port 8080 (ADR-013), so Reports stops publishing that port and the URLs in §3, the execution record and the `ci.yml` smoke steps change with it.

First authenticated business slice: seeded synthetic event and users, one help-request button, Reports persistence/outbox, stored-status read and broker round trip. Implement domain/security/isolation and recovery tests with that slice. No cross-service shared database or synchronous domain calls are introduced. Identity/gateway work stays coordinated with ADR-006 and ADR-013; platform/CI decisions with ADR-010.

The minimal observability plan remains two extra containers, Prometheus/Grafana plus Actuator/Micrometer and correlated logs; #148 owns its actual implementation. The AI decision remains local Ollama/Qwen3 with human approval and measured evaluation. Neither is claimed running by this one-service demo.

6. Infrastructure answers retained

G1–G3: local only, cloud costs/alternatives retained, no cloud region or paid provisioning; explain the professor's cost authorisation in the report. G4–G6: isolated local dev/test/demo projects and versioned Compose/Dockerfiles; full stack/configuration comes in Sprint B. F2/C1: local React and RabbitMQ then, not deployed in this Sprint A cut. G7: R0 rollback rebuilds the previous reviewed tag. No tag exists yet: `v0.1.0` (guide H7) is to be created on `main` at the end of Sprint A. Rollback is then `git checkout v0.1.0` followed by `VERSION=0.1.0 docker compose -p farol-skeleton -f infra/compose.yaml up --build -d --wait`. `ci.yml` does not yet set `VERSION` on tag builds, so CI still names the image `farol-reports:dev`; database dumps, expand/contract migrations and measured restore/RPO apply when persistence exists. Existing NFR recovery targets are not measured by this in-memory demo.

Walking skeleton execution record · 31 October 2026

Issue: #166 · **Branch:** `chore/166-minimal-walking-skeleton` · **Scope:** minimum Sprint A diagnostic Reports service; not the Sprint B application.

Host build and automated HTTP test

Command: `mvn --batch-mode --no-transfer-progress -f backend/reports/pom.xml verify`.

Observed locally on Java 21.0.5, Maven 3.9.9: **BUILD SUCCESS**, one test, zero failures/errors/skips; first build 26.024 s. `StatusEndpointTest` started the real Spring Boot 4.0.9 HTTP server on a random port and verified the diagnostic JSON response. Maven created the executable `target/reports.jar`; build output is ignored by Git.

Container and CI evidence

Observed locally with Docker Engine 27.4.0:

```
docker compose -p farol-skeleton-166 -f infra/compose.yaml up --build -d --wait --wait-timeout 120
curl --fail --include http://127.0.0.1:8080/api/v1/status
docker compose -p farol-skeleton-166 -f infra/compose.yaml ps
```

- Multi-stage build completed, including the same one-test Maven verification inside the builder; runtime image ID `sha256:490a74e399c7f7b3df0fb99b1df70c4c15e221f6b3d4d1cdca92c05c0273777a`.
- Container `farol-skeleton-166-reports-1`: **healthy**, user `10001:10001`, published only at `127.0.0.1:8080`.
- Live response: **HTTP 200**, `Content-Type: application/json`, `{"service":"reports","status":"ok"}`.
- One idle `docker stats --no-stream` sample: 110.1 MiB / 512 MiB limit, CPU 0.85%. This is an observation, not a capacity benchmark or minimum-RAM guarantee.
- The Docker Desktop credential helper stalled on public-image metadata lookup. The local verification used an isolated temporary anonymous Docker config pointed at the same daemon; no credentials/settings were changed. Standard commands above apply normally when the user's Docker helper is functioning. Both official base manifests are pinned by digest in the Dockerfile.

The CI workflow executes the build/test and container smoke on each PR. Its GitHub run result is linked from the corrective PR after it completes; local success alone is not reported as remote CI success. No model, database, auth, dashboard, resilience or SLO result is supplied by this slice.

Review checks

Markdown links/anchors, XML/YAML syntax, Compose configuration and ignore rules were checked. #58/#148 remain open; the board was corrected from Done to In progress (#58) and Backlog (#148). #166's PR closes only the corrective issue and requires the repository's normal Copilot/team review. **Update (31 October 2026):** #58 was closed at 13:07 UTC, after #167 merged; #148 is still open.

Swagger UI follow-up (31 October 2026, #166)

Springdoc 3.0.3 adds `/swagger-ui.html` with the existing handwritten contract served at `/openapi.yaml`. Maven packages that file directly from `contracts/`; Docker includes the same input. Automatic `/v3/api-docs` generation is disabled while the UI configuration endpoint remains available.

Host `mvn verify` passed **two HTTP tests, zero failures/errors/skips** (14.881 s): the existing status response plus UI redirect/HTML, configuration pointing to the handwritten contract, exact served-contract contents and absence of a competing generated spec. CI also checks UI HTML and the packaged YAML against the repository file. The running local container is rebuilt for browser verification; CI results are linked in PR #167.

Product-related synthetic slice (31 October 2026, #166)

The 31 October review (#166) asked for a meaningful Reports operation instead of using only the diagnostic as the demonstration. The new demo-profile routes create and read a synthetic help request in memory. Status is `Submitted`; `triage.state` is `NotTriaged`, matching the production terminology without claiming AI ran. Identity/event are fixed on the server; every record marks `demo: true`. No real request is sent, persisted durably or dispatched.

- Host `mvn verify`: **10 HTTP test cases, 0 failures/errors/skips**, 23.350 s. Coverage includes create→read the same record, invalid category, missing category, blank/oversized description, forged identity field rejection, missing record, disabled demo routes without the profile, and the existing status/UI/contract checks.
- Actual rebuilt Compose container: POST `/api/v1/demo/help-requests` returned **201** with generated UUID, demo identity/event, Submitted and NotTriaged. GET of the Location returned **200** and the exact same record.
- Reports was restarted and waited healthy; GET of the old ID returned **404**, confirming temporary in-memory storage. The restarted demo is left running.
- CI now performs the same synthetic create/read flow against the actual image, in addition to status and Swagger/contract checks. The latest GitHub result is recorded on PR #167; earlier CI success is not substituted for this follow-up's result.

ALSO ASKED FOR BY THE PROFESSOR

Docs/Deliverables/operating-cost-estimation.md

Operating Cost Estimation

Course: LABDSOF 2026/27, Sprint A · **Issue:** #134 · **Date:** 23 October 2026 · **Updated:** 31 October 2026 (#162)
Asked for by: the professor, meeting of 22 October 2026 (P7, minutes): salaries and infrastructure, simple, reasonable assumptions.
Infrastructure figures: from the cloud research of #53 (`deployment.md`, prices of **22 October 2026** in the price snapshot).

What Farol would cost to build and to keep running if the team were a startup paying salaries, and why it is delivered locally instead of in the cloud. It is an estimate with stated assumptions, not a quote or a business plan.

1. Summary (one slide)

All amounts in euros, without VAT, rounded to the euro. Totals are computed from the unrounded amounts, so a rounded figure × 12 can differ from the yearly total by a few euros (e.g. 377 € × 12 = 4,524 €, against 376.53 € × 12 = 4,518 €).

Phase	Salaries	Infrastructure	Total
Development, per month (6 roles, full time)	17,360	102 (local; Copilot only)	**17,462**
Development, whole project (24 Sep 2026 – 3 Jan 2027, 3.35 months)	58,170	341	**58,511**
Same, if the demo ran in the cloud (Azure, 100 h/month, Sprint B and C)	*58,170*	*642*	*58,812*
Maintenance, per month (1 FTE, production in the cloud, 730 h)	3,029	377	**3,406**
Maintenance, first year (12 months)	36,348	4,518	**40,866**
Development + first year			**≈ 99,380**

Why local: salaries are a notional cost for a student team; infrastructure is the only cost the team would actually pay. Running locally costs 0 € beyond Copilot, which is the same in both scenarios. The cloud demo would cost about 301 €, and continuous cloud operation 358 € per month, against a one-off Azure for Students credit of 89 € ($100). See §5.

2. Assumptions

#	Assumption	Value	Source or reason
A1	Staffing	**6 people, one per role**, full time: Product Owner, Scrum Master, Tech lead/architect, QA lead, DevOps lead, AI lead	Roles in `team-and-process.md` §2. A startup would hire one person per role; the student team covers the six roles with 4 people, so this is about 50% above the team's size
A2	Development period	24 Sep 2026 – 3 Jan 2027 = 102 days = **3.35 months** (102 / 30.44)	Start: first commit of the repository (Sprint A has no written start date). End: Sprint C review (`features.md` §8.1)
A3	Period with code to deploy	2 Nov 2026 – 3 Jan 2027 = 63 days = **2.07 months**	Sprint B and C (`features.md` §8.1)
A4	Gross salary, Junior	**1,816 € per month**, paid 14 times a year	Teamlyzer salary calculator, average of all IT roles in Portugal by seniority, updated 23 Oct 2026, 23,585 salaries. No public source gives junior salaries per role (Hays and Michael Page guides need registration)
A5	Gross salary, Middle	**2,380 € per month**, paid 14 times a year	Same source as A4
A6	Level per role	**Middle:** Tech lead/architect, Product Owner (they decide the architecture and the product). **Junior:** Scrum Master, QA, DevOps, AI lead	Team decision for this estimate: a newly created startup
A7	Employer cost	Gross × 14 / 12 × **1.2375** (employer social security, TSU 23.75%)	Portuguese general regime. Meal allowance, insurance and equipment are not included
A8	Region	National average used for Porto	Porto is 0.5% above the national average in the same source (2,447 € vs 2,434 €): negligible
A9	Maintenance team	**1 FTE**: 0.5 DevOps (Junior) + 0.5 Tech lead (Middle)	Fixes, dependency updates and operation. Team decision for this estimate
A10	Currency	USD → EUR at **1.1205 USD per euro**	ECB reference rate of 22 Oct 2026, the date of the price snapshot
A11	Taxes	All amounts **without VAT**	The cloud prices in the snapshot are pre-tax retail prices

3. Salaries

Employer cost per month (A4, A5, A7): Junior 1,816 × 14 / 12 × 1.2375 = **2,622 €**; Middle 2,380 × 14 / 12 × 1.2375 = **3,436 €**.

Role	Level (A6)	Development, per month	Maintenance, per month (A9)
Product Owner	Middle	3,436	–
Tech lead / architect	Middle	3,436	0.5 × 3,436 = 1,718

Role	Level (A6)	Development, per month	Maintenance, per month (A9)
Scrum Master	Junior	2,622	–
QA lead	Junior	2,622	–
DevOps lead	Junior	2,622	0.5 × 2,622 = 1,311
AI lead	Junior	2,622	–
Total per month		**17,360**	**3,029**
Total for the phase		× 3.35 months (A2) = **58,170**	× 12 months = **36,348**

Check against the backlog: 6 people × 3.35 months × about 21 working days ≈ 420 person-days. The 62 user stories take 62 to 186 person-days (each is "one PR of one to three days", `features.md` §8.5). The rest goes to Sprint A (research, product and architecture documents), the walking skeleton, tests, the AI evaluation and the presentations.

4. Infrastructure

4.1 Local scenario (what is actually used)

Delivery is local only (ADR-011): everything runs with Docker Compose on one team member's laptop, with a second laptop as a backup for the demo, and the phones open the web app over the LAN (`local-delivery.md`, #129).

Item	Per month	Assumption
Laptops (demo and backup), Docker, PostgreSQL (one instance, six databases: five services and Keycloak, ADR-003), RabbitMQ	0 €	Equipment the team already owns. Electricity and wear are not measured, so they are not zero, only not counted (ADR-011)
Container registry	0 €	No registry: images built locally from the git tag (guide H7, decided on 23 October 2026, ADR-010)
App stores, mobile builds	0 €	No mobile app: one React web app (guide F1, #125)
GitHub Free, public repository, GitHub Actions, SonarQube Cloud	0 €	Actions is free for public repositories; the repository was private until 31 October 2026, and the pipeline was designed to fit the 2,000 free minutes per month. SonarQube Cloud on its Free plan; no paid feature (ADR-010 point 10, setup)
AI triage with local Ollama	0 €	No API charges; runs on the demo laptop; guide K1 chose it (Ollama with `qwen3:4b`, ADR-005, local AI plan)
GitHub Copilot Business (automatic code review, README §5)	6 × $19 = $114 = **102 €**	Copilot Business, the plan for organisations, $19 per user per month (GitHub plans, checked 26 Oct 2026), one licence per person (A1). Notional, like the salaries: the team uses Copilot Pro free through GitHub Education, so the real cost is 0 €
Total per month	**102 €**	× 3.35 months (A2) = **341 €**

4.2 Cloud scenario (researched in #53, not used)

Azure Container Apps for the five services, Azure Database for PostgreSQL Flexible Server (one server; #53 priced five databases, the design now has six with Keycloak's, on the same server, ADR-003), a VM for RabbitMQ, Azure Container Registry, West Europe. All figures from `deployment.md` §*Cost and budget gates*, prices of 22 Oct 2026:

Scenario	USD per month	EUR per month (A10)	Used for
Demo, 100 h/month of application time; database, broker and registry always on	162.84	**145**	Development: Sprint B and C, × 2.07 months (A3) = **301 €**, plus Copilot (341 €) = 642 €
Continuous, 730 h/month	400.98	**358**	Maintenance: a production system runs all the time

Both include $30 of unquoted allowances (disks, networking, telemetry). They do **not** include AI, VAT, an API gateway, Keycloak, the observability stack (Prometheus and Grafana, ADR-012), overlapping revisions or private-network charges (deployment.md), so the real bill would be higher. In the cloud scenario a registry is needed, because the platform pulls images from it; the "no registry" decision (guide H7) is for local delivery.

4.3 AI (ADR-005)

Option	Cost	Assumption
Local Ollama (`qwen3:4b`)	0 € of API	Runs on the laptop; in the cloud it would need its own host (unquoted, ADR-005)
Hosted Mistral Small 4	**$0.000192 per request** = 0.00017 €	$0.15 per million input tokens, $0.60 per million output, 800 input + 120 output tokens per request (`responsible-ai.md`, research date)
Hosted, 10,000 help requests per month	$1.92 = **2 €** per month	Example volume from the same source; counted in the maintenance month

Even at hosted prices the AI is negligible next to hosting. The choice of provider (guide K1) depends on quality, latency and privacy, not on cost.

4.4 Maintenance month

Item	Per month
Cloud, continuous (730 h)	358 €
GitHub Copilot Business, 1 licence (A9)	17 €
Hosted AI, 10,000 requests	2 €
Total infrastructure	**377 €** (× 12 = 4,518 €)

5. Why the delivery is local

1. **Salaries are notional; infrastructure is real money.** The team is not paid, so the only cash it would spend is on infrastructure. That is the part that decides where the system runs.
2. **The cloud does not fit the available credit.** Azure for Students gives a one-off credit of $100 (89 €), not $100 per month (`deployment.md`). The reduced demo would cost about 301 € over Sprint B and C, 3.4 times the credit; continuous operation would cost 358 € every month, 4 times the credit, with costs still missing from the quote (§4.2).
3. **Locally the extra cost is 0 € beyond Copilot**, which is the same in both scenarios (and free for the team through GitHub Education). From Sprint B, the same five independent services, six databases (five services and Keycloak) and RabbitMQ will run on a laptop the team already has, which shows independent deployment without a cloud bill. Sprint A runs one of them, Reports (walking skeleton, #167).
4. **The professor agreed.** After discussing these costs, the professor authorised local delivery, provided the report and presentation explain that cost is the reason (clarification, ADR-011).

What this costs the project: cloud operation, managed scaling and cloud recovery are not demonstrated. The local topology, the smoke test and the degraded-mode tests are shown instead (`local-delivery.md`).

6. Limits of this estimate

- Salaries come from a seniority average for all IT roles, not from a per-role junior figure; per-role junior salaries are not public.
- The cloud figures are the dated scenarios of #53, not a quote, and they leave costs out (§4.2).
- Not included: meal allowance, insurance, equipment, office, electricity, VAT, recruitment, legal costs, and the Porto Digital API (open data, free).
- Out of scope: a formal quote or a business plan.

Non-Functional Requirements

> LABDSOF 2026/27 · Sprint A · Assignment §6 (Common Technical Requirements), §8.3–8.5 and §9 (*Requirements and quality attributes*)
> **Status (24 October 2026):** one list of the quality attributes already spread across the repository, for the Sprint A deck (meeting with the professor of 22 October 2026, *Presentation contents*; #137). **No new decisions:** every target comes from the linked source. **[PROPOSED]** targets are not agreed by the team yet; **[PENDING]** means no target exists yet, with the owner from the architecture decisions guide (owners table). **Updated the same day:** the success-criteria targets confirmed, SC6 raised to 15 s, the outbox alert and the accessibility target set (§4). **Updated on 26 October 2026:** the resilience values are merged (#52, #43, 25 October 2026) and checked against `architecture.md` §6, §9.1 and §10.1, unchanged; NFR-03 follows the triage coverage aligned in #145; NFR-19 takes the retention and consent proposals of the Security and Privacy Assessment (#143). **Validated with the team on 26 October 2026:** every **[PROPOSED]** target is confirmed, and the **[PENDING]** ones get a value (§5); no target is left open (§3).
> Legend: see `product-vision.md`.

Functional requirements (features and user stories) are in `features.md`. This page lists how well Farol must do them: each requirement has a measurable target and the test, demo or metric that proves it. Categories follow the ISO/IEC 25010 quality characteristics; observability (assignment §6.7) and privacy (GDPR) are listed on their own because the assignment asks for them separately. The architecture drivers behind most of them are in `architecture.md` §1.

1. Summary

Category	NFRs	Count
Performance efficiency	NFR-01 to NFR-06	6
Reliability: fault tolerance and availability	NFR-07 to NFR-12	6
Reliability: recoverability	NFR-13	1
Security	NFR-14 to NFR-18	5
Privacy and compliance (GDPR)	NFR-19 to NFR-21	3
Functional correctness (AI)	NFR-22	1
Usability and accessibility	NFR-23 to NFR-25	3
Maintainability	NFR-26 to NFR-29	4
Portability and compatibility	NFR-30 to NFR-32	3
Observability	NFR-33 to NFR-34	2
Total		**34**

2. Requirements

ID	Category	Requirement	Measurable target	How it is verified	Source
NFR-01	Performance efficiency	The participant app opens fast, even on a weak venue network	QR scan to venue map, logged in: ≤ 5 s cold, ≤ 1 s from cache	Automated test with network throttling (slow 3G)	SC1 · driver D3
NFR-02	Performance efficiency	A help request reaches the event team at once, without reloading	Submit to visible in the backoffice: p95 ≤ 5 s; same ≤ 5 s for a routed request on a responder's phone	Automated end-to-end test; under load in k6 scenario L3	SC2 · US-31
NFR-03	Performance efficiency	The AI triage suggestion arrives in time	Submission to stored suggestion ≤ **30 s** (team decision); in operation ≥ 95 % valid suggestions stored within 30 s over a one-hour window; errors, timeouts and abstentions count as misses, including an abstention on a request with no text (definition, #145)	Timer in the triage worker; load test with the evaluation set; SLI on the dashboard	SC3 · ADR-005 · observability
NFR-04	Performance efficiency	Announcements reach participants who have the app open	Publish to visible on an open app ≤ 15 s (polling every 10 s plus the server time)	Automated end-to-end test; during k6 scenario L2	SC6 · US-44
NFR-05	Performance efficiency	Hold the read peak at the end of a concert with one instance per service	≈ 1,300 read req/s for 10 min plus 1 help request/s: errors < 1 %, server p95 < 500 ms, p99 < 1.5 s, 0 lost help requests; stress limit goal ≥ 1.5 × peak	k6 scenarios L1–L5, run by hand in Sprint C (*performance evidence*, §7.3.2 item 7)	driver D2 · guide J3, C5 · `architecture.md` §10.1
NFR-06	Performance efficiency	Saving a help request is fast	API ingress to database/outbox commit: p95 ≤ 2 s over 15 min	Histogram metric	ADR-012 · observability
NFR-07	Reliability: fault tolerance	No help request is lost when the AI fails (degraded mode D1)	100 % of requests shown to the operator at once, with the participant's category and "*Not triaged*", with the AI off or slow	Automated test with the AI stopped (WireMock in front of the provider, on every PR); D1 demo	SC5 · D1 · driver D6

ID	Category	Requirement	Measurable target	How it is verified	Source
NFR-08	Reliability: fault tolerance	When the transport source is down, late or invalid, users see the last valid data and a warning (degraded mode D2)	Last valid data with "*Data from HH:MM*" and a warning; invalid data never replaces valid data; event shuttles unaffected. Import every 30 s, freshness limit 5 min (guide J1, J2)	Automated tests against the simulator: source stopped, late, invalid (US-42, US-43); D2 demo	D2 · ADR-008 · `architecture.md` §7
NFR-09	Reliability: fault tolerance	A service or the broker being down does not stop the others	Other services keep working; with RabbitMQ down, help requests are still saved (outbox) and catch up when it returns	Integration tests that stop the broker and the AI separately	`architecture.md` §9 · ADR-004 · ADR-011
NFR-10	Reliability: fault tolerance	The asynchronous workflow loses no messages and handles duplicates	Outbox in the same transaction (0 lost events); idempotent consumers (duplicates ignored); then DLQ. Retries: 4 attempts, backoff 1 s → 2 s → 4 s (guide C3)	Integration tests with duplicate and poison messages; DLQ on the dashboard (NFR-34)	ADR-004 · `architecture.md` §6 · guide C3
NFR-11	Reliability: fault tolerance	Every external call has a timeout and, where useful, a retry and a circuit breaker	Local AI (Ollama, K1): connect 2 s, response 20 s, one attempt, whole call ≤ 22 s; circuit opens at ≥ 50 % failed or slow (> 16 s) calls; at most 5 s waiting before inference, which starts only with ≥ 23 s of the 30 s left (AI plan, #153). Porto Digital: connect 3 s, response 5 s, 3 attempts, circuit open 60 s. Keycloak keys: connect 2 s, response 3 s, no retry (guide J1)	WireMock integration tests on every PR (delay, 503, dropped connection, invalid body)	guide J1 · `architecture.md` §9.1 · ADR-004
NFR-12	Reliability: availability	Help requests can be created during service hours	Successful durable creates ≥ 99.5 % over a rolling 7 days, in advertised service hours	SLI on the dashboard; empty windows count as *no data*, not 100 %	ADR-012 · observability

ID	Category	Requirement	Measurable target	How it is verified	Source
NFR-13	Reliability: recoverability	A bad release or lost state can be recovered	Code rollback ≤ 10 min; stateful restore ≤ 60 min; a process restart loses no committed rows. Backup: a dump of the six databases (Keycloak's included) before every demo and every update, kept at most 30 days; messages queued in RabbitMQ are not backed up; on disk loss, the state of the last dump is recovered and anything saved after it is lost, so no RPO is guaranteed (`local-delivery.md` §6)	Recovery drill with measured RTO/RPO; Flyway *expand and contract* lets the previous image run	deployment research · ADR-011 · guide D6, G7
NFR-14	Security	Each event's data stays private to its members	0 successful cross-event access attempts; `FarolAdmin` sees no event data	Integration test per service (a member of X cannot read or change Y); DoD item	SC8 · driver D1 · guide D3
NFR-15	Security	Every user is authenticated; tokens are validated	Login mandatory for every user, through **Keycloak** (Authorization Code + PKCE); access tokens of 5 min, refresh tokens rotated on every use; a role or permission change applies in ≤ 5 s	Tests with rejected and expired tokens; integration test of the role change	ADR-006 · `architecture.md` §8 · assignment §6.6 · guide E1, E2
NFR-16	Security: accountability	Every change to a help request is recorded	100 % of request changes with an audit record (who, when, from, to)	Database query after the test suite	SC9 · `architecture.md` §8
NFR-17	Security	Abuse is limited without blocking people at the venue	Limits per account, never per IP: every API call 120 req/min; help requests 5 per 10 min; community reports 5 per 10 min per event (US-48); venue-map upload ≤ 20 MB. A refused request gets 429 with `Retry-After`	Rate-limit tests (429 with `Retry-After`); k6 scenarios	ADR-013 · US-48 · guide C6

ID	Category	Requirement	Measurable target	How it is verified	Source
NFR-18	Security	Secrets and service-to-service access are protected	No secret in the repository (DoD); secrets from environment variables (`.env` locally, GitHub Actions secrets in CI); no HTTP between services, one RabbitMQ user per service limited to its own exchanges and queues	DoD review; gitleaks secret scan in CI, planned for Sprint B (ADR-010 point 9)	ADR-006 · guide E3, E4 · DoD · assignment §8.4
NFR-19	Privacy and compliance (GDPR)	Location and help requests (possibly health data) are handled as sensitive data	Consent: no account without accepting the terms (US-03); help-request consent asked when joining an event, with a notice that the app is not an emergency service and the first-aid points shown if it is declined (no mention of 112: meeting of 12 October 2026, T5); account deletion removes personal data, memberships and badges (US-05; deletion D7). The account holds an e-mail and a display name only. Retention: help-request content deleted 30 days after the event closes, then only anonymous statistics; Keycloak login events 30 days; telemetry as in the observability plan (#148, #153): container logs rotated at 10 MiB × 2 files, sanitised log exports deleted within 14 days, metrics 1 day or 512 MiB, no traces stored	Acceptance tests of US-03 and US-05; retention clean-up jobs (Sprint C); security review (§7.3.2 item 6)	driver D5 · report §5 · F2 · Security and Privacy Assessment §7, §8

ID	Category	Requirement	Measurable target	How it is verified	Source
NFR-20	Privacy and compliance (GDPR)	The AI receives only what it needs	Input limited to category, sanitised text and priority floor; no identifiers or exact location; the AI runs on local Ollama with `qwen3:4b` (AI plan, #153), with cloud features off and no hosted fallback, so no text leaves the demo machine. A hosted provider would need a new cost and privacy approval, an ADR-005 update and a security review before real text (assessment §8.5)	Prompt and schema review; evaluation package	ADR-005 · Responsible AI
NFR-21	Privacy and compliance (GDPR)	Telemetry carries no personal data	No help-request text, prompts, exact location, names, tokens or credentials in logs; no user, request or event IDs as metric labels	Log inspection (ADR-012 acceptance evidence)	ADR-012 · observability
NFR-22	Functional correctness (AI)	The AI triage is correct where it matters and never overrides the operator	0 urgent cases under-prioritised; category accuracy ≥ 80 %; a suggestion never overwrites an operator's decision or lowers a priority	Evaluation set compared with the non-AI baseline (guide K2); integration test	SC4 · ADR-005 · D1
NFR-23	Usability	Asking for help is quick	Opening *Ask for help* to submitted: median ≤ 30 s	Usability test with students	SC11
NFR-24	Usability	Information and limitations are communicated clearly	100 % of information items show their last-update time; degraded states show their banner or warning (D1, D2)	Automated UI test; D1 and D2 demos	SC7 · driver D4 · assignment §6.8
NFR-25	Usability: accessibility	The interface follows accessibility and usability principles	**WCAG 2.1 AA** on the main pages: *Ask for help*, the venue map and the backoffice list of help requests (24 October 2026)	Accessibility review against WCAG 2.1 AA (automated checker plus a manual keyboard and screen-reader check) and usability plan (Sprint C, §7.3.2 items 4 and 5)	Assignment §6.1

ID	Category	Requirement	Measurable target	How it is verified	Source
NFR-26	Maintainability: modularity	Backend services are independently deployable and loosely coupled	5 services, each with its own `main`, Dockerfile and database; no shared code, only contracts (OpenAPI and events); hexagonal layout	Each service builds and is replaced on its own; review of dependencies	ADR-001 · ADR-003 · driver D7 · guide A3, A4
NFR-27	Maintainability: modifiability	APIs are contract-first and versioned	Hand-written OpenAPI per service under `/api/v1`, code generated; a breaking change is versioned	Generated code builds in CI; PR review	ADR-004 · Spec-Driven Development · guide C7
NFR-28	Maintainability: testability	Every change is built and tested automatically	Every service checked on every PR, merge and tag; tests cover the acceptance criteria; CI green to merge	GitHub Actions; branch protection; DoD	guide H4 · DoD · README §6 · assignment §8.5
NFR-29	Maintainability	Database changes do not break the previous release	Flyway migrations compatible with the previous image (*expand and contract*)	Run the previous image against the migrated database	guide D6 · `architecture.md` §10
NFR-30	Portability: installability	Farol runs locally from containers, without cloud	Reproducible local startup/update with Compose on one machine and a smoke test, following the procedure and smoke test of `local-delivery.md` §5 and §7. No time limit	Local startup and smoke test from a clean checkout	ADR-011 · guide G1, G6 · assignment §8.4
NFR-31	Portability: adaptability	External systems are replaceable by configuration	AI provider and Porto Digital URLs are configuration: the same code runs against the real system or the simulator	Tests and demo run against the simulator (guide J4: WireMock)	ADR-004 decision 5 · ADR-008 decision 4
NFR-32	Compatibility	One web app works on phones and laptops	Phone pages on Safari (iOS) and Chrome (Android), touch; laptop pages with mouse and wheel	Spike on a real iPhone and Android phone before the map is built	ADR-002 · ADR-007

ID	Category	Requirement	Measurable target	How it is verified	Source
NFR-33	Observability	The team can see the operational state of every container	Structured logs, health checks, metrics, correlation IDs across HTTP and AMQP, error reporting, one dashboard. Stack (observability plan, #148, #153): Micrometer/Actuator, Prometheus (scrape every 15 s) and Grafana, two extra containers; JSON logs in Docker with a `correlationId`; no trace backend, Collector, Alertmanager or log store	The same correlation ID in the HTTP, outbox, consumer and outcome logs of one help request	Assignment §6.7 · ADR-012 · guide I1
NFR-34	Observability	Failures are visible and alerted	SLIs: fallback rate, outbox oldest age, DLQ additions, transport data age, circuit-breaker state. Three alerts in Grafana, as in the observability plan §4 (#148, #153), evaluated every 15 s: an expected service or its telemetry down for 2 min; outbox oldest pending row above 60 s for 2 min (24 October 2026); useful triage below 95 % in a complete one-hour cohort with at least 20 eligible requests, for 5 min. The other signals of the plan (request not visible, urgent suggestion failure, triage stuck, DLQ, broker, stale transport) are dashboard and acceptance checks. Shown in the local Grafana only, no external notifications	Alert drill (trigger and resolve); degraded-mode demos	observability plan §4 · observability · D1 · `architecture.md` §6

3. Open points

No target is open after 26 October 2026 (§5). One thing to watch:

- **If K1 moves to a hosted AI provider**, it needs a new cost and privacy approval and an ADR-005 update; NFR-20 needs a security review before real text is sent (checklist in the assessment §8.5), and NFR-11 goes back to the hosted values of `architecture.md` §9.1.

4. Decided on 24 October 2026

- **Success criteria confirmed:** SC1, SC2, SC4, SC6 and SC11 (`product-vision.md` §5) and the report rate limit of US-48 (5 per 10 min per account). Only SC3 (30 s) had been set by the team before.
- **SC6 raised from 10 s to 15 s** (NFR-04): participant pages keep polling every 10 s, so the worst case (one interval plus the server time) fits, and the J3 peak of about 1,300 req/s does not change.
- **Outbox alert:** oldest pending row above **60 s for 2 min** (NFR-34), joining the 60 s of PR #52 and the 2 min window of the observability research, so a short peak does not alert.
- **Accessibility (NFR-25):** WCAG 2.1 AA on the main pages (*Ask for help*, the venue map and the backoffice list of help requests).
- **Availability and recovery (NFR-12, NFR-13)** stay, measured from local runs and a recovery drill, because delivery is local only (ADR-011); no cloud high availability is claimed.
- **Weak-network mode (F11, SC10)** stays out of the list while it is a Could; it is added only if F11 is built.

5. Validated with the team on 26 October 2026

The proposed targets were checked with the owners of their sources (guide owners table). All confirmed:

- **NFR-03:** ≥ 95 % valid suggestions stored within 30 s over a one-hour window, as defined in ADR-005 (#145). **Abstentions count as misses**, including an abstention on a help request with no text, so the target asks the model to suggest whenever it can.
- **NFR-06:** saving a help request, p95 ≤ 2 s over 15 min. **NFR-12:** ≥ 99.5 % successful creates over a rolling 7 days, in service hours.
- **NFR-11:** the AI timeout, retry and circuit-breaker values of `architecture.md` §9.1 (connect 2 s, response 10 s, 2 attempts), whatever model K1 picks.
- **NFR-13:** code rollback ≤ 10 min, stateful restore ≤ 60 min, no committed rows lost on a restart.
- **NFR-19:** the help-request consent when joining an event, an account with an e-mail and a display name only, and the retention periods of the assessment §7 (help-request content 30 days after the event closes, logs 14 days, traces 7 days, Keycloak login events 30 days).
- **NFR-31:** WireMock as the simulator of the Porto Digital API and the AI provider (guide J4).
- **NFR-33:** Micrometer/OpenTelemetry with Prometheus/Grafana, run locally (ADR-012).

Then, the same day, the items that had no value yet:

- **NFR-13, backup:** the proposal of PR #130 (`local-delivery.md` §6): a dump of the six databases before every demo and every update, queued messages not covered; on disk loss the last dump is recovered, with no RPO guaranteed for what came after it; dumps kept at most 30 days, so account deletions reach them (assessment §7).
- **NFR-30:** the procedure and smoke test of PR #130, **with no time limit**.
- **NFR-34:** the six other alerts proposed in the observability research, with their triggers; only on the Grafana of the demo machine, no external notifications. The spend alert is dropped: delivery is local.
- **NFR-20: local Ollama for now** (K1), so the hosted-provider review is not needed. If K1 moves to a hosted provider, the review applies again (§3).

Then, still on 26 October 2026, the list was aligned with the plans merged that day (#148, #149), which are now the reference:

- **NFR-19, telemetry retention:** traces 24 h instead of 7 days; container logs rotated at 10 MiB × 2 files, and sanitised log exports deleted within 14 days, instead of 14 days of application logs; metrics 7 days or 2 GiB. The help-request, account and Keycloak periods do not change.
- **NFR-33:** the stack of the observability plan, with Tempo for traces, the OpenTelemetry Collector and Alertmanager, and no Loki.
- **NFR-34:** the alert rules of the observability plan §4 replace the six alerts of the research: the create-failure alert (> 5 % over 5 min) is dropped; useful triage below 95 %, urgent suggestion failure, missing telemetry and stale transport data are added; the DLQ and broker triggers follow the plan.
- **NFR-20:** local Ollama stays for now; the final provider and model are chosen through the evaluation plan (#149).

On 27 October 2026, both plans were simplified (#153, PR #154) and the list follows them:

- **NFR-11:** the local AI profile replaces the confirmed 10 s and 2 attempts: response 20 s, one attempt, slow calls above 16 s, at most 5 s waiting and a start only with ≥ 23 s left. Two attempts of 20 s would not fit in the 30 s of SC3. The hosted values stay in `architecture.md` §9.1 as the alternative.
- **NFR-20:** Ollama with `qwen3:4b`, cloud features off and no hosted fallback.
- **NFR-33:** only Prometheus and Grafana are added; correlation IDs in the logs replace distributed tracing, so Tempo, the Collector and Alertmanager are dropped.
- **NFR-34:** three alerts in Grafana (service or telemetry down, outbox, useful triage); the other signals become dashboard and acceptance checks.
- **NFR-19, telemetry retention:** metrics 1 day or 512 MiB, no traces stored; logs unchanged.

Team Roles and Work Process

> LABDSOF 2026/27 · Sprint A (Sprint 1) · Assignment §8.1, §8.2 and §10
> **Status:** agreed in the team meeting of 8 October 2026 (minutes). The work process is defined in the README and summarised here. Roles are identified by what they do. People are not assigned to them on purpose: everyone codes, reviews and can take any role.
> Legend: see `product-vision.md`.

1. Team

What each member worked on, from the pull requests they wrote; open investigations and their owners are in the decisions guide. Counts on 1 November 2026, from GitHub (48 merged pull requests): pull requests written and merged, and pull requests the member approved.

Member	GitHub	Student Number	Areas (pull requests)	PRs merged	PRs approved
Tomás Oliveira	`tomasoliveira21`	1230213	Repository owner. Product vision (#19, #30); domain model, with Guerra (#37, #171); architecture research on deployment, observability and AI (#54, #146, #151, #152, #154); walking skeleton (#150, #167); CI with SonarQube Cloud (#169)	13	15
Tiago Teixeira	`tteixeira30`	1240493	Problem and Opportunity Report and survey script (#18); Sprint 1 presentation and personas (#21, #27); venue map, ADR-007 (#51); resilience, degraded modes and load testing (#52); role model (#57); backlog issues and board (#122, #140, #142); non-functional requirements (#139); survey results and the Sprint A deck (#165)	11	5
Bruno de Abreu Lopes	`BrunoLopes1190`	1240441	Contributing guidelines and process (#1, #33); market and competitor analysis (#20); main features (#25); decisions guide and ADR-002/003/004 (#36, #49); architecture diagrams and ADR-009 (#50, #157); meeting records (#23, #96); API outline (#124); one web app (#126); versioning, local delivery and ADR-010 CI/CD (#128, #130, #132); operating costs (#136); account deletion (#158); documentation corrections (#163)	18	9

Member	GitHub	Student Number	Areas (pull requests)	PRs merged	PRs approved
António Guerra	`AntonioGuerra1190409`	1190409	User workflows slide (#31); identity and API gateway, ADR-006 and ADR-013 (#47); security and privacy assessment (#144, #160); Porto Digital simulator in WireMock (#156); Sprint A deck split for the T/TP and PL classes (#174)	6	9

2. Roles

The assignment asks the team to work as a startup (§1) and evaluates each student individually, including their contribution and understanding of the whole product (§10). These are the roles the project needs and what each one does. Roles are **not assigned to a member**: any member can take a role for a given task, and everyone writes code and reviews.

Role	What it does	Focus in this project
Product Owner	Owns the backlog and its priority; accepts stories (DoD); speaks for the users; records investor feedback	Personas, success criteria, survey and interview
Scrum Master / facilitator	Runs the meetings, keeps the GitHub Projects board up to date, removes blockers, watches the process conventions	Board, sprint goals, retrospective actions
Tech lead / architect	Owns the architecture views and the ADRs; makes sure technical decisions are justified	Architecture, domain model, API outline
QA lead	Owns the test strategy and the quality evidence (assignment §8.3)	Test strategy, authorisation-isolation tests, AI evaluation dataset
DevOps lead	Owns the pipeline, containers, deployment and observability (assignment §8.4, §8.5, §6.7)	CI/CD, Docker, logs, health checks, dashboard
AI lead	Owns the AI-assisted workflow, its evaluation and the Responsible AI assessment (assignment §5.6)	AI help-request triage, fallback, evaluation report

3. Work process

Defined in the README; summarised here for the presentation.

Area	Convention	Source
Workflow	**GitHub Flow**: every piece of work starts from an issue; short-lived branch from `main`; PR to `main`; squash merge; branch deleted. `main` is always deployable, no direct pushes	README §1
Issues	Title `[<Type>] <Title>`, e.g. `[Documentation] Architecture decisions`. Types: `[Documentation]`, `[Back Office]`, `[Front Office]`, `[Technicians]`	README §1
Branches	`<prefix>/<short-description>`, prefixes `feature/`, `bugfix/`, `refactor/`, `test/`, `docs/`, `ci/`, `chore/`	README §2

Area	Convention	Source
Commits	`<prefix> (#<issue>): <Title>`, title ≤ 50 characters, imperative mood	README §3
Pull requests	Title `<Prefix>: <Description>`; description with `Closes #<issue>`, summary and how it was tested. **Every PR has an issue**	README §4
Code review	One review by a team member; GitHub Copilot is requested automatically but does not count, for lack of quota (1 November 2026, #175); review within 1 working day; blocking comments start with `blocking:`	README §5
Branch protection	No direct pushes, Copilot review, 1 approval, stale approvals dismissed, conversations resolved, CI must pass, branch up to date. **Active since 30 October 2026** (repository ruleset; checked on 31 October 2026: `protected: true`), without required checks, dismissal of stale approvals or resolved conversations yet (§4)	README §6
Definition of Ready / Done	See `definition-of-ready-and-done.md`. One user story = one issue = one PR	—
Language	All branches, commits, PRs and repository documentation in **English**. Exceptions: survey questions and presentation speaker notes, which are for a Portuguese audience (decided 12 October 2026)	README (introduction)

Sprints and meetings

The assignment fixes **three sprints** (§1, §7). The team calls them A, B and C.

Sprint	Objective (assignment §7)	Dates
Sprint A (1)	Discover, validate and design	Until **1 November 2026** (milestone *Sprint A*; the Sprint 1 presentation was on 12 October 2026)
Sprint B (2)	Build and operate the vertical slice	**2 November 2026 – 6 December 2026**
Sprint C (3)	Evaluate, harden and present	**7 December 2026 – 3 January 2027**

The team meets **twice a week**:

Meeting	When	Output
Planning	Weekly, during the class	Week's goal; Ready stories pulled from the backlog; at sprint start, the sprint goal; at sprint end, a short retrospective (1–3 actions)
Stand-up	Weekly, at the weekend	Progress and blockers, updated on the board
Sprint review (with investors)	End of each sprint	Feedback recorded on the board; at least one decision revised in response (assignment §5.7)

Tools

Tool	Use	Status
GitHub (`tomasoliveira21/LABDSOF`)	Code, documentation, issues, pull requests, reviews	In use
GitHub Projects	Backlog, sprints, decisions, sprint goals, investor feedback (the "Jira or equivalent" of assignment §8.2)	Chosen on 8 October 2026; in use: board, linked from the README; field rules in `features.md` §8.3
GitHub Copilot	Automatic PR review	The ruleset asks for it on every PR, but every review so far failed for lack of quota (README §5), and on PR #172 no request appeared at all (timeline checked on 31 October 2026)

Deployment scope clarification (recorded 22 October 2026)

As relayed to the team, the professor authorised **local-only delivery because of cloud costs**, which must be explained in the final report/presentation (record, ADR-011). Keep the cloud cost/hosting evaluation as justification. Builds, tests, independently replaceable service images and local smoke/failure demonstrations remain required; the earlier automatic cloud-production deployment expectation is superseded, and the local delivery handoff is adapted to local execution; no unattended access to a developer's machine is assumed.

4. Process health check (what the repository shows today)

Check	Finding	Action (agreed 8 October 2026)
Branch protection on `main`	A repository ruleset protects `main` since 30 October 2026, 18:43 UTC: pull request with 1 approval, Copilot review, no force push (checked on 31 October 2026: `protected: true`). Not on yet: required checks (the list is empty), dismissal of stale approvals, resolved conversations (README §6). There were **no direct pushes**: every commit on `main` came through a PR, except the initial commit (checked on 30 October 2026). The commits once listed here as direct pushes belong to PRs: `5ef2c4a` and its merge commit `3a4ba04` to PR #1, `e853854` to PR #21	Turn the rules on in the repository settings, then tick the boxes in README §6. **Ruleset on since 30 October 2026**; still to do: the unticked items of README §6
Merge method	README §4 requires **squash merge**, but PRs #18 to #21, #23, #25, #27 and #30 were merged with merge commits. By 30 October 2026, 39 PRs in all had been merged with a merge commit (#1 included), the last #160 (`git log --first-parent --merges`); squash-only since then	Restrict the repository to squash merging (Settings → General → Pull Requests). **Done**: only squash merging is allowed (checked on 31 October 2026)
PR template	README §7 says to save it as `.github/pull_request_template.md`, but the file did not exist	Created in PR #21

Check	Finding	Action (agreed 8 October 2026)
Issue templates	The DoR and the story-level DoD were not in the tool	Since 12 October 2026: issue forms in `.github/ISSUE_TEMPLATE/` (user story and task)
Commit message format	Several commits on `main` have no issue number or prefix (`3a4ba04` , `5159ff4` , `5ef2c4a` , `e853854` , and the commits of PR #21; later `58fb4e0`). Branch `feat/UpdateProductVisionPP` (PR #30) uses a prefix that does not exist	Agreed: every PR has an issue, and PR titles follow `<Prefix>: <Description>` . With squash merging, the PR title becomes the commit on `main` when the PR has more than one commit; a one-commit PR keeps its commit title, because the repository's squash title setting is *default message* (`COMMIT_OR_PR_TITLE` , checked on 31 October 2026), e.g. PR #171 → `4571cb4 docs (#170): …` . Since 12 October 2026 the **Conventions** workflow (`.github/workflows/conventions.yml`) checks the branch name, the PR title and the `Closes #<issue>` line on every PR
Documentation language	The README requires English, but the three deliverables in `Docs/Deliverables/` are in Portuguese	**Decided 12 October 2026:** all documentation is in English and stays that way. The three deliverables were translated

Definition of Ready and Definition of Done

> LABDSOF 2026/27 · Sprint A (Sprint 1) · Assignment §8 (Development and Delivery Practices)
> **Status:** agreed in the team meeting of 8 October 2026 (minutes).
> Legend: see `product-vision.md`.

Why both

- **Definition of Ready (DoR):** a backlog item can enter a sprint only when the team understands it well enough to start it.
- **Definition of Done (DoD):** an item is finished only when it meets the same quality bar as every other item. It applies at two levels: the pull request and the user story.

Rule: one user story = one backlog item (GitHub issue) = one pull request. A story that needs more than one PR is too big and must be split into smaller stories.

Definition of Ready (backlog item)

A user story or task is *Ready* when:

- [ ] It is written as a user story (*As a … I want … so that …*) or as a task with a clear outcome
- [ ] It names the persona it serves (P1 to P5, see `personas.md`) or the technical goal
- [ ] It has acceptance criteria in *Given / When / Then* that can be tested
- [ ] Dependencies (other stories, external data, decisions) are identified, and none of them blocks it
- [ ] It has a GitHub issue on the GitHub Projects board, whose number goes in the branch and commits (README §1, §3)
- [ ] If it uses AI: the fallback behaviour is described (assignment §5.6.2)

Definition of Done

Level 1 · Pull request

From the README (§7, PR template):

- [ ] I reviewed my own diff before requesting a review
- [ ] CI pipeline is green
- [ ] Commits follow the conventions
- [ ] Tests added/updated and passing
- [ ] Documentation updated (if applicable)
- [ ] No secrets committed

Plus the review rules in README §5: one approval from another team member, every comment addressed (Copilot's too, if it reviewed), all conversations resolved.

Level 2 · User story

A story is *Done* when its PR is merged and:

- [ ] All acceptance criteria pass
- [ ] Automated tests cover the acceptance criteria at the appropriate level (unit, integration or end-to-end; assignment §8.3)
- [ ] Authorisation is tested where the story touches event data: a user from another event cannot read or change it (report §5, technical risks)
- [ ] Logs for the new behaviour are structured and carry a correlation ID (assignment §6.7)
- [ ] The pipeline on `main` is green after the merge, including the image build and the smoke test (assignment §8.4, §8.5; delivery is local, ADR-010)
- [ ] The Product Owner has accepted it
- [ ] The GitHub issue is closed by the PR (`Closes #<issue>`)

> **Note:** the items about CI and deployment have applied since the walking skeleton (#167, 31 October 2026): `ci.yml` builds, tests and runs the Compose smoke test on every pull request. Logs and the rest of the observability apply once they exist (#148).

In the tool

Rule	Where it lives
DoR	Issue form `.github/ISSUE_TEMPLATE/user-story.yml` : the story, persona, acceptance criteria and dependencies are required fields, and the DoR is a checklist at the end. Work that is not a story uses `task.yml` . Blank issues are disabled
DoD, Level 1 (PR)	`.github/pull_request_template.md` , plus the **Conventions** check (`.github/workflows/conventions.yml`)
DoD, Level 2 (story)	Checklist pre-filled in every user-story issue, ticked before the item moves to "Done"
Board	GitHub Projects *Status* field: `Backlog → Ready → In progress → In review → Done` . The *Ready* option is described as "Meets the DoR" and *Done* as "Meets the DoD", both linking to this page

> ⚠️ **Board status (checked 24 October 2026, #138).**
> **Done:** the board exists with the *Status* options above, the *Priority* (P0–P2) and *Size* (XS–XL) fields, and the built-in workflows on (auto-add, item added, item closed, pull request linked, pull request merged); issues #58 to #121 are on it, and its link is in the README. The DoR box "On the GitHub Projects board" is ticked in the 62 story issues. *Priority*, *Size* and *Status* applied to #58 to #121 with the rules of `features.md` §8.3 (approved on 24 October 2026, #138). The sprint is the issue's milestone (*Sprint A*, *B*, *C*) instead of an *Iteration* field; *Sprint A* is due on 1 November 2026. The *Ready* and *Done* options are described as "Meets

> the DoR" and "Meets the DoD", with a link to this page.
> **Still to do:**
>
> - The board is private: give the professor access, or make it public (repository owner).

ANNEX: STUDENT SURVEY SCRIPT

`Docs/Research/inquerito-estudantes-google-forms.gs`

inquerito-estudantes-google-forms.gs

```
/**
 * Creates the student survey in Google Forms, with a response sheet
 * in Google Sheets.
 *
 * The questions are in Portuguese on purpose: the respondents are Portuguese
 * students. Comments in this file are in English, like the rest of the repository.
 *
 * Goal: validate the critical assumptions of the Problem and Opportunity Report.
 *   A2 · Participants check the application during the night     → questions 6, 7 and 8
 *   A3 · They adopt one application shared by events, no install  → questions 7 and 9
 *   A4 · Asking for help is slow or unclear today                 → questions 3, 5 and 10
 *   Problem · Information is scattered and hard to find           → questions 3, 4 and 6
 *
 * Anyone who has never been to a Queima and does not plan to go ends on the first page.
 * Anyone who has never been but plans to go skips the questions about the experience.
 *
 * How to use:
 *   1. Open https://script.google.com and create a "New project"
 *      (or add a new file to a project you already have).
 *   2. Paste this whole file.
 *   3. Choose the function criarInqueritoEstudantes and click "Run".
 *   4. The links to the form and to the responses appear in the "Execution log".
 */

function criarInqueritoEstudantes() {
  const TITULO = 'Queima das Fitas: como vives a noite do evento?';

  const DESCRICAO =
    'Somos estudantes do ISEP e estamos a estudar como melhorar a experiência e a segurança ' +
    'em grandes eventos académicos, como a Queima das Fitas.\n\n' +
    'Demora cerca de 3 minutos. É anónimo: não recolhemos o teu nome nem o teu email, ' +
    'e as respostas são usadas apenas de forma agregada, num trabalho académico (LABDSOF).';

  const form = FormApp.create(TITULO)
    .setDescription(DESCRICAO)
    .setProgressBar(true)
    .setCollectEmail(false)
    .setLimitOneResponsePerUser(false)
    .setConfirmationMessage('Obrigado! A tua resposta ajuda-nos muito.');

  // ---------- Page 1 · Filter ----------
  const q1 = form.addMultipleChoiceItem()
    .setTitle('1. Já foste a alguma Queima das Fitas ou semana académica?')
    .setRequired(true);

  // ---------- Page 2 · Your experience ----------
  const paginaExperiencia = form.addPageBreakItem()
    .setTitle('A tua experiência')
    .setHelpText('Pensa na última vez que foste a uma Queima ou semana académica.');

  form.addCheckboxItem()
    .setTitle('2. A que eventos já foste?')
    .setChoiceValues([
```

```
      'Queima das Fitas do Porto',
      'Queima das Fitas de Coimbra',
      'Enterro da Gata (Braga)',
    ])
    .showOtherOption(true)
    .setRequired(true);

  form.addCheckboxItem()
    .setTitle('3. Na última vez que foste, passaste por alguma destas situações?')
    .setHelpText('Escolhe todas as que se aplicam.')
    .setChoiceValues([
      'Esperei mais de 30 minutos para entrar',
      'Não sabia onde ficava alguma coisa no recinto (WC, palco, posto médico)',
      'Perdi-me do meu grupo',
      'Não sabia como voltar para casa ou a que horas havia transporte',
      'Precisei de ajuda (saúde, segurança, roubo) e não sabia a quem pedir',
      'Fui vítima ou testemunha de um roubo',
      'Nenhuma destas',
    ])
    .showOtherOption(true)
    .setRequired(true);

  form.addCheckboxItem()
    .setTitle('4. Onde procuraste informação sobre o evento (transportes, horários, recinto)?')
    .setHelpText('Escolhe todas as que se aplicam.')
    .setChoiceValues([
      'Instagram ou outras redes sociais da organização',
      'App da federação académica (ex.: app FAP)',
      'Site oficial do evento',
      'Amigos ou grupos de WhatsApp',
      'Placas e staff no recinto',
      'Não procurei',
    ])
    .showOtherOption(true)
    .setRequired(true);

  form.addMultipleChoiceItem()
    .setTitle('5. Se precisasses de ajuda dentro do recinto, o que farias primeiro?')
    .setChoiceValues([
      'Procurava um segurança ou alguém do staff',
      'Ligava para o 112',
      'Pedia ajuda aos meus amigos',
      'Ia ao posto médico',
      'Não sei',
    ])
    .showOtherOption(true)
    .setRequired(true);

  form.addScaleItem()
    .setTitle('6. Durante a noite do evento, quão fácil foi encontrar a informação de que precisavas?')
    .setBounds(1, 5)
    .setLabels('Muito difícil', 'Muito fácil')
    .setRequired(true);

  // ---------- Page 3 · An application for the event ----------
  const paginaApp = form.addPageBreakItem()
    .setTitle('Uma aplicação para o evento')
    .setHelpText(
```

```
      'Imagina uma aplicação que reúne a informação de vários eventos académicos. ' +
      'Abre-se por link ou por QR code no bilhete, sem ser obrigatório instalar.'
    );

  form.addMultipleChoiceItem()
    .setTitle('7. Usarias uma aplicação destas durante a noite do evento?')
    .setChoiceValues([
      'Sim, de certeza',
      'Provavelmente sim',
      'Provavelmente não',
      'Não',
    ])
    .setRequired(true);

  form.addCheckboxItem()
    .setTitle('8. Quais destas funcionalidades te seriam mais úteis?')
    .setHelpText('Escolhe até 3.')
    .setChoiceValues([
      'Planta do recinto (WC, palcos, postos de apoio)',
      'Horários e localização dos transportes de regresso',
      'Tempo de espera previsto nas entradas',
      'Botão de ajuda (saúde, segurança, roubo) que avisa a organização',
      'Avisos da organização em tempo real (alterações, alertas)',
      'Bilhete eletrónico na aplicação',
      'Assistente para tirar dúvidas sobre o evento',
    ])
    .setValidation(
      FormApp.createCheckboxValidation()
        .setHelpText('Escolhe no máximo 3.')
        .requireSelectAtMost(3)
        .build()
    )
    .setRequired(true);

  form.addCheckboxItem()
    .setTitle('9. O que te levaria a NÃO usar uma aplicação destas?')
    .setHelpText('Escolhe até 2.')
    .setChoiceValues([
      'Ter de instalar mais uma app',
      'Gastar bateria',
      'Ter de partilhar a minha localização',
      'Rede fraca no recinto',
      'Não confiar que a informação está atualizada',
      'Nada, usaria',
    ])
    .setValidation(
      FormApp.createCheckboxValidation()
        .setHelpText('Escolhe no máximo 2.')
        .requireSelectAtMost(2)
        .build()
    )
    .showOtherOption(true)
    .setRequired(true);

  form.addMultipleChoiceItem()
    .setTitle('10. Se pedisses ajuda pela aplicação, partilharias a tua localização com a organização?'
    .setChoiceValues([
      'Sim, a localização do telemóvel',
```

```
      'Só indicando eu o local na planta do recinto',
      'Não',
    ])
    .setRequired(true);

  form.addParagraphTextItem()
    .setTitle('11. Há alguma coisa que mudarias na experiência da Queima? (opcional)');

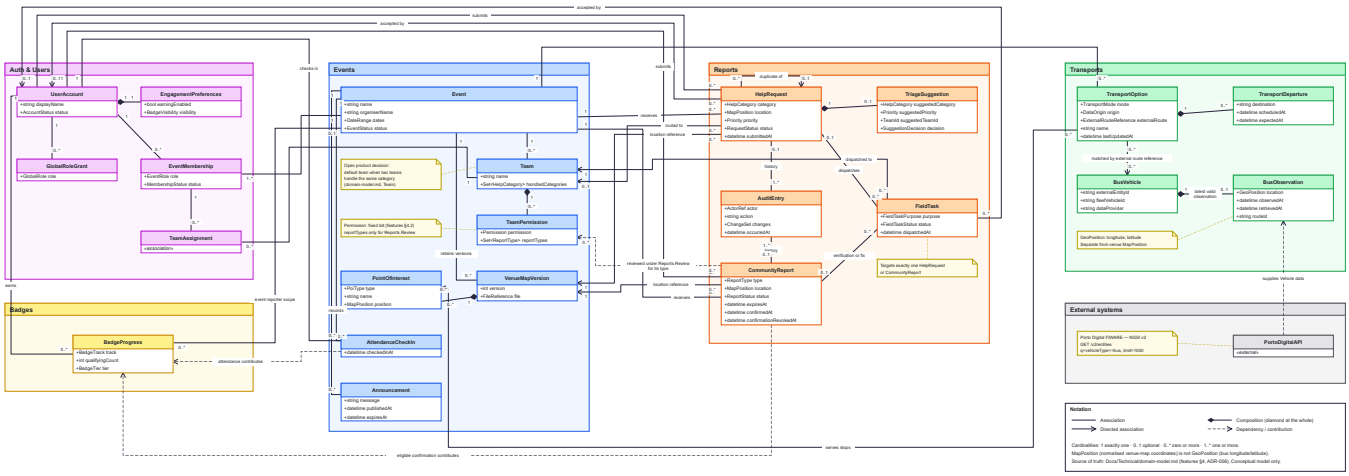
  // Question 1 navigation: depends on whether they have been before.
  q1.setChoices([
    q1.createChoice('Sim, várias vezes', paginaExperiencia),
    q1.createChoice('Sim, uma vez', paginaExperiencia),
    q1.createChoice('Não, mas tenciono ir', paginaApp),
    q1.createChoice('Não, nem tenciono ir', FormApp.PageNavigationType.SUBMIT),
  ]);

  const folha = SpreadsheetApp.create('Respostas · ' + TITULO);
  form.setDestination(FormApp.DestinationType.SPREADSHEET, folha.getId());

  Logger.log('Link para divulgar:   ' + form.getPublishedUrl());
  Logger.log('Editar o formulário:  ' + form.getEditUrl());
  Logger.log('Folha de respostas:   ' + folha.getUrl());
}
```

Docs/Technical/domain-model.svg

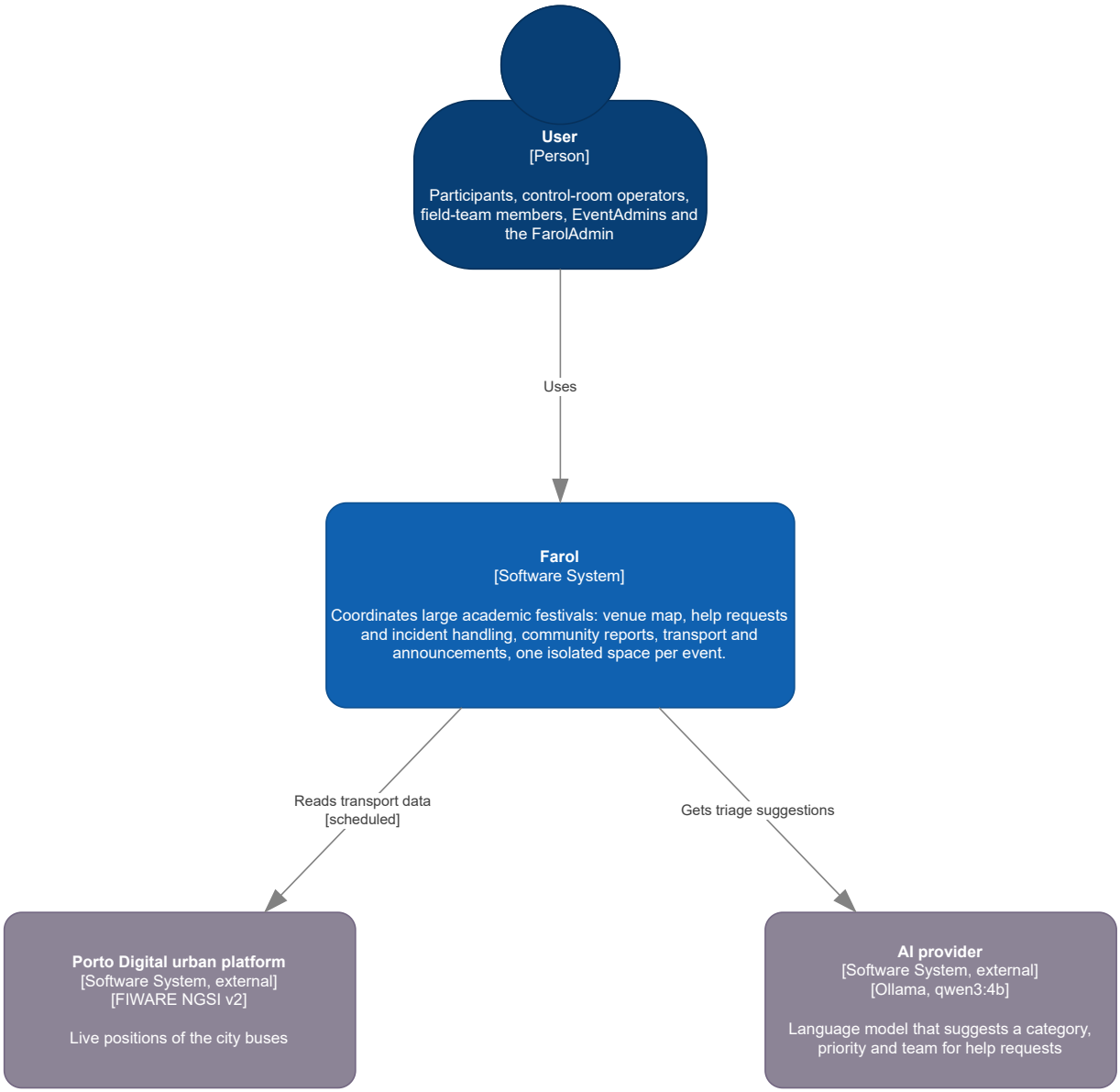
domain-model.svg



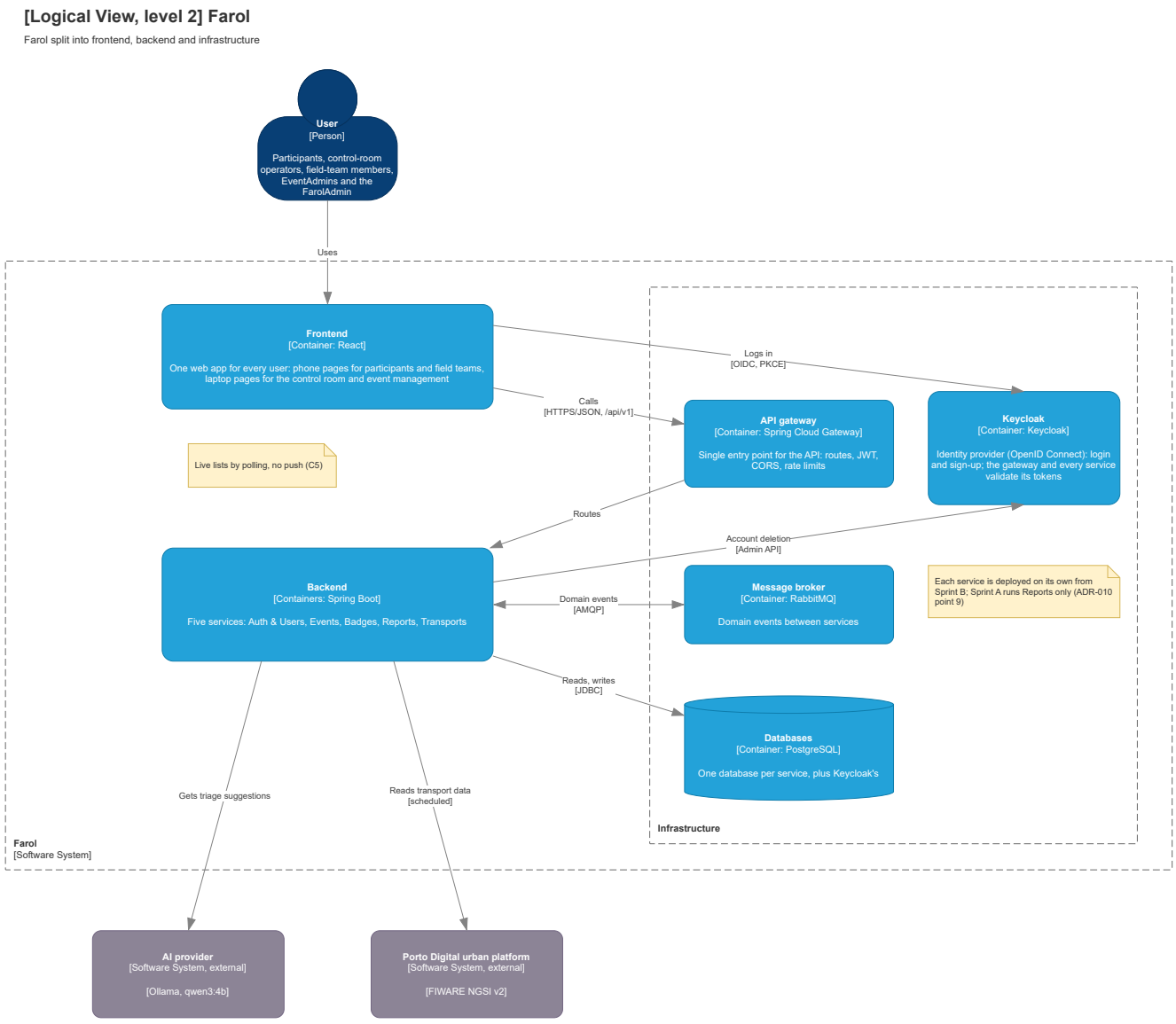
logical-view-level-1.drawio.svg

[Logical View, level 1] Farol

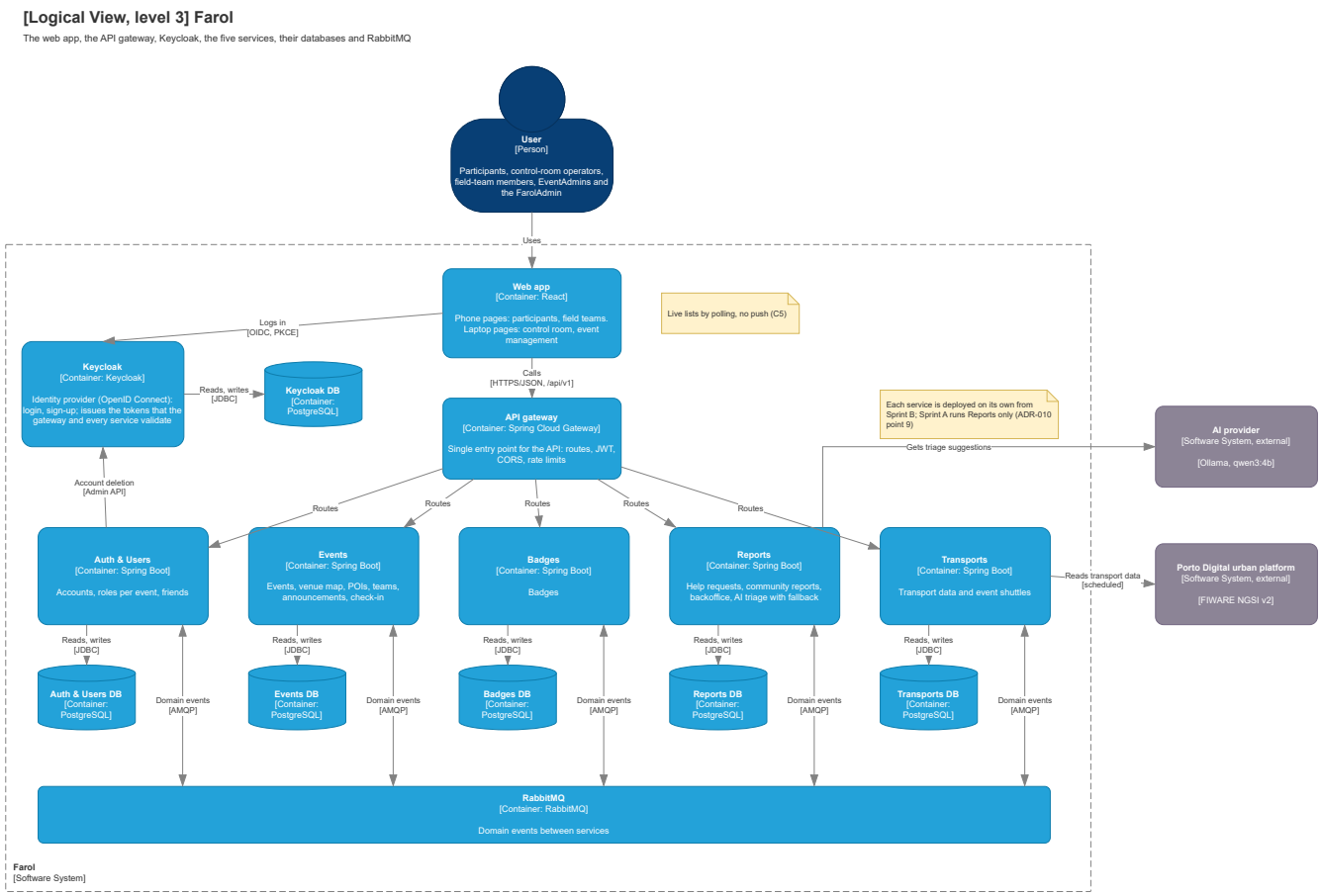
Who uses Farol and which external systems it depends on



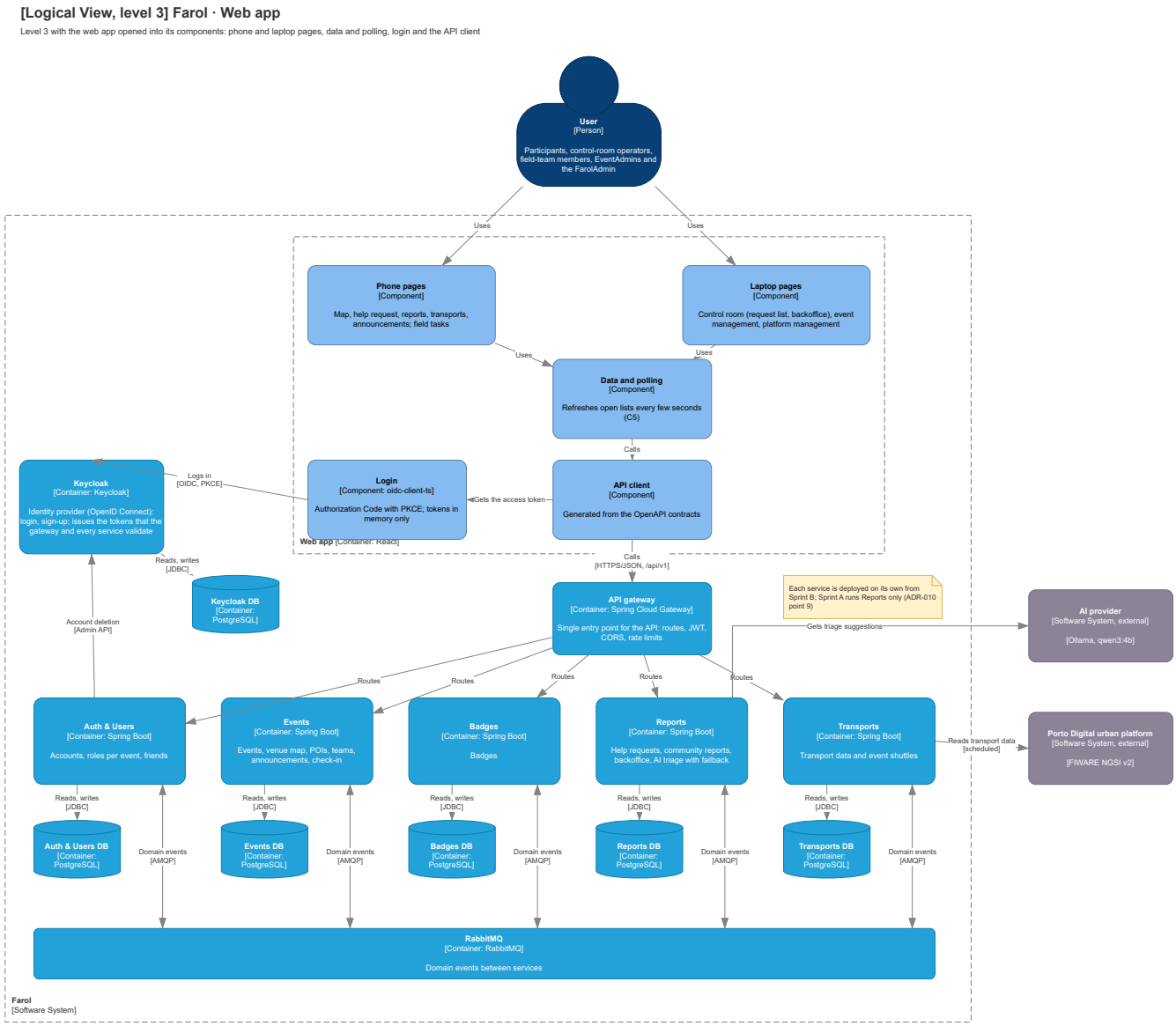
logical-view-level-2.drawio.svg



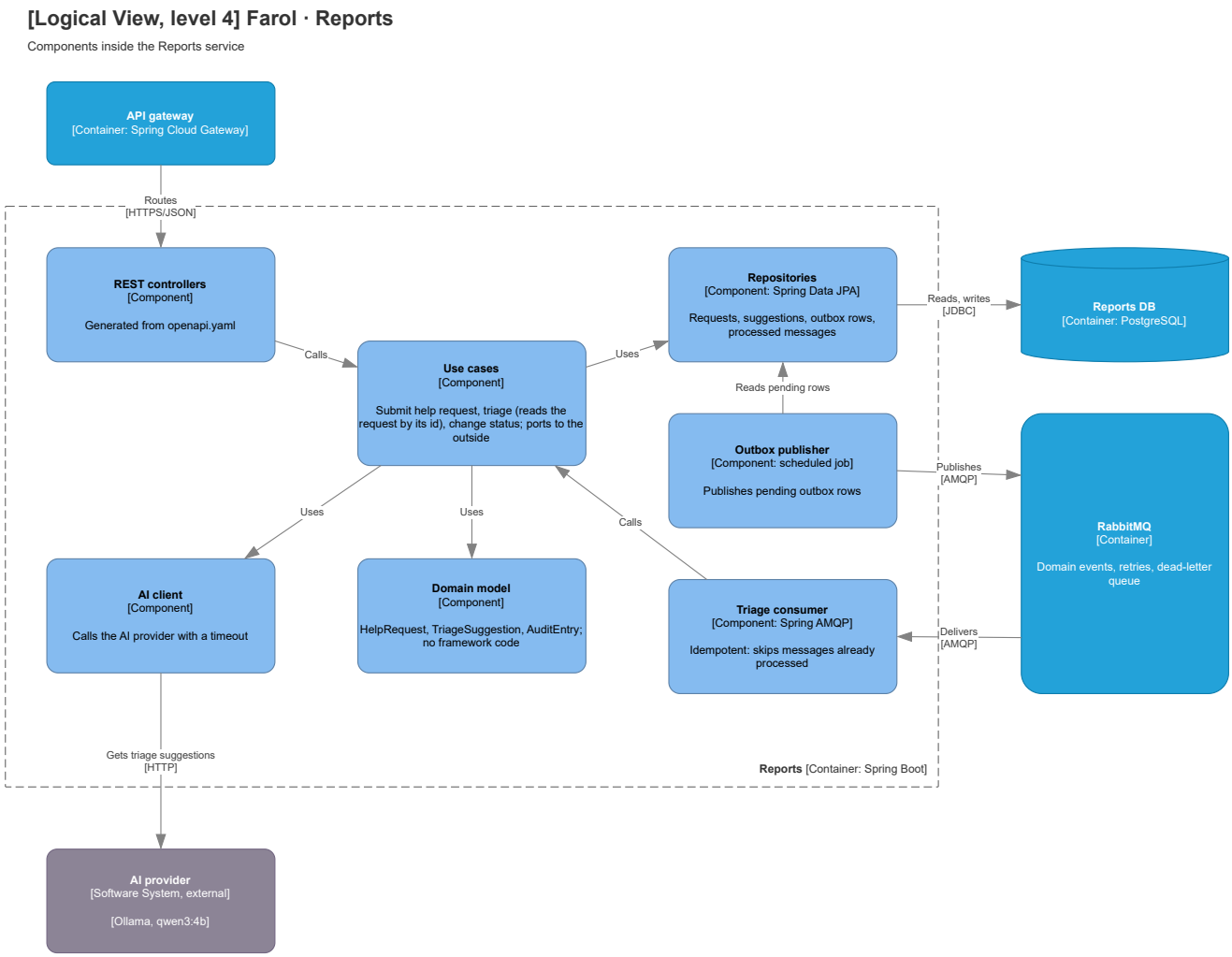
logical-view-level-3.drawio.svg



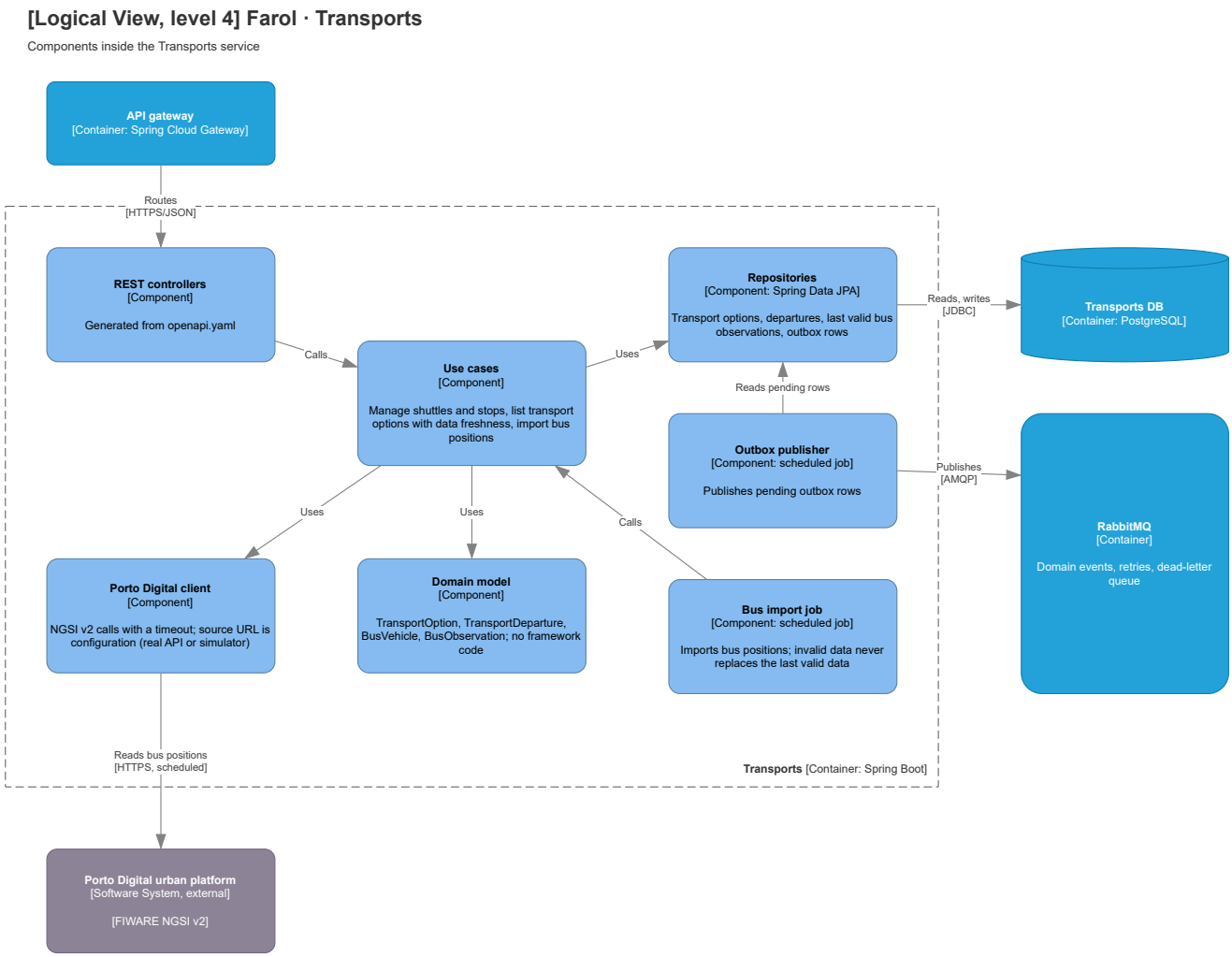
logical-view-level-3-web.drawio.svg



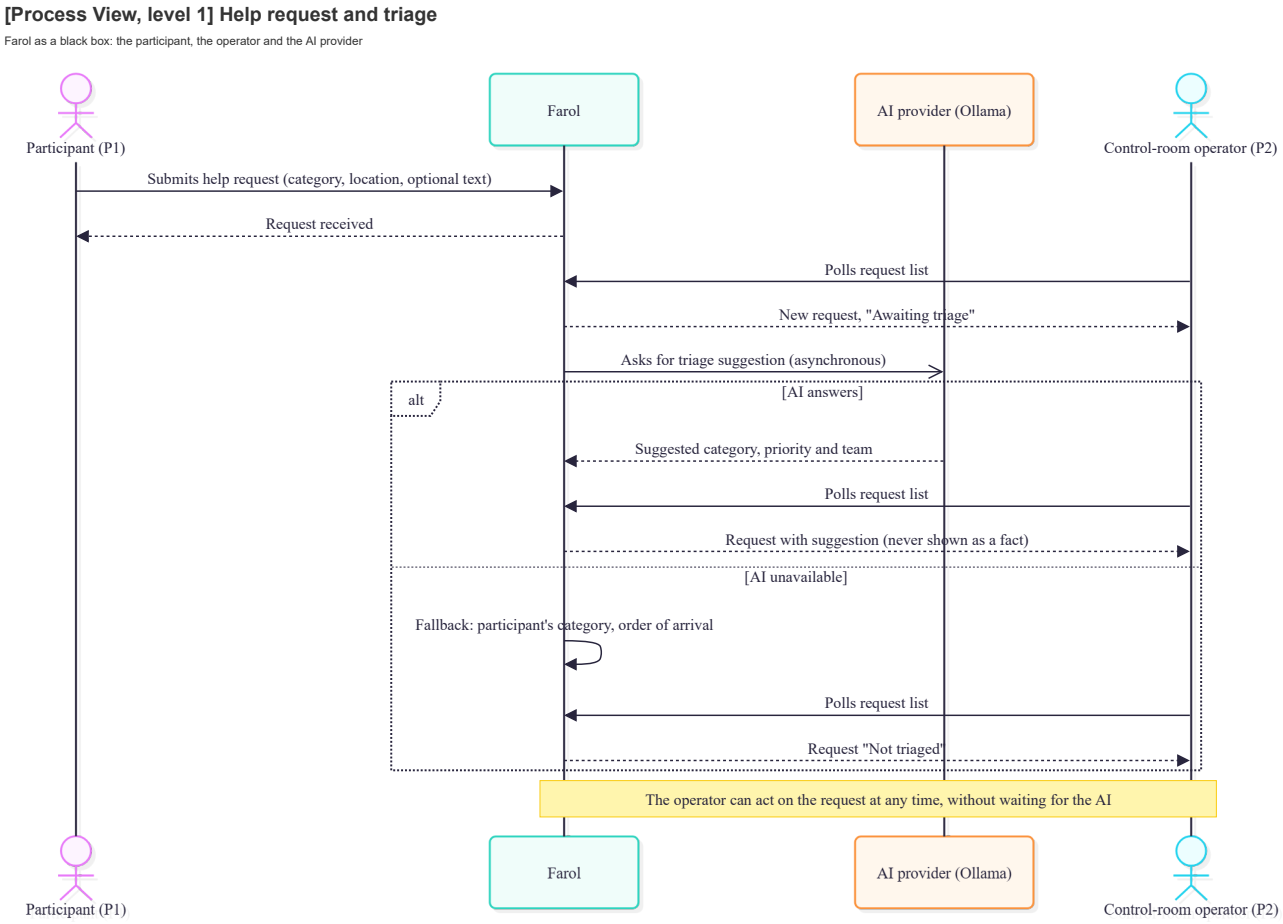
logical-view-level-4-reports.drawio.svg



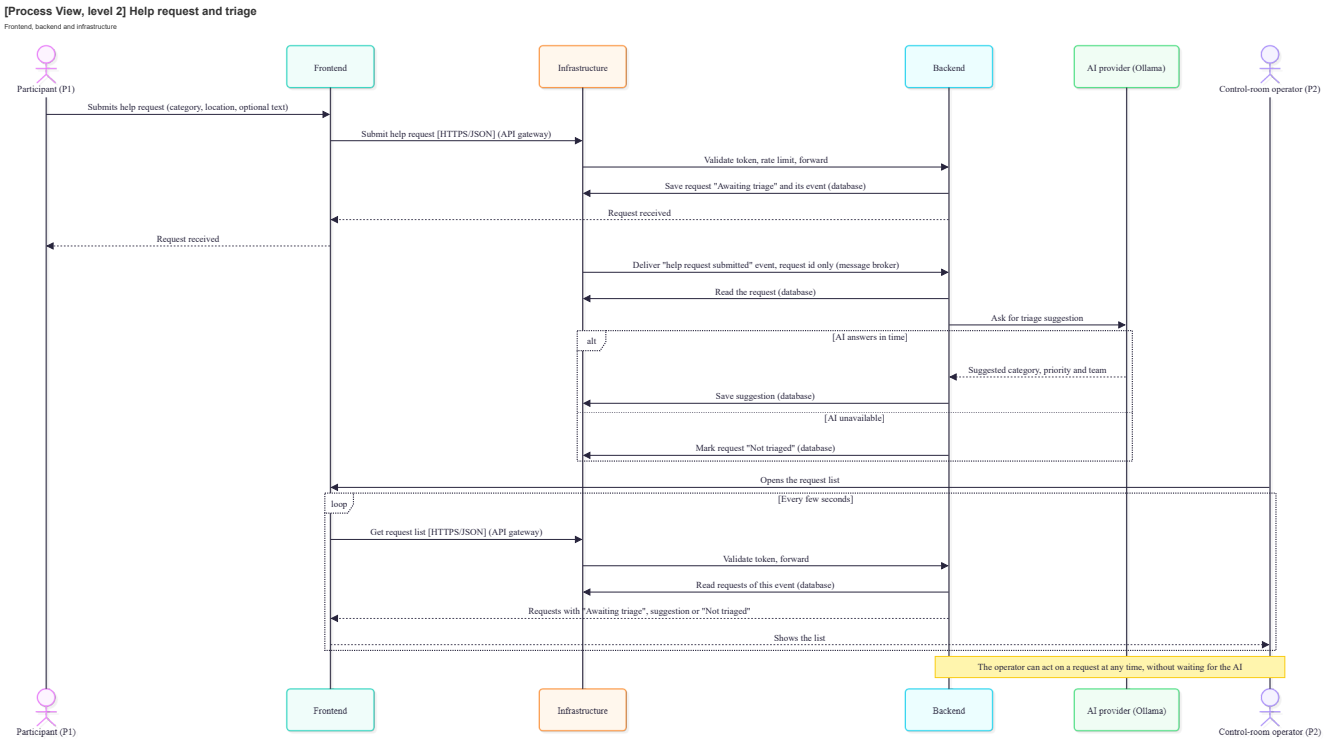
logical-view-level-4-transports.drawio.svg



process-view-level-1-help-request.drawio.svg

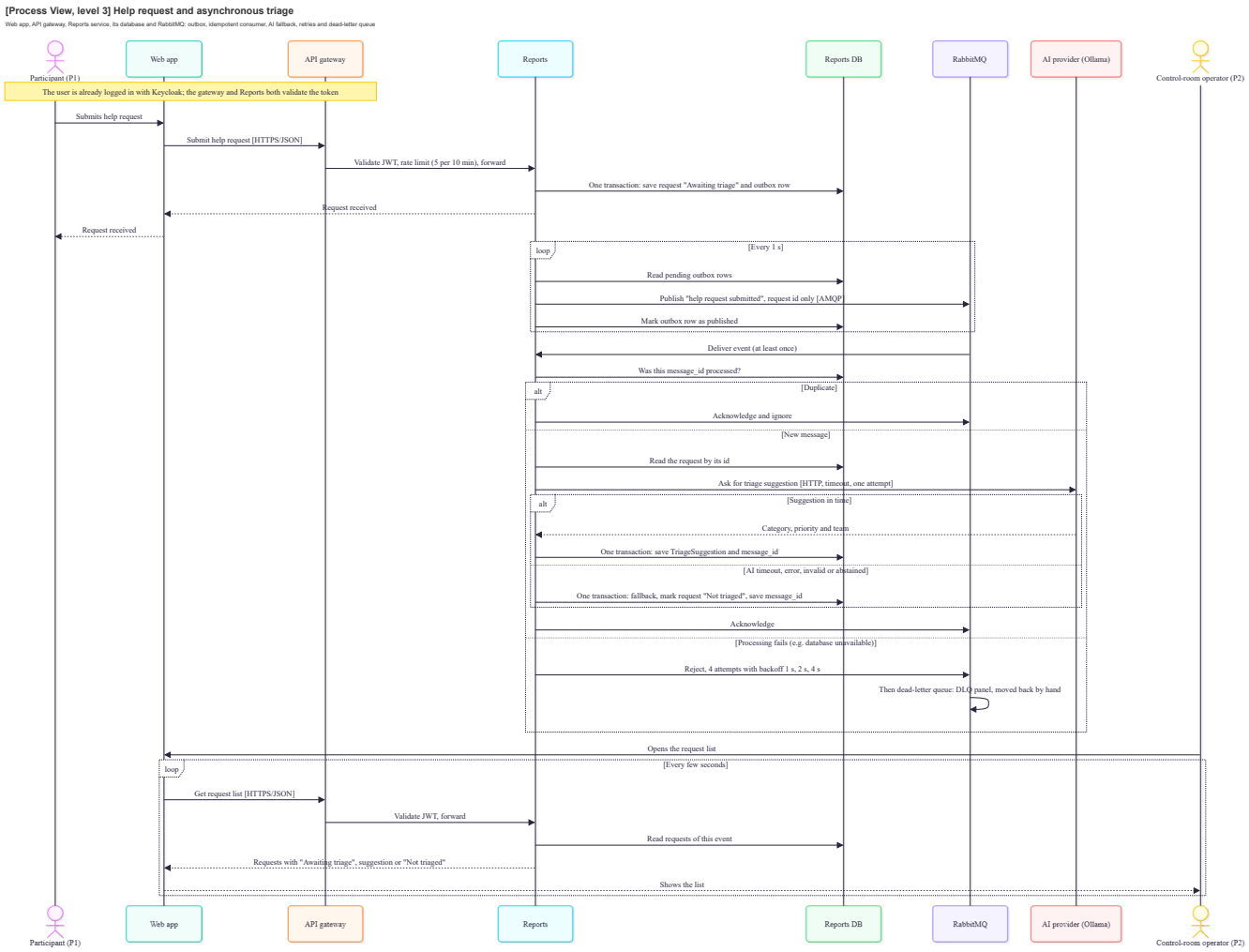


process-view-level-2-help-request.drawio.svg



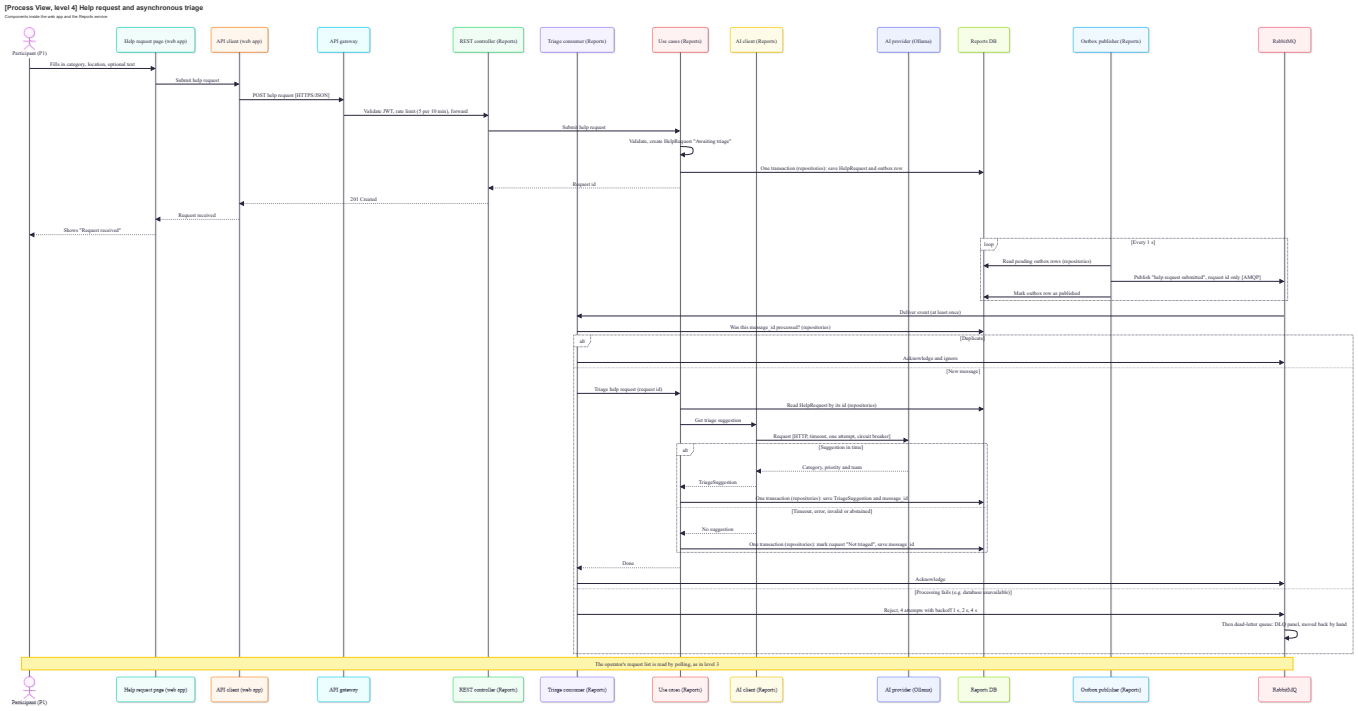
Docs/Technical/diagrams/process-view-level-3-help-request.drawio.svg

process-view-level-3-help-request.drawio.svg

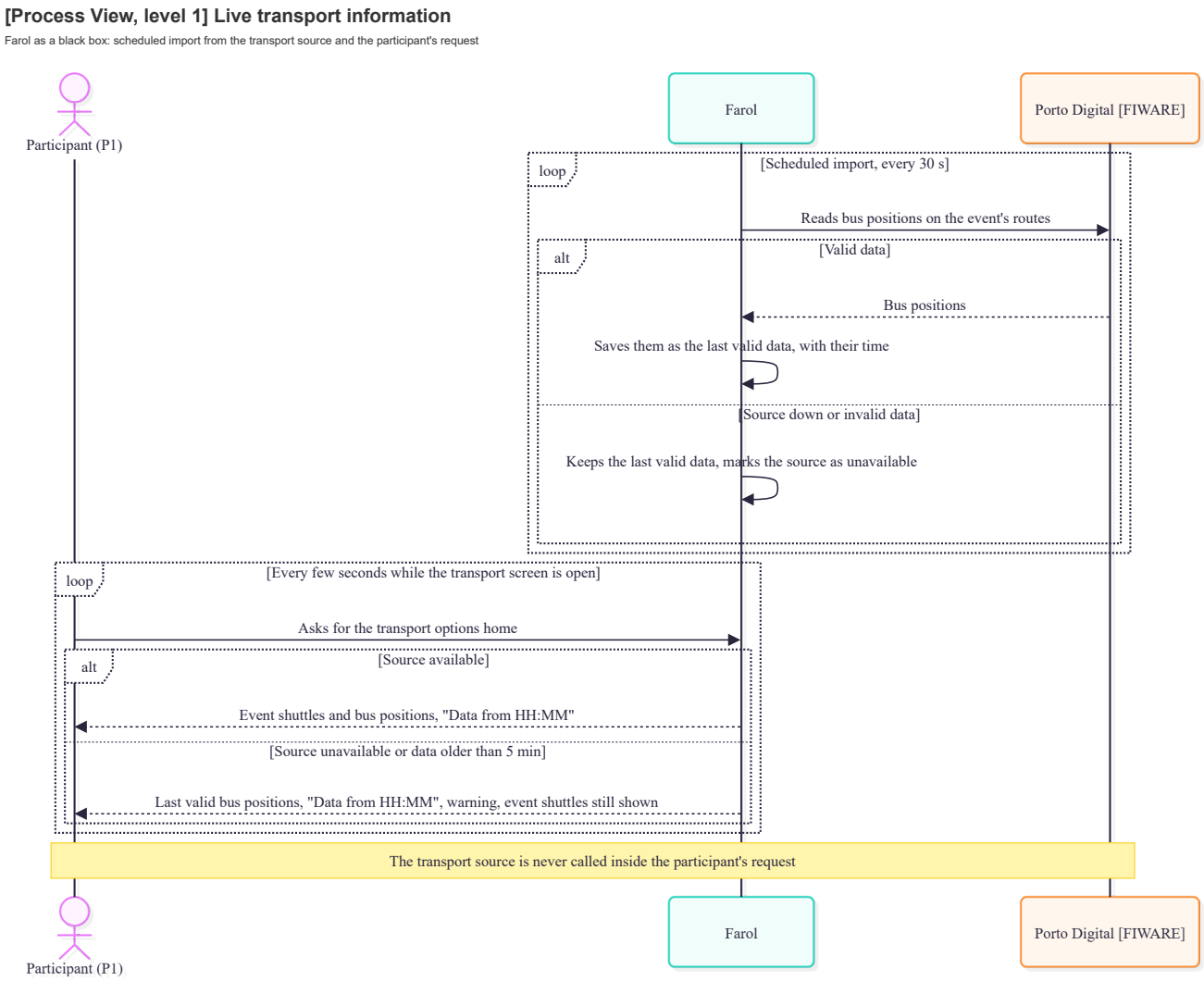


Docs/Technical/diagrams/process-view-level-4-help-request.drawio.svg

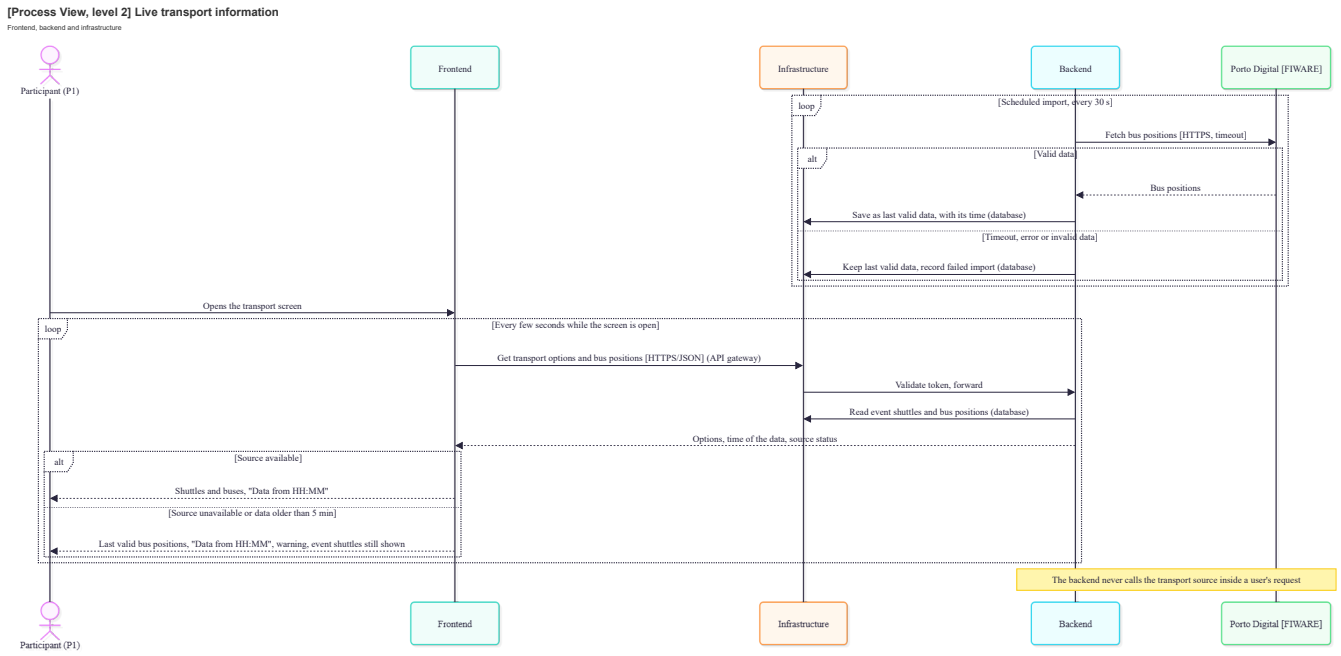
process-view-level-4-help-request.drawio.svg



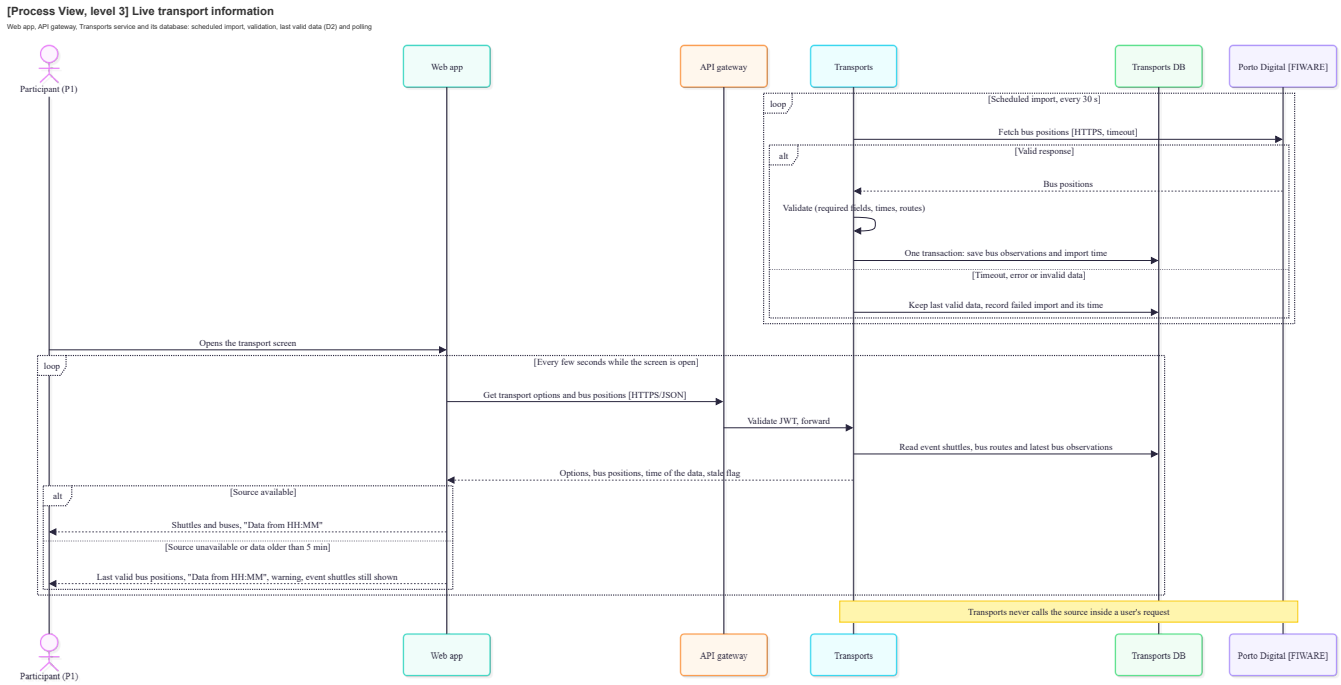
process-view-level-1-transports.drawio.svg



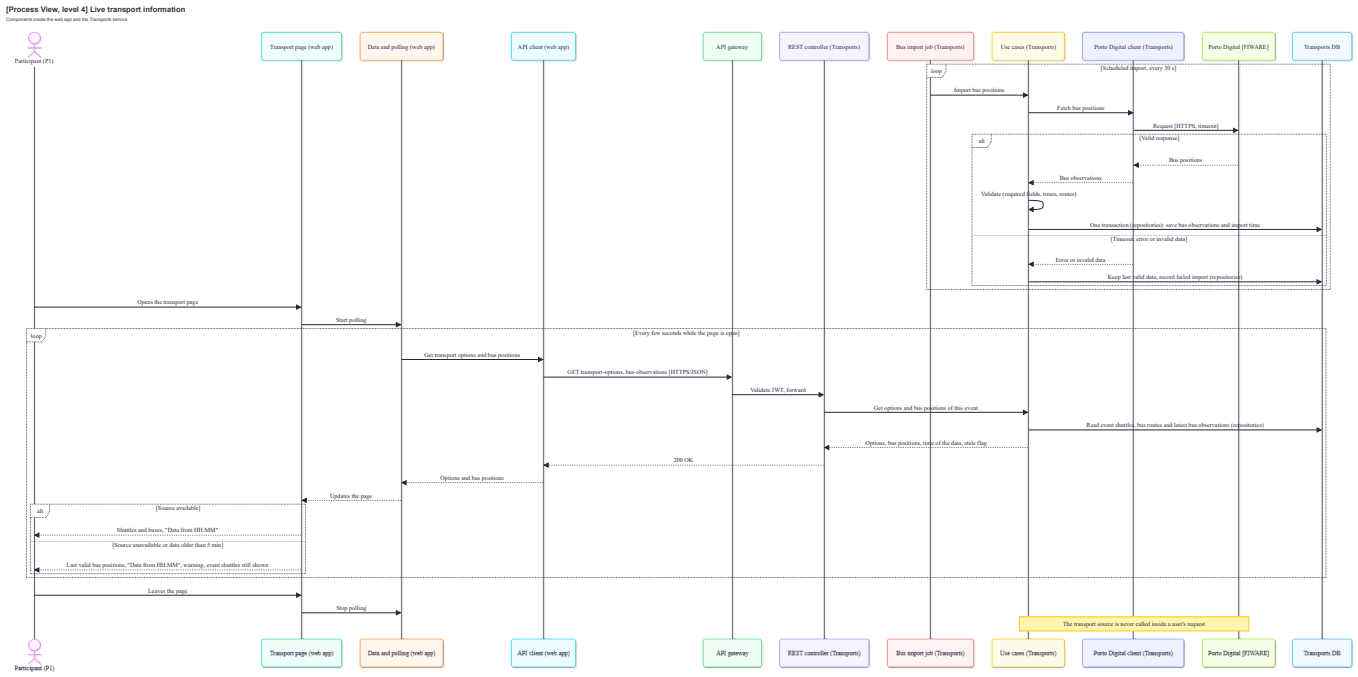
process-view-level-2-transports.drawio.svg



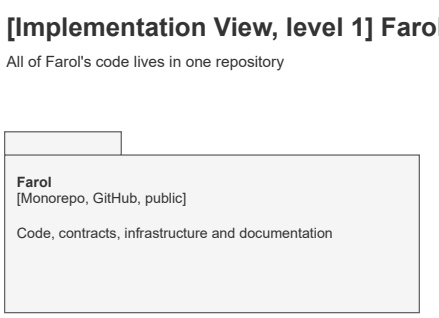
process-view-level-3-transports.drawio.svg



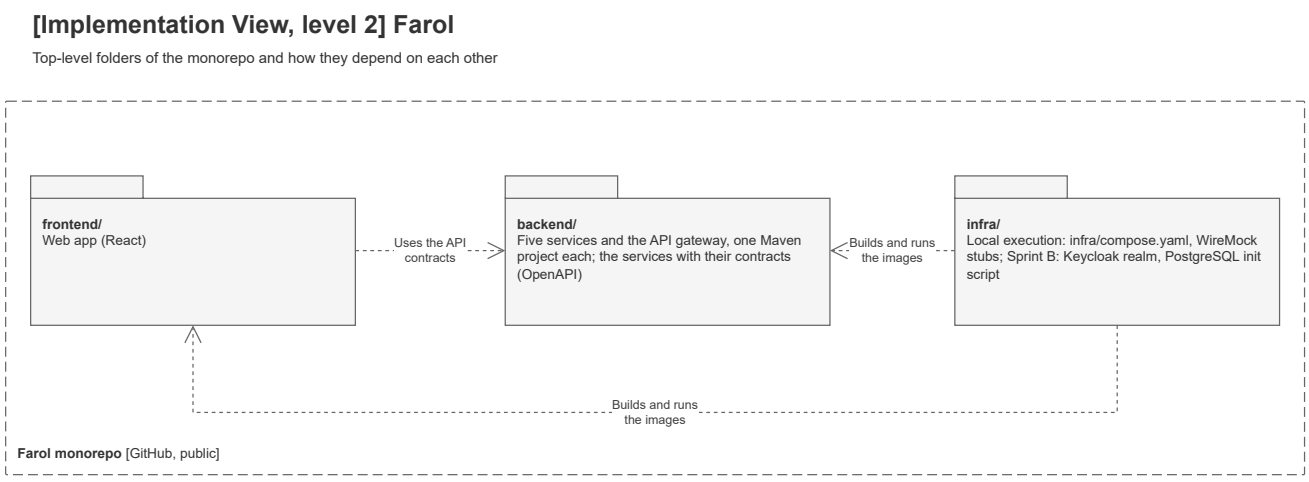
process-view-level-4-transports.drawio.svg



implementation-view-level-1.drawio.svg



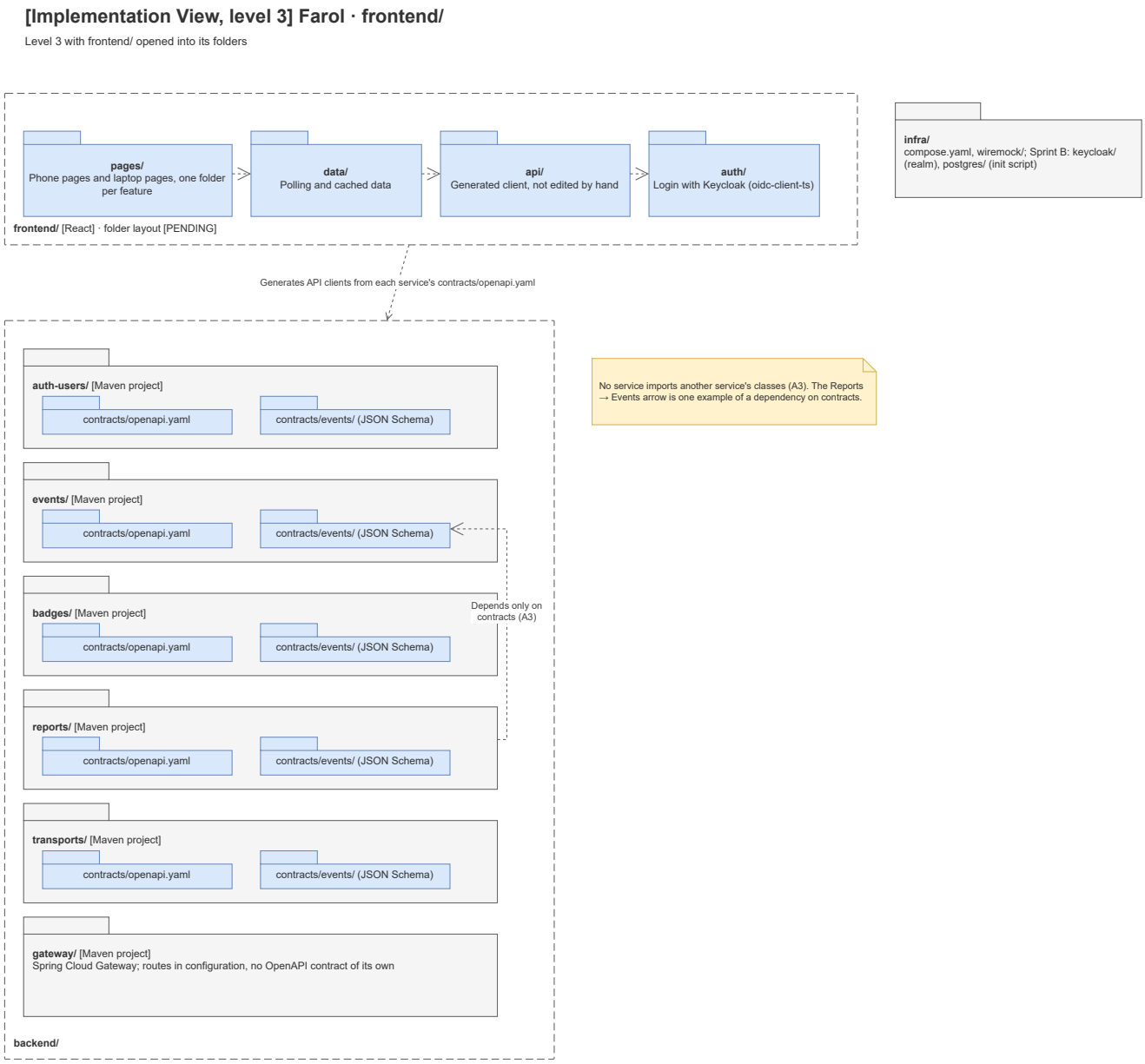
implementation-view-level-2.drawio.svg



implementation-view-level-3.drawio.svg

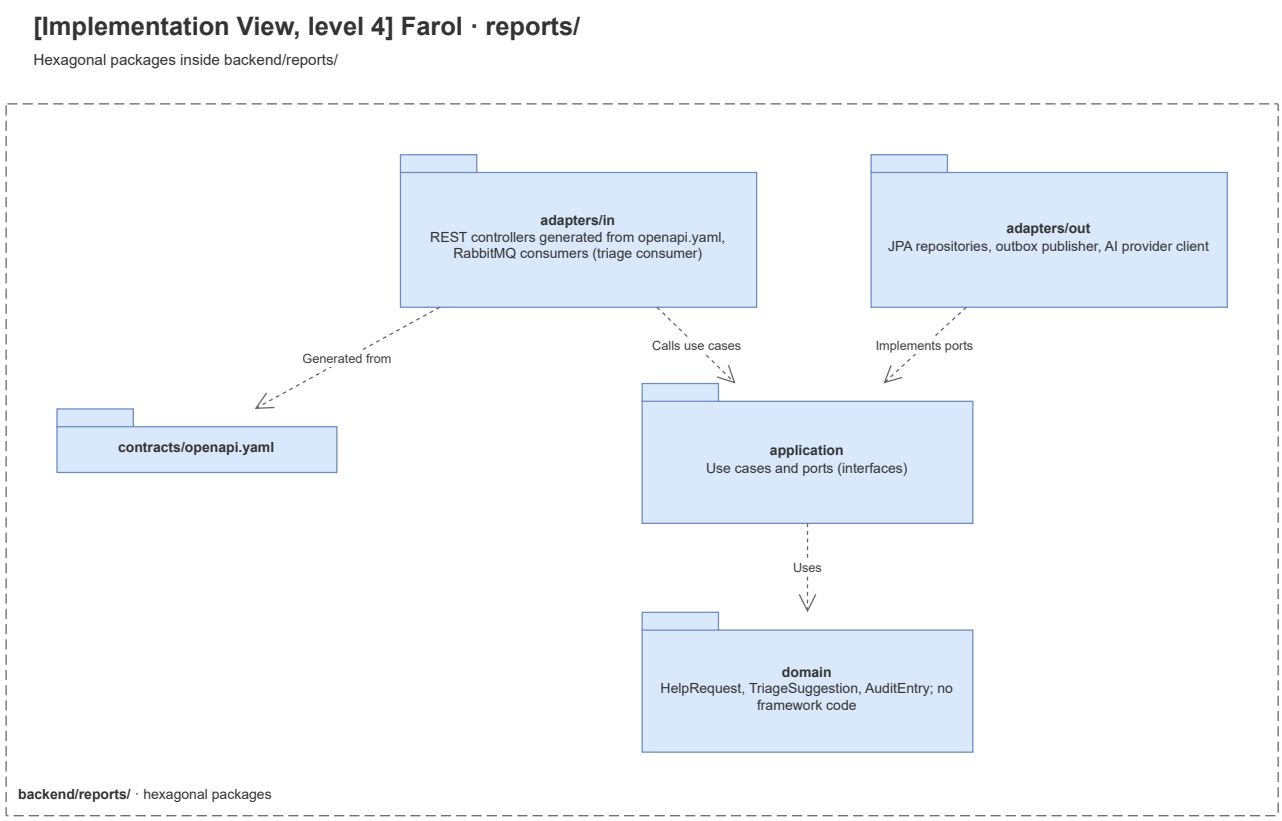


implementation-view-level-3-web.drawio.svg

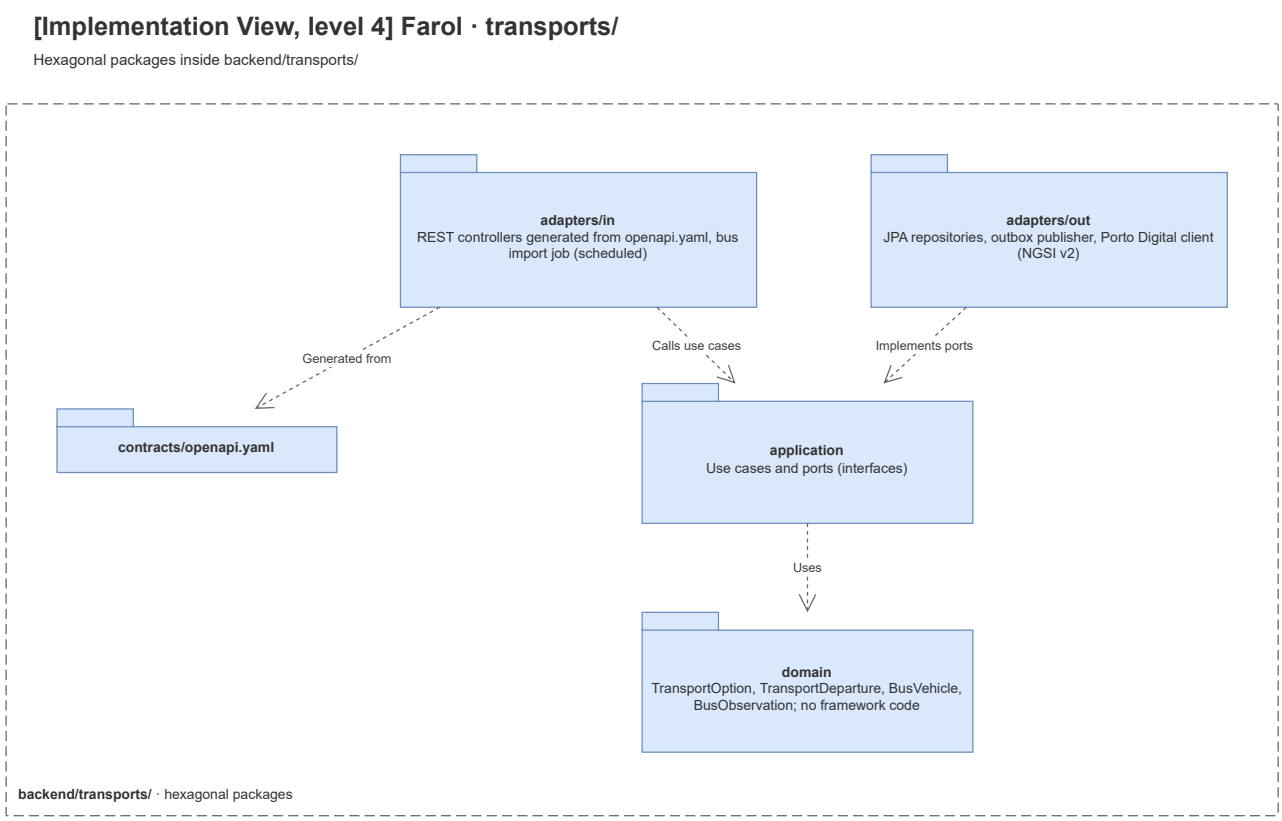


Docs/Technical/diagrams/implementation-view-level-4-reports.drawio.svg

implementation-view-level-4-reports.drawio.svg



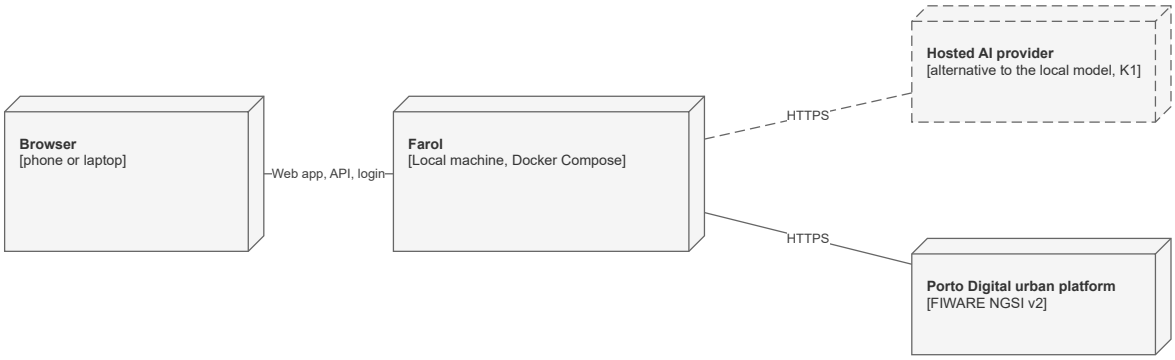
implementation-view-level-4-transports.drawio.svg



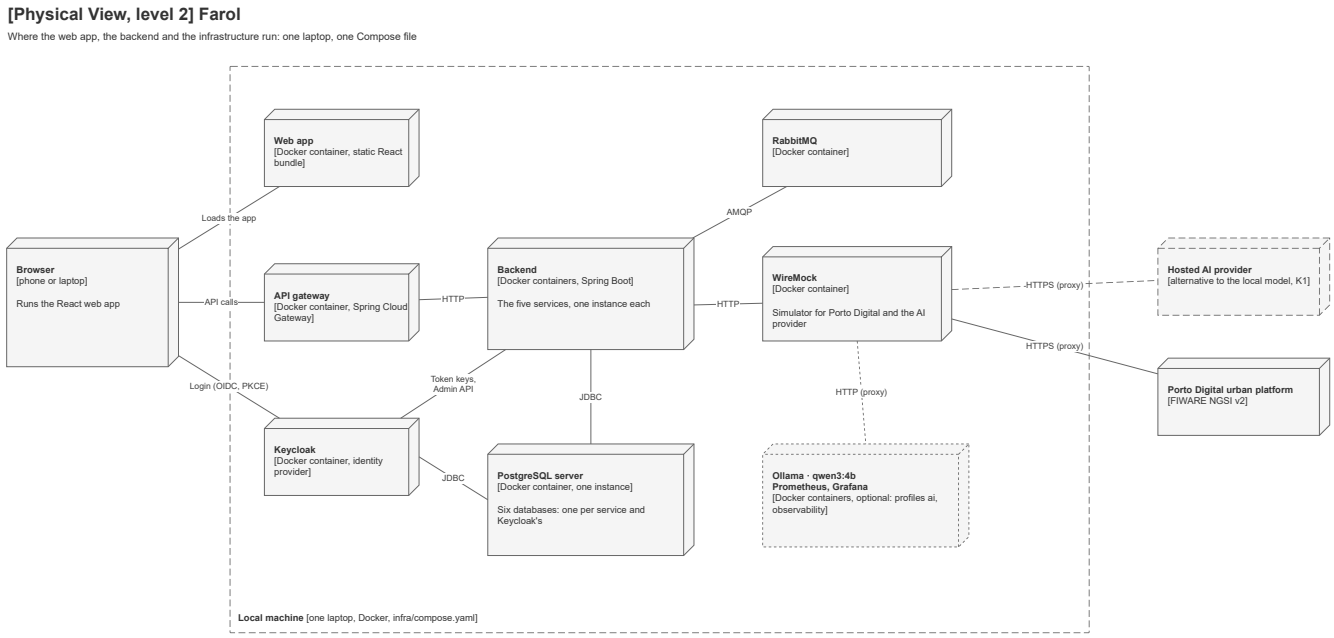
physical-view-level-1.drawio.svg

[Physical View, level 1] Farol

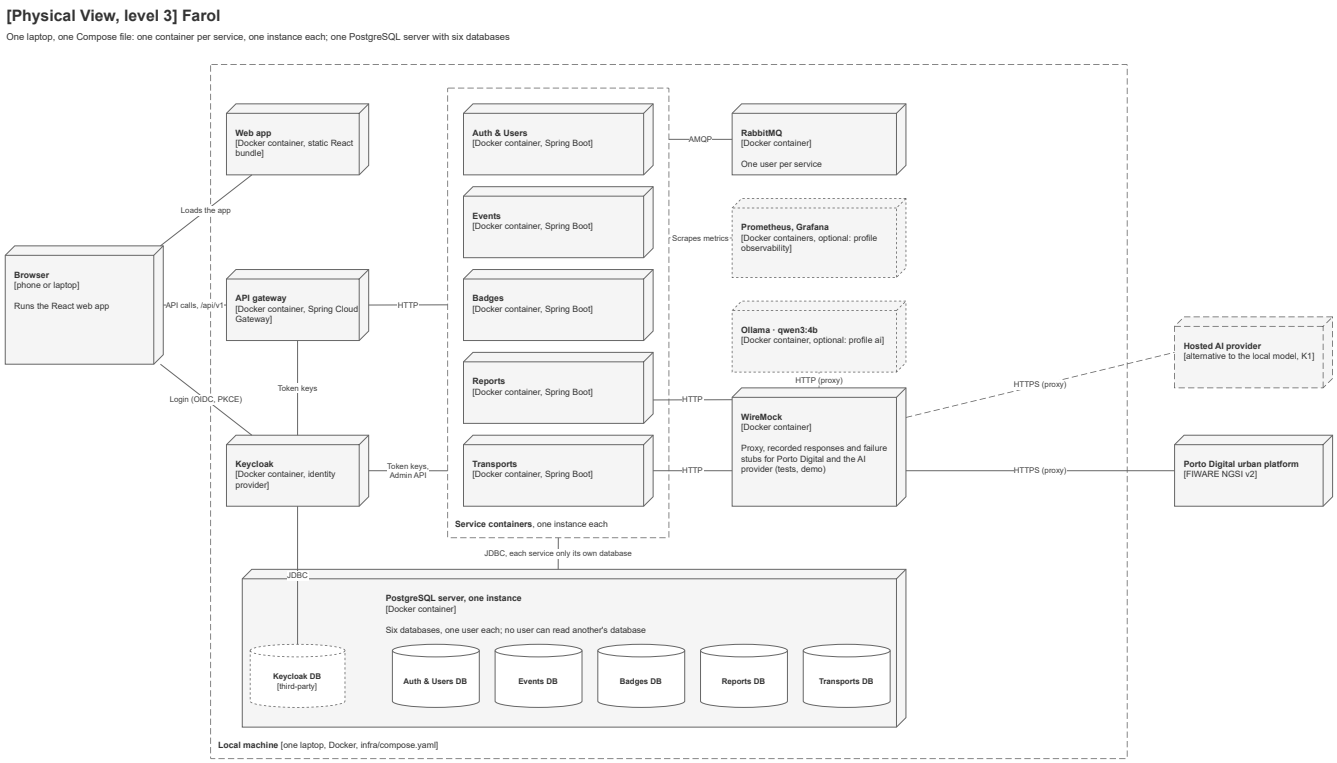
Farol on one local machine, the browser that uses it and the external servers it calls



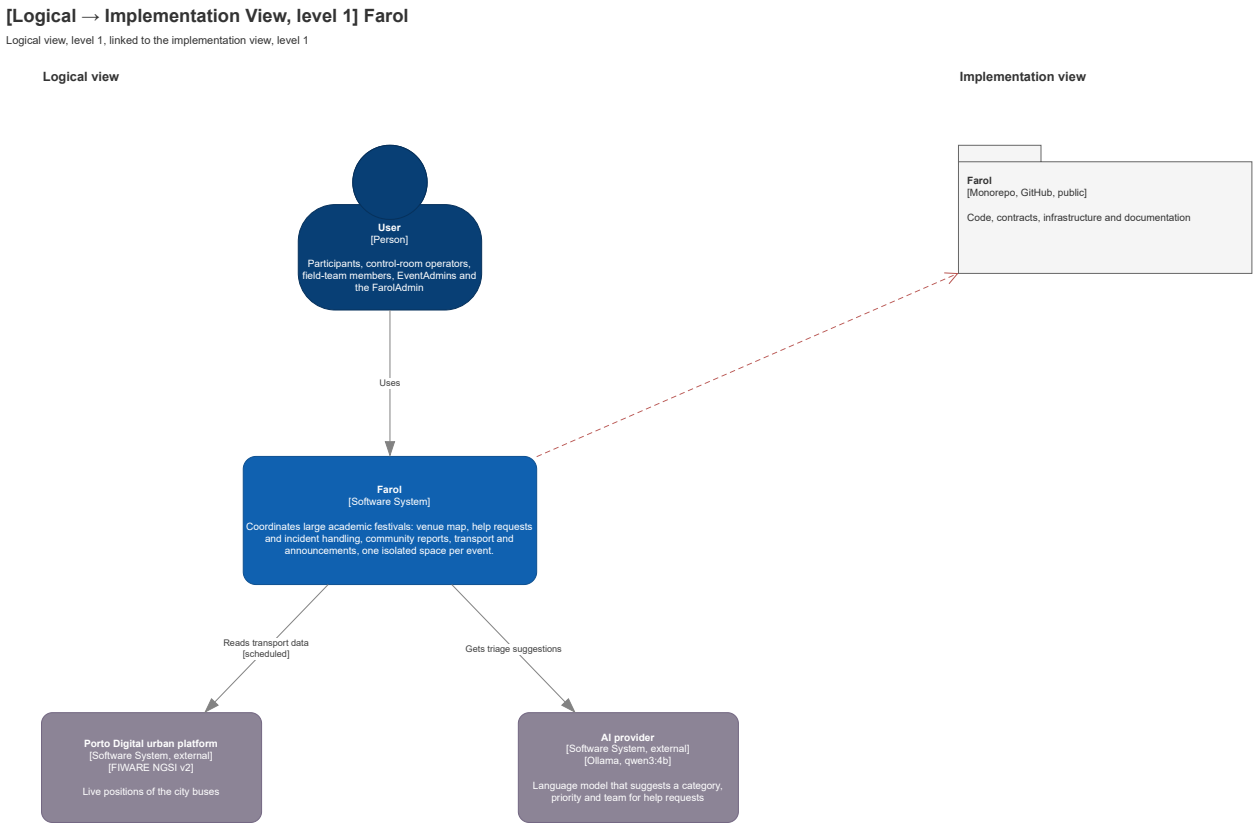
physical-view-level-2.drawio.svg



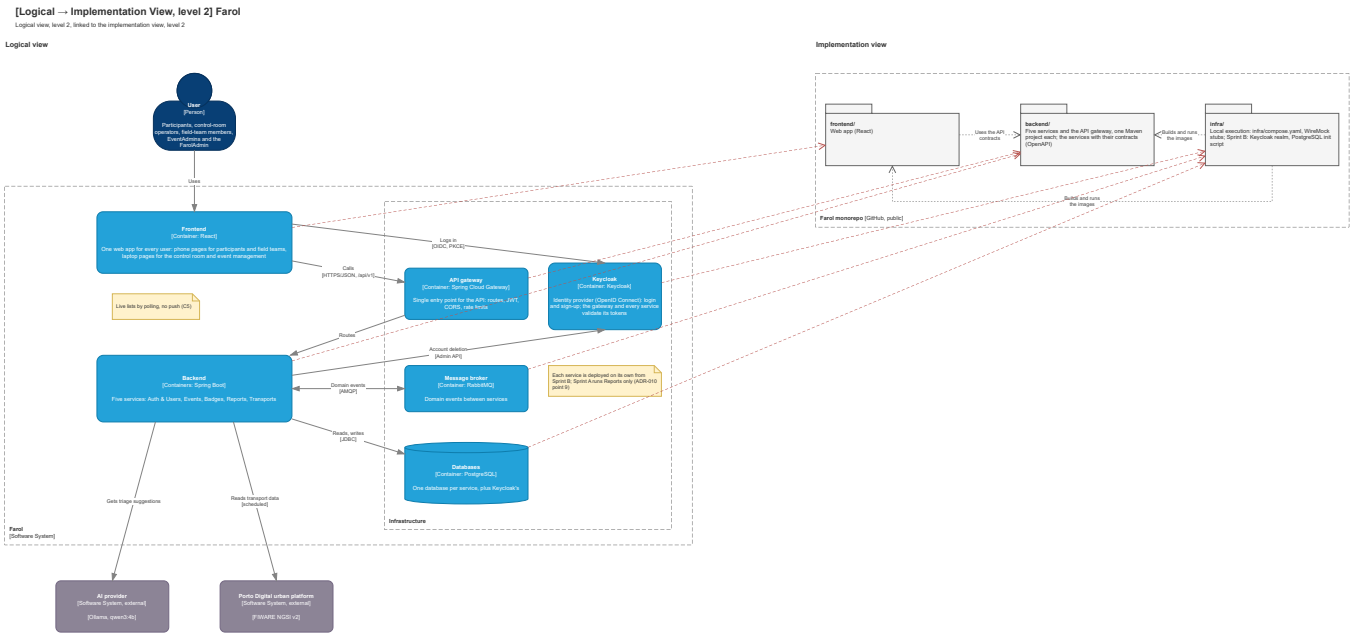
physical-view-level-3.drawio.svg



mapping-logical-implementation-level-1.drawio.svg

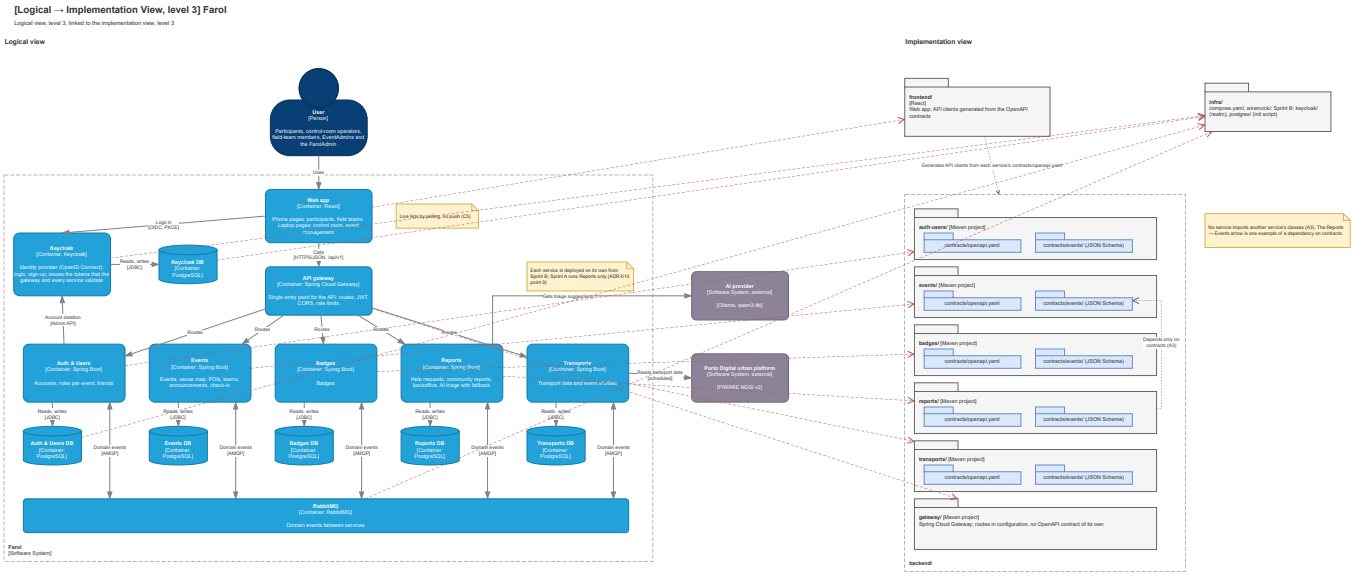


mapping-logical-implementation-level-2.drawio.svg

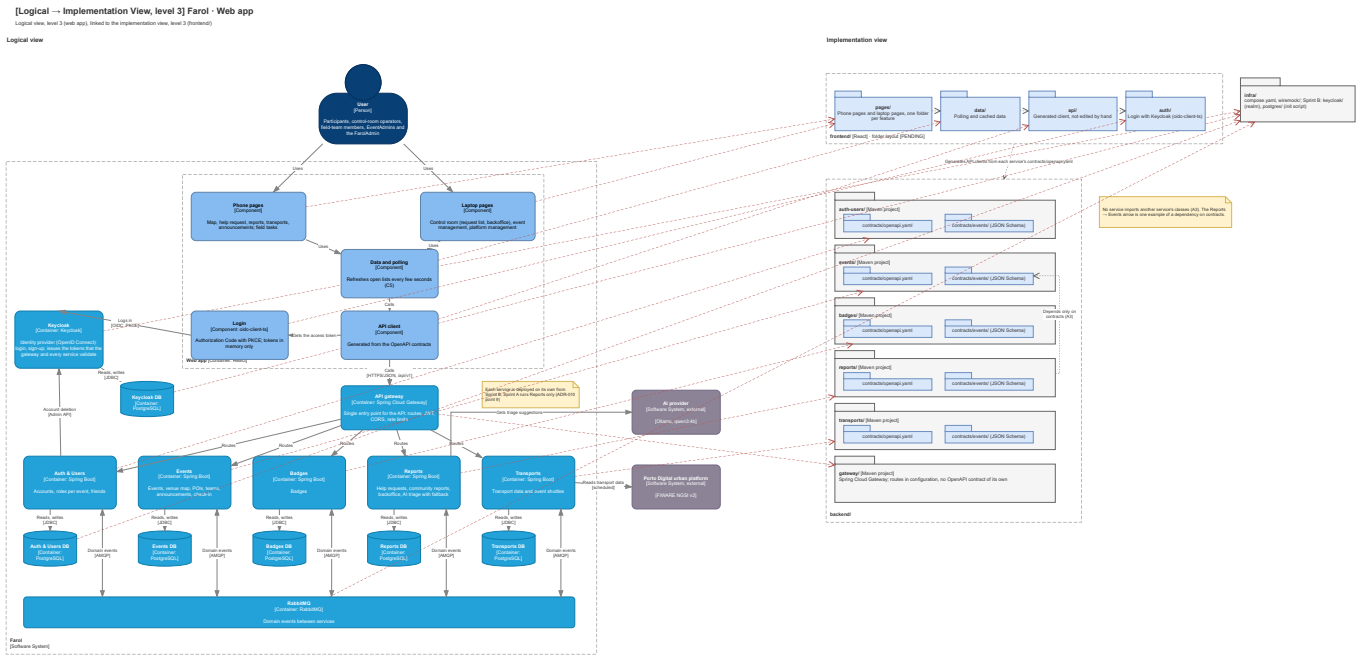


Docs/Technical/diagrams/mapping-logical-implementation-level-3.drawio.svg

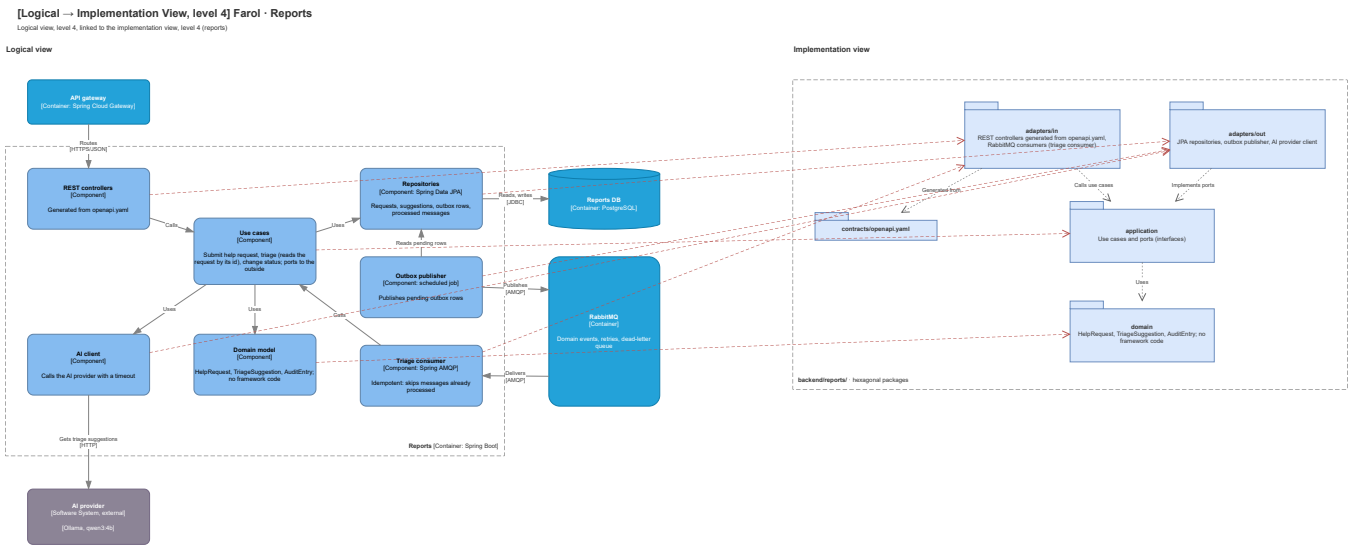
mapping-logical-implementation-level-3.drawio.svg



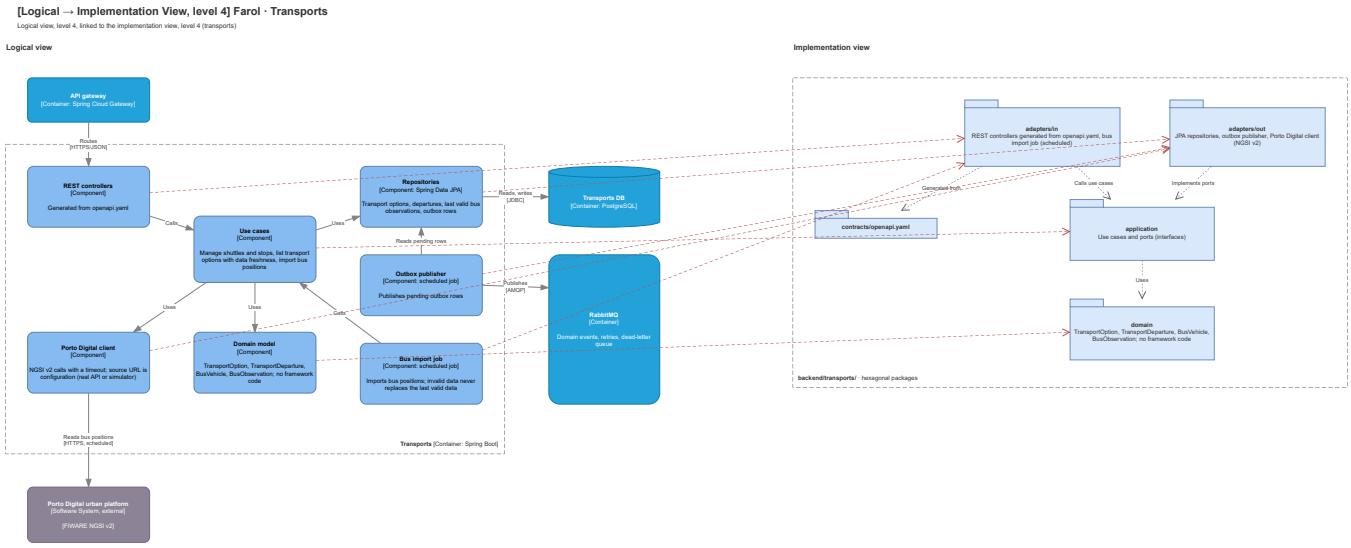
mapping-logical-implementation-level-3-web.drawio.svg



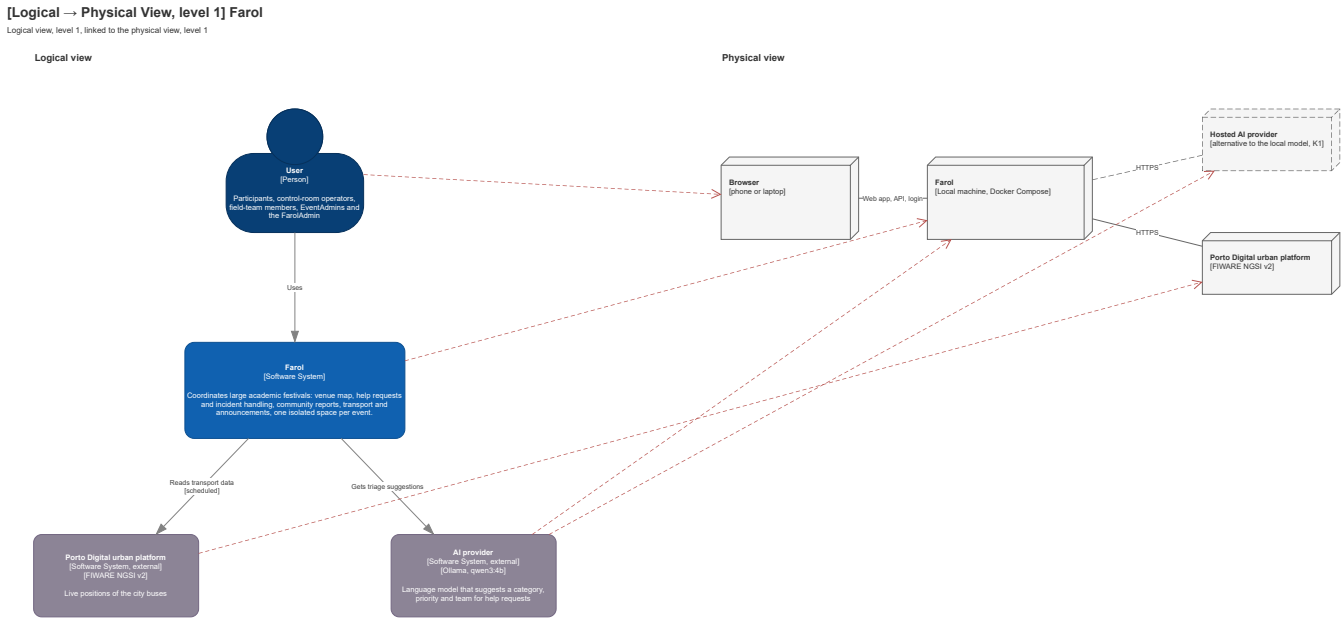
mapping-logical-implementation-level-4-reports.drawio.svg



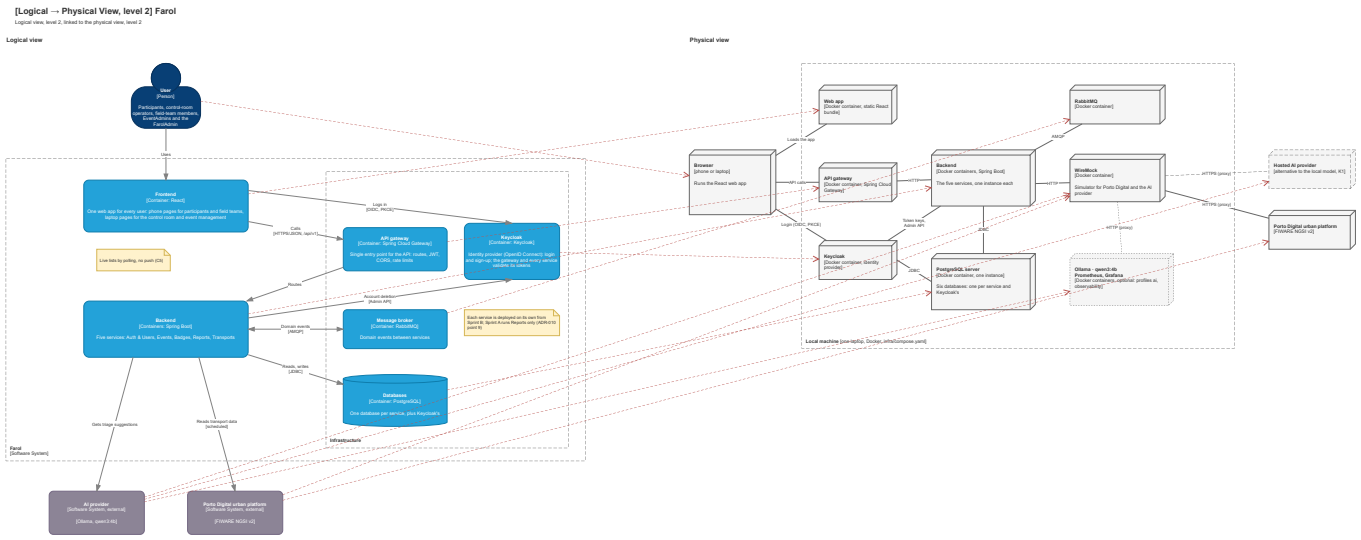
mapping-logical-implementation-level-4-transports.drawio.svg



mapping-logical-physical-level-1.drawio.svg



mapping-logical-physical-level-2.drawio.svg



Docs/Technical/diagrams/mapping-logical-physical-level-3.drawio.svg

mapping-logical-physical-level-3.drawio.svg

